

Effective C++: A Practical Study Guide

Ajay Kumar

August 2026

Contents

1	Chapter 1: Accustoming Yourself to C++	3
1.1	Chapter Flow	3
1.2	Item 1: View C++ as a Federation of Languages	3
1.3	Visual Summary	3
1.4	Item 2: Prefer <code>const</code> , <code>enum</code> , and <code>inline</code> to <code>#define</code>	7
1.5	Visual Summary	7
1.6	Item 3: Use <code>const</code> Whenever Possible	10
1.7	Visual Summary	10
1.8	Item 4: Make Sure That Objects Are Initialized Before They're Used	13
1.9	Visual Summary	13
2	Chapter 2: Constructors, Destructors, and Assignment Operators	17
2.1	Chapter Flow	17
2.2	Item 5: Know What Functions C++ Silently Writes and Calls	18
2.3	Visual Summary	18
2.4	Item 6: Explicitly Disallow the Use of Compiler-Generated Functions You Do Not Want	24
2.5	Visual Summary	24
2.6	Item 7: Declare Destructors Virtual in Polymorphic Base Classes	30
2.7	Visual Summary	30
2.8	Item 8: Prevent Exceptions from Leaving Destructors	36
2.9	Visual Summary	36
2.10	Item 9: Never Call Virtual Functions During Construction or Destruction	42
2.11	Visual Summary	42
2.12	Item 10: Have Assignment Operators Return a Reference to <code>*this</code>	49
2.13	Visual Summary	49
2.14	Item 11: Handle Assignment to Self in <code>operator=</code>	53
2.15	Visual Summary	53
2.16	Item 12: Copy All Parts of an Object	59
2.17	Visual Summary	59
3	Chapter 3: Resource Management	67
3.1	Chapter Flow	67
3.2	Item 13: Use objects to manage resources	68
3.3	Visual Summary	68
3.4	Item 14: Think carefully about copying behavior in resource-managing classes	74
3.5	Visual Summary	74
3.6	Item 15: Provide access to raw resources in resource-managing classes	81
3.7	Visual Summary	81
3.8	Item 16: Use the same form in corresponding uses of <code>new</code> and <code>delete</code>	85
3.9	Visual Summary	85
3.10	Item 17: Store newed objects in smart pointers in standalone statements	90
3.11	Visual Summary	90
3.12	Summary of Chapter 3	95
4	Chapter 4: Designs and Declarations	95
4.1	Chapter Flow	95
4.2	Item 18: Make Interfaces Easy to Use Correctly and Hard to Use Incorrectly	95

4.3	Visual Summary	95
4.4	Item 19: Treat Class Design as Type Design	100
4.5	Visual Summary	100
4.6	Item 20: Prefer Pass-by-Reference-to-const to Pass-by-Value	104
4.7	Visual Summary	104
4.8	Item 21: Don't Try to Return a Reference When You Must Return an Object	108
4.9	Visual Summary	108
4.10	Item 22: Declare Data Members Private	112
4.11	Visual Summary	112
4.12	Item 23: Prefer Non-Member Non-Friend Functions to Member Functions	116
4.13	Visual Summary	116
4.14	Item 24: Declare Non-Member Functions When Type Conversions Should Apply to All Parameters	121
4.15	Visual Summary	121
4.16	Item 25: Consider Support for a Non-Throwing Swap	125
4.17	Visual Summary	125
4.18	Item 26: Postpone Variable Definitions as Long as Possible	130
4.19	Visual Summary	130
4.20	Summary	135
5	Chapter 5: Implementations	135
5.1	Chapter Flow	135
5.2	Item 27: Minimize casting	136
5.3	Visual Summary	136
5.4	Item 28: Avoid returning "handles" to object internals	143
5.5	Visual Summary	143
5.6	Item 29: Strive for exception-safe code	148
5.7	Visual Summary	148
5.8	Item 30: Understand the ins and outs of inlining	154
5.9	Visual Summary	154
5.10	Item 31: Minimize compilation dependencies between files	159
5.11	Visual Summary	159
6	Chapter 6: Inheritance and Object-Oriented Design	169
6.1	Chapter Flow	169
6.2	Item 32: Make sure public inheritance models "is-a"	169
6.3	Visual Summary	169
6.4	Item 33: Avoid hiding inherited names	173
6.5	Visual Summary	173
6.6	Item 34: Differentiate between inheritance of interface and inheritance of implementation	177
6.7	Visual Summary	177
6.8	Item 35: Consider alternatives to virtual functions	182
6.9	Visual Summary	182
6.10	Item 36: Never redefine an inherited non-virtual function	188
6.11	Visual Summary	188
6.12	Item 37: Never redefine a function's inherited default parameter value	191
6.13	Visual Summary	191
6.14	Item 38: Model "has-a" or "is-implemented-in-terms-of" through composition	194
6.15	Visual Summary	194
6.16	Item 39: Use private inheritance judiciously	198
6.17	Visual Summary	198
6.18	Item 40: Use multiple inheritance judiciously	202
6.19	Visual Summary	202
6.20	Summary	209
7	Chapter 7: Templates and Generic Programming	209
7.1	Chapter Flow	209
7.2	Item 41: Understand implicit interfaces and compile-time polymorphism	210
7.3	Visual Summary	210
7.4	Item 42: Understand the two meanings of typename	213
7.5	Visual Summary	213
7.6	Item 43: Know how to access names in templated base classes	217

7.7	Visual Summary	217
7.8	Item 44: Factor parameter-independent code out of templates	221
7.9	Visual Summary	221
7.10	Item 45: Use member function templates to accept “all compatible types”	225
7.11	Visual Summary	225
7.12	Item 46: Define non-member functions inside templates when type conversions are desired	231
7.13	Visual Summary	231
7.14	Item 47: Use traits classes for information about types	235
7.15	Visual Summary	235
7.16	Item 48: Be aware of template metaprogramming	242
7.17	Visual Summary	242
8	Chapter 8: Customizing new and delete	252
8.1	Chapter Flow	252
8.2	Item 49: Understand the Behavior of the new-handler	252
8.3	Visual Summary	252
8.4	Item 50: Understand When It Makes Sense to Replace new and delete	258
8.5	Visual Summary	258
8.6	Item 51: Adhere to Convention When Writing new and delete	266
8.7	Visual Summary	266
8.8	Item 52: Write Placement delete If You Write Placement new	273
8.9	Visual Summary	273
8.10	Summary of Chapter 8	282
9	Chapter 9: Miscellany	282
9.1	Chapter Flow	282
9.2	Item 53: Pay Attention to Compiler Warnings	282
9.3	Visual Summary	282
9.4	Item 54: Familiarize Yourself with the Standard Library, Including TR1	293
9.5	Visual Summary	293
9.6	Item 55: Familiarize Yourself with Boost	313
9.7	Visual Summary	313
9.8	Summary of Chapter 9	321

1 Chapter 1: Accustoming Yourself to C++

1.1 Chapter Flow

CHAPTER 1: ACCUSTOMING YOURSELF TO C++
Item 1 -> choose the right C++ sub-language mental model.
Item 2 -> replace preprocessor tricks with compiler-visible constructs.
Item 3 -> use const to express and enforce non-mutation.
Item 4 -> initialize before use; avoid cross-file static-order traps.

Items 1–4: The philosophical and practical foundation for effective C++ programming.

1.2 Item 1: View C++ as a Federation of Languages

1.3 Visual Summary

ITEM 1: VIEW C++ AS A FEDERATION OF LANGUAGES

1. Problem: treating C++ like one language makes rules feel inconsistent.
2. C layer -> built-ins, pointers, macros: cheap values, manual discipline.
3. OO layer -> classes and virtual dispatch: interfaces, invariants, refs.
4. Template layer -> type parameters: compile-time contracts and code gen.
5. STL layer -> containers + iterators + algorithms: value-like conventions.
6. Meaning: first identify the sub-language, then choose the matching rule.

1.3.1 Core Concept

C++ is not a single unified language — it is a **federation of four sub-languages**, each with its own conventions, idioms, and best practices. The rules for effective programming change depending on which sub-language you're working in.

1.3.2 The Four Sub-Languages

```
// C-style programming: blocks, statements, preprocessor, built-in types,  
// arrays, pointers, pass-by-value  
void process_data(int* array, int size) {  
    for (int i = 0; i < size; ++i) {  
        array[i] *= 2; // Direct pointer/array manipulation  
    }  
}  
  
// Preprocessor macros (C heritage)  
#define MAX(a, b) ((a) > (b) ? (a) : (b))  
// Pitfall: macro arguments evaluated multiple times  
int x = 5, y = 3;  
int result = MAX(++x, y); // x incremented TWICE! Undefined behavior territory
```

1.3.2.1 1. C — The Foundation

```

// Classes, encapsulation, inheritance, polymorphism, virtual functions
class Shape {
public:
    virtual ~Shape() = default;
    virtual double area() const = 0;
    virtual void draw() const = 0;

protected:
    std::string color_;
};

class Circle : public Shape {
public:
    explicit Circle(double radius) : radius_(radius) {}

    double area() const override {
        return M_PI * radius_ * radius_;
    }

    void draw() const override {
        std::cout << "Drawing circle with radius " << radius_ << "\n";
    }

private:
    double radius_;
};

// Polymorphic usage
void render(const Shape& shape) {
    shape.draw(); // Virtual dispatch – runtime polymorphism
    std::cout << "Area: " << shape.area() << "\n";
}

```

1.3.2.2 2. Object-Oriented C++

```

// Generic programming and template metaprogramming
template<typename T>
class Stack {
public:
    void push(const T& value) {
        data_.push_back(value);
    }

    T pop() {
        if (data_.empty()) throw std::runtime_error("Stack empty");
        T top = data_.back();
        data_.pop_back();
        return top;
    }

    bool empty() const { return data_.empty(); }

private:
    std::vector<T> data_;
};

// Template metaprogramming – compile-time computation
template<unsigned N>

```

```

struct Factorial {
    static constexpr unsigned value = N * Factorial<N - 1>::value;
};

template<>
struct Factorial<0> {
    static constexpr unsigned value = 1;
};

// Factorial<5>::value == 120, computed at COMPILE TIME
static_assert(Factorial<5>::value == 120, "5! should be 120");

```

1.3.2.3 3. Template C++

```

// Containers, iterators, algorithms, function objects
#include <vector>
#include <algorithm>
#include <numeric>
#include <functional>

void stl_example() {
    std::vector<int> nums = {5, 3, 1, 4, 2};

    // Algorithm + iterator paradigm
    std::sort(nums.begin(), nums.end());

    // Function objects
    auto sum = std::accumulate(nums.begin(), nums.end(), 0);

    // Lambda as function object
    auto count_even = std::count_if(nums.begin(), nums.end(),
        [](int n) { return n % 2 == 0; });

    // STL convention: pass by value for iterators and function objects
    // (they are designed to be cheap to copy)
}

```

1.3.2.4 4. The STL

1.3.3 Why This Matters: Rules Change by Sub-Language

```

// RULE: Pass-by-value vs. pass-by-reference depends on the sub-language

// C sub-language: pass by value for built-in types
void increment(int x) { /* efficient */ }

// OO C++: pass by reference-to-const for user-defined types
void process(const Widget& w) { /* avoids expensive copy */ }

// Template C++: depends on the type parameter
template<typename T>
void handle(const T& t) { /* safe default */ }

// STL: pass iterators and function objects by value
std::for_each(v.begin(), v.end(), MyFunctor());

```

1.3.4 Things to Remember

- C++ rules vary depending on which part of C++ you're using

- Pass-by-value is fine for built-in types and STL iterators/function objects
 - Pass-by-reference-to-const is preferred for user-defined types
 - For template code, the right approach depends on the template parameter
-

1.4 Item 2: Prefer const, enum, and inline to #define

1.5 Visual Summary

ITEM 2: PREFER `CONST`, `ENUM`, AND `INLINE` TO `#DEFINE`

1. Macro enters preprocessor -> compiler sees only substituted text.
2. Consequence -> no scope, weak type checking, poor debugger names.
3. Constant value? -> use const or enum so compiler owns the symbol.
4. Function-like macro? -> use inline/template function for one evaluation.
5. Meaning: prefer language constructs because the compiler can protect you.

1.5.1 Core Concept

“Prefer the compiler to the preprocessor.” #define operates before the compiler even sees your code, leading to confusing errors, no type safety, and no scope control.

1.5.2 Problem: #define for Constants

```
// BAD: Preprocessor macro
#define ASPECT_RATIO 1.653
// Problem 1: "ASPECT_RATIO" may never be seen by compiler – error messages
//           will refer to "1.653", making debugging confusing
// Problem 2: No type information
// Problem 3: No scope – pollutes entire translation unit

// GOOD: typed constant
const double AspectRatio = 1.653;
// – Appears in symbol table – better error messages and debugger support
// – Type-checked by the compiler
// – Respects scope rules
```

1.5.3 Special Cases for Constants

```

// For constant char* strings in headers, you need TWO consts
const char* const authorName = "Scott Meyers";
// ^^^^^      ^^^^^
// |           pointer itself is const
// what pointer points to is const

// Better: use std::string
const std::string authorName("Scott Meyers");

```

1.5.3.1 Constant Pointers

```

class GamePlayer {
private:
    // Declaration of a class-specific constant
    static const int NumTurns = 5; // In-class initialization (integral types only)
    int scores[NumTurns];          // Use of the constant

    // For non-integral types or when you need an address:
    static const double FudgeFactor; // Declaration only
};

// Definition in .cpp file (required if address is ever taken)
const int GamePlayer::NumTurns; // Definition; no value given (already in declaration)
const double GamePlayer::FudgeFactor = 1.35; // Definition with value

```

1.5.3.2 Class-Specific Constants

1.5.4 The enum Hack

```

class GamePlayer {
private:
    // When your compiler won't allow in-class initialization:
    enum { NumTurns = 5 }; // "the enum hack"
    int scores[NumTurns]; // Works fine

    // Advantages of enum hack:
    // 1. It's impossible to take the address of an enum (more like #define)
    // 2. No memory allocated for the enum value
    // 3. Pragmatic – widely used in template metaprogramming
};

// enum hack in template metaprogramming
template<typename T>
struct TypeTraits {
    enum { isPointer = 0 };
};

template<typename T>
struct TypeTraits<T*> {
    enum { isPointer = 1 }; // Specialization for pointer types
};

```

1.5.5 Problem: #define for Function-Like Macros


```

// BAD: macro "function"
#define CALL_WITH_MAX(a, b) f((a) > (b) ? (a) : (b))

// Despite all the parentheses, this is STILL broken:
int a = 5, b = 0;
CALL_WITH_MAX(++a, b);      // a incremented TWICE
CALL_WITH_MAX(++a, b + 10); // a incremented ONCE
// Behavior depends on what b is compared to! Nightmare.

// GOOD: inline template function
template<typename T>
inline void callWithMax(const T& a, const T& b) {
    f(a > b ? a : b);
    // - Type safe
    // - Predictable evaluation (arguments evaluated exactly once)
    // - Respects scope and access control
    // - Can be debugged with a debugger
}

```

1.5.6 Comprehensive Example: Replacing All #define Patterns

```

// ===== BEFORE: #define everywhere =====
#define PI 3.14159265358979
#define MAX_BUFFER_SIZE 1024
#define MIN(a, b) ((a) < (b) ? (a) : (b))
#define SQUARE(x) ((x) * (x))
#define DEBUG_LOG(msg) std::cout << __FILE__ << ":" << __LINE__ << " " << msg << "\n"

// ===== AFTER: Modern C++ =====
constexpr double Pi = 3.14159265358979;
constexpr int MaxBufferSize = 1024;

template<typename T>
inline const T& minOf(const T& a, const T& b) {
    return a < b ? a : b;
}

template<typename T>
inline T square(const T& x) {
    return x * x;
}

// For debug logging, consider a proper logging framework
// or at minimum an inline function:
inline void debugLog(const char* file, int line, const std::string& msg) {
    std::cout << file << ":" << line << " " << msg << "\n";
}
#define DEBUG_LOG(msg) debugLog(__FILE__, __LINE__, msg)
// ^-- One of the FEW legitimate uses of #define:
//      __FILE__ and __LINE__ MUST be macros to get caller context

```

1.5.7 Things to Remember

- For simple constants, prefer const objects or enums to #defines
- For function-like macros, prefer inline functions to #defines
- #define doesn't respect scope, lacks type safety, and produces confusing errors
- The enum hack is a useful technique for compile-time integer constants

1.6 Item 3: Use const Whenever Possible

1.7 Visual Summary

ITEM 3: USE `CONST` WHENEVER POSSIBLE

1. You know something should not change -> say it with const.
2. Pointer constness -> decide whether pointer, pointee, or both are fixed.
3. Member function const -> callable on const objects and promises no logical change.
4. Return/parameter const -> blocks accidental writes and documents ownership.
5. Meaning: const turns design intent into compile-time checking.

1.7.1 Core Concept

const is one of C++'s most powerful and versatile tools. It allows you to communicate and enforce the semantic constraint that an object should not be modified. The compiler enforces this constraint, catching errors at compile time.

1.7.2 const with Pointers

```
char greeting[] = "Hello";

char* p = greeting;           // non-const pointer, non-const data
const char* p = greeting;     // non-const pointer, const data
char* const p = greeting;     // const pointer, non-const data
const char* const p = greeting; // const pointer, const data

// Rule: If const appears to the LEFT of the *, the data is const
// Rule: If const appears to the RIGHT of the *, the pointer is const

// These two are equivalent:
const char* p;    // pointer to const char
char const* p;    // pointer to const char (same thing, different style)
```

1.7.3 const with Iterators

```
#include <vector>

std::vector<int> vec = {1, 2, 3, 4, 5};

// const iterator = const pointer (T* const)
// The iterator itself can't change, but what it points to can
const std::vector<int>::iterator iter = vec.begin();
*iter = 10;    // OK: changes what iter points to
// ++iter;    // ERROR: iter is const

// iterator to const = pointer to const (const T*)
// The iterator can change, but what it points to can't
std::vector<int>::const_iterator cIter = vec.begin();
// *cIter = 10; // ERROR: *cIter is const
++cIter;    // OK: cIter itself isn't const
```

1.7.4 const Member Functions

```

class TextBlock {
public:
    // Two overloaded operator[]s – one for const objects, one for non-const
    const char& operator[](std::size_t position) const {
        // ... bounds checking, logging, etc.
        return text[position];
    }

    char& operator[](std::size_t position) {
        // ... bounds checking, logging, etc.
        return text[position];
    }

private:
    std::string text;
};

// Usage:
void print(const TextBlock& ctb) {
    std::cout << ctb[0]; // Calls const operator[]
    // ctb[0] = 'x';      // ERROR: const char& can't be assigned to
}

void modify(TextBlock& tb) {
    std::cout << tb[0]; // Calls non-const operator[]
    tb[0] = 'x';        // OK: char& can be assigned to
}

```

1.7.5 Bitwise vs. Logical Constness

```

// Bitwise constness: the compiler's definition
// A member function is const if it doesn't modify any data members
// BUT this can be too strict OR too lenient:

// Too lenient: bitwise const but logically non-const
class CTextBlock {
public:
    char& operator[](std::size_t position) const {
        return pText[position]; // Compiles! But allows modification through pointer
    }

private:
    char* pText; // The pointer isn't modified, but what it points to can be!
};

const CTextBlock cctb("Hello");
char* pc = &cctb[0];
*pc = 'J'; // Now cctb has value "Jello" – a "const" object was modified!

// Solution: logical constness with mutable
class TextBlock {
public:
    std::size_t length() const {
        if (!lengthIsValid) {
            textLength = std::strlen(text.c_str()); // OK: mutable members
            lengthIsValid = true;                    // can be modified in const functions
        }
        return textLength;
    }
}

```

```
private:
    std::string text;
    mutable std::size_t textLength;    // These may always be modified,
    mutable bool lengthIsValid;        // even in const member functions
};
```

1.7.6 Avoiding Duplication Between const and Non-const Member Functions

```
class TextBlock {
public:
    const char& operator[](std::size_t position) const {
        // Bounds checking
        if (position >= text.size()) {
            throw std::out_of_range("TextBlock::operator[]");
        }
        // Logging
        // Data integrity validation
        // ... lots of shared logic ...
        return text[position];
    }

    char& operator[](std::size_t position) {
        // Cast away const from the return value of the const version
        // This is safe because we KNOW the object is non-const
        // (otherwise the non-const version wouldn't have been called)
        return const_cast<char&>(
            static_cast<const TextBlock&>(*this)[position]
        );
        // Step 1: static_cast<const TextBlock&>(*this) – add const to *this
        // Step 2: call const operator[] – safe, we're calling const version
        // Step 3: const_cast<char&>(…) – remove const from return value
    }

private:
    std::string text;
};

// NEVER do it the other way around (having const call non-const)!
// That would cast away the const-ness of the object, which is dangerous.
```

1.7.7 const in Function Declarations

```

class Rational {
public:
    Rational(int numerator = 0, int denominator = 1);

    int numerator() const;
    int denominator() const;
};

// Return const value to prevent meaningless assignments
const Rational operator*(const Rational& lhs, const Rational& rhs);

// Why const return value?
Rational a, b, c;
// (a * b) = c;    // ERROR with const return value – catches typo
// Programmer probably meant: if (a * b == c)

// const parameters – prevent accidental modification
bool operator==(const Rational& lhs, const Rational& rhs) {
    return lhs.numerator() * rhs.denominator() ==
           rhs.numerator() * lhs.denominator();
}

```

1.7.8 Things to Remember

- Declaring something const helps compilers detect usage errors
- Compilers enforce bitwise constness, but you should program using logical constness
- When const and non-const member functions have essentially identical implementations, use the non-const version calling the const version to avoid duplication
- const can be applied to objects, function parameters, return types, and member functions

1.8 Item 4: Make Sure That Objects Are Initialized Before They're Used

1.9 Visual Summary

ITEM 4: MAKE SURE THAT OBJECTS ARE INITIALIZED BEFORE THEY'RE USED

1. Object storage appears -> members/bases must receive valid starting values.
2. Assignment inside constructor body -> default first, then overwrite.
3. Initialization list -> construct directly with intended value.
4. Non-local statics across files -> order is uncertain.
5. Meaning: initialize at construction point; use function-local statics for order safety.

1.9.1 Core Concept

Reading uninitialized values yields **undefined behavior** — anything can happen. C++ is inconsistent about when things get initialized, so the best approach is to **always initialize everything**.

1.9.2 The Initialization Rules

```

// Built-in types: NOT guaranteed to be initialized
int x;           // Uninitialized in some contexts!
int* p;          // Uninitialized – points to garbage

// User-defined types: constructors handle initialization
std::string s;   // Initialized to "" by default constructor
std::vector<int> v; // Initialized to empty by default constructor

// The inconsistency:
class Point {
    int x, y;    // NOT initialized unless constructor does it
};

void foo() {
    int x;       // Uninitialized
    Point p;     // x and y are uninitialized!
}

// Arrays of built-in types:
int arr[100];    // NOT initialized to 0 in local scope
int arr2[100] = {}; // Zero-initialized (the {} trick)

```

1.9.3 Initialization vs. Assignment in Constructors

```

class PhoneNumber { /* ... */ };

class ABEntry {
public:
    // BAD: This is ASSIGNMENT, not initialization
    ABEntry(const std::string& name, const std::string& address,
            const std::list<PhoneNumber>& phones) {
        theName = name;           // These are all ASSIGNMENTS
        theAddress = address;     // The members were already DEFAULT-CONSTRUCTED
        thePhones = phones;       // before the body of the constructor executed
        numTimesConsulted = 0;
        // Cost: default construction + assignment = 2 operations per member
    }

    // GOOD: Member initialization list
    ABEntry(const std::string& name, const std::string& address,
            const std::list<PhoneNumber>& phones)
        : theName(name),           // These are true INITIALIZATIONS
          theAddress(address),     // Copy constructor called directly
          thePhones(phones),       // No default construction + assignment
          numTimesConsulted(0) {   // Even built-in types should be initialized
        // Body is empty – all work done in init list
        // Cost: 1 copy construction per member
    }

private:
    std::string theName;
    std::string theAddress;
    std::list<PhoneNumber> thePhones;
    int numTimesConsulted;
};

```

1.9.4 Order of Initialization

```

class Widget {
public:
    Widget(int val)
        : b(val),      // WARNING: despite appearing first in the init list,
          a(b) {        // 'a' is initialized FIRST because it's declared first!
        // If b hasn't been initialized yet when a(b) runs,
        // 'a' gets an uninitialized value. Bug!
    }

private:
    int a; // Declared first → initialized first
    int b; // Declared second → initialized second
};

// RULE: Members are initialized in the order of DECLARATION, not the
// order in which they appear in the initialization list.
// ALWAYS write initialization list in declaration order to avoid confusion.

class WidgetFixed {
public:
    WidgetFixed(int val)
        : a(val),      // First in declaration order → first in init list
          b(val) {      // Second in declaration order → second in init list
    }

private:
    int a;
    int b;
};

```

1.9.5 The Static Initialization Order Problem

```

// === file: FileSystem.cpp ===
class FileSystem {
public:
    std::size_t numDisks() const;
    // ...
};
extern FileSystem tfs; // Object for clients to use

// === file: Directory.cpp ===
class Directory {
public:
    Directory() {
        std::size_t disks = tfs.numDisks(); // Uses tfs!
        // BUT: Is tfs initialized yet? MAYBE NOT.
        // If Directory's constructor runs before FileSystem's constructor,
        // this is undefined behavior!
    }
};
Directory tempDir; // Another non-local static object

// PROBLEM: The relative order of initialization of non-local static objects
// defined in different translation units is UNDEFINED.

```

1.9.6 The Solution: Meyers' Singleton (Local Static Objects)

```
// === file: FileSystem.cpp ===
class FileSystem {
public:
    std::size_t numDisks() const;
    // ...
};

FileSystem& tfs() {
    static FileSystem fs; // Local static – initialized on first call
    return fs;           // Return reference to the singleton
    // Guaranteed to be initialized before first use!
    // C++11 guarantees thread-safe initialization of local statics
}

// === file: Directory.cpp ===
class Directory {
public:
    Directory() {
        std::size_t disks = tfs().numDisks(); // Note: tfs() not tfs
        // tfs() returns a reference to the FileSystem singleton
        // The first call initializes it; subsequent calls just return it
    }
};

Directory& tempDir() {
    static Directory td;
    return td;
}

// This technique is known as the "Construct On First Use" idiom
// It replaces non-local statics with functions returning references
// to local statics, giving you full control over initialization order.
```

1.9.7 Complete Example: Safe Initialization Patterns

```
#include <string>
#include <vector>
#include <iostream>
#include <mutex>

// Pattern 1: Always use member initialization lists
class DatabaseConnection {
public:
    DatabaseConnection(const std::string& host, int port,
                      const std::string& dbName)
        : host_(host),
          port_(port),
          dbName_(dbName),
          connectionCount_(0), // Even built-in types
          isConnected_(false) {} // Don't skip any member

private:
    std::string host_;
    int port_;
    std::string dbName_;
    int connectionCount_;
    bool isConnected_;
};
```



```

// Pattern 2: Meyers' Singleton for cross-TU dependencies
class Logger {
public:
    void log(const std::string& msg) {
        std::lock_guard<std::mutex> lock(mutex_);
        std::cout << msg << "\n";
    }

    // Non-copyable
    Logger(const Logger&) = delete;
    Logger& operator=(const Logger&) = delete;

    static Logger& instance() {
        static Logger logger; // Thread-safe in C++11+
        return logger;
    }

private:
    Logger() = default;
    std::mutex mutex_;
};

// Pattern 3: Default member initializers (C++11)
class Config {
    int maxRetries_ = 3; // Default right in declaration
    double timeout_ = 30.0;
    std::string logLevel_ = "INFO";
    bool verbose_ = false;

public:
    Config() = default; // All members have sensible defaults

    Config(int retries, double timeout)
        : maxRetries_(retries), // Override only what needs changing
          timeout_(timeout) {} // logLevel_ and verbose_ keep defaults
};

// Usage
void example() {
    auto& logger = Logger::instance();
    logger.log("Application started");

    Config defaultConfig; // Uses all defaults
    Config customConfig(5, 60.0); // Overrides retries and timeout
}

```

1.9.8 Things to Remember

- Manually initialize objects of built-in type — C++ doesn't always do it for you
- In constructors, prefer member initialization lists over assignment in the body; list members in the same order they're declared
- Avoid the initialization order problem across translation units by replacing non-local static objects with functions returning references to local static objects (Meyers' Singleton)
- C++11 default member initializers are a convenient modern alternative for default values

2 Chapter 2: Constructors, Destructors, and Assignment Operators

2.1 Chapter Flow

CHAPTER 2: CONSTRUCTORS, DESTRUCTORS, AND ASSIGNMENT OPERATORS

Item 5 -> know compiler-generated lifecycle functions.
Item 6 -> delete/disallow lifecycle operations that violate your type.
Item 7 -> virtual destructor when deleting polymorphically.
Item 8 -> destructors must not leak exceptions.
Item 9 -> no virtual calls while constructing/destructing.
Items 10-12 -> assignment returns `*this`, handles self-assignment, copies every part.

> Items 5-12: The special member functions that control object lifecycle — creation, destruction, and copying.
> Getting these right is fundamental to writing correct, efficient, and exception-safe C++.

2.2 Item 5: Know What Functions C++ Silently Writes and Calls

2.3 Visual Summary

ITEM 5: KNOW WHAT FUNCTIONS C++ SILENTLY WRITES AND CALLS

1. You declare a class -> compiler may synthesize special member functions.
2. Default ctor, dtor, copy ctor, copy assignment appear only when needed/allowed.
3. Generated copy -> memberwise copy, not semantic resource ownership.
4. References/const members/base restrictions can block assignment generation.
5. Meaning: know the invisible API before relying on class behavior.

2.3.1 Core Concept

When you declare an empty class, C++ does not leave it truly empty. The compiler will **automatically generate** certain special member functions if you don't declare them yourself. Understanding exactly what gets generated — and under what conditions — is essential to avoiding subtle bugs.

2.3.2 What the Compiler Generates

For an apparently empty class:

```
class Empty {};
```

The compiler behaves as though you had written:

```
class Empty {
public:
    Empty() { } // default constructor
    Empty(const Empty& rhs) { } // copy constructor
    ~Empty() { } // destructor
    Empty& operator=(const Empty& rhs) { } // copy assignment operator
};
```

These are generated **only if they are needed** (i.e., only if your code actually calls them). They are all public and inline.

2.3.3 The Default Constructor

The compiler generates a default constructor only if you declare **no constructors at all**.

```
// CASE 1: Compiler generates a default constructor
class Widget {
    int value_; // left uninitialized by generated default ctor!
    std::string name_; // default-constructed (empty string)
};

Widget w; // Fine – compiler-generated default ctor is called
// w.value_ is UNINITIALIZED (undefined behavior to read it)
// w.name_ is ""

// CASE 2: No default constructor generated – you declared a constructor
class Gadget {
public:
    Gadget(int id) : id_(id) {} // user-declared constructor
private:
    int id_;
};

Gadget g1(42); // Fine
// Gadget g2; // ERROR! No default constructor available
// The compiler does NOT generate one because you declared a ctor
```

Real-world consequence — STL containers:

```
class DatabaseRecord {
public:
    // Only this constructor – no default ctor generated
    DatabaseRecord(int id, const std::string& table) : id_(id), table_(table) {}
private:
    int id_;
    std::string table_;
};

// This is a problem for certain container operations:
std::vector<DatabaseRecord> records;
records.resize(10); // ERROR! resize() needs to default-construct objects
// No default constructor available

// FIX: Provide a default constructor or use a different approach
std::vector<DatabaseRecord> records2;
records2.reserve(10); // reserve does NOT construct objects – OK
records2.push_back(DatabaseRecord(1, "users")); // explicit construction
```

2.3.4 The Copy Constructor

The compiler-generated copy constructor performs **member-wise copy construction** — it copies each non-static data member from the source object to the new object.

```
class NamedObject {
public:
    NamedObject(const std::string& name, int value)
        : name_(name), value_(value) {}

    // Compiler generates:
    // NamedObject(const NamedObject& rhs)
    //     : name_(rhs.name_),      // calls std::string copy ctor
    //     value_(rhs.value_) {}    // copies the int bitwise

private:
    std::string name_; // has its own copy constructor
    int value_;        // built-in type – bitwise copy
};

NamedObject obj1("Widget", 42);
NamedObject obj2(obj1); // Compiler-generated copy ctor called
// obj2.name_ is "Widget" (independent copy)
// obj2.value_ is 42
```

When member-wise copy is dangerous:

```
// BAD: Raw pointer member with compiler-generated copy ctor
class RawBuffer {
public:
    RawBuffer(size_t size) : data_(new char[size]), size_(size) {
        std::memset(data_, 0, size_);
    }
    ~RawBuffer() { delete[] data_; } // free the memory

    // Compiler generates a copy ctor that copies the POINTER, not the data:
    // RawBuffer(const RawBuffer& rhs) //     : data_(rhs.data_),    // SHALLOW COPY! Two objects
    //     size_(rhs.size_) {}      // share the same memory!

private:
    char* data_;
    size_t size_;
};

void disaster() {
    RawBuffer buf1(1024);
    RawBuffer buf2(buf1); // Both buf1 and buf2 point to the same memory!
    // When buf2 is destroyed: delete[] data_ frees the memory
    // When buf1 is destroyed: delete[] data_ frees ALREADY-FREED memory
    // DOUBLE FREE – undefined behavior, likely crash
}

// GOOD: Proper deep copy
class SafeBuffer {
public:
    SafeBuffer(size_t size) : data_(new char[size]), size_(size) {
        std::memset(data_, 0, size_);
    }

    SafeBuffer(const SafeBuffer& rhs) : data_(new char[rhs.size_]), // allocate own memory
        size_(rhs.size_) {}
}
```

```

    std::memcpy(data_, rhs.data_, size_); // copy the DATA, not the pointer
}

~SafeBuffer() { delete[] data_; }

private:
    char* data_;
    size_t size_;
};

```

2.3.5 The Copy Assignment Operator

The generated copy assignment operator also performs member-wise copy, but via assignment rather than construction. It behaves the same way — and has the same dangers with pointer members.

```

class Employee {
public:
    Employee(const std::string& name, int id) : name_(name), id_(id) {}
    // Compiler generates:
    // Employee& operator=(const Employee& rhs) {
    //     name_ = rhs.name_;    // calls std::string::operator=
    //     id_ = rhs.id_;        // bitwise copy of int
    //     return *this;
    // }
private:
    std::string name_;
    int id_;
};

Employee e1("Alice", 1001);
Employee e2("Bob", 1002);
e2 = e1; // e2 is now a copy of e1

```

2.3.6 When the Compiler REFUSES to Generate Copy Assignment

The compiler will not generate `operator=` in certain cases where member-wise assignment would be illegal or ambiguous.

Case 1: Reference members

```

class BindingRef {
public:
    BindingRef(std::string& ref, int val) : nameRef_(ref), value_(val) {}

    // Compiler WILL NOT generate operator=
    // Because you cannot rebind a reference – nameRef_ was bound at construction
    // and C++ references cannot be reseated

private:
    std::string& nameRef_; // reference member
    int value_;
};

std::string s1 = "hello", s2 = "world";
BindingRef br1(s1, 1);
BindingRef br2(s2, 2);
// br1 = br2; // ERROR: compiler cannot generate operator=
// Should it rebind the reference? (Illegal in C++)
// Should it modify the referred-to object? (Surprising semantics)
// Compiler's answer: refuse to decide – you write it

```

Case 2: const members

```
class ImmutableId {
public:
    ImmutableId(int id, const std::string& name) : id_(id), name_(name) {}

    // Compiler WILL NOT generate operator=
    // Because const members cannot be assigned to

private:
    const int id_;           // const member – cannot be modified after construction
    std::string name_;
};

ImmutableId a(1, "alpha");
ImmutableId b(2, "beta");
// a = b; // ERROR: cannot assign to const member id_
```

Case 3: Base class with private/deleted operator=

```
class Uncopyable {
public:
    Uncopyable() = default;
private:
    Uncopyable& operator=(const Uncopyable&); // private – inaccessible
};

class Derived : public Uncopyable {
public:
    Derived(int x) : x_(x) {}
    // Compiler WILL NOT generate operator= for Derived
    // because it would need to call Uncopyable::operator=, which is private
private:
    int x_;
};
```

2.3.7 The Destructor

The compiler-generated destructor is **non-virtual** (unless the class inherits from a base class that already has a virtual destructor). It invokes the destructors of each non-static data member and each base class in reverse order of construction.

```
class Composite {
public:
    Composite() : name_("default"), id_(0) {}
    // Compiler generates:
    // ~Composite() {
    //     // id_ is int – no destructor to call
    //     // name_ is std::string – calls std::string::~~string()
    //     // Then base class destructors (if any) in reverse order
    // }
private:
    std::string name_;
    int id_;
};
```

Critical: the generated destructor is NOT virtual:

```

class Base {
public:
    // Compiler-generated destructor is NON-VIRTUAL
    // ~Base() {} // implicitly generated
};

class Derived : public Base {
public:
    Derived() : data_(new int[100]) {}
    ~Derived() { delete[] data_; }
private:
    int* data_;
};

// This leaks memory!
Base* ptr = new Derived();
delete ptr; // Calls Base::~~Base() only – Derived::~~Derived() never runs
// The int[100] is leaked
// (Technically undefined behavior)

```

2.3.8 Real-World Example: Understanding All Four Together

```

#include <string>
#include <iostream>

class LogEntry {
public:
    // User-declared constructor – no default ctor generated
    LogEntry(const std::string& msg, int severity)
        : message_(msg), severity_(severity) {
        std::cout << "LogEntry constructed: " << message_ << "\n";
    }

    // No copy ctor declared – compiler generates one (member-wise copy)
    // No operator= declared – compiler generates one (member-wise assignment)
    // No destructor declared – compiler generates one (non-virtual)

    void print() const {
        std::cout << "[" << severity_ << "]" << message_ << "\n";
    }

private:
    std::string message_; // copy ctor/assignment via std::string's
    int severity_;        // copy ctor/assignment via bitwise copy
};

void demonstrate() {
    LogEntry entry1("System started", 0); // user-declared ctor
    LogEntry entry2(entry1);              // compiler-generated copy ctor
    LogEntry entry3("Placeholder", 99);   // user-declared ctor
    entry3 = entry1;                      // compiler-generated operator=

    // LogEntry entry4; // ERROR: no default ctor (because user declared a ctor)

    entry2.print(); // [0] System started
    entry3.print(); // [0] System started
}
// All three LogEntry objects destroyed here by compiler-generated destructors

```

2.3.9 Things to Remember

- Compilers may implicitly generate a default constructor, copy constructor, copy assignment operator, and destructor for a class.
- The default constructor is generated only if you declare no constructors at all.
- The generated copy operations perform member-wise copy — shallow, not deep. This is dangerous for classes managing raw resources (pointers, file handles, etc.).
- The generated destructor is non-virtual unless the class inherits from a base with a virtual destructor.
- The compiler will refuse to generate `operator=` if the class contains reference members, const members, or if a base class has an inaccessible `operator=`.

2.4 Item 6: Explicitly Disallow the Use of Compiler-Generated Functions You Do Not Want

2.5 Visual Summary

ITEM 6: EXPLICITLY DISALLOW THE USE OF COMPILER-GENERATED FUNCTIONS YOU DO

1. Class owns identity/resource -> copying may be wrong.
2. If you say nothing -> compiler may generate copy operations.
3. C++11+ path -> delete unwanted copy/move operations.
4. Older path -> declare private and do not define.
5. Meaning: forbidden operations should fail at compile time, not at runtime.

2.5.1 Core Concept

Sometimes copying an object makes no sense. A class representing a unique system resource — a database connection, a singleton, a log file handle — should not be copyable. But if you don't take action, the compiler will happily generate copy operations for you. You must **explicitly prevent** unwanted copying.

2.5.2 The Problem: Accidental Copies of Unique Resources

```
// BAD: This class manages a unique resource but is implicitly copyable!
class DatabaseConnection {
public:
    DatabaseConnection(const std::string& connStr) {
        handle_ = db_connect(connStr.c_str()); // acquire unique resource
    }
    ~DatabaseConnection() {
        db_disconnect(handle_); // release resource
    }

    void executeQuery(const std::string& sql) { /* ... */ }

private:
    db_handle_t handle_;
    // Compiler generates copy ctor and operator= that copy the handle!
    // Two objects would share the same connection handle
    // Double-close on destruction – undefined behavior
};

void oops() {
    DatabaseConnection conn("host=localhost dbname=mydb");
    DatabaseConnection conn2 = conn; // Compiles! But it shouldn't!
```



```

// Both conn and conn2 now hold the same db handle
// When conn2 is destroyed: db_disconnect(handle_)
// When conn is destroyed: db_disconnect(handle_) again – DOUBLE CLOSE!
}

```

2.5.3 Solution 1: Declare Private and Don't Define (C++03 Approach)

The classic Effective C++ technique — declare the copy operations private and don't provide a definition.

```

class DatabaseConnection {
public:
    DatabaseConnection(const std::string& connStr) {
        handle_ = db_connect(connStr.c_str());
    }
    ~DatabaseConnection() {
        db_disconnect(handle_);
    }

    void executeQuery(const std::string& sql) { /* ... */ }

private:
    // Declared private – prevents external code from copying
    // Not defined – prevents member functions and friends from copying
    DatabaseConnection(const DatabaseConnection&);           // no definition
    DatabaseConnection& operator=(const DatabaseConnection&); // no definition

    db_handle_t handle_;
};

void test() {
    DatabaseConnection conn("host=localhost");
    // DatabaseConnection conn2(conn);           // ERROR: copy ctor is private
    // DatabaseConnection conn3 = conn;           // ERROR: copy ctor is private
    // conn3 = conn;                             // ERROR: operator= is private
}

```

If a member function or friend tries to copy, the error surfaces as a **link-time** error (undefined symbol) rather than a compile-time error — not ideal, but it still catches the mistake.

```

class DatabaseConnection {
    // ... same as above ...

    friend void backup(DatabaseConnection& conn) {
        DatabaseConnection copy(conn); // Compiles (friend can access private)
        // but LINKER ERROR: undefined reference
    }
};

```

2.5.4 Solution 2: Inherit from an Uncopyable Base Class (C++03)

Move the private-and-undefined trick into a reusable base class:

```

class Uncopyable {
protected:
    Uncopyable() {} // allow construction
    ~Uncopyable() {} // allow destruction

private:
    Uncopyable(const Uncopyable&); // prevent copying
    Uncopyable& operator=(const Uncopyable&); // prevent assignment
};

// Now any class that inherits from Uncopyable is non-copyable
class UniqueFile : private Uncopyable { // private inheritance - is-implemented-in-terms-of
public:
    UniqueFile(const std::string& path) {
        file_ = fopen(path.c_str(), "r");
    }
    ~UniqueFile() {
        if (file_) fclose(file_);
    }

private:
    FILE* file_;
};

void test() {
    UniqueFile f1("/etc/passwd");
    // UniqueFile f2(f1); // ERROR: copy ctor of Uncopyable is private
    // f2 = f1; // ERROR: operator= of Uncopyable is private
}

```

Boost provides `boost::noncopyable` for exactly this purpose:

```

#include <boost/noncopyable.hpp>

class Singleton : private boost::noncopyable {
public:
    static Singleton& instance() {
        static Singleton s;
        return s;
    }
    void doWork() { /* ... */ }

private:
    Singleton() {} // private ctor for singleton pattern
};

```

2.5.5 Solution 3: = delete (C++11 and Beyond — Preferred)

Modern C++ provides a much cleaner mechanism: explicitly deleting functions.

```

class DatabaseConnection {
public:
    DatabaseConnection(const std::string& connStr) {
        handle_ = db_connect(connStr.c_str());
    }
    ~DatabaseConnection() {
        db_disconnect(handle_);
    }

    // Explicitly deleted – clear intent, better error messages
    DatabaseConnection(const DatabaseConnection&) = delete;
    DatabaseConnection& operator=(const DatabaseConnection&) = delete;

    void executeQuery(const std::string& sql) { /* ... */ }

private:
    db_handle_t handle_;
};

void test() {
    DatabaseConnection conn("host=localhost");
    // DatabaseConnection conn2(conn); // ERROR: "use of deleted function 'DatabaseConnection(c
    DatabaseConnection&""
    // Much clearer error message than "private within this context"
}

```

Advantages of = delete over the private-and-undefined approach:

```

class NetworkSocket {
public:
    NetworkSocket(int port) : fd_(socket(AF_INET, SOCK_STREAM, 0)) {
        // bind and listen...
    }
    ~NetworkSocket() { close(fd_); }

    // = delete works even when declared public – and SHOULD be public
    // Why public? Because the error message is better:
    // "deleted function" vs "private within this context"
    // The user sees immediately that copying was intentionally forbidden,
    // not accidentally hidden
    NetworkSocket(const NetworkSocket&) = delete;
    NetworkSocket& operator=(const NetworkSocket&) = delete;

    // You can also delete specific overloads to prevent implicit conversions:
    void send(const char* data, size_t len);
    void send(int) = delete;           // prevent accidental send(42)
    void send(double) = delete;        // prevent accidental send(3.14)
    void send(bool) = delete;          // prevent accidental send(true)

private:
    int fd_;
};

void test() {
    NetworkSocket sock(8080);
    sock.send("hello", 5); // Fine
    // sock.send(42);       // ERROR: use of deleted function
    // sock.send(3.14);     // ERROR: use of deleted function
    // Without the deleted overloads, 42 would silently convert to const char*!
}

```

2.5.6 Real-World Example: Thread-Safe Logger (Non-Copyable but Movable)

```
#include <mutex>
#include <fstream>
#include <string>

class Logger {
public:
    explicit Logger(const std::string& filepath)
    : file_(filepath, std::ios::app) {
        if (!file_.is_open()) {
            throw std::runtime_error("Cannot open log file: " + filepath);
        }
    }

    ~Logger() {
        if (file_.is_open()) {
            file_.flush();
            file_.close();
        }
    }

    // Copying a logger makes no sense – two loggers writing to the same file
    // with separate mutexes would cause interleaved output
    Logger(const Logger&) = delete;
    Logger& operator=(const Logger&) = delete;

    // Moving is fine – transfer ownership of the file handle
    Logger(Logger&& other) noexcept
    : file_(std::move(other.file_)) {}

    Logger& operator=(Logger&& other) noexcept {
        if (this != &other) {
            file_ = std::move(other.file_);
        }
        return *this;
    }

    void log(const std::string& message) {
        std::lock_guard<std::mutex> lock(mutex_);
        file_ << message << "\n";
    }

private:
    std::ofstream file_;
    std::mutex mutex_; // std::mutex is itself non-copyable and non-movable
};

void test() {
    Logger logger("app.log");
    logger.log("Application started");

    // Logger copy = logger;           // ERROR: deleted
    Logger moved = std::move(logger); // Fine – ownership transferred
    moved.log("Continuing after move");
}
```

2.5.7 Real-World Example: Unique Handle Wrapper

```

// A RAII wrapper for OS handles that must not be duplicated
template<typename HandleTraits>
class UniqueHandle {
public:
    explicit UniqueHandle(typename HandleTraits::Type h = HandleTraits::Invalid())
        : handle_(h) {}

    ~UniqueHandle() {
        if (handle_ != HandleTraits::Invalid()) {
            HandleTraits::Close(handle_);
        }
    }

    // Non-copyable
    UniqueHandle(const UniqueHandle&) = delete;
    UniqueHandle& operator=(const UniqueHandle&) = delete;

    // Movable
    UniqueHandle(UniqueHandle&& other) noexcept : handle_(other.handle_) {
        other.handle_ = HandleTraits::Invalid();
    }

    UniqueHandle& operator=(UniqueHandle&& other) noexcept {
        if (this != &other) {
            if (handle_ != HandleTraits::Invalid()) {
                HandleTraits::Close(handle_);
            }
            handle_ = other.handle_;
            other.handle_ = HandleTraits::Invalid();
        }
        return *this;
    }

    typename HandleTraits::Type get() const { return handle_; }
    bool isValid() const { return handle_ != HandleTraits::Invalid(); }

private:
    typename HandleTraits::Type handle_;
};

// Usage with file descriptors
struct FileDescriptorTraits {
    using Type = int;
    static Type Invalid() { return -1; }
    static void Close(Type fd) { ::close(fd); }
};

using UniqueFd = UniqueHandle<FileDescriptorTraits>;

void example() {
    UniqueFd fd(open("/tmp/test.txt", O_RDONLY));
    // UniqueFd fd2 = fd;           // ERROR: deleted copy ctor
    UniqueFd fd2 = std::move(fd);    // Fine – ownership transfer
    // fd is now invalid, fd2 owns the file descriptor
}

```

2.5.8 Things to Remember

- The compiler will generate copy constructor and copy assignment operator if you don't declare them. If copying is inappropriate for your class, you must explicitly prevent it.
- In C++03, declare copy operations `private` and don't define them, or inherit from an `Uncopyable` base class.
- In C++11 and later, use `= delete` — it's clearer, produces better error messages, and works for any function, not just copy operations.
- Declare deleted functions `public` for the best error messages.
- Consider whether your non-copyable class should still be **movable** (like `std::unique_ptr`).

2.6 Item 7: Declare Destructors Virtual in Polymorphic Base Classes

2.7 Visual Summary

ITEM 7: DECLARE DESTRUCTORS VIRTUAL IN POLYMORPHIC BASE CLASSES

1. Base pointer points to derived object → delete through base pointer.
2. Base destructor non-virtual → only base part is destroyed: undefined behavior risk.
3. Base destructor virtual → derived destructor runs, then base destructor.
4. No polymorphic deletion needed → non-virtual destructor is fine and smaller.
5. Meaning: virtual functions in a base usually imply a virtual destructor.

2.7.1 Core Concept

When a derived class object is deleted through a base class pointer, and the base class has a **non-virtual destructor**, the behavior is **undefined**. In practice, the derived class's destructor typically never runs, leading to resource leaks and corruption. Any class designed to be used polymorphically must have a virtual destructor.

2.7.2 The Problem: Non-Virtual Destructor with Polymorphic Deletion

```
// BAD: Non-virtual destructor in a polymorphic base class
class TimeKeeper {
public:
    TimeKeeper() {}
    ~TimeKeeper() {} // NON-VIRTUAL destructor!

    virtual int getCurrentTime() const = 0; // virtual function => polymorphic use
};

class AtomicClock : public TimeKeeper {
public:
    AtomicClock() : calibrationData_(new double[1000]) {
        // Expensive calibration data
    }
    ~AtomicClock() {
        delete[] calibrationData_; // Free calibration data
        std::cout << "AtomicClock resources freed\n";
    }
    int getCurrentTime() const override { return /* atomic time */0; }
```

```

private:
    double* calibrationData_;
};

class WaterClock : public TimeKeeper {
public:
    WaterClock() : waterLevel_(new float[500]) {}
    ~WaterClock() {
        delete[] waterLevel_;
        std::cout << "WaterClock resources freed\n";
    }
    int getCurrentTime() const override { return /* water time */0; }

private:
    float* waterLevel_;
};

// Factory function returns base class pointer
TimeKeeper* getTimeKeeper(const std::string& type) {
    if (type == "atomic") return new AtomicClock();
    if (type == "water") return new WaterClock();
    return nullptr;
}

void disaster() {
    TimeKeeper* tk = getTimeKeeper("atomic");
    // ... use tk ...
    delete tk; // UNDEFINED BEHAVIOR!
    // TimeKeeper::~~TimeKeeper() is non-virtual
    // Only the TimeKeeper part is destroyed
    // AtomicClock::~~AtomicClock() is NOT called
    // calibrationData_ is LEAKED
    // The "AtomicClock resources freed" message never prints
}

```

2.7.3 The Fix: Virtual Destructor

```

// GOOD: Virtual destructor in polymorphic base class
class TimeKeeper {
public:
    TimeKeeper() {}
    virtual ~TimeKeeper() {} // VIRTUAL destructor

    virtual int getCurrentTime() const = 0;
};

class AtomicClock : public TimeKeeper {
public:
    AtomicClock() : calibrationData_(new double[1000]) {}
    ~AtomicClock() override { // 'override' for safety (C++11)
        delete[] calibrationData_;
        std::cout << "AtomicClock resources freed\n";
    }
    int getCurrentTime() const override { return 0; }

private:
    double* calibrationData_;
};

```

```

void correct() {
    TimeKeeper* tk = getTimeKeeper("atomic");
    delete tk; // CORRECT: virtual dispatch calls AtomicClock::~~AtomicClock()
    // then TimeKeeper::~~TimeKeeper()
    // "AtomicClock resources freed" prints
    // No leak
}

```

2.7.4 The Rule: Virtual Destructor IFF Polymorphic

The rule is not “always make destructors virtual.” It is: **if the class has any virtual functions, the destructor should be virtual.**

Why not make ALL destructors virtual?

```

// BAD: Gratuitous virtual destructor on a non-polymorphic class
class Point {
public:
    Point(int x, int y) : x_(x), y_(y) {}
    virtual ~Point() {} // Unnecessary virtual – this class is not polymorphic

private:
    int x_, y_;
};

// sizeof(Point) without virtual: typically 8 bytes (two ints)
// sizeof(Point) with virtual: typically 16 bytes (two ints + vptr)
// That's a 100% overhead! And it breaks C-compatibility for layout.

// GOOD: No virtual destructor – this class is not meant for polymorphism
class Point {
public:
    Point(int x, int y) : x_(x), y_(y) {}
    // No virtual destructor – and no virtual functions at all
    // sizeof(Point) == 8 bytes, C-compatible layout

private:
    int x_, y_;
};

```

A virtual function adds a **vptr** (virtual table pointer) to each object instance, typically 8 bytes on a 64-bit system. For small value-type objects, this overhead is unacceptable.

2.7.5 The Dangers of Inheriting from Non-Virtual-Destructor Classes

Standard library classes like `std::string`, `std::vector`, and `std::unordered_map` do **not** have virtual destructors. Inheriting from them is dangerous:


```

// BAD: Inheriting from std::string (which has a non-virtual destructor)
class SpecialString : public std::string {
public:
    SpecialString(const char* s) : std::string(s), metadata_(new int(42)) {}
    ~SpecialString() { delete metadata_; } // cleanup

private:
    int* metadata_;
};

void danger() {
    std::string* sp = new SpecialString("hello");
    delete sp; // UNDEFINED BEHAVIOR!
    // std::string::~~string() is non-virtual
    // SpecialString::~~SpecialString() never runs
    // metadata_ is leaked
}

// BAD: Inheriting from std::vector
class AuditedVector : public std::vector<int> {
public:
    ~AuditedVector() {
        logToAuditTrail("Vector destroyed with " + std::to_string(size()) + " elements");
    }
};
// Same problem: deleting through a std::vector<int>* skips the audit log

```

The safe alternative — composition over inheritance:

```

// GOOD: Use composition instead of inheriting from standard containers
class AuditedVector {
public:
    void push_back(int val) {
        data_.push_back(val);
        logToAuditTrail("Element added: " + std::to_string(val));
    }

    size_t size() const { return data_.size(); }

    ~AuditedVector() {
        logToAuditTrail("Vector destroyed with " + std::to_string(data_.size()) + " elements");
    }

private:
    std::vector<int> data_; // composition, not inheritance
};

```

2.7.6 Pure Virtual Destructor for Abstract Base Classes

Sometimes you want an abstract class but have no natural pure virtual function. You can make the destructor pure virtual — but you **must still provide a definition**.

```

class AbstractAnimal {
public:
    virtual ~AbstractAnimal() = 0; // pure virtual destructor
    // Makes the class abstract – cannot be instantiated directly
};

// You MUST provide a definition! Derived class destructors call the base destructor.
AbstractAnimal::~~AbstractAnimal() {
    // Base cleanup (if any)
    // This body can be empty, but the definition must exist
}

class Dog : public AbstractAnimal {
public:
    Dog(const std::string& name) : name_(name) {}
    ~Dog() override {
        std::cout << name_ << " destroyed\n";
        // After this, AbstractAnimal::~~AbstractAnimal() is called automatically
    }
private:
    std::string name_;
};

void test() {
    // AbstractAnimal a; // ERROR: cannot instantiate abstract class
    AbstractAnimal* pet = new Dog("Rex");
    delete pet; // Correctly calls Dog::~~Dog() then AbstractAnimal::~~AbstractAnimal()
}

```

2.7.7 Real-World Example: Plugin System

```

// A plugin system where plugins are loaded dynamically and destroyed
// through base class pointers – virtual destructor is ESSENTIAL

class Plugin {
public:
    virtual ~Plugin() = default; // MUST be virtual

    virtual std::string name() const = 0;
    virtual void initialize() = 0;
    virtual void execute() = 0;
    virtual void shutdown() = 0;
};

class ImageProcessorPlugin : public Plugin {
public:
    ImageProcessorPlugin() : buffer_(nullptr), bufferSize_(0) {}

    ~ImageProcessorPlugin() override {
        delete[] buffer_; // Must run! Only runs if base dtor is virtual.
        std::cout << "ImageProcessor plugin destroyed, buffer freed\n";
    }

    std::string name() const override { return "ImageProcessor"; }

    void initialize() override {
        bufferSize_ = 1024 * 1024; // 1MB
        buffer_ = new unsigned char[bufferSize_];
    }
}

```

```

void execute() override {
    // Process image data in buffer_
}

void shutdown() override {
    // Graceful shutdown
}

private:
    unsigned char* buffer_;
    size_t bufferSize_;
};

class PluginManager {
public:
    void loadPlugin(std::unique_ptr<Plugin> plugin) {
        plugin->initialize();
        plugins_.push_back(std::move(plugin));
    }

    ~PluginManager() {
        for (auto& p : plugins_) {
            p->shutdown();
        }
        // unique_ptr calls delete on Plugin* pointers
        // Virtual destructor ensures derived destructors run correctly
        plugins_.clear();
    }

private:
    std::vector<std::unique_ptr<Plugin>> plugins_;
};

```

2.7.8 Real-World Example: Shape Hierarchy

```

class Shape {
public:
    virtual ~Shape() = default; // Virtual destructor – this is a polymorphic base

    virtual double area() const = 0;
    virtual double perimeter() const = 0;
    virtual void draw() const = 0;

    // Non-virtual interface pattern (NVI) – see Item 35
    std::string describe() const {
        return "Shape with area=" + std::to_string(area()) +
            " perimeter=" + std::to_string(perimeter());
    }
};

class Circle : public Shape {
public:
    explicit Circle(double radius) : radius_(radius) {}
    // No explicit destructor needed – default is fine
    // But it IS virtual because base class destructor is virtual

    double area() const override { return M_PI * radius_ * radius_; }
    double perimeter() const override { return 2 * M_PI * radius_; }
    void draw() const override { /* ... */ }
};

```

```

private:
    double radius_;
};

class Polygon : public Shape {
public:
    Polygon(std::vector<Point> vertices)
        : vertices_(std::move(vertices)),
          texture_(new Texture("default.png")) {} // heap resource

    ~Polygon() override {
        delete texture_; // Properly called through virtual dispatch
    }

    double area() const override { /* shoelace formula */ return 0; }
    double perimeter() const override { /* sum of edge lengths */ return 0; }
    void draw() const override { /* ... */ }

private:
    std::vector<Point> vertices_;
    Texture* texture_;
};

void render(const std::vector<std::unique_ptr<Shape>>& shapes) {
    for (const auto& shape : shapes) {
        std::cout << shape->describe() << "\n";
    }
}
// When shapes vector is destroyed, each unique_ptr calls delete on Shape*
// Virtual destructor ensures Polygon::~~Polygon runs and texture_ is freed

```

2.7.9 Things to Remember

- Polymorphic base classes should declare virtual destructors. If a class has any virtual functions, it should have a virtual destructor.
- Classes not designed to be base classes or not designed for polymorphic use should **not** declare virtual destructors.
- Never inherit from standard library container classes (`std::string`, `std::vector`, etc.) — they have non-virtual destructors.
- A pure virtual destructor makes a class abstract but must still have a definition (the body can be empty).

2.8 Item 8: Prevent Exceptions from Leaving Destructors

2.9 Visual Summary

ITEM 8: PREVENT EXCEPTIONS FROM LEAVING DESTRUCTORS

1. Destructor runs during cleanup -> another exception may already be active.
2. Destructor throws -> program may terminate immediately.
3. Option A -> catch/log/swallow when cleanup cannot be fixed by caller.
4. Option B -> expose close()/commit() so caller handles failure before destruction.
5. Meaning: destructors release; explicit operations report errors.

2.9.1 Core Concept

Destructors should **never** emit exceptions. If an exception escapes a destructor while another exception is already propagating (during stack unwinding), C++ calls `std::terminate()`, which kills the program immediately. Even without double-exception scenarios, throwing destructors make it impossible to write correct cleanup code.

2.9.2 Why Throwing Destructors Are Catastrophic

```
// BAD: Destructor that throws
class DatabaseSession {
public:
    DatabaseSession() { /* open connection */ }

    ~DatabaseSession() {
        // What if commit() throws? commit(); // might throw std::runtime_error
        disconnect();
    }

    void commit() {
        if (!changes_.empty()) {
            // Might fail - network error, constraint violation, etc.
            throw std::runtime_error("Commit failed");
        }
    }

    void disconnect() { /* close connection */ }

private:
    std::vector<std::string> changes_;
};

void catastrophe() {
    try {
        DatabaseSession s1;
        DatabaseSession s2;
        // ... do work ...
        throw std::runtime_error("Application error");
        // Stack unwinding begins:
        // s2.~DatabaseSession() is called - if commit() throws here,
        // we now have TWO active exceptions
        // C++ calls std::terminate() - program dies immediately
    } catch (...) {
        // We never reach this handler
    }
}
```

The double-exception problem with containers:

```
// Even worse with containers
void containerProblem() {
    std::vector<DatabaseSession> sessions(10);
    // Vector destruction calls ~DatabaseSession() for each element
    // If the 3rd destructor throws, stack unwinding destroys elements 4-10
    // If the 7th destructor ALSO throws during that unwinding:
    //     std::terminate() – immediate program death
}
```

2.9.3 Solution 1: Swallow the Exception (Log and Continue)

```
class DatabaseSession {
public:
    DatabaseSession() : connected_(true) {}

    ~DatabaseSession() {
        try {
            if (connected_) {
                commit();
                disconnect();
                connected_ = false;
            }
        } catch (const std::exception& e) {
            // Log the error – don't let it escape
            std::cerr << "WARNING: Exception in ~DatabaseSession: " << e.what() << "\n";
            // Swallow the exception – destructor completes normally
            // This is acceptable when the failure is non-critical
        } catch (...) {
            std::cerr << "WARNING: Unknown exception in ~DatabaseSession\n";
        }
    }

    void commit() { /* might throw */ }
    void disconnect() { connected_ = false; }

private:
    bool connected_;
    std::vector<std::string> changes_;
};
```

2.9.4 Solution 2: Abort on Failure (When Continuing Is Dangerous)

```

class CriticalResource {
public:
    CriticalResource() { /* acquire */ }

    ~CriticalResource() {
        try {
            release();
        } catch (...) {
            // If we can't release a critical resource, the program state
            // is corrupted. Better to abort cleanly than continue in an
            // undefined state.
            std::cerr << "FATAL: Cannot release critical resource. Aborting.\n";
            std::abort(); // Immediate, clean termination
        }
    }

    void release() { /* might throw */ }
};

```

2.9.5 Solution 3: Give Clients a Chance to Handle It (Preferred)

The best approach: provide a separate function that clients can call to perform the operation that might throw, then have the destructor serve as a fallback safety net.

```

// GOOD: Two-phase close pattern
class DatabaseSession {
public:
    DatabaseSession(const std::string& connStr)
        : connected_(true), closed_(false) {
        // Open connection
    }

    // Public close function – clients SHOULD call this
    // It CAN throw, and clients can handle the exception
    void close() {
        if (!closed_) {
            commit(); // might throw – client can catch
            disconnect(); // might throw – client can catch
            closed_ = true;
        }
    }

    // Destructor – safety net, never throws
    ~DatabaseSession() {
        if (!closed_) {
            try {
                commit();
                disconnect();
                closed_ = true;
            } catch (const std::exception& e) {
                std::cerr << "WARNING: Cleanup failed in destructor: "
                    << e.what() << "\n";
                // Swallow – destructor must not throw
                // The client had their chance to call close() and handle errors
            } catch (...) {
                std::cerr << "WARNING: Unknown cleanup failure in destructor\n";
            }
        }
    }
};

```

```

void addChange(const std::string& change) {
    changes_.push_back(change);
}

private:
void commit() { /* might throw */ }
void disconnect() { connected_ = false; }

bool connected_;
bool closed_;
std::vector<std::string> changes_;
};

// Client code – proper usage
void properUsage() {
    DatabaseSession session("host=localhost");
    session.addChange("INSERT INTO users ...");
    session.addChange("UPDATE accounts ...");

    try {
        session.close(); // Try to close explicitly
    } catch (const std::exception& e) {
        std::cerr << "Failed to commit: " << e.what() << "\n";
        // Handle the error – retry, rollback, alert user, etc.
    }
    // Even if close() threw, the destructor will try again as a safety net
    // But the client had the opportunity to handle the error properly
}

```

2.9.6 Real-World Example: File Writer with Flush

```

#include <fstream>
#include <stdexcept>

class BufferedFileWriter {
public:
    explicit BufferedFileWriter(const std::string& path)
        : file_(path, std::ios::binary), flushed_(false) {
        if (!file_.is_open()) {
            throw std::runtime_error("Cannot open file: " + path);
        }
    }

    // Write data to buffer
    void write(const char* data, size_t len) {
        buffer_.append(data, len);
        if (buffer_.size() > flushThreshold_) {
            flush(); // auto-flush when buffer is large
        }
    }

    // Public flush – CAN throw, client CAN handle
    void flush() {
        if (!buffer_.empty()) {
            file_.write(buffer_.data(), buffer_.size());
            if (file_.fail()) {
                throw std::runtime_error("Write to file failed");
            }
            file_.flush();
        }
    }
}

```



```

        if (file_.fail()) {
            throw std::runtime_error("Flush to file failed");
        }
        buffer_.clear();
        flushed_ = true;
    }
}

// Public close – CAN throw, client CAN handle
void close() {
    flush(); // write remaining data
    file_.close();
    if (file_.fail()) {
        throw std::runtime_error("Close file failed");
    }
}

// Destructor – safety net, NEVER throws
~BufferedFileWriter() {
    if (file_.is_open()) {
        try {
            if (!buffer_.empty()) {
                file_.write(buffer_.data(), buffer_.size());
                file_.flush();
            }
            file_.close();
        } catch (...) {
            // Last resort – data may be lost, but program doesn't crash
            std::cerr << "WARNING: Could not flush/close file in destructor\n";
        }
    }
}

private:
    std::ofstream file_;
    std::string buffer_;
    bool flushed_;
    static constexpr size_t flushThreshold_ = 4096;
};

void example() {
    BufferedFileWriter writer("/tmp/output.bin");
    writer.write("important data", 14);
    writer.write("more data", 9);

    try {
        writer.close(); // Explicit close – can handle errors
        std::cout << "Data safely written to disk\n";
    } catch (const std::exception& e) {
        std::cerr << "WRITE FAILED: " << e.what() << "\n";
        // Take corrective action: retry, write to backup location, etc.
    }
}

```

2.9.7 Real-World Example: Transaction Scope Guard

```

class TransactionGuard {
public:
    explicit TransactionGuard(Database& db)
    : db_(db), committed_(false) {
        db_.beginTransaction();
    }

    void commit() {
        db_.commitTransaction(); // might throw – client handles it
        committed_ = true;
    }

    // Destructor rolls back if not committed – safe fallback
    ~TransactionGuard() {
        if (!committed_) {
            try {
                db_.rollbackTransaction();
            } catch (const std::exception& e) {
                // Rollback failed in destructor – log but don't throw
                std::cerr << "CRITICAL: Rollback failed: " << e.what() << "\n";
                // At this point we may have an inconsistent DB state
                // but throwing from destructor would be worse
            }
        }
    }

private:
    Database& db_;
    bool committed_;
};

void transferFunds(Database& db, int from, int to, double amount) {
    TransactionGuard txn(db);

    db.execute("UPDATE accounts SET balance = balance - ? WHERE id = ?",
               amount, from);
    db.execute("UPDATE accounts SET balance = balance + ? WHERE id = ?",
               amount, to);

    txn.commit(); // If this throws, the destructor will rollback
    // If any db.execute() throws, destructor runs and rolls back automatically
}

```

2.9.8 Things to Remember

- Destructors should never emit exceptions. If functions called in a destructor may throw, the destructor should catch them and either swallow them or terminate the program.
- If clients need the ability to react to exceptions thrown during a cleanup operation, provide a separate function (like `close()` or `flush()`) that they can call before the destructor runs.
- The destructor then serves as a last-resort safety net, not the primary mechanism for error-prone operations.

2.10 Item 9: Never Call Virtual Functions During Construction or Destruction

2.11 Visual Summary

ITEM 9: NEVER CALL VIRTUAL FUNCTIONS DURING CONSTRUCTION OR DESTRUCTION

1. Base constructor runs -> derived part is not constructed yet.
2. Virtual call here -> dispatches as base, not derived.
3. Base destructor runs -> derived part is already gone.
4. Need customization -> pass data to base constructor or use non-virtual hooks after construction.
5. Meaning: virtual dispatch is unsafe while object identity is changing.

2.11.1 Core Concept

During base class construction, virtual functions **do not** behave polymorphically. When a base class constructor is executing, the object's dynamic type is the **base class**, not the derived class. Virtual function calls resolve to the base class version. The same is true during base class destruction. This is not a bug — it is a deliberate design decision, because the derived class's members have not yet been initialized (during construction) or have already been destroyed (during destruction).

2.11.2 The Problem: Virtual Calls in Constructors

```
// BAD: Calling a virtual function in a constructor
class Transaction {
public:
    Transaction() {
        // ... setup code ...
        logTransaction(); // Virtual call – but which version runs?
    }

    virtual void logTransaction() const {
        std::cout << "Logging base Transaction\n";
    }

    virtual ~Transaction() {}
};

class BuyTransaction : public Transaction {
public:
    BuyTransaction(const std::string& stock, int shares)
        : stock_(stock), shares_(shares) {
        // Before this body runs, Transaction::Transaction() already ran
        // and called logTransaction() – but BuyTransaction's version? NO!
    }

    void logTransaction() const override {
        // This version is NEVER called from the base constructor
        std::cout << "BUY " << shares_ << " shares of " << stock_ << "\n";
    }

private:
    std::string stock_;
    int shares_;
};

void demonstrate() {
    BuyTransaction bt("AAPL", 100);
    // Output: "Logging base Transaction"
```

```
// NOT: "BUY 100 shares of AAPL"
//
// During Transaction::Transaction(), the object's type IS Transaction,
// not BuyTransaction. BuyTransaction's members (stock_, shares_) don't
// even exist yet – they haven't been initialized!
}
```

2.11.3 Why This Behavior Exists

The rationale is safety. During `Transaction::Transaction()`:

1. The `BuyTransaction` part of the object has not been constructed yet.
2. `BuyTransaction::stock_` and `BuyTransaction::shares_` are **uninitialized**.
3. If `BuyTransaction::logTransaction()` were called, it would access `stock_` and `shares_` — reading uninitialized memory.
4. C++ prevents this by treating the object as a `Transaction` during `Transaction`'s constructor.

```
// What would happen if C++ DID allow derived virtual calls in base ctor:
class Derived : public Base {
public:
    Derived() : data_(new int[100]) {} // data_ not yet allocated

    void doWork() override {
        // If called from Base::Base(), data_ is UNINITIALIZED
        data_[0] = 42; // Writing to a random memory address – CRASH
    }

private:
    int* data_;
};
```

2.11.4 The Problem Gets Worse: Indirect Virtual Calls

The danger is not always obvious. The constructor might call a non-virtual function that internally calls a virtual function:

```
// BAD: Indirect virtual call – harder to spot
class Transaction {
public:
    Transaction() {
        init(); // Non-virtual – but calls a virtual function inside!
    }

    virtual ~Transaction() {}

private:
    void init() {
        // ... common setup ...
        logTransaction(); // VIRTUAL CALL during construction!
        // Still resolves to Transaction::logTransaction()
    }

    virtual void logTransaction() const = 0; // pure virtual!
};

class SellTransaction : public Transaction {
public:
    SellTransaction() {}

    void logTransaction() const override {
        std::cout << "SELL transaction logged\n";
    }
};
```

```

    }
};

void test() {
    // SellTransaction st; // RUNTIME ERROR or UNDEFINED BEHAVIOR
    // Transaction::Transaction() calls init() which calls logTransaction()
    // logTransaction() is pure virtual in Transaction
    // Calling a pure virtual function => typically std::terminate() / crash
    // Some compilers: "pure virtual function called" error message
}

```

2.11.5 The Same Problem in Destructors

Virtual functions during destruction have the same issue, in reverse order:

```

class Transaction {
public:
    virtual ~Transaction() {
        logTransaction(); // Virtual call during destruction
        // By the time Transaction::~~Transaction() runs,
        // the derived part (BuyTransaction) has ALREADY been destroyed
        // So this calls Transaction::logTransaction(), not the derived version
    }

    virtual void logTransaction() const {
        std::cout << "Base transaction cleanup\n";
    }
};

class BuyTransaction : public Transaction {
public:
    BuyTransaction() : stock_("AAPL") {}

    ~BuyTransaction() override {
        std::cout << "BuyTransaction destroyed\n";
        // After this, stock_ is destroyed, then Transaction::~~Transaction() runs
    }

    void logTransaction() const override {
        std::cout << "Logging BUY of " << stock_ << "\n"; // stock_ is destroyed!
    }

private:
    std::string stock_;
};

void test() {
    BuyTransaction* bt = new BuyTransaction();
    delete bt;
    // Output:
    // "BuyTransaction destroyed"      (BuyTransaction::~~BuyTransaction)
    // "Base transaction cleanup"      (Transaction::~~Transaction calls
    //                                Transaction::logTransaction, NOT
    //                                BuyTransaction::logTransaction)
}

```

2.11.6 Solution 1: Pass Information Up to the Base Class

Instead of calling down (via virtual functions), pass information **up** (via constructor parameters).

```

// GOOD: Pass derived-class-specific data up to the base constructor
class Transaction {
public:
    explicit Transaction(const std::string& logInfo) {
        logTransaction(logInfo); // Non-virtual call – safe
    }

    virtual ~Transaction() {}

    // Non-virtual – no polymorphism needed
    void logTransaction(const std::string& info) const {
        std::cout << "Transaction: " << info << "\n";
        // Write to log file, database, etc.
    }
};

class BuyTransaction : public Transaction {
public:
    BuyTransaction(const std::string& stock, int shares)
        : Transaction(createLogString(stock, shares)), // pass info UP
          stock_(stock), shares_(shares) {}

private:
    // Static helper – can be called before the object is fully constructed
    // because it doesn't access any member variables
    static std::string createLogString(const std::string& stock, int shares) {
        return "BUY " + std::to_string(shares) + " shares of " + stock;
    }

    std::string stock_;
    int shares_;
};

class SellTransaction : public Transaction {
public:
    SellTransaction(const std::string& stock, int shares)
        : Transaction(createLogString(stock, shares)),
          stock_(stock), shares_(shares) {}

private:
    static std::string createLogString(const std::string& stock, int shares) {
        return "SELL " + std::to_string(shares) + " shares of " + stock;
    }

    std::string stock_;
    int shares_;
};

void test() {
    BuyTransaction bt("AAPL", 100);
    // Output: "Transaction: BUY 100 shares of AAPL" – correct!

    SellTransaction st("GOOG", 50);
    // Output: "Transaction: SELL 50 shares of GOOG" – correct!
}

```

Note the use of a static helper function: The function `createLogString` is static, so it doesn't depend on any member of the not-yet-constructed derived object. This is critical for safety.

2.11.7 Solution 2: Post-Construction Initialization

Use a two-phase initialization pattern where virtual dispatch works correctly:

```
// GOOD: Two-phase initialization with factory function
class Widget {
public:
    virtual ~Widget() = default;

    // Factory method ensures init() is called AFTER construction
    template<typename T, typename... Args>
    static std::unique_ptr<T> create(Args&&... args) {
        auto widget = std::unique_ptr<T>(new T(std::forward<Args>(args)...));
        widget->init(); // Virtual call AFTER full construction - safe!
        return widget;
    }

    virtual void doWork() = 0;
};

protected:
    Widget() {} // Protected - force use of factory

    // Called after construction - virtual dispatch works correctly
    virtual void init() {
        std::cout << "Widget base init\n";
    }
};

class FancyWidget : public Widget {
public:
    void doWork() override {
        std::cout << "FancyWidget doing work with buffer of size " << bufferSize_ << "\n";
    }
};

protected:
    FancyWidget() : bufferSize_(0), buffer_(nullptr) {}

    void init() override {
        Widget::init(); // call base init
        bufferSize_ = 1024;
        buffer_ = new char[bufferSize_];
        std::cout << "FancyWidget initialized with buffer\n";
    }

    friend class Widget; // allow Widget::create to call constructor
};

private:
    size_t bufferSize_;
    char* buffer_;
};

void test() {
    auto w = Widget::create<FancyWidget>();
    // Output:
    // "Widget base init"
    // "FancyWidget initialized with buffer"
    w->doWork();
    // Output: "FancyWidget doing work with buffer of size 1024"
}
```

2.11.8 Real-World Example: GUI Widget Hierarchy

```
// BAD version – virtual calls in constructor
class GUIWidget {
public:
    GUIWidget(int x, int y, int width, int height)
    : x_(x), y_(y), width_(width), height_(height) {
        // These virtual calls don't dispatch to derived classes!
        applyDefaultStyle();           // virtual – BAD
        calculateLayout();             // virtual – BAD
        registerEventHandlers();       // virtual – BAD
    }

    virtual ~GUIWidget() = default;
    virtual void applyDefaultStyle() { /* base style */ }
    virtual void calculateLayout() { /* base layout */ }
    virtual void registerEventHandlers() { /* base handlers */ }

protected:
    int x_, y_, width_, height_;
};

class Button : public GUIWidget {
public:
    Button(int x, int y, int w, int h, const std::string& label)
    : GUIWidget(x, y, w, h), label_(label) {}

    void applyDefaultStyle() override {
        // This is NEVER called from GUIWidget's constructor!
        // label_ is uninitialized when GUIWidget ctor runs
        std::cout << "Button style for: " << label_ << "\n";
    }

    void calculateLayout() override {
        // Calculates text position – but label_ doesn't exist yet!
        textWidth_ = label_.length() * 8; // ACCESSING UNINITIALIZED MEMBER
    }

    void registerEventHandlers() override {
        onClick_ = [this]() { std::cout << "Clicked: " << label_ << "\n"; };
    }

private:
    std::string label_;
    int textWidth_;
    std::function<void()> onClick_;
};

// GOOD version – pass info up, or use post-construction init
class GUIWidget {
public:
    virtual ~GUIWidget() = default;

    // Factory with post-construction initialization
    template<typename T, typename... Args>
    static std::unique_ptr<T> create(Args&&... args) {
        auto widget = std::unique_ptr<T>(new T(std::forward<Args>(args)...));
        widget->applyDefaultStyle();
        widget->calculateLayout();
        widget->registerEventHandlers();
    }
};
```



```

    return widget;
}

virtual void applyDefaultStyle() {}
virtual void calculateLayout() {}
virtual void registerEventHandlers() {}

protected:
    GUIWidget(int x, int y, int width, int height)
        : x_(x), y_(y), width_(width), height_(height) {}

    int x_, y_, width_, height_;
};

class Button : public GUIWidget {
protected:
    friend class GUIWidget;
    Button(int x, int y, int w, int h, const std::string& label)
        : GUIWidget(x, y, w, h), label_(label) {}

public:
    void applyDefaultStyle() override {
        std::cout << "Button style for: " << label_ << "\n"; // label_ is valid!
    }

    void calculateLayout() override {
        textWidth_ = label_.length() * 8; // Safe - label_ exists
    }

    void registerEventHandlers() override {
        onClick_ = [this]() { std::cout << "Clicked: " << label_ << "\n"; };
    }

private:
    std::string label_;
    int textWidth_ = 0;
    std::function<void()> onClick_;
};

void test() {
    auto btn = GUIWidget::create<Button>(10, 20, 100, 30, "OK");
    // All virtual functions called AFTER full construction - correct behavior
}

```

2.11.9 Things to Remember

- Don't call virtual functions during construction or destruction. During base class construction/destruction, virtual functions resolve to the base class version, never the derived class version.
- This behavior exists for safety: during base class construction, derived class members are uninitialized; during base class destruction, they have already been destroyed.
- Instead of virtual calls in constructors, pass derived-class-specific information **up** to the base class constructor (using static helper functions if necessary).
- Alternatively, use a post-construction initialization pattern (e.g., factory functions that call virtual `init()` after the object is fully constructed).

2.12 Item 10: Have Assignment Operators Return a Reference to `*this`

2.13 Visual Summary

ITEM 10: HAVE ASSIGNMENT OPERATORS RETURN A REFERENCE TO `\*THIS`

1. Assignment expression executes -> object on left is modified.
2. Return void/value -> breaks normal chaining or adds unnecessary copy.
3. Return *this by reference -> a = b = c works like built-in types.
4. Applies to +=, -=, *= and similar assignment operators.
5. Meaning: user-defined assignment should behave like built-in assignment.

2.13.1 Core Concept

C++ built-in types support **chained assignment**:

```
int x, y, z;  
x = y = z = 15; // Right-to-left: z = 15, then y = z, then x = y
```

This works because assignment returns a reference to the left-hand operand. For your user-defined types to support the same convention, your assignment operators should return a reference to *this.

2.13.2 The Convention

```
class Widget {  
public:  
    // Copy assignment operator  
    Widget& operator=(const Widget& rhs) {  
        // ... do the assignment work ...  
        return *this; // return reference to left-hand object  
    }  
  
    // Move assignment operator  
    Widget& operator=(Widget&& rhs) noexcept {  
        // ... do the move ...  
        return *this; // same convention  
    }  
  
    // Assignment from other types  
    Widget& operator=(int value) {  
        // ... assign from int ...  
        return *this; // same convention  
    }  
};
```

2.13.3 Why This Matters

```

// Without returning *this, chaining doesn't work
class BadWidget {
public:
    void operator=(const BadWidget& rhs) { // returns void – BAD
        data_ = rhs.data_;
        // no return statement
    }
private:
    int data_;
};

BadWidget a, b, c;
// a = b = c; // ERROR: b = c returns void, can't assign void to a

// With returning *this, chaining works naturally
class GoodWidget {
public:
    GoodWidget& operator=(const GoodWidget& rhs) { // returns reference – GOOD
        data_ = rhs.data_;
        return *this;
    }
private:
    int data_;
};

GoodWidget x, y, z;
x = y = z; // Works: z assigned to y (returns y), then y assigned to x (returns x)

```

2.13.4 Applies to ALL Assignment Operators

This convention applies to all assignment-like operators, not just operator=:

```

class Matrix {
public:
    Matrix(int rows, int cols)
        : rows_(rows), cols_(cols), data_(rows * cols, 0.0) {}

    // Copy assignment
    Matrix& operator=(const Matrix& rhs) {
        if (this != &rhs) {
            rows_ = rhs.rows_;
            cols_ = rhs.cols_;
            data_ = rhs.data_;
        }
        return *this;
    }

    // Compound assignment operators – same convention
    Matrix& operator+=(const Matrix& rhs) {
        for (size_t i = 0; i < data_.size(); ++i) {
            data_[i] += rhs.data_[i];
        }
        return *this;
    }

    Matrix& operator-=(const Matrix& rhs) {
        for (size_t i = 0; i < data_.size(); ++i) {
            data_[i] -= rhs.data_[i];
        }
        return *this;
    }
}

```

```

}

Matrix& operator*=(double scalar) {
    for (auto& val : data_) {
        val *= scalar;
    }
    return *this;
}

// Assignment from initializer list
Matrix& operator=(std::initializer_list<double> values) {
    size_t i = 0;
    for (auto val : values) {
        if (i >= data_.size()) break;
        data_[i++] = val;
    }
    return *this;
}

private:
    int rows_, cols_;
    std::vector<double> data_;
};

void example() {
    Matrix a(2, 2), b(2, 2), c(2, 2);
    a = b = c;           // chained assignment
    (a += b) *= 2.0;      // compound assignment chaining
    a = {1.0, 2.0, 3.0, 4.0}; // initializer list assignment
}

```

2.13.5 Real-World Example: String Class

```

class MyString {
public:
    MyString() : data_(nullptr), len_(0) {}

    MyString(const char* s) {
        len_ = std::strlen(s);
        data_ = new char[len_ + 1];
        std::strcpy(data_, s);
    }

    ~MyString() { delete[] data_; }

    // Copy assignment – returns *this
    MyString& operator=(const MyString& rhs) {
        if (this != &rhs) {
            delete[] data_;
            len_ = rhs.len_;
            data_ = new char[len_ + 1];
            std::strcpy(data_, rhs.data_);
        }
        return *this;
    }

    // Move assignment – returns *this
    MyString& operator=(MyString&& rhs) noexcept {
        if (this != &rhs) {

```

```

        delete[] data_;
        data_ = rhs.data_;
        len_ = rhs.len_;
        rhs.data_ = nullptr;
        rhs.len_ = 0;
    }
    return *this;
}

// Assignment from C-string - returns *this
MyString& operator=(const char* s) {
    MyString temp(s);
    std::swap(data_, temp.data_);
    std::swap(len_, temp.len_);
    return *this;
}

// Append - returns *this
MyString& operator+=(const MyString& rhs) {
    size_t newLen = len_ + rhs.len_;
    char* newData = new char[newLen + 1];
    std::strcpy(newData, data_);
    std::strcat(newData, rhs.data_);
    delete[] data_;
    data_ = newData;
    len_ = newLen;
    return *this;
}

private:
    char* data_;
    size_t len_;
};

void test() {
    MyString a("Hello"), b, c;

    // All of these work because each operator returns *this:
    c = b = a;           // chain assignment
    (a += MyString(" World")) += MyString("!"); // chain append
    a = "Reset";         // assign from C-string
}

```

2.13.6 Things to Remember

- Have assignment operators return a reference to `*this`. This is a convention followed by all built-in types and all standard library types (like `std::string`, `std::vector`, etc.).
- This applies to all forms of assignment: `operator=`, `operator+=`, `operator-=`, `operator*=`, and so forth.
- While returning by value or `const` reference would technically compile, it breaks chaining and contradicts universal convention.

2.14 Item 11: Handle Assignment to Self in `operator=`

2.15 Visual Summary

ITEM 11: HANDLE ASSIGNMENT TO SELF IN `OPERATOR=`

1. `x = x` or aliasing assignment -> source and destination may be same object.
2. Delete old resource first -> source data may be destroyed before copy.
3. Guard check -> if `this == &rhs`, return `*this`.
4. Stronger path -> copy-and-swap handles self-assignment and exceptions together.
5. Meaning: assignment must survive aliases and partial failure.

2.15.1 Core Concept

Self-assignment (`a = a`) happens more often than you might think, and it can be catastrophic if your `operator=` doesn't handle it. When an object manages resources, a naive `operator=` that deletes the old resource before copying the new one will **delete the resource it's supposed to copy from** during self-assignment.

2.15.2 How Self-Assignment Happens

Self-assignment is not always as obvious as `w = w`:

```
Widget w;  
w = w; // Obvious self-assignment  
  
// Less obvious cases:  
Widget* a = &w;  
Widget* b = &w; // a and b point to the same object  
*a = *b; // Self-assignment!  
  
// Through references:  
void process(Widget& lhs, Widget& rhs) {  
    lhs = rhs; // Self-assignment if lhs and rhs refer to same object  
}  
process(w, w); // Oops  
  
// Through containers:  
std::vector<Widget> widgets(10);  
int i = 3, j = 3;  
widgets[i] = widgets[j]; // Self-assignment if i == j  
  
// Through inheritance:  
class Derived : public Base { /* ... */ };  
Derived d;  
Base& br = d;  
br = d; // Self-assignment! Both refer to same object
```

2.15.3 The Problem: Naive `operator=`

```

// BAD: Unsafe self-assignment
class Bitmap { /* large image data */ };

class Widget {
public:
    Widget(Bitmap* pb) : pb_(pb) {}

    Widget& operator=(const Widget& rhs) {
        delete pb_; // Step 1: delete current bitmap
        pb_ = new Bitmap(*rhs.pb_); // Step 2: copy rhs's bitmap
        return *this;
    }

    ~Widget() { delete pb_; }

private:
    Bitmap* pb_; // heap-allocated bitmap
};

void disaster() {
    Bitmap* bm = new Bitmap();
    Widget w(bm);
    w = w; // SELF-ASSIGNMENT!
    // Step 1: delete pb_ - destroys the only Bitmap
    // Step 2: pb_ = new Bitmap(*rhs.pb_) - rhs.pb_ IS pb_ IS DELETED
    // Dereferencing a deleted pointer - UNDEFINED BEHAVIOR
}

```

2.15.4 Solution 1: Identity Test (Partial Fix)

```

// PARTIAL FIX: Identity test
class Widget {
public:
    Widget& operator=(const Widget& rhs) {
        if (this == &rhs) return *this; // Identity test - handle self-assignment

        delete pb_;
        pb_ = new Bitmap(*rhs.pb_); // What if this throws? pb_ is dangling!
        return *this;
    }

private:
    Bitmap* pb_;
};

```

This handles self-assignment but is **not exception-safe**. If `new Bitmap(*rhs.pb_)` throws (out of memory, for example), `this->pb_` is left pointing to deleted memory.

2.15.5 Solution 2: Copy-and-Swap (Preferred — Both Self-Assignment-Safe AND Exception-Safe)

```

// GOOD: Copy-and-swap idiom
class Widget {
public:
    Widget(Bitmap* pb = nullptr) : pb_(pb) {}

    Widget(const Widget& rhs) : pb_(new Bitmap(*rhs.pb_)) {}

    Widget& operator=(const Widget& rhs) {
        Widget temp(rhs);      // Step 1: copy rhs into a temporary
        swap(temp);            // Step 2: swap this with the temporary
        return *this;          // Step 3: temp (holding old data) is destroyed
    }

    // Or even more concise – pass by value (copy made by caller):
    // Widget& operator=(Widget rhs) { // rhs is a copy
    //     swap(rhs);
    //     return *this;
    // }

    ~Widget() { delete pb_; }

    void swap(Widget& other) noexcept {
        using std::swap;
        swap(pb_, other.pb_);
    }

private:
    Bitmap* pb_;
};

void test() {
    Widget w1(new Bitmap());
    w1 = w1; // Safe!
    // temp is constructed as a copy of w1
    // swap exchanges pb_ pointers
    // temp destroys the old pb_ – which is a valid copy
    // No dangling pointers, no double deletes

    Widget w2(new Bitmap());
    w1 = w2; // Also safe, and exception-safe
    // If new Bitmap() in copy ctor throws, w1 is unchanged
}

```

2.15.6 Solution 3: Careful Ordering (Exception-Safe Without Copy-and-Swap)


```

// GOOD: Exception-safe via careful ordering
class Widget {
public:
    Widget& operator=(const Widget& rhs) {
        Bitmap* pOrig = pb_;           // Step 1: remember old pointer
        pb_ = new Bitmap(*rhs.pb_);    // Step 2: copy rhs's bitmap FIRST
        // If this throws, pb_ still points
        // to the original bitmap – safe!
        delete pOrig;                  // Step 3: delete old bitmap
        return *this;
    }

private:
    Bitmap* pb_;
};

// Self-assignment analysis:
// w = w;
// pOrig = pb_           (save pointer to our bitmap)
// pb_ = new Bitmap(*pb_) (copy our own bitmap – it's still valid)
// delete pOrig          (delete old bitmap – it's a different object now)
// Everything is fine!

```

This approach handles self-assignment correctly **without** an identity test, and it's exception-safe. The identity test (`if (this == &rhs)`) can still be added as an optimization for the self-assignment case, but it's not required for correctness.

2.15.7 Real-World Example: Resource-Managing Class with Multiple Resources

```

class TextDocument {
public:
    TextDocument(const std::string& content, const std::string& fontPath)
        : text_(new std::string(content)),
          font_(new Font(fontPath)),
          metadata_(new Metadata()) {}

    TextDocument(const TextDocument& rhs)
        : text_(new std::string(*rhs.text_)),
          font_(new Font(*rhs.font_)),
          metadata_(new Metadata(*rhs.metadata_)) {}

    ~TextDocument() {
        delete text_;
        delete font_;
        delete metadata_;
    }

    // BAD: Unsafe with multiple resources
    // If font copy throws after text was already replaced, we have
    // a partially-assigned, inconsistent object
    TextDocument& BAD_operator_assign(const TextDocument& rhs) {
        if (this == &rhs) return *this;
        delete text_;
        text_ = new std::string(*rhs.text_); // OK so far
        delete font_;
        font_ = new Font(*rhs.font_);        // If this THROWS: text_ has new
        // value but font_ is deleted!
        // Object is in an inconsistent state
        delete metadata_;
    }
}

```

```

    metadata_ = new Metadata(*rhs.metadata_);
    return *this;
}

// GOOD: Copy-and-swap – atomic, exception-safe, self-assignment-safe
TextDocument& operator=(const TextDocument& rhs) {
    TextDocument temp(rhs); // All-or-nothing copy
    swap(temp);              // noexcept swap
    return *this;            // temp cleans up old data
}

void swap(TextDocument& other) noexcept {
    using std::swap;
    swap(text_, other.text_);
    swap(font_, other.font_);
    swap(metadata_, other.metadata_);
}

private:
    std::string* text_;
    Font* font_;
    Metadata* metadata_;
};

```

2.15.8 Real-World Example: Smart Array with Copy-and-Swap

```

template<typename T>
class Array {
public:
    explicit Array(size_t size = 0)
        : size_(size), data_(size ? new T[size] : nullptr) {}

    Array(const Array& rhs)
        : size_(rhs.size_), data_(rhs.size_ ? new T[rhs.size_] : nullptr) {
            std::copy(rhs.data_, rhs.data_ + size_, data_);
        }

    // Pass-by-value copy-and-swap (combines copy ctor + swap)
    Array& operator=(Array rhs) { // rhs is a COPY – copy already made
        swap(rhs);                // swap with the copy
        return *this;              // old data destroyed when rhs goes out of scope
    }

    // Move constructor
    Array(Array&& rhs) noexcept
        : size_(rhs.size_), data_(rhs.data_) {
            rhs.size_ = 0;
            rhs.data_ = nullptr;
        }

    ~Array() { delete[] data_; }

    T& operator[](size_t index) { return data_[index]; }
    const T& operator[](size_t index) const { return data_[index]; }
    size_t size() const { return size_; }

    void swap(Array& other) noexcept {
        using std::swap;
        swap(size_, other.size_);
    }
};

```

```

    swap(data_, other.data_);
}

private:
    size_t size_;
    T* data_;
};

// ADL-findable swap for use with std::swap
template<typename T>
void swap(Array<T>& a, Array<T>& b) noexcept {
    a.swap(b);
}

void test() {
    Array<int> a(100);
    for (size_t i = 0; i < 100; ++i) a[i] = static_cast<int>(i);

    a = a; // Self-assignment – safe!
    // operator= receives a copy of a (by value)
    // Swaps internal state with the copy
    // Copy (holding old data) is destroyed
    // a still contains 0..99

    Array<int> b(50);
    b = std::move(a); // Move – also works through copy-and-swap
    // rhs is move-constructed (no copy), then swapped
}

```

2.15.9 Things to Remember

- Make sure operator= is well-behaved when an object is assigned to itself. Techniques include comparing addresses of source and target objects, careful statement ordering, and copy-and-swap.
- Make sure that any function operating on more than one object behaves correctly if two or more of the objects are actually the same object.
- The copy-and-swap idiom is the gold standard — it provides self-assignment safety and exception safety in one clean technique.

2.16 Item 12: Copy All Parts of an Object

2.17 Visual Summary

ITEM 12: COPY ALL PARTS OF AN OBJECT

1. Copy operation runs -> every meaningful member and base subobject must be copied.
2. Add new member later -> old copy code silently omits it.
3. Derived copy -> call base copy constructor/assignment explicitly.
4. Avoid duplication -> share copy logic via helper, not by one copy op calling another.
5. Meaning: partial copies create objects that look valid but carry stale state.

2.17.1 Core Concept

When you write your own copy constructor or copy assignment operator, you are taking full responsibility for copying. The compiler will not warn you if you forget to copy a member. Two common bugs arise: **(1)** forgetting to copy a newly added data member, and **(2)** forgetting to copy the base class part of a derived class. Both lead to **partial copies** — objects that look fully constructed but have uninitialized or default-initialized members.

2.17.2 Bug 1: Forgetting to Copy New Members

```
class Customer {
public:
    Customer(const std::string& name) : name_(name) {}

    Customer(const Customer& rhs) : name_(rhs.name_) {}

    Customer& operator=(const Customer& rhs) {
        name_ = rhs.name_;
        return *this;
    }

private:
    std::string name_;
};
// Everything is fine so far...

// Months later, a new member is added:
class Customer {
public:
    Customer(const std::string& name, int priority)
        : name_(name), priority_(priority) {}

    // OOPS: Copy constructor was NOT updated!
    Customer(const Customer& rhs) : name_(rhs.name_) {
        // priority_ is NOT copied – it's default-initialized (0 or garbage)
    }

    // OOPS: Copy assignment was NOT updated!
    Customer& operator=(const Customer& rhs) {
        name_ = rhs.name_;
        // priority_ is NOT assigned – the target keeps its old value
        return *this;
    }

private:
    std::string name_;
```

```

    int priority_;    // NEW MEMBER – but copy operations don't know about it!
};

void demonstrate() {
    Customer c1("Alice", 5);    // priority_ = 5
    Customer c2(c1);            // c2.priority_ = 0 or garbage – NOT 5!
    Customer c3("Bob", 1);
    c3 = c1;                    // c3.priority_ is still 1, NOT 5!
}

```

The compiler will **not** warn you about this. It generated the copy operations itself before, and it considers the fact that you've written them to mean you know what you're doing.

The fix is obvious but easy to forget:

```

// GOOD: Copy ALL members
class Customer {
public:
    Customer(const std::string& name, int priority)
        : name_(name), priority_(priority) {}

    Customer(const Customer& rhs)
        : name_(rhs.name_),
          priority_(rhs.priority_) {} // DON'T FORGET THIS!

    Customer& operator=(const Customer& rhs) {
        name_ = rhs.name_;
        priority_ = rhs.priority_;    // DON'T FORGET THIS!
        return *this;
    }

private:
    std::string name_;
    int priority_;
};

```

2.17.3 Bug 2: Forgetting to Copy the Base Class Part

This is the more insidious bug. When writing copy operations for a derived class, you must explicitly copy the base class portion.

```

class PriorityCustomer : public Customer {
public:
    PriorityCustomer(const std::string& name, int priority, int vipLevel)
        : Customer(name, priority), vipLevel_(vipLevel) {}

    // BAD: Copies derived part but FORGETS base part
    PriorityCustomer(const PriorityCustomer& rhs)
        : vipLevel_(rhs.vipLevel_) {
        // Customer base class is DEFAULT-CONSTRUCTED (if possible)
        // rhs.name_ and rhs.priority_ are NOT copied!
        // The base class portion of this object has empty name and 0 priority
    }

    // BAD: Assigns derived part but FORGETS base part
    PriorityCustomer& operator=(const PriorityCustomer& rhs) {
        vipLevel_ = rhs.vipLevel_;
        // Customer::operator= is NOT called!
        // name_ and priority_ are NOT assigned!
        return *this;
    }
}

```

```

private:
    int vipLevel_;
};

void bug() {
    PriorityCustomer pc1("VIP Alice", 10, 3);
    PriorityCustomer pc2(pc1);
    // pc2.vipLevel_ == 3 (copied)
    // pc2.name_ == "" (NOT copied – default constructed)
    // pc2.priority_ == 0 (NOT copied – default constructed)

    PriorityCustomer pc3("Regular", 1, 1);
    pc3 = pc1;
    // pc3.vipLevel_ == 3 (assigned)
    // pc3.name_ is still "Regular" (NOT assigned!)
    // pc3.priority_ is still 1 (NOT assigned!)
}

```

2.17.4 The Fix: Explicitly Call Base Class Copy Operations

```

// GOOD: Copy ALL parts – including the base class
class PriorityCustomer : public Customer {
public:
    PriorityCustomer(const std::string& name, int priority, int vipLevel)
        : Customer(name, priority), vipLevel_(vipLevel) {}

    // Copy constructor: invoke base class copy constructor
    PriorityCustomer(const PriorityCustomer& rhs)
        : Customer(rhs),                // IMPORTANT: copy the base class part!
          vipLevel_(rhs.vipLevel_) {    // copy the derived class part
        // Customer's copy ctor receives a PriorityCustomer& but takes
        // a const Customer& – slicing extracts the Customer part
    }

    // Copy assignment: invoke base class operator=
    PriorityCustomer& operator=(const PriorityCustomer& rhs) {
        Customer::operator=(rhs);      // IMPORTANT: assign the base class part!
        vipLevel_ = rhs.vipLevel_;     // assign the derived class part
        return *this;
    }

private:
    int vipLevel_;
};

```

2.17.5 Don't Implement One in Terms of the Other

A common temptation is to avoid code duplication by having the copy constructor call `operator=` or vice versa. Both are wrong:

```
// BAD: Copy constructor calling operator=
class Widget {
public:
    Widget(const Widget& rhs) {
        *this = rhs; // Calls operator= on a not-yet-fully-constructed object!
        // operator= may delete resources that were never allocated
        // (because ctor hasn't finished initializing them)
    }

    Widget& operator=(const Widget& rhs) {
        if (this == &rhs) return *this;
        delete data_; // Deletes uninitialized pointer in copy ctor!
        data_ = new int(*rhs.data_);
        return *this;
    }

private:
    int* data_;
};

// BAD: operator= calling copy constructor (via placement new)
class Widget {
public:
    Widget& operator=(const Widget& rhs) {
        this->~Widget(); // Destroy current object
        new (this) Widget(rhs); // Construct a new one in the same memory
        // If the copy ctor throws, the object is in a destroyed state!
        // Any subsequent access (including the destructor) is undefined behavior
        return *this;
    }
};
```

The correct approach: extract common code into a private helper:

```
// GOOD: Common code in a private init/copyFrom function
class Widget {
public:
    Widget(int value, const std::string& name)
    : data_(new int(value)), name_(name), cache_(nullptr) {
        rebuildCache();
    }

    Widget(const Widget& rhs)
    : data_(new int(*rhs.data_)), // allocate and copy
      name_(rhs.name_),
      cache_(nullptr) {
        rebuildCache(); // shared logic in private helper
    }

    Widget& operator=(const Widget& rhs) {
        if (this == &rhs) return *this;

        int* newData = new int(*rhs.data_); // copy first (exception safety)
        delete data_; // then delete old
        data_ = newData;
        name_ = rhs.name_;
        rebuildCache(); // shared logic in private helper
        return *this;
    }

    ~Widget() {
```

```

    delete data_;
    delete cache_;
}

private:
    void rebuildCache() {
        delete cache_;
        cache_ = new CacheData(*data_, name_);
    }

    int* data_;
    std::string name_;
    CacheData* cache_;
};

```

2.17.6 Real-World Example: Deep Hierarchy

```

class Shape {
public:
    Shape(const Color& color, double opacity)
        : color_(color), opacity_(opacity) {}

    Shape(const Shape& rhs)
        : color_(rhs.color_), opacity_(rhs.opacity_) {}

    Shape& operator=(const Shape& rhs) {
        color_ = rhs.color_;
        opacity_ = rhs.opacity_;
        return *this;
    }

    virtual ~Shape() = default;

protected:
    Color color_;
    double opacity_;
};

class Polygon : public Shape {
public:
    Polygon(const Color& color, double opacity, std::vector<Point> vertices)
        : Shape(color, opacity), vertices_(std::move(vertices)) {}

    Polygon(const Polygon& rhs)
        : Shape(rhs),                      // copy Shape part
          vertices_(rhs.vertices_) {}      // copy Polygon part

    Polygon& operator=(const Polygon& rhs) {
        Shape::operator=(rhs);             // assign Shape part
        vertices_ = rhs.vertices_;         // assign Polygon part
        return *this;
    }

protected:
    std::vector<Point> vertices_;
};

class TexturedPolygon : public Polygon {
public:

```



```

TexturedPolygon(const Color& color, double opacity,
                std::vector<Point> vertices,
                const std::string& texturePath)
    : Polygon(color, opacity, std::move(vertices)),
      texture_(new Texture(texturePath)),
      uvCoords_(vertices_.size()) {}

// Copy constructor – must copy ALL THREE levels
TexturedPolygon(const TexturedPolygon& rhs)
    : Polygon(rhs), // copies Shape AND Polygon parts
      texture_(new Texture(*rhs.texture_)), // deep copy of texture
      uvCoords_(rhs.uvCoords_) {} // copy UV coordinates

// Copy assignment – must assign ALL THREE levels
TexturedPolygon& operator=(const TexturedPolygon& rhs) {
    Polygon::operator=(rhs); // assigns Shape AND Polygon parts

    Texture* newTex = new Texture(*rhs.texture_); // copy first (exception safe)
    delete texture_;
    texture_ = newTex;
    uvCoords_ = rhs.uvCoords_;

    return *this;
}

~TexturedPolygon() override {
    delete texture_;
}

private:
    Texture* texture_;
    std::vector<UV> uvCoords_;
};

void test() {
    TexturedPolygon tp1(Color::Red, 0.8,
                       {{0,0}, {1,0}, {1,1}},
                       "brick.png");

    TexturedPolygon tp2(tp1); // Copies ALL parts:
    // Shape: color_ = Red, opacity_ = 0.8
    // Polygon: vertices_ = {{0,0},{1,0},{1,1}}
    // TexturedPolygon: texture_ = deep copy of brick texture,
    // uvCoords_ = copied

    TexturedPolygon tp3(Color::Blue, 0.5,
                       {{0,0}, {2,0}, {2,2}, {0,2}},
                       "stone.png");
    tp3 = tp1; // Assigns ALL parts – nothing is left behind
}

```

2.17.7 Real-World Example: Configuration Object with Many Members

```

class ServerConfig {
public:
    ServerConfig()
        : host_("localhost"), port_(8080), maxConnections_(100),
          timeout_(30), useTLS_(false), logLevel_(LogLevel::INFO),
          threadPoolSize_(4), maxRequestSize_(1 << 20),
          keepAliveEnabled_(true), keepAliveTimeout_(60),
          compressionEnabled_(false),
          certPath_(""), keyPath_("") {}

    // When there are many members, it's easy to miss one.
    // A disciplined approach: list members in the SAME ORDER as declaration.

    ServerConfig(const ServerConfig& rhs)
        : host_(rhs.host_),
          port_(rhs.port_),
          maxConnections_(rhs.maxConnections_),
          timeout_(rhs.timeout_),
          useTLS_(rhs.useTLS_),
          logLevel_(rhs.logLevel_),
          threadPoolSize_(rhs.threadPoolSize_),
          maxRequestSize_(rhs.maxRequestSize_),
          keepAliveEnabled_(rhs.keepAliveEnabled_),
          keepAliveTimeout_(rhs.keepAliveTimeout_),
          compressionEnabled_(rhs.compressionEnabled_),
          certPath_(rhs.certPath_),
          keyPath_(rhs.keyPath_) {
        // Every. Single. Member. Copied.
        // If someone adds a new member and forgets to add it here,
        // the compiler won't warn – this is a maintenance burden.
    }

    ServerConfig& operator=(const ServerConfig& rhs) {
        host_ = rhs.host_;
        port_ = rhs.port_;
        maxConnections_ = rhs.maxConnections_;
        timeout_ = rhs.timeout_;
        useTLS_ = rhs.useTLS_;
        logLevel_ = rhs.logLevel_;
        threadPoolSize_ = rhs.threadPoolSize_;
        maxRequestSize_ = rhs.maxRequestSize_;
        keepAliveEnabled_ = rhs.keepAliveEnabled_;
        keepAliveTimeout_ = rhs.keepAliveTimeout_;
        compressionEnabled_ = rhs.compressionEnabled_;
        certPath_ = rhs.certPath_;
        keyPath_ = rhs.keyPath_;
        return *this;
    }

    // Modern alternative: if compiler-generated copy is correct
    // (no raw pointers, all members are copyable), just use = default:
    // ServerConfig(const ServerConfig&) = default;
    // ServerConfig& operator=(const ServerConfig&) = default;
    // The compiler copies ALL members – no risk of forgetting one!

private:
    std::string host_;
    int port_;
    int maxConnections_;

```

```

int timeout_;
bool useTLS_;
LogLevel logLevel_;
int threadPoolSize_;
size_t maxRequestSize_;
bool keepAliveEnabled_;
int keepAliveTimeout_;
bool compressionEnabled_;
std::string certPath_;
std::string keyPath_;
};

```

2.17.8 Checklist: When Writing Copy Operations

Every time you write a copy constructor or copy assignment operator, go through this checklist:

1. **Copy every data member** declared in the class, in the same order they're declared.
2. **Invoke the base class copy operation** — `Base( rhs )` in the copy ctor's initializer list, `Base::operator=( rhs )` in `operator=`.
3. **If a new member is added later**, update both the copy constructor AND `operator=`.
4. **Don't implement one in terms of the other**. Extract shared code into a private helper function instead.
5. **Consider whether `= default` is sufficient**. If your class doesn't manage raw resources, compiler-generated copy operations may be correct and maintainable.

```

// Modern approach: when possible, use = default and let the compiler do it
class ModernConfig {
public:
    ModernConfig() = default;
    ModernConfig(const ModernConfig&) = default;           // copies ALL members
    ModernConfig& operator=(const ModernConfig&) = default; // assigns ALL members
    ~ModernConfig() = default;

    // If you add a new member, the compiler automatically includes it
    // in the generated copy operations. No maintenance burden!

private:
    std::string host_ = "localhost";
    int port_ = 8080;
    bool useTLS_ = false;
    // Add a new member here – copy operations automatically updated
};

```

2.17.9 Things to Remember

- Copying functions should be sure to copy all of an object's data members and all of its base class parts.
- When adding a new data member to a class, update every copy constructor and every copy assignment operator. The compiler will not warn you if you forget.
- Don't try to implement one of the copying functions in terms of the other. Instead, put common functionality in a third function that both call.
- When your class doesn't manage raw resources, prefer `= default` to hand-written copy operations — the compiler will automatically include all members, including ones added later.

3 Chapter 3: Resource Management

3.1 Chapter Flow

CHAPTER 3: RESOURCE MANAGEMENT

Item 13 -> acquire resource into RAII object immediately.
Item 14 -> define copy behavior for resource owners.
Item 15 -> expose raw handles only as borrowed access.
Item 16 -> pair new/delete forms exactly.
Item 17 -> put newed objects into smart pointers in standalone statements.

C++ programs manage many kinds of resources: dynamically allocated memory, file descriptors, mutex locks, database connections, network sockets, GUI fonts and brushes. Regardless of the resource type, the fundamental challenge is the same: once you acquire a resource, you must eventually release it. Failure to do so leads to resource leaks, which degrade performance, cause data corruption, or crash programs.

This chapter covers five items that together form a coherent philosophy for resource management in C++. The central idea is **RAII** (Resource Acquisition Is Initialization): bind the lifetime of a resource to the lifetime of an object, so that C++'s deterministic destruction guarantees cleanup even in the presence of exceptions or early returns.

3.2 Item 13: Use objects to manage resources

3.3 Visual Summary

ITEM 13: USE OBJECTS TO MANAGE RESOURCES

1. Raw resource acquired -> every exit path must release it.
2. Manual delete/close -> return, exception, or edit can skip cleanup.
3. RAII wrapper constructed -> resource ownership moves into an object.
4. Scope exits any way -> destructor releases automatically.
5. Meaning: bind resource lifetime to object lifetime.

3.3.1 The Problem: Manual Resource Management

Consider a factory function that returns a pointer to a dynamically allocated object:

```
class Investment {
public:
    virtual ~Investment() {}
    virtual double netAssetValue() const = 0;
    // ...
};

class Stock : public Investment { /* ... */ };
class Bond : public Investment { /* ... */ };

// Factory function
Investment* createInvestment() {
    // Analyze parameters, return appropriate derived type
    return new Stock(); // simplified
}
```

Callers of `createInvestment` are responsible for deleting the returned object. This seems straightforward, but in practice it is remarkably fragile.

```

void f() {
    Investment* pInv = createInvestment();

    // ... use pInv ...

    delete pInv;    // Release the resource
}

```

3.3.1.1 BAD: Manual delete This looks correct, but there are numerous ways the delete can fail to execute:

1. A premature return statement somewhere in the “use pInv” block.
2. The “use pInv” code throws an exception.
3. A continue or goto jumps past the delete.
4. A maintainer later restructures the function and accidentally removes the delete.

```

// BAD: Exception-unsafe manual management
void f() {
    Investment* pInv = createInvestment();

    doSomethingRisky();    // If this throws, pInv is never deleted!

    if (someCondition) {
        return;            // pInv is leaked here too
    }

    // Imagine 50 more lines of code...

    delete pInv;          // Reached only on the "happy path"
}

```

Even diligent programmers eventually forget or are defeated by exceptions. The fundamental issue is that **relying on manual cleanup is fragile and unscalable**.

3.3.2 The Solution: RAII (Resource Acquisition Is Initialization)

The C++ language guarantees that destructors of local objects are called when control leaves their scope, regardless of *how* it leaves – whether by normal flow, an exception, or a return statement. RAII exploits this guarantee: put the resource inside an object, and let the destructor release it.

Two critical aspects of RAII:

1. **Resources are acquired and immediately turned over to resource-managing objects.** The resource is acquired in the same statement that initializes the managing object. “Resource Acquisition Is Initialization” – the resource is acquired during the object’s initialization (construction).
2. **Resource-managing objects use their destructors to ensure resources are released.** Destructors are called automatically when objects go out of scope, so resources are released regardless of how control leaves a block.

```

void f() {
    std::auto_ptr<Investment> pInv(createInvestment());

    // Use pInv exactly as before, via -> and *
    double nav = pInv->netAssetValue();

    // auto_ptr's destructor deletes pInv automatically
    // -- no leak, even if an exception is thrown
}

```

3.3.2.1 GOOD: Using auto_ptr (C++98/03 style) auto_ptr was the original standard smart pointer. It has a peculiar property: **copying an auto_ptr transfers ownership** (the source becomes null). This means auto_ptrs cannot be stored in standard containers, and passing one by value silently strips ownership from the caller.

```

// auto_ptr's unusual copy behavior
std::auto_ptr<Investment> pInv1(createInvestment());
std::auto_ptr<Investment> pInv2(pInv1);    // pInv2 owns it; pInv1 is now null!
pInv1 = pInv2;                            // pInv1 owns it; pInv2 is now null!

```

3.3.2.2 GOOD: Using shared_ptr (C++11 and TR1) shared_ptr is a reference-counted smart pointer. Multiple shared_ptrs can point to the same object, and the object is destroyed when the last shared_ptr pointing to it is destroyed or reset.

```

void f() {
    std::shared_ptr<Investment> pInv(createInvestment());

    // Use pInv exactly as before
    double nav = pInv->netAssetValue();

    // shared_ptr's destructor decrements the reference count.
    // When the count reaches zero, the object is deleted.
}

```

Unlike auto_ptr, shared_ptr behaves like a normal value during copying:

```

std::shared_ptr<Investment> pInv1(createInvestment());
std::shared_ptr<Investment> pInv2(pInv1);    // Both point to same object, refcount = 2
pInv1 = pInv2;                             // Still both point to same object

// Object is deleted when both pInv1 and pInv2 are destroyed

```

This makes shared_ptr safe for use in STL containers:

```
std::vector<std::shared_ptr<Investment>> portfolio;
portfolio.push_back(std::make_shared<Stock>());
portfolio.push_back(std::make_shared<Bond>());
// All Investments are automatically deleted when portfolio is destroyed
```

3.3.2.3 GOOD: Using unique_ptr (Modern C++11+) `unique_ptr` is the modern replacement for `auto_ptr`. It enforces sole ownership and cannot be copied – only moved. This eliminates the confusing implicit ownership transfer of `auto_ptr`.

```
void f() {
    std::unique_ptr<Investment> pInv(createInvestment());

    // Use pInv normally
    double nav = pInv->netAssetValue();

    // unique_ptr's destructor deletes the object
}
```

```
std::unique_ptr<Investment> pInv1(createInvestment());

// std::unique_ptr<Investment> pInv2(pInv1); // ERROR: copy constructor is deleted
std::unique_ptr<Investment> pInv2(std::move(pInv1)); // OK: explicit ownership transfer
// pInv1 is now null
```

`unique_ptr` can be stored in move-aware containers:

```
std::vector<std::unique_ptr<Investment>> portfolio;
portfolio.push_back(std::make_unique<Stock>()); // C++14
portfolio.push_back(std::make_unique<Bond>());
// All Investments are automatically deleted when portfolio is destroyed
```

3.3.3 RAII Applied to Other Resource Types

RAII is not limited to heap memory. It applies to any resource that must be acquired and then released.

```
// BAD: Manual file management
void processFile(const std::string& filename) {
    FILE* fp = fopen(filename.c_str(), "r");
    if (!fp) return;

    processData(fp); // If this throws, fp is leaked!

    fclose(fp);
}

// GOOD: RAII wrapper for FILE*
class FileHandle {
public:
    explicit FileHandle(const std::string& filename, const char* mode)
        : fp_(fopen(filename.c_str(), mode))
    {
        if (!fp_) throw std::runtime_error("Cannot open file: " + filename);
    }

    ~FileHandle() {
        if (fp_) fclose(fp_);
    }

    FILE* get() const { return fp_; }
}
```

```

    // Prevent copying
    FileHandle(const FileHandle&) = delete;
    FileHandle& operator=(const FileHandle&) = delete;

private:
    FILE* fp_;
};

void processFile(const std::string& filename) {
    FileHandle fh(filename, "r");
    processData(fh.get());
    // fclose happens automatically, even if processData throws
}

```

3.3.3.1 Example: File Handles

```

// BAD: Manual lock management
void processSharedData() {
    mutex.lock();

    // ... process data ...
    // If an exception is thrown here, the mutex is never unlocked!
    // Other threads will deadlock.

    mutex.unlock();
}

// GOOD: RAII lock guard
void processSharedData() {
    std::lock_guard<std::mutex> guard(mutex); // Lock acquired

    // ... process data ...
    // If an exception is thrown, ~lock_guard() unlocks the mutex

    // Mutex automatically unlocked when guard goes out of scope
}

```

3.3.3.2 Example: Mutex Locks

```

// GOOD: RAII for database connections
class DBConnection {
public:
    static DBConnection open(const std::string& connStr) {
        DBConnection conn;
        conn.handle_ = db_connect(connStr.c_str());
        if (!conn.handle_) throw std::runtime_error("DB connection failed");
        return conn;
    }

    ~DBConnection() {
        if (handle_) db_disconnect(handle_);
    }

    // Move-only semantics
    DBConnection(DBConnection&& other) noexcept : handle_(other.handle_) {
        other.handle_ = nullptr;
    }
    DBConnection& operator=(DBConnection&& other) noexcept {

```



```

        if (this != &other) {
            if (handle_) db_disconnect(handle_);
            handle_ = other.handle_;
            other.handle_ = nullptr;
        }
        return *this;
    }
    DBConnection(const DBConnection&) = delete;
    DBConnection& operator=(const DBConnection&) = delete;

private:
    DBConnection() : handle_(nullptr) {}
    db_handle_t handle_;
};

void doWork() {
    auto conn = DBConnection::open("host=db.example.com;db=mydb");
    // Use conn...
    // Connection automatically closed when conn goes out of scope
}

```

3.3.3.3 Example: Database Connections

3.3.4 Using shared_ptr with Custom Deleters

Both `auto_ptr` and `unique_ptr` call `delete` in their destructors. But not all resources are released by calling `delete`. For non-new resources, `shared_ptr` (and `unique_ptr`) can be given custom deleters:

```

// Using shared_ptr with a custom deleter for a C-style resource
void processFile(const std::string& filename) {
    // shared_ptr with a custom deleter that calls fclose
    std::shared_ptr<FILE> fp(
        fopen(filename.c_str(), "r"),
        [](FILE* f) {
            if (f) fclose(f);
        }
    );

    if (!fp) throw std::runtime_error("Cannot open file");

    // Use fp.get() to access the raw FILE*
    char buffer[256];
    while (fgets(buffer, sizeof(buffer), fp.get())) {
        // process buffer
    }
    // fclose is called automatically via the custom deleter
}

```

```

// Using unique_ptr with a custom deleter
struct FileCloser {
    void operator()(FILE* f) const {
        if (f) fclose(f);
    }
};

using UniqueFile = std::unique_ptr<FILE, FileCloser>;

void processFile(const std::string& filename) {
    UniqueFile fp(fopen(filename.c_str(), "r"));
    if (!fp) throw std::runtime_error("Cannot open file");
    // ...
    // fclose called automatically
}

```

3.3.5 An Important Warning

Both `auto_ptr` and `shared_ptr` (and `unique_ptr`) use `delete` in their destructors, not `delete[]`. This means that using them with dynamically allocated arrays is a bad idea (though `unique_ptr` has a partial specialization for arrays):

```

// BAD: undefined behavior with auto_ptr and shared_ptr
std::auto_ptr<std::string> aps(new std::string[10]); // delete called, not delete[]
std::shared_ptr<int> spi(new int[1024]);           // Same problem

// OK: unique_ptr has array specialization
std::unique_ptr<int[]> upi(new int[1024]);           // delete[] called correctly
// upi[0] = 42; // operator[] is available

// OK: shared_ptr with custom deleter for arrays
std::shared_ptr<int> spi(new int[1024], std::default_delete<int[]>());

// BEST: Use std::vector or std::array instead of raw arrays
std::vector<int> v(1024);

```

3.3.6 Things to Remember

- To prevent resource leaks, use RAII objects that acquire resources in their constructors and release them in their destructors.
- Two commonly useful RAII classes are `shared_ptr` and `unique_ptr` (formerly `auto_ptr`). `shared_ptr` uses reference counting and supports copying. `unique_ptr` enforces sole ownership and supports only moving. `auto_ptr` is deprecated and should not be used in new code.
- `auto_ptr`'s copy operations transfer ownership, making the source null. This makes it unsuitable for use in containers. Prefer `unique_ptr` instead.

3.4 Item 14: Think carefully about copying behavior in resource-managing classes

3.5 Visual Summary

ITEM 14: THINK CAREFULLY ABOUT COPYING BEHAVIOR IN RESOURCE-MANAGING CLASSES

1. RAII class owns a resource -> copying asks: what happens to ownership?
2. Forbid copy -> unique ownership.
3. Reference count -> shared ownership.
4. Deep copy -> independent resources with same value.
5. Transfer/move -> source gives up ownership explicitly.
6. Meaning: copying policy is part of the resource contract.

3.5.1 Beyond Heap Memory

Not all resources live on the heap. For non-heap resources, smart pointers like `shared_ptr` and `weak_ptr` are often inappropriate as resource handlers. When that is the case, you need to craft your own resource-managing class, and when you do, you must confront a fundamental question: **what should happen when an RAII object is copied?**

3.5.2 The Motivating Example: Mutex Locks

Consider a class that manages the locking of a Mutex:

```
// A C-style API for mutexes
void lock(Mutex* pm);    // Lock the mutex pointed to by pm
void unlock(Mutex* pm); // Unlock the mutex

// RAII class for mutex locks
class Lock {
public:
    explicit Lock(Mutex* pm) : mutexPtr_(pm) {
        lock(mutexPtr_); // Acquire resource (lock) in constructor
    }
    ~Lock() {
        unlock(mutexPtr_); // Release resource (unlock) in destructor
    }

private:
    Mutex* mutexPtr_;
};
```

Usage follows the RAII pattern:

```

Mutex m;

{
    Lock m1(&m);    // Lock the mutex
    // ... critical section ...
}                  // Automatically unlock the mutex at end of block

```

But what should happen if a Lock object is copied?

```

Lock m1(&m);
Lock m2(m1);    // What should happen here?

```

This question arises for every RAII class. There are four common strategies.

3.5.3 Strategy 1: Prohibit Copying

In many cases, it makes no sense to copy an RAII object. A mutex lock is owned by exactly one scope; copying it is semantically meaningless. The right response is to make copying illegal.

```

// C++11 style: delete the copy operations
class Lock {
public:
    explicit Lock(Mutex* pm) : mutexPtr_(pm) {
        lock(mutexPtr_);
    }
    ~Lock() {
        unlock(mutexPtr_);
    }

    // Prohibit copying
    Lock(const Lock&) = delete;
    Lock& operator=(const Lock&) = delete;

private:
    Mutex* mutexPtr_;
};

// C++98 style: declare copy operations private and don't define them
// (or inherit from boost::noncopyable / a custom Uncopyable base class)
class Lock : private Uncopyable {
public:
    // ...
};

```

This is the approach taken by `std::lock_guard`, `std::unique_lock` (it supports moving but not copying), and `std::unique_ptr`.

```
Lock m1(&m);
// Lock m2(m1);    // Compile-time error: copy constructor is deleted
```

3.5.4 Strategy 2: Reference-Count the Underlying Resource

Sometimes you want to hold on to a resource until the last object using it is destroyed. In that case, copying an RAI object should increment a reference count. This is the behavior of `std::shared_ptr`.

Conveniently, `shared_ptr` allows a custom deleter, so you can use it directly:

```
class Lock {
public:
    explicit Lock(Mutex* pm) : mutexPtr_(pm, unlock) {
        lock(mutexPtr_.get());
    }
    // No need for a destructor! shared_ptr's destructor calls unlock(mutexPtr_.get())
    // when the reference count drops to zero.

private:
    std::shared_ptr<Mutex> mutexPtr_;
};
```

Now copying works naturally:

```
Mutex m;
Lock m1(&m);    // Locks the mutex, refcount = 1
{
    Lock m2(m1); // Copies the shared_ptr, refcount = 2
    // Both m1 and m2 "share" the lock
}
// m2 destroyed, refcount = 1 -- mutex NOT unlocked yet
// ...
// m1 destroyed, refcount = 0 -- mutex IS unlocked
```

Note that the `Lock` class no longer declares a destructor. The default destructor destroys `mutexPtr_`, which is a `shared_ptr` whose custom deleter calls `unlock`. This is an elegant example of composition.

3.5.5 Strategy 3: Copy the Underlying Resource (Deep Copy)

Sometimes you can – and should – copy the managed resource itself. This is “deep copying.” When you copy the RAI object, you also create a copy of the resource it wraps.

The canonical example is `std::string`. When you copy a `std::string`, the underlying character buffer is duplicated:

```
// Deep-copying RAI class
class Bitmap {
public:
    Bitmap(int width, int height)
        : width_(width), height_(height),
          data_(new unsigned char[width * height * 4])
    {
        std::memset(data_, 0, width * height * 4);
    }

    ~Bitmap() {
        delete[] data_;
    }

    // Deep copy: allocate new buffer and copy contents
    Bitmap(const Bitmap& rhs)
        : width_(rhs.width_), height_(rhs.height_),
          data_(new unsigned char[width_ * height_ * 4])
    {
```

```

        std::memcpy(data_, rhs.data_, width_ * height_ * 4);
    }

    Bitmap& operator=(const Bitmap& rhs) {
        if (this != &rhs) {
            // Allocate new buffer first (strong exception safety)
            unsigned char* newData = new unsigned char[rhs.width_ * rhs.height_ * 4];
            std::memcpy(newData, rhs.data_, rhs.width_ * rhs.height_ * 4);

            // Now safe to modify *this
            delete[] data_;
            data_ = newData;
            width_ = rhs.width_;
            height_ = rhs.height_;
        }
        return *this;
    }

private:
    int width_, height_;
    unsigned char* data_;
};

```

```

class TempFile {
public:
    explicit TempFile(const std::string& prefix) {
        filename_ = prefix + "_" + generateUniqueId() + ".tmp";
        fp_ = fopen(filename_.c_str(), "w+");
        if (!fp_) throw std::runtime_error("Cannot create temp file");
    }

    ~TempFile() {
        if (fp_) fclose(fp_);
        std::remove(filename_.c_str()); // Delete the file from disk
    }

    // Deep copy: create a new temp file and copy contents
    TempFile(const TempFile& rhs) : fp_(nullptr) {
        filename_ = rhs.filename_ + ".copy_" + generateUniqueId();
        fp_ = fopen(filename_.c_str(), "w+");
        if (!fp_) throw std::runtime_error("Cannot create temp file copy");

        // Copy the contents of the original file
        if (rhs.fp_) {
            long pos = ftell(rhs.fp_); // Save current position
            fseek(rhs.fp_, 0, SEEK_SET); // Rewind
            char buffer[4096];
            size_t n;
            while ((n = fread(buffer, 1, sizeof(buffer), rhs.fp_)) > 0) {
                fwrite(buffer, 1, n, fp_);
            }
            fseek(rhs.fp_, pos, SEEK_SET); // Restore original position
        }
    }

    // Assignment: similar logic with self-assignment check
    TempFile& operator=(const TempFile& rhs);
};

```

```

FILE* get() const { return fp_; }
const std::string& filename() const { return filename_; }

private:
    static std::string generateUniqueId();
    std::string filename_;
    FILE* fp_;
};

```

3.5.5.1 A Complete Deep-Copy RAII Example: Managing a Temporary File

3.5.6 Strategy 4: Transfer Ownership of the Underlying Resource

In rare cases, you want to ensure that only one RAII object ever refers to a raw resource, and copying transfers ownership from the source to the destination, leaving the source in a null or empty state. This is what `auto_ptr` does and what `unique_ptr` achieves via move semantics.

```

// Move-only RAII class (modern C++ approach)
class SocketConnection {
public:
    explicit SocketConnection(const std::string& host, int port)
        : fd_(connectToHost(host, port))
    {
        if (fd_ < 0) throw std::runtime_error("Connection failed");
    }

    ~SocketConnection() {
        if (fd_ >= 0) close(fd_);
    }

    // Move constructor: transfer ownership
    SocketConnection(SocketConnection&& other) noexcept : fd_(other.fd_) {
        other.fd_ = -1; // Source no longer owns the socket
    }

    // Move assignment: transfer ownership
    SocketConnection& operator=(SocketConnection&& other) noexcept {
        if (this != &other) {
            if (fd_ >= 0) close(fd_);
            fd_ = other.fd_;
            other.fd_ = -1;
        }
        return *this;
    }

    // Copying is prohibited -- there is only one socket
    SocketConnection(const SocketConnection&) = delete;
    SocketConnection& operator=(const SocketConnection&) = delete;

    int fd() const { return fd_; }

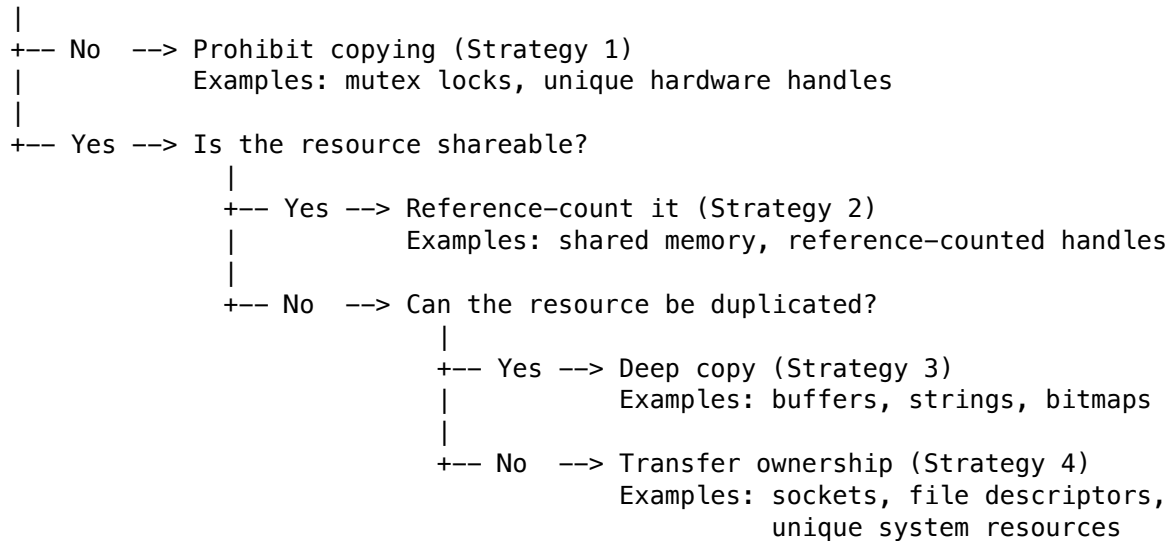
private:
    int fd_;
};

// Usage:
SocketConnection conn1("example.com", 80);
// SocketConnection conn2(conn1); // ERROR: cannot copy
SocketConnection conn2(std::move(conn1)); // OK: ownership transferred
// conn1.fd() is now -1; conn2 owns the socket

```

3.5.7 Choosing the Right Strategy: A Decision Framework

Is copying semantically meaningful?



3.5.8 A Real-World Example: Combining Strategies

In practice, a single class might use different strategies for different resources:

```
class RenderContext {
public:
    RenderContext(int width, int height, const std::string& shaderFile)
        : framebuffer_(new unsigned char[width * height * 4]), // Deep copy
          width_(width), height_(height),
          shaderProgram_(loadShader(shaderFile), deleteShader), // Ref-counted
          gpuContext_(acquireGPUContext()) // Non-copyable
    {}

    // The compiler-generated copy constructor would:
    // - Deep copy framebuffer_ (if we use a vector)
    // - Share shaderProgram_ (shared_ptr copies increment refcount)
    // - Fail on gpuContext_ (unique_ptr is non-copyable)
    //
    // Since gpuContext_ is non-copyable, the whole class is non-copyable
    // unless we explicitly decide what to do.

    RenderContext(const RenderContext&) = delete;
    RenderContext& operator=(const RenderContext&) = delete;

    // But we can support move semantics
    RenderContext(RenderContext&&) = default;
    RenderContext& operator=(RenderContext&&) = default;

private:
    std::vector<unsigned char> framebuffer_; // Deep-copyable (via vector)
    std::shared_ptr<ShaderProgram> shaderProgram_; // Reference-counted
    std::unique_ptr<GPUContext> gpuContext_; // Non-copyable, movable
};
```

3.5.9 Things to Remember

- Copying an RAII object entails copying the resource it manages, so the copying behavior of the resource determines the copying behavior of the RAII object.
- Common RAII class copying behaviors are: disallowing copying (the most common), reference counting, deep copying, and transferring ownership.

3.6 Item 15: Provide access to raw resources in resource-managing classes

3.7 Visual Summary

ITEM 15: PROVIDE ACCESS TO RAW RESOURCES IN RESOURCE-MANAGING CLASSES

- | |
|--|
| <ol style="list-style-type: none">1. RAII wrapper protects resource -> APIs may still require raw handle.2. Explicit get() -> caller consciously borrows the raw resource.3. Implicit conversion -> convenient but can hide lifetime mistakes.4. Never give ownership accidentally -> raw access should not mean raw ownership.5. Meaning: expose handles for interoperability while preserving ownership semantics. |
|--|

3.7.1 The Reality of APIs

RAII classes are wonderful, but the world is full of APIs that deal in raw resources. If you are going to use RAII, you need a way to convert an RAII object into the raw resource it wraps, because sooner or later you will need to pass it to an API that expects the raw type.

```
// Suppose we have an Investment hierarchy managed by shared_ptr
std::shared_ptr<Investment> pInv(createInvestment());

// And an API function that takes a raw pointer:
int daysHeld(const Investment* pi);           // How many days has this been held?
double creditRating(const Investment* pi); // What is the credit rating?

// We need to convert from shared_ptr<Investment> to const Investment*
```

3.7.2 Explicit Conversion: The get() Member Function

Smart pointers provide a get() member function that returns a copy of the raw pointer inside the smart pointer:

```
std::shared_ptr<Investment> pInv(createInvestment());

int days = daysHeld(pInv.get());           // Pass raw pointer to the API
double rating = creditRating(pInv.get());
```

Smart pointers also overload operator-> and operator*, so you can use them like raw pointers in most contexts:

```

class Investment {
public:
    bool isTaxFree() const;
    double currentValue() const;
};

std::shared_ptr<Investment> pInv(createInvestment());

bool taxFree = pInv->isTaxFree();           // operator->
double val = (*pInv).currentValue();       // operator*

```

3.7.3 Designing Your Own RAII Classes: Explicit vs. Implicit Conversion

When you write your own RAII class, you must decide how clients will access the underlying resource. There are two approaches: explicit conversion and implicit conversion.

```

// A RAII wrapper for a C-style font handle
class Font {
public:
    explicit Font(const std::string& name, int size)
        : handle_(createFont(name.c_str(), size))
    {
        if (!handle_) throw std::runtime_error("Failed to create font");
    }

    ~Font() {
        if (handle_) destroyFont(handle_);
    }

    // Explicit conversion: client must call get()
    FontHandle get() const { return handle_; }

    Font(const Font&) = delete;
    Font& operator=(const Font&) = delete;

private:
    FontHandle handle_;    // Raw C handle
};

```

3.7.3.1 Explicit Conversion via get() Usage:

```

// C API function
void drawText(FontHandle font, const char* text, int x, int y);

Font f("Arial", 12);
drawText(f.get(), "Hello, World!", 10, 20); // Must explicitly call get()

```

This is safe but verbose. Every time you use the font with a C API, you type `.get()`.

3.7.3.2 Implicit Conversion via operator You can provide an implicit conversion operator:

```

class Font {
public:
    explicit Font(const std::string& name, int size)
        : handle_(createFont(name.c_str(), size))
    {}

    ~Font() {
        if (handle_) destroyFont(handle_);
    }

    // Implicit conversion to FontHandle
    operator FontHandle() const { return handle_; }

    Font(const Font&) = delete;
    Font& operator=(const Font&) = delete;

private:
    FontHandle handle_;
};

```

Usage becomes seamless:

```

Font f("Arial", 12);
drawText(f, "Hello, World!", 10, 20); // Implicit conversion -- looks natural

```

But implicit conversions open the door to accidental misuse:

```

// BAD: Implicit conversion can lead to dangling handles
Font f1("Arial", 12);
FontHandle h = f1;           // Implicit conversion -- h is a copy of the raw handle

// Now suppose f1 is destroyed (goes out of scope)...
// h is a dangling handle! Using it is undefined behavior.

// BAD: Accidentally passing the wrong type
void changeFontSize(FontHandle fh, int newSize); // C API

Font f("Arial", 12);
changeFontSize(f, 14); // Compiles fine due to implicit conversion
// But did the programmer mean to modify the font managed by f?
// The C API might reallocate the handle, leaving f holding a stale value.

```

3.7.4 The Trade-off: Safety vs. Convenience

The choice between explicit and implicit conversion is a design decision that involves a trade-off:

Aspect	Explicit (get())	Implicit (operator T())
Safety	Higher – conversions are visible	Lower – accidental conversions possible
Convenience	Lower – verbose	Higher – seamless with C APIs
Dangling risk	Lower – deliberate action	Higher – easy to extract and outlive

Meyers' recommendation: Lean toward explicit conversion (get()), because the cost of inadvertent type conversions usually outweighs the inconvenience of explicit calls. However, the right choice depends on the specific use case and how the RAII class interacts with existing APIs.

3.7.5 Real-World Examples from the Standard Library

The standard library generally favors explicit conversion:

```

// std::shared_ptr and std::weak_ptr use get()
std::shared_ptr<Widget> sp = std::make_shared<Widget>();
Widget* raw = sp.get();    // Explicit

// std::string provides c_str() for explicit conversion
std::string s = "hello";
const char* cstr = s.c_str(); // Explicit conversion to C string

// std::vector provides data() for explicit conversion
std::vector<int> v = {1, 2, 3};
int* arr = v.data();        // Explicit conversion to raw array

```

But some classes do provide implicit conversions:

```

// std::string has an implicit conversion via operator basic_string_view (C++17)
// std::reference_wrapper has an implicit conversion to T&
std::reference_wrapper<int> ref(someInt);
int& r = ref;    // Implicit conversion

```

3.7.6 Complete Example: A RAII Wrapper with Both Access Methods

```

// A RAII class managing an OpenGL texture
class Texture {
public:
    explicit Texture(int width, int height, const unsigned char* pixels)
        : id_(0)
    {
        glGenTextures(1, &id_);
        glBindTexture(GL_TEXTURE_2D, id_);
        glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, width, height,
                    0, GL_RGBA, GL_UNSIGNED_BYTE, pixels);
    }

    ~Texture() {
        if (id_ != 0) {
            glDeleteTextures(1, &id_);
        }
    }

    // Explicit access -- preferred
    GLuint id() const { return id_; }

    // Move semantics (textures are GPU resources, not copyable)
    Texture(Texture&& other) noexcept : id_(other.id_) {
        other.id_ = 0;
    }

    Texture& operator=(Texture&& other) noexcept {
        if (this != &other) {
            if (id_ != 0) glDeleteTextures(1, &id_);
            id_ = other.id_;
            other.id_ = 0;
        }
        return *this;
    }

    Texture(const Texture&) = delete;
    Texture& operator=(const Texture&) = delete;

private:

```

```

    GLuint id_;
};

// Usage:
Texture tex(256, 256, pixelData);
glBindTexture(GL_TEXTURE_2D, tex.id()); // Explicit -- clear and safe

```

3.7.7 Accessing Raw Resources Does Not Violate Encapsulation

It might seem that providing access to the raw resource defeats the purpose of the RAI class. But the purpose of the RAI class is not to encapsulate the resource – it is to **ensure the resource is released**. Encapsulation is a secondary concern. If providing access to the raw resource is necessary for the class to be useful, that access does not undermine the class’s primary mission.

Think of RAI classes as a careful layer on top of the resource, not a wall around it. `shared_ptr` and `unique_ptr` both provide `get()`, and no one considers them poorly designed.

Some RAI classes combine both roles (resource management AND encapsulation), but these are the minority. Most RAI classes exist solely to guarantee cleanup.

3.7.8 Things to Remember

- APIs often require access to raw resources, so each RAI class should offer a way to get at the resource it manages.
- Access may be via explicit conversion (e.g., a `get()` member function) or implicit conversion (e.g., `operator RawType()`). Explicit conversion is generally safer; implicit conversion is more convenient for callers.
- Providing access to the raw resource does not violate encapsulation. RAI classes exist to guarantee resource release, not to hide the resource.

3.8 Item 16: Use the same form in corresponding uses of new and delete

3.9 Visual Summary

ITEM 16: USE THE SAME FORM IN CORRESPONDING USES OF NEW AND DELETE

1. `new T` → one object constructed → `delete T` destroys one object.
2. `new T[n]` → `n` objects constructed plus array bookkeeping.
3. `delete[]` required → destroys every element and uses right deallocation form.
4. Mismatch → undefined behavior, leaks, or corrupted heap metadata.
5. Meaning: allocation form and deallocation form are a matched pair.

3.9.1 The Rule

This item has the simplest guideline in the chapter, yet violations cause some of the most insidious bugs in C++:

If you use `[]` in a new expression, you must use `[]` in the corresponding delete expression. If you do not use `[]` in a new expression, you must not use `[]` in the corresponding delete expression.

3.9.2 Understanding Memory Layout

When you use `new`, two things happen: 1. Memory is allocated (via `operator new`). 2. One or more constructors are called on that memory.

When you use `delete`, two things happen (in reverse): 1. One or more destructors are called. 2. Memory is deallocated (via operator `delete`).

The critical question for `delete` is: **how many objects reside in the memory being deleted?** This determines how many destructors to call.

A single object and an array of objects have different memory layouts:

Single object:

```
+-----+
|      Object      |
+-----+
```

Array of objects:

```
+---+-----+-----+-----+-----+
| n |      Object 0      |      Object 1      | ... |      Object n-1      |
+---+-----+-----+-----+-----+
```

^

|-- Array size (stored by the implementation, typically before the first object)

When you say `delete[]`, the runtime reads the array size `n` from this header and calls destructors for each of the `n` objects. When you say `delete` (without `[]`), the runtime assumes there is a single object and calls one destructor.

3.9.3 What Goes Wrong

```
std::string* stringArray = new std::string[100];

// ...

delete stringArray; // UNDEFINED BEHAVIOR!
// Only one destructor is called (for stringArray[0]).
// The other 99 std::string objects are never destroyed.
// Their internal memory (heap-allocated character buffers) is leaked.
// The memory layout is also misinterpreted, potentially corrupting the heap.
```

3.9.3.1 BAD: Using `delete` on an array

```
std::string* stringPtr = new std::string("hello");

// ...

delete[] stringPtr; // UNDEFINED BEHAVIOR!
// The runtime tries to read an array size from memory before the object.
// That memory contains garbage (or part of another allocation).
// It then tries to call destructors on "objects" that do not exist.
// This can corrupt memory, crash, or cause silent data corruption.
```

3.9.3.2 BAD: Using `delete[]` on a single object

3.9.3.3 Both are undefined behavior The C++ Standard says the behavior is undefined in both cases. In practice:

- `delete` on an array may leak resources held by all but the first element.
- `delete[]` on a single object may read garbage as an array count and corrupt memory.
- Some implementations may appear to “work” for built-in types (like `int`) because their destructors are trivial, but the behavior is still technically undefined.

3.9.4 The Correct Pairings

```

// Correct: new with delete
std::string* ps = new std::string("hello");
delete ps;

// Correct: new[] with delete[]
std::string* psa = new std::string[100];
delete[] psa;

// Correct: Built-in types follow the same rule
int* pi = new int(42);
delete pi;

int* pia = new int[100];
delete[] pia;

```

3.9.5 The Typedef Trap

Typedefs can obscure whether a type is an array, making this rule harder to follow:

```

// BAD: A typedef that hides an array
typedef std::string AddressLines[4];

// This looks like a single object allocation, but it is an array!
std::string* pal = new AddressLines;
// This is equivalent to: new std::string[4]

// What form of delete is correct?
delete pal;      // UNDEFINED BEHAVIOR! This is actually an array.
delete[] pal;    // Correct, but non-obvious because AddressLines hides the array.

```

This is a compelling reason to prefer `std::array` or `std::vector` over raw arrays and array typedefs:

```

// GOOD: No ambiguity with std::array or std::vector
using AddressLines = std::array<std::string, 4>;

AddressLines* pal = new AddressLines;
delete pal; // Correct: AddressLines is a single object (a struct containing an array)

// BETTER: No new/delete at all
AddressLines pal; // Stack-allocated, no manual cleanup needed

// BEST: Use a vector if the size is dynamic
std::vector<std::string> pal(4);

```

3.9.6 Smart Pointers and the new/delete Mismatch

This issue affects smart pointers as well:

```

// BAD: shared_ptr uses delete by default, not delete[]
std::shared_ptr<int> sp(new int[100]); // Will call delete, not delete[]!

// GOOD: Use a custom deleter
std::shared_ptr<int> sp(new int[100], std::default_delete<int[]>());

// GOOD: unique_ptr has array specialization
std::unique_ptr<int[]> up(new int[100]); // Correctly calls delete[]
up[5] = 42; // operator[] is available

// BEST: Avoid raw arrays entirely
auto v = std::make_shared<std::vector<int>>(100);

```

3.9.7 Practical Impact: A Debugging Nightmare

Consider this class hierarchy:

```

class Widget {
public:
    Widget() { data_ = new char[1024]; }
    virtual ~Widget() { delete[] data_; }
private:
    char* data_;
};

class SpecialWidget : public Widget {
public:
    SpecialWidget() { extra_ = new char[2048]; }
    ~SpecialWidget() override { delete[] extra_; }
private:
    char* extra_;
};

```

```

// BAD: Mismatched new/delete with polymorphic types
Widget* widgets = new SpecialWidget[10];
delete[] widgets; // UNDEFINED BEHAVIOR even with delete[]!
// The compiler uses sizeof(Widget) to compute element offsets,
// but the actual objects are SpecialWidget (which is larger).
// Destructors are called at wrong addresses. Catastrophic.

// GOOD: Use a container of smart pointers
std::vector<std::unique_ptr<Widget>> widgets;
for (int i = 0; i < 10; ++i) {
    widgets.push_back(std::make_unique<SpecialWidget>());
}
// Each widget is individually allocated and correctly destroyed

```

3.9.8 Complete Example: Demonstrating the Mismatch


```

#include <iostream>
#include <memory>

class Tracked {
public:
    Tracked(int id) : id_(id) {
        std::cout << "Tracked(" << id_ << ") constructed\n";
    }
    ~Tracked() {
        std::cout << "Tracked(" << id_ << ") destroyed\n";
    }
private:
    int id_;
};

int main() {
    // --- Correct usage ---
    std::cout << "=== Correct: new[] with delete[] ===\n";
    Tracked* arr = new Tracked[3]{{1}, {2}, {3}};
    delete[] arr;
    // Output:
    // Tracked(1) constructed
    // Tracked(2) constructed
    // Tracked(3) constructed
    // Tracked(3) destroyed
    // Tracked(2) destroyed
    // Tracked(1) destroyed

    std::cout << "\n=== Correct: new with delete ===\n";
    Tracked* single = new Tracked(42);
    delete single;
    // Output:
    // Tracked(42) constructed
    // Tracked(42) destroyed

    // --- Smart pointer approaches ---
    std::cout << "\n=== unique_ptr with array ===\n";
    {
        std::unique_ptr<Tracked[]> uarr(new Tracked[3]{{10}, {20}, {30}});
        // Automatically calls delete[] when uarr goes out of scope
    }

    std::cout << "\n=== Best: vector ===\n";
    {
        std::vector<Tracked> v;
        v.reserve(3);
        v.emplace_back(100);
        v.emplace_back(200);
        v.emplace_back(300);
        // Automatically destroyed when v goes out of scope
    }

    return 0;
}

```

3.9.9 Things to Remember

- If you use `[]` in a `new` expression, you must use `[]` in the corresponding `delete` expression. If you do not use `[]` in `new`, do not use `[]` in `delete`.

- **Typedefs can obscure the array nature of a type.** Prefer `std::vector`, `std::array`, or smart pointers to avoid the ambiguity entirely.
- **When in doubt, avoid `new[]` altogether.** Use `std::vector` for dynamic arrays and `std::array` for fixed-size arrays. These manage their own memory and eliminate the `new/delete` mismatch problem.

3.10 Item 17: Store newed objects in smart pointers in standalone statements

3.11 Visual Summary

ITEM 17: STORE NEWED OBJECTS IN SMART POINTERS IN STANDALONE STATEMENTS

1. Function call arguments may be evaluated in flexible order.
2. `new` object happens -> another argument throws before smart pointer construction.
3. Raw pointer is stranded -> resource leaks.
4. Standalone smart pointer statement -> ownership captured before risky work.
5. Meaning: never leave a freshly newed object temporarily unowned.

3.11.1 The Subtle Bug

Consider this seemingly innocent code:

```
int priority();
void processWidget(std::shared_ptr<Widget> pw, int priority);
```

You might call `processWidget` like this:

```
// BAD: Potential resource leak!
processWidget(std::shared_ptr<Widget>(new Widget), priority());
```

This looks safe – we are using a smart pointer. But it can leak!

3.11.2 Why It Leaks: Evaluation Order

The C++ standard gives compilers significant freedom in the order they evaluate function arguments. In the call above, three things must happen before `processWidget` can be called:

1. Execute `new Widget`
2. Construct the `shared_ptr<Widget>`
3. Call `priority()`

The C++ standard requires that step 2 happens after step 1 (the `shared_ptr` constructor needs the result of `new Widget`). But **step 3 can happen at any point** – before step 1, between steps 1 and 2, or after step 2.

A compiler might choose this order:

1. Execute `new Widget` – `Widget` is allocated on the heap
2. Call `priority()` – **If this throws, the `Widget` is leaked!**
3. Construct `shared_ptr<Widget>` – This never executes

The `Widget` was newed in step 1, but the `shared_ptr` that would manage it is not constructed until step 3. If `priority()` throws in step 2, the `Widget` leaks because nothing is responsible for deleting it.

3.11.3 The Fix: Standalone Statements

The solution is to separate the creation of the smart pointer into its own statement:

```
// GOOD: Store the newed object in a smart pointer in a standalone statement
std::shared_ptr<Widget> pw(new Widget); // Statement 1: no leak possible
processWidget(pw, priority());          // Statement 2: no leak possible
```

Now the sequence is deterministic: 1. `new Widget` is executed and immediately handed to the `shared_ptr` constructor. 2. `priority()` is called. 3. `processWidget` is called.

If `priority()` throws, `pw` has already been constructed and its destructor will delete the `Widget`.

3.11.4 The Modern Fix: `make_shared` and `make_unique`

C++11's `std::make_shared` and C++14's `std::make_unique` eliminate this problem entirely:

```
// BEST: make_shared combines allocation and smart pointer construction
processWidget(std::make_shared<Widget>(), priority());
```

With `make_shared`, the allocation and the `shared_ptr` construction are a single, indivisible operation. There is no window where the `Widget` exists but is not yet managed by a smart pointer.

```
// BEST: make_unique (C++14)
void processWidget(std::unique_ptr<Widget> pw, int priority);

processWidget(std::make_unique<Widget>(), priority());
```

3.11.5 Detailed Walkthrough: Why the Order Matters

Let us trace through the problem step by step with a concrete example:

```
class Widget {
public:
    Widget() {
        std::cout << "Widget constructed at " << this << "\n";
        data_ = new int[1000]; // Widget allocates its own resources
    }
    ~Widget() {
        std::cout << "Widget destroyed at " << this << "\n";
        delete[] data_;
    }
private:
    int* data_;
};

int priority() {
    // Imagine this reads from a database, a file, or does complex computation
    throw std::runtime_error("Priority database unavailable!");
    return 0; // Never reached
}

void processWidget(std::shared_ptr<Widget> pw, int p) {
    std::cout << "Processing widget with priority " << p << "\n";
}
```

```

try {
    // Compiler may evaluate in this order:
    // 1. new Widget          -> Widget constructed, raw pointer exists
    // 2. priority()          -> THROWS! Stack unwinding begins.
    // 3. shared_ptr(...)     -> Never reached. Widget is leaked.
    processWidget(std::shared_ptr<Widget>(new Widget), priority());
}
catch (const std::exception& e) {
    std::cout << "Caught: " << e.what() << "\n";
    // Output: "Widget constructed at 0x..."
    //         "Caught: Priority database unavailable!"
    // Note: NO "Widget destroyed" message -- the Widget leaked!
}

```

3.11.5.1 BAD path (compiler chooses unfortunate evaluation order):

```

try {
    std::shared_ptr<Widget> pw(new Widget); // Widget is safely managed
    processWidget(pw, priority());          // If priority() throws...
}
catch (const std::exception& e) {
    std::cout << "Caught: " << e.what() << "\n";
    // Output: "Widget constructed at 0x..."
    //         "Widget destroyed at 0x..." <-- pw's destructor cleans up!
    //         "Caught: Priority database unavailable!"
}

```

3.11.5.2 GOOD path (standalone statement):

3.11.6 This Problem Extends Beyond Function Arguments

The same issue can arise in any expression where a new and other potentially-throwing operations are interleaved:

```

// BAD: Multiple news in one expression
auto p = std::make_pair(
    std::shared_ptr<Widget>(new Widget),
    std::shared_ptr<Gadget>(new Gadget)
);
// If the second new succeeds but the first shared_ptr hasn't been constructed yet
// (or vice versa), a throw would leak.

// GOOD: Separate statements
auto pw = std::make_shared<Widget>();
auto pg = std::make_shared<Gadget>();
auto p = std::make_pair(pw, pg);

```

```
// BAD: new in a ternary expression
std::shared_ptr<Widget> pw(
    condition ? new SpecialWidget : new Widget
);
// This is actually fine because only one new is evaluated, and it is directly
// passed to the shared_ptr constructor. But it is easier to reason about:

// GOOD: Clear and unambiguous
std::shared_ptr<Widget> pw;
if (condition) {
    pw = std::make_shared<SpecialWidget>();
} else {
    pw = std::make_shared<Widget>();
}
```

3.11.7 C++17 Changes

Starting with C++17, the evaluation order of function arguments was tightened. Specifically, the expressions associated with a single parameter must be fully evaluated before the evaluation of any other parameter begins. This means:

```
// In C++17 and later, this is safe:
processWidget(std::shared_ptr<Widget>(new Widget), priority());
// Because: either (new Widget + shared_ptr construction) happens entirely before
// priority(), or priority() happens entirely before (new Widget + shared_ptr construction).
```

However, even in C++17, using `make_shared` / `make_unique` is still recommended because:

1. It communicates intent more clearly.
2. It avoids the `new/delete` asymmetry (no raw `new` at all).
3. `make_shared` can be more efficient (single allocation for object + control block).
4. Code may need to compile on pre-C++17 compilers.

3.11.8 Complete Example: A Factory with Exception Safety

```
#include <memory>
#include <string>
#include <stdexcept>
#include <iostream>

class Config {
public:
    explicit Config(const std::string& filename) {
        std::cout << "Loading config from " << filename << "\n";
        // Might throw if file not found
    }
};

class Logger {
public:
    explicit Logger(const std::string& logfile) {
        std::cout << "Opening log: " << logfile << "\n";
    }
};

class Database {
public:
    Database(const std::string& connStr, int timeout) {
        std::cout << "Connecting to " << connStr << "\n";
        if (timeout < 0) throw std::runtime_error("Invalid timeout");
    }
};
```

```

    }
};

int computeTimeout(const Config& cfg) {
    // Might throw
    return 30;
}

// BAD: Multiple potential leaks
void initializeSystem_bad() {
    processResources(
        std::shared_ptr<Config>(new Config("app.cfg")),
        std::shared_ptr<Logger>(new Logger("app.log")),
        std::shared_ptr<Database>(new Database("db://host",
                                              computeTimeout(*new Config("app.cfg"))))
    );
    // This is a mess: raw new of Config that is never managed,
    // multiple unsequenced new operations, etc.
}

// GOOD: Each resource created in a standalone statement
void initializeSystem_good() {
    auto config = std::make_shared<Config>("app.cfg");
    auto logger = std::make_shared<Logger>("app.log");

    int timeout = computeTimeout(*config); // Might throw -- but nothing leaks
    auto database = std::make_shared<Database>("db://host", timeout);

    processResources(config, logger, database);
    // If any step throws, all previously created resources are cleaned up
    // by their shared_ptr destructors during stack unwinding.
}

```

3.11.9 Guideline Summary

Rule of thumb:

NEVER write "new" inside a function call's argument list.
 Always store the result of "new" in a named smart pointer first,
 or use `make_shared` / `make_unique`.

```

// BAD patterns:
f(shared_ptr<T>(new T), g());
f(unique_ptr<T>(new T), g());
f(shared_ptr<T>(new T), shared_ptr<U>(new U));

```

```

// GOOD patterns:
auto p = make_shared<T>(); // or make_unique<T>()
f(p, g());

```

```

auto p1 = make_shared<T>();
auto p2 = make_shared<U>();
f(p1, p2);

```

3.11.10 Things to Remember

- Store newed objects in smart pointers in standalone statements. Failure to do this can lead to subtle resource leaks when exceptions are thrown, because compilers have latitude to reorder operations within a single statement.
- Prefer `std::make_shared` (C++11) and `std::make_unique` (C++14) over raw `new` with smart

pointer constructors. These functions combine allocation and smart pointer construction into a single, atomic operation, eliminating the window for leaks.

3.12 Summary of Chapter 3

Item	Key Principle
13	Use RAII objects to manage resources. Acquire in constructors, release in destructors.
14	Choose the right copying strategy: prohibit, reference-count, deep copy, or transfer ownership.
15	Provide get () or conversion operators so RAII objects work with raw-resource APIs.
16	Match new with delete and new[] with delete[].
17	Prefer containers over raw arrays.
	Store newed objects in smart pointers in standalone statements. Prefer make_shared/make_unique.

The overarching theme: **make resource management automatic and exception-safe by tying resource lifetimes to object lifetimes**. When you follow RAII consistently, resource leaks become nearly impossible, and your code becomes simpler, safer, and easier to maintain.

4 Chapter 4: Designs and Declarations

4.1 Chapter Flow

CHAPTER 4: DESIGNS AND DECLARATIONS
Items 18–19 → design APIs/types so invalid use is hard.
Items 20–21 → avoid unnecessary copies, return real objects when needed.
Items 22–24 → protect representation and use non-members for symmetry/encapsulation.
Items 25–26 → provide efficient no-throw swap and delay variable creation.

This chapter is the heart of “Effective C++” – it addresses the design decisions that determine whether your C++ software is correct, efficient, and maintainable. Good interfaces are easy to use correctly and hard to use incorrectly. Good class designs mirror the thought that goes into designing built-in types. The items here cover parameter passing, return values, encapsulation, namespace design, swap mechanics, and variable lifetime – all critical to writing professional C++.

4.2 Item 18: Make Interfaces Easy to Use Correctly and Hard to Use Incorrectly

4.3 Visual Summary

ITEM 18: MAKE INTERFACES EASY TO USE CORRECTLY AND HARD TO USE INCORRECTLY

1. Caller sees API -> wrong usage should be hard to express.
2. Ambiguous primitive parameters -> easy to swap or pass invalid values.
3. Strong types, factories, RAII -> encode valid states and ownership.
4. Consistent behavior -> match built-in and standard library expectations.
5. Meaning: good interfaces prevent bugs before tests run.

Good interfaces are a joy to use. Bad interfaces lead to bugs that compile without warnings and blow up at runtime. The cardinal rule of interface design: if a client can use your interface incorrectly, the interface shares at least part of the blame. You should design interfaces that **prevent** misuse rather than merely documenting correct usage.

4.3.1 The Date Constructor Problem

Consider a class for representing dates:

```
// BAD: Easy to misuse -- what order are the parameters?
class Date {
public:
    Date(int month, int day, int year);
    // ...
};

// Client code -- spot the bugs:
Date d1(30, 3, 1995);    // Oops: day and month swapped (should be 3, 30, 1995)
Date d2(3, 40, 1995);    // Oops: day 40 doesn't exist
Date d3(2, 30, 1995);    // Oops: Feb 30 doesn't exist

// All three compile without error. All three are wrong.
```

The problem is that `int` carries no semantic meaning. A month is not just an integer – it is a value in the range [1, 12]. A day is not just an integer – it is a value whose valid range depends on the month and year. Using raw `int` for both loses this information at the type level.

4.3.2 Solution: Introduce Wrapper Types

```
// GOOD: Distinct types prevent argument transposition errors
struct Day {
    explicit Day(int d) : val(d) {}
    int val;
};

struct Month {
    explicit Month(int m) : val(m) {}
    int val;
};

struct Year {
    explicit Year(int y) : val(y) {}
    int val;
};

class Date {
public:
```



```

    Date(const Month& m, const Day& d, const Year& y);
    // ...
};

// Now the compiler catches transposition errors:
Date d1(30, 3, 1995); // Error! Can't convert int to Month
Date d2(Day(30), Month(3), Year(1995)); // Error! Wrong argument types
Date d3(Month(3), Day(30), Year(1995)); // OK -- reads naturally

```

4.3.3 Restricting the Value Space with Enums or Static Factory Methods

Even with wrapper types, Month(13) still compiles. We can restrict the set of valid values:

```

// GOOD: Month as a class with only 12 valid values
class Month {
public:
    static Month Jan() { return Month(1); }
    static Month Feb() { return Month(2); }
    static Month Mar() { return Month(3); }
    static Month Apr() { return Month(4); }
    static Month May() { return Month(5); }
    static Month Jun() { return Month(6); }
    static Month Jul() { return Month(7); }
    static Month Aug() { return Month(8); }
    static Month Sep() { return Month(9); }
    static Month Oct() { return Month(10); }
    static Month Nov() { return Month(11); }
    static Month Dec() { return Month(12); }

    int asInt() const { return val_; }

private:
    explicit Month(int m) : val_(m) {} // Private! Only the static methods can create Months.
    int val_;
};

// Usage is clear and impossible to misuse:
Date d(Month::Mar(), Day(30), Year(1995));

// Month(13) won't compile -- the constructor is private.
// The only way to get a Month is through the 12 named factory functions.

```

Why functions returning Month instead of static const Month objects? The latter risks the “static initialization order fiasco” (see Item 4). Functions returning local statics avoid this.

4.3.4 Restricting Operations – Multiplication of Ints

```

// BAD: if operator* returns a bare int, the user can write:
//      if (a * b = c) ...    // Assignment instead of comparison -- compiles!
//
// GOOD: return const to prevent assignment to temporaries
const Rational operator*(const Rational& lhs, const Rational& rhs);

// Now:
Rational a, b, c;
if (a * b = c) ... // Error! Can't assign to a const Rational

```

4.3.5 Consistent Interfaces – Behave Like Built-in Types

One of the most important rules: **make your types behave consistently with built-in types**. If users already know how `int` works, they should be able to guess how your type works.

```

// BAD: inconsistent naming across container types
class Array {
public:
    int length() const;    // "length"
};

class LinkedList {
public:
    int size() const;      // "size" -- different name, same concept
};

class HashTable {
public:
    int count() const;    // "count" -- yet another name
};

// GOOD: use consistent naming (the STL does this with size())
class Array {
public:
    size_t size() const;
};

class LinkedList {
public:
    size_t size() const;
};

class HashTable {
public:
    size_t size() const;
};

```

4.3.6 Eliminating Client Resource Management with Smart Pointers

One of the most impactful applications of this principle: don't force clients to manage resources. If a factory function returns a raw pointer, the client must remember to delete it – and must delete it with the right mechanism.

```

// BAD: raw pointer -- client must remember to delete
Investment* createInvestment();

// The client might:
// 1. Forget to delete entirely (memory leak)
// 2. Delete twice (undefined behavior)
// 3. Use delete[] instead of delete (undefined behavior)
// 4. Delete but then continue using the pointer (dangling pointer)

void f() {
    Investment* pInv = createInvestment();
    // ... code that might throw or return early ...
    delete pInv; // Might never execute!
}

// GOOD: return a smart pointer -- resource management is automatic
std::shared_ptr<Investment> createInvestment() {
    // The deleter can be baked in at creation time
    std::shared_ptr<Investment> retVal(new Stock(...),
                                      getRidOfInvestment); // Custom deleter!
    return retVal;
}

void f() {
    std::shared_ptr<Investment> pInv = createInvestment();
    // ... use pInv ...
    // No delete needed. Automatic cleanup, even if exceptions are thrown.
    // Even the custom deleter is handled automatically.
}

```

This is especially powerful because the **custom deleter is embedded in the smart pointer at creation time**. The client doesn't need to know or care what cleanup mechanism is needed. The factory function's author – who knows how the resource was allocated – bakes in the correct cleanup strategy.

4.3.7 Cross-DLL Resource Management

A particularly nasty bug: an object allocated with `new` in one DLL but deleted in another. On many platforms this causes undefined behavior because each DLL may have its own heap. `std::shared_ptr` solves this: the deleter is captured at construction time, so `delete` is always called in the same DLL that called `new`.

```

// GOOD: shared_ptr ensures delete is called in the right DLL
std::shared_ptr<Investment> createInvestment() {
    // 'new' happens in this DLL
    // The default deleter (which calls 'delete') is bound here
    return std::shared_ptr<Investment>(new Stock(...));
    // Even if the shared_ptr crosses DLL boundaries,
    // delete will be called using this DLL's delete operator.
}

```

4.3.8 Real-World Example: Database Connection Handle

```

// BAD: raw handle -- client must remember to close
class DatabaseConnection {
public:
    static DBHandle open(const std::string& connStr);
    // Client must call close(handle) when done. Will they?
};

// GOOD: RAII wrapper with interface that prevents misuse
class DatabaseConnection {
public:
    // Factory returns a managed connection
    static std::shared_ptr<DatabaseConnection> open(const std::string& connStr) {
        auto conn = std::shared_ptr<DatabaseConnection>{
            new DatabaseConnection(connStr),
            [](DatabaseConnection* c) {
                c->close();    // Guaranteed cleanup
                delete c;
            }
        };
        return conn;
    }

    void executeQuery(const std::string& sql);

private:
    DatabaseConnection(const std::string& connStr); // Private -- must use open()
    void close(); // Private -- handled by deleter
};

// Client code is clean and safe:
void processData() {
    auto conn = DatabaseConnection::open("host=localhost;db=mydb");
    conn->executeQuery("SELECT * FROM users");
    // Connection is automatically closed when conn goes out of scope.
    // No possibility of forgetting to close.
}

```

4.3.9 Things to Remember

- Good interfaces are easy to use correctly and hard to use incorrectly. You should strive for these characteristics in all your interfaces.
- Ways to facilitate correct use include consistency in interfaces and behavioral compatibility with built-in types.
- Ways to prevent errors include creating new types, restricting operations on types, constraining object values, and eliminating client resource management responsibilities.
- `std::shared_ptr` supports custom deleters. This prevents the cross-DLL problem, can be used to automatically unlock mutexes (see Item 14), and more.

4.4 Item 19: Treat Class Design as Type Design

4.5 Visual Summary

ITEM 19: TREAT CLASS DESIGN AS TYPE DESIGN

1. New class → new type in the language of your program.
2. Decide construction, destruction, copying, comparison, conversion, invariants.
3. Decide performance, exception safety, ownership, and threading assumptions.
4. Only then implement members.
5. Meaning: class design is semantic design, not just data plus functions.

In C++, defining a new class defines a new type. You are not just a class designer – you are a **type designer**. Overloaded functions and operators, controlling memory allocation and deallocation, defining object initialization and finalization – it's all in your hands. You should approach class design with the same care that language designers lavish on the design of built-in types.

Designing good classes is challenging because designing good types is challenging. Good types have natural syntax, intuitive semantics, and one or more efficient implementations. Here are the questions you should consider every time you design a class:

4.5.1 The Comprehensive Design Checklist

1. How should objects of your new type be created and destroyed?

This determines the design of your constructors, destructor, and memory allocation/deallocation functions (operator new, operator delete, operator new[], operator delete[]) if you write them.

```
// Example: A MemoryPool type that manages its own allocation
class MemoryPool {
public:
    // Custom allocation for pooled objects
    static void* operator new(size_t size);
    static void operator delete(void* ptr, size_t size);

    // Constructor initializes the pool
    explicit MemoryPool(size_t blockSize, size_t numBlocks);

    // Destructor releases all pooled memory
    ~MemoryPool();

    // No copy -- pools are not copyable resources
    MemoryPool(const MemoryPool&) = delete;
    MemoryPool& operator=(const MemoryPool&) = delete;

    // Move is fine -- transfer ownership
    MemoryPool(MemoryPool&& other) noexcept;
    MemoryPool& operator=(MemoryPool&& other) noexcept;
};
```

2. How should object initialization differ from object assignment?

This determines how your constructors differ from your assignment operators. Don't confuse initialization with assignment – they correspond to different function calls.

```

class Widget {
public:
    Widget(int value);           // Initialization
    Widget(const Widget& rhs);    // Initialization (copy ctor)
    Widget& operator=(const Widget& rhs); // Assignment

    // These are fundamentally different operations.
    // Initialization creates a new object; assignment changes an existing one.
};

Widget w1(10);           // Calls constructor -- initialization
Widget w2 = w1;          // Calls copy constructor -- initialization (NOT assignment!)
w1 = w2;                 // Calls operator= -- assignment

```

3. What does it mean for objects of your new type to be passed by value?

The copy constructor defines what pass-by-value means for your type. If copying is expensive or doesn't make semantic sense, consider disabling it or making it explicit.

```

// BAD: Large class that is expensive to copy, yet allows pass-by-value silently
class LargeMatrix {
    double data[1000][1000]; // ~8 MB per copy!
public:
    // Implicit copy constructor copies 8 MB of data every time
    // the matrix is passed by value or returned. Probably not what you want.
};

// GOOD: Either make it non-copyable or provide explicit clone
class LargeMatrix {
    std::unique_ptr<double[]> data_;
    size_t rows_, cols_;
public:
    LargeMatrix(const LargeMatrix&) = delete;           // No accidental copies
    LargeMatrix& operator=(const LargeMatrix&) = delete;

    LargeMatrix(LargeMatrix&&) noexcept = default;     // Move is fine
    LargeMatrix& operator=(LargeMatrix&&) noexcept = default;

    LargeMatrix clone() const; // Explicit copy when you really need one
};

```

4. What are the restrictions on legal values for your new type?

Not all combinations of data member values are valid. The invariants determine the error checking inside your member functions, especially constructors, assignment operators, and setters.

```

class Probability {
public:
    explicit Probability(double value) : value_(value) {
        if (value < 0.0 || value > 1.0) {
            throw std::out_of_range("Probability must be in [0.0, 1.0]");
        }
    }

    Probability operator+(const Probability& rhs) const {
        return Probability(std::min(value_ + rhs.value_, 1.0));
    }

    // Every mutating operation must maintain the invariant.

private:
    double value_; // Invariant: 0.0 <= value_ <= 1.0
};

```

5. Does your new type fit into an inheritance graph?

If you inherit from existing classes, you are constrained by those classes – particularly whether their functions are virtual or non-virtual. If you intend others to inherit from your class, that affects whether you declare functions virtual, especially the destructor.

```

// If this is a base class that will be used polymorphically:
class Shape {
public:
    virtual ~Shape(); // Virtual destructor -- essential!
    virtual double area() const = 0; // Pure virtual -- must override
    virtual void draw() const; // Virtual -- may override
    int objectID() const; // Non-virtual -- should NOT override
};

// If this is a concrete class NOT meant to be inherited from:
class Point {
public:
    // No virtual functions, no virtual destructor
    // Consider making it final in C++11 and later:
    // class Point final { ... };
    double x, y;
};

```

6. What kind of type conversions are allowed?

Do you want implicit conversions? If so, write a non-explicit single-argument constructor or an implicit conversion function. If you want only explicit conversions, declare them explicit.

```

class Rational {
public:
    // Allows implicit conversion from int to Rational:
    Rational(int numerator = 0, int denominator = 1);

    // This permits: Rational r = 42;    -- int implicitly converts to Rational
    // This permits: doSomething(42);    -- if doSomething takes a Rational
};

class BigNumber {
public:
    // Only explicit conversion from int -- no surprises
    explicit BigNumber(int value);

    // BigNumber b = 42;          -- Error! No implicit conversion.
    // BigNumber b(42);           -- OK, explicit construction.
    // BigNumber b = BigNumber(42); -- OK, explicit construction.
};

```

7. What operators and functions make sense for the new type?

Some will be member functions, some will be non-members. See Items 23, 24, and 46.

8. What standard functions should be disallowed?

Those are the ones you declare deleted (or in older C++, declare private and don't implement).

```

class Uncopyable {
public:
    Uncopyable() = default;
    Uncopyable(const Uncopyable&) = delete;
    Uncopyable& operator=(const Uncopyable&) = delete;
};

```

9. Who should have access to the members of your new type?

This determines which members are public, protected, and private. It also determines which classes and functions are friends, and whether nesting one class inside another makes sense.

10. What is the “undeclared interface” of your new type?

What guarantees does your type offer with respect to performance, exception safety (see Item 29), and resource usage (e.g., locks, dynamic memory)? These guarantees constrain your implementation.

11. How general is your new type?

Maybe you're not really defining a new type – maybe you're defining a whole **family** of types. If so, you should define a class template rather than a class.

12. Is a new type really what you need?

If you're defining a new derived class only to add functionality to an existing class, perhaps non-member functions or templates would achieve the goal better without creating a new type.

4.5.2 Things to Remember

- Class design is type design. Before defining a new type, be sure to consider all the issues discussed in this Item.

4.6 Item 20: Prefer Pass-by-Reference-to-const to Pass-by-Value

4.7 Visual Summary

ITEM 20: PREFER PASS-BY-REFERENCE-TO-CONST TO PASS-BY-VALUE

1. Pass by value -> copy constructor and destructor may run.
2. Large/user-defined object -> cost and slicing risk.
3. Pass const& -> no copy and no mutation by callee.
4. Small built-ins/iterators/function objects -> value is often better.
5. Meaning: avoid expensive or lossy copies unless value semantics are intended.

By default, C++ passes objects to and from functions by value (a trait inherited from C). Unless you tell it otherwise, function parameters are initialized with **copies** of the actual arguments, and function callers get back a **copy** of the value returned by the function. These copies are produced by the objects' copy constructors. This can make pass-by-value an expensive operation.

4.7.1 The Cost of Pass-by-Value

```
class Person {
public:
    Person();
    virtual ~Person();

private:
    std::string name;
    std::string address;
};

class Student : public Person {
public:
    Student();
    ~Student();

private:
    std::string schoolName;
    std::string schoolAddress;
};
```

Now consider this function:

```
// BAD: pass by value -- expensive!
bool validateStudent(Student s);

Student plato;
bool platoIsOK = validateStudent(plato);
```

What happens when `validateStudent` is called? The parameter `s` is initialized by calling the `Student` copy constructor with `plato` as the argument. Similarly, `s` is destroyed when `validateStudent` returns. The cost of passing by value:

1. One call to the `Student` copy constructor
2. One call to the `Student` destructor
3. But `Student` contains two `std::string` objects, so that's two more copy constructions and two more destructions
4. `Student` derives from `Person`, which also has two `std::string` objects – two more copy constructions, two more destructions
5. Plus the `Person` copy construction and destruction

Total: 6 constructors and 6 destructors just to pass a single parameter.

```
// GOOD: pass by reference-to-const -- no copies at all
bool validateStudent(const Student& s);

// Same calling syntax, but no constructors or destructors are invoked.
// The const guarantees that validateStudent won't modify the caller's Student.
```

4.7.2 The Slicing Problem

Pass-by-value doesn't just hurt performance – it causes a subtle and dangerous bug called the **slicing problem**. When a derived class object is passed by value to a function expecting a base class object, the derived class's data members and virtual function implementations are “sliced off.”

```
class Window {
public:
    std::string name() const;
    virtual void display() const; // Base class version
};

class WindowWithScrollBars : public Window {
public:
    virtual void display() const; // Overridden version -- draws scroll bars
};
```

Now consider a function that prints window information:

```
// BAD: pass by value -- causes slicing!
void printNameAndDisplay(Window w) {
    std::cout << w.name();
    w.display(); // ALWAYS calls Window::display, never WindowWithScrollBars::display!
}

WindowWithScrollBars wwsb;
printNameAndDisplay(wwsb);
// wwsb is SLICED: it becomes a plain Window inside the function.
// The WindowWithScrollBars-specific data is chopped off.
// Virtual dispatch is broken -- display() calls the base class version.
```

The parameter `w` is constructed as a `Window` object (it's passed by value, and the parameter type is `Window`). All the specialization that makes `wwsb` a `WindowWithScrollBars` is sliced off. Inside `printNameAndDisplay`, `w` always behaves like a `Window`, regardless of the type of the object actually passed. In particular, `w.display()` calls `Window::display`, not `WindowWithScrollBars::display`.

```
// GOOD: pass by reference-to-const -- preserves polymorphic behavior
void printNameAndDisplay(const Window& w) {
    std::cout << w.name();
    w.display(); // Calls the correct version via virtual dispatch!
}

WindowWithScrollBars wwsb;
printNameAndDisplay(wwsb);
// No slicing. w refers to the original wwsb object.
// w.display() correctly calls WindowWithScrollBars::display.
```

4.7.3 A More Dramatic Slicing Example

```

class Shape {
public:
    virtual ~Shape() {}
    virtual double area() const = 0;
    virtual std::string description() const { return "Shape"; }
};

class Circle : public Shape {
    double radius_;
public:
    explicit Circle(double r) : radius_(r) {}
    double area() const override { return 3.14159 * radius_ * radius_; }
    std::string description() const override {
        return "Circle(r=" + std::to_string(radius_) + ")";
    }
};

class Rectangle : public Shape {
    double width_, height_;
public:
    Rectangle(double w, double h) : width_(w), height_(h) {}
    double area() const override { return width_ * height_; }
    std::string description() const override {
        return "Rectangle(" + std::to_string(width_) + "x" + std::to_string(height_) + ")";
    }
};

// BAD: Taking a vector of Shapes by value is impossible (Shape is abstract),
// but even if it weren't, it would slice all derived objects.

// BAD: Taking Shape by value in a utility function
// void logShape(Shape s); // Won't even compile -- Shape is abstract!

// GOOD: Take by reference-to-const
void logShape(const Shape& s) {
    std::cout << s.description() << " has area " << s.area() << "\n";
}

Circle c(5.0);
Rectangle r(3.0, 4.0);
logShape(c); // "Circle(r=5.000000) has area 78.539750"
logShape(r); // "Rectangle(3.000000x4.000000) has area 12.000000"

```

4.7.4 When Pass-by-Value Is Acceptable

References are typically implemented as pointers under the hood. For **small, built-in types** and **STL iterators and function objects**, pass-by-value is often more efficient:

```
// These are fine to pass by value:
void f(int x);           // Built-in type -- cheaper to copy than to indirect
void f(double x);        // Built-in type
void f(char c);          // Built-in type

// STL iterators are designed to be passed by value:
void processRange(std::vector<int>::iterator begin,
                  std::vector<int>::iterator end);

// STL function objects are designed to be passed by value:
void sortWithComparator(std::vector<int>& v, std::less<int> comp);
```

But be careful. Just because a type is small does not mean pass-by-value is cheap. A class with a single `std::string*` is only the size of a pointer, but copying it might trigger a deep copy if the class's copy constructor does so. Furthermore, just because a class is small today does not mean it will be small tomorrow – the implementation may grow.

```
// DANGEROUS ASSUMPTION: "It's small, so pass by value is fine"
class SmallButExpensive {
    std::shared_ptr<HugeDataStructure> data_;
    // Only 8 bytes (one pointer)!
    // But copying increments a reference count (atomic operation)
    // and may involve other bookkeeping.
public:
    SmallButExpensive(const SmallButExpensive&); // Might be more expensive than you think
};

// Safe default: pass by reference-to-const
void process(const SmallButExpensive& obj); // No copies, no surprises
```

4.7.5 The General Rule

```
// For user-defined types, the safe default is always reference-to-const:
void doWork(const MyClass& obj);

// Only pass by value when ALL of these are true:
// 1. The type is a built-in type (int, double, char, pointers), OR
// 2. The type is an STL iterator or function object, OR
// 3. The type is specifically designed to be passed by value and you've measured
//    that pass-by-value is actually more efficient.
```

4.7.6 Things to Remember

- Prefer pass-by-reference-to-const over pass-by-value. It's typically more efficient and it avoids the slicing problem.
- The rule doesn't apply to built-in types and STL iterator and function object types. For them, pass-by-value is usually appropriate.

4.8 Item 21: Don't Try to Return a Reference When You Must Return an Object

4.9 Visual Summary

ITEM 21: DON'T TRY TO RETURN A REFERENCE WHEN YOU MUST RETURN AN OBJECT

1. Need a result value -> ask where the referenced object would live.
2. Local object -> reference dangles after return.
3. Heap object -> caller must delete; leak risk.
4. Static object -> shared mutable state and wrong repeated results.
5. Return by value -> compiler can optimize with RVO/move.
6. Meaning: references are aliases, not storage for new results.

Once programmers learn about the efficiency costs of pass-by-value (Item 20), they sometimes become crusaders, determined to eliminate all pass-by-value from their code – including in contexts where returning by reference leads to disaster. The result: returning references to objects that no longer exist.

The key insight: **a reference is just a name for an existing object.** Whenever you see a reference, you should ask yourself what object it is another name for. If there is no such object, the reference is dangling and the program has undefined behavior.

4.9.1 The Rational Number Example

```
class Rational {
public:
    Rational(int numerator = 0, int denominator = 1);

private:
    int n, d; // numerator and denominator

    friend const Rational operator*(const Rational& lhs, const Rational& rhs);
};
```

The natural implementation of `operator*` would return a new `Rational`. But a performance-obsessed programmer might try to avoid the copy by returning a reference:

```
// BAD Attempt #1: Return a reference to a local stack object
const Rational& operator*(const Rational& lhs, const Rational& rhs) {
    Rational result(lhs.n * rhs.n, lhs.d * rhs.d);
    return result; // DISASTER! result is destroyed when the function exits.
                  // The caller receives a dangling reference.
}

// Any use of the returned reference is undefined behavior:
Rational a(1, 2);
Rational b(3, 5);
Rational c = a * b; // c is initialized from a reference to a destroyed object.
                   // Might appear to work, might crash, might corrupt memory.
```

```

// BAD Attempt #2: Return a reference to a heap-allocated object
const Rational& operator*(const Rational& lhs, const Rational& rhs) {
    Rational* result = new Rational(lhs.n * rhs.n, lhs.d * rhs.d);
    return *result; // Who deletes this? The caller? How?
}

// Memory leak -- the caller has no way to delete the object:
Rational w, x, y, z;
w = x * y * z; // This calls operator* twice:
                // temp = operator*(x, y)    -- allocates on heap (leak #1)
                // w = operator*(temp, z)    -- allocates on heap (leak #2)
                // We have a reference to the second result (assigned to w),
                // but the first result is leaked forever.
                // There is no way to retrieve a pointer to it.

// BAD Attempt #3: Return a reference to a local static object
const Rational& operator*(const Rational& lhs, const Rational& rhs) {
    static Rational result; // Only one instance, shared across all calls!
    result = Rational(lhs.n * rhs.n, lhs.d * rhs.d);
    return result;
}

// This is broken in a subtle way:
Rational a(1, 2);
Rational b(3, 4);
Rational c(5, 6);
Rational d(7, 8);

if ((a * b) == (c * d)) {
    // This is ALWAYS true!
    // Both calls to operator* modify the SAME static object.
    // By the time == is evaluated, both references point to the same object
    // (which holds the value from the most recent call, c * d).
    // So the comparison is: static_result == static_result
    // which is always true.
    std::cout << "This always prints!\n";
} else {
    std::cout << "This never prints!\n";
}

```

Even an array of statics wouldn't fix the problem – you'd need to know how many simultaneous results might be needed, and the comparison issue remains.

4.9.2 The Correct Solution: Return by Value

```
// GOOD: Just return a new object by value
const Rational operator*(const Rational& lhs, const Rational& rhs) {
    return Rational(lhs.n * rhs.n, lhs.d * rhs.d);
}
```

Yes, this incurs the cost of constructing and destroying the return value. But that cost is **correct** – you are paying for the creation of a new object, which is exactly what you need. Moreover, compilers are allowed to (and routinely do) apply **Return Value Optimization (RVO)** and **Named Return Value Optimization (NRVO)**, which eliminate the copy entirely by constructing the result directly in the caller's memory.

```
// With RVO, this code:
Rational c = a * b;

// is optimized to construct the result directly in c's memory.
// No copy constructor is called. The cost is just one constructor call.
```

In C++11 and later, move semantics provide an additional optimization: even when RVO doesn't apply, the return value is **moved** rather than copied.

4.9.3 Real-World Example: String Concatenation

```
// BAD: trying to avoid copies leads to bugs
class MyString {
public:
    // Don't do this!
    const MyString& operator+(const MyString& rhs) const {
        // Where does the result live? Can't be on the stack (dangling).
        // Can't be on the heap (leak). Can't be static (shared state).
        // There is no good answer.
    }
};

// GOOD: return by value
class MyString {
public:
    MyString operator+(const MyString& rhs) const {
        MyString result;
        result.data_ = data_ + rhs.data_;
        return result; // RVO will likely eliminate the copy.
    }

private:
    std::string data_;
};
```

4.9.4 When References ARE Appropriate to Return

References are appropriate when the object already exists and will outlive the reference:

```

class Container {
public:
    // GOOD: the element exists in the container and will outlive the call
    int& operator[](size_t index) { return data_[index]; }
    const int& operator[](size_t index) const { return data_[index]; }

    // GOOD: returning *this for chaining
    Container& add(int value) {
        data_.push_back(value);
        return *this;
    }

private:
    std::vector<int> data_;
};

// GOOD: singleton pattern -- the object is static and lives forever
DatabaseManager& DatabaseManager::instance() {
    static DatabaseManager mgr;
    return mgr;
}

```

4.9.5 Things to Remember

- Never return a pointer or reference to a local stack object, a reference to a heap-allocated object, or a pointer or reference to a local static object if there is a chance that more than one such object will be needed. (Item 4 provides an example of a design where returning a reference to a local static is reasonable: the singleton pattern for avoiding the static initialization order problem.)

4.10 Item 22: Declare Data Members Private

4.11 Visual Summary

ITEM 22: DECLARE DATA MEMBERS PRIVATE
1. Public data -> every caller depends on representation. 2. Need validation/lazy computation/thread safety later -> impossible without breaking users. 3. Private data + accessors -> stable interface around changing internals. 4. Protected data also leaks representation to derived classes. 5. Meaning: encapsulation preserves your ability to change implementation.

This item is about encapsulation. It argues that data members should **always** be private. The reasoning is based on access control, flexibility, and a fundamental truth about software evolution: implementations change, but interfaces should be stable.

4.11.1 Why Not Public?

Reason 1: Syntactic consistency. If data members aren't public, the only way for clients to access an object is via member functions. The client never has to wonder whether to use parentheses or not – it's always parentheses.


```

// BAD: public data members lead to inconsistent access syntax
class SpreadsheetCell {
public:
    double value;           // Accessed as: cell.value
    double cachedValue();  // Accessed as: cell.cachedValue()
    // Client must remember: is it 'value' or 'value()'?
};

// GOOD: all access through functions -- consistent syntax
class SpreadsheetCell {
public:
    double value() const;    // Always use ()
    double cachedValue() const; // Always use ()
private:
    double value_;
    mutable double cachedValue_;
    mutable bool cacheValid_;
};

```

Reason 2: Fine-grained access control. With member functions, you can implement no access, read-only access, write-only access, or read-write access. Public data members give you only read-write – all or nothing.

```

class AccessControlled {
public:
    // Read-only: temperature can be read but not set externally
    double temperature() const { return temperature_; }

    // Write-only: password can be set but never read back
    void setPassword(const std::string& pwd) { passwordHash_ = hash(pwd); }

    // Read-write: name can be read and written
    std::string name() const { return name_; }
    void setName(const std::string& n) { name_ = n; }

    // No access: internalState is completely hidden
    // (no getter or setter provided)

private:
    double temperature_;
    size_t passwordHash_;
    std::string name_;
    int internalState_;
};

```

Reason 3: Encapsulation – the ability to change the implementation.

If data members are public, changing them breaks all client code that uses them. If data members are private and accessed through functions, you can change the internal representation without changing the interface.

```

// Version 1: SpeedOMeter stores speed as mph
class SpeedOMeter {
public:
    double speedInMph() const { return speed_; }
    double speedInKph() const { return speed_ * 1.60934; }
    void setSpeed(double mph) { speed_ = mph; }

private:
    double speed_; // stored in mph
};

// Version 2: Decision to store in kph instead (internal change)
// Client code doesn't change AT ALL
class SpeedOMeter {
public:
    double speedInMph() const { return speed_ / 1.60934; } // Compute from kph
    double speedInKph() const { return speed_; }           // Direct return
    void setSpeed(double mph) { speed_ = mph * 1.60934; } // Convert to kph

private:
    double speed_; // now stored in kph -- internal change only
};

```

4.11.2 Encapsulation and Breakage: The Quantitative Argument

The encapsulation of a data member is inversely proportional to the amount of code that might be broken if that data member changes. If a data member is public, the amount of code that could be affected is **all client code** – an unknowably large amount. If it's private, the amount of code that could be affected is limited to the **member functions and friends** of the class.

```

// If x is public:
class Point {
public:
    double x, y; // Every piece of code that uses Point::x is coupled
                // to the fact that x is a double data member.
                // Changing it breaks an unknowable number of clients.
};

// If x is private with accessor:
class Point {
public:
    double x() const { return x_; }
    double y() const { return y_; }
    void setX(double newX) { x_ = newX; }
    void setY(double newY) { y_ = newY; }

private:
    double x_, y_;
    // Can later change to:
    // double r_, theta_; (polar coordinates)
    // without changing the public interface.
};

```

4.11.3 Why Not Protected?

The same arguments apply to protected data members. Protected is not much more encapsulated than public.

```
// BAD: Protected data members are almost as bad as public
class Base {
protected:
    int protectedData; // All derived classes can access this directly.
                        // Changing or removing it breaks all derived classes.
};

class Derived1 : public Base {
    void f() { protectedData = 42; } // Direct access
};

class Derived2 : public Base {
    void g() { int x = protectedData; } // Direct access
};

// If we need to change protectedData (rename it, change its type, compute it
// on the fly instead of storing it), ALL derived classes break.
// In a class hierarchy with many derived classes, this can be catastrophic.
```

```
// GOOD: Private data with protected accessor
class Base {
public:
    int getData() const { return data_; }

protected:
    void setData(int d) { data_ = d; } // Only derived classes can set

private:
    int data_;
    // Can change implementation without breaking derived classes,
    // as long as getData() and setData() maintain their contracts.
};
```

4.11.4 Real-World Example: Validating Invariants

```
// BAD: Public members can't enforce invariants
class Rectangle {
public:
    double width, height;
    // Nothing stops a client from writing:
    // rect.width = -5; // Negative width?!
};

// GOOD: Private members with validation
class Rectangle {
public:
    double width() const { return width_; }
    double height() const { return height_; }

    void setWidth(double w) {
        if (w <= 0) throw std::invalid_argument("Width must be positive");
        width_ = w;
        updateCachedArea();
    }

    void setHeight(double h) {
        if (h <= 0) throw std::invalid_argument("Height must be positive");
        height_ = h;
        updateCachedArea();
    }
};
```

```

    }

    double area() const { return cachedArea_; } // O(1) lookup

private:
    double width_;
    double height_;
    double cachedArea_;

    void updateCachedArea() { cachedArea_ = width_ * height_; }
};

```

4.11.5 Real-World Example: Thread-Safe Access

```

// BAD: Public data is impossible to make thread-safe
class Counter {
public:
    int count; // Data races galore
};

// GOOD: Private data can be made thread-safe without changing the interface
class Counter {
public:
    int count() const {
        std::lock_guard<std::mutex> lock(mutex_);
        return count_;
    }

    void increment() {
        std::lock_guard<std::mutex> lock(mutex_);
        ++count_;
    }

private:
    mutable std::mutex mutex_;
    int count_ = 0;
};

// Clients never know (or need to know) about the locking.
// Thread safety is an implementation detail hidden behind the interface.

```

4.11.6 Things to Remember

- Declare data members private. It gives clients syntactically uniform access to data, affords fine-grained access control, allows invariants to be enforced, and offers class authors implementation flexibility.
- protected is no more encapsulated than public.

4.12 Item 23: Prefer Non-Member Non-Friend Functions to Member Functions

4.13 Visual Summary

ITEM 23: PREFER NON-MEMBER NON-FRIEND FUNCTIONS TO MEMBER FUNCTIONS

1. Function needs no private access -> it does not need to be a member.
2. Member function -> increases class surface and coupling.
3. Non-member non-friend -> same capability with better encapsulation.
4. Group related helpers in namespaces/headers for discoverability.
5. Meaning: fewer privileged functions means stronger encapsulation.

This item contains one of the most counterintuitive pieces of advice in the book. Many programmers believe that putting a function inside a class (making it a member) increases encapsulation because the function “belongs” to the class. In fact, the opposite is true: in many cases, a **non-member non-friend function** provides **more** encapsulation, **more** packaging flexibility, and **more** functional extensibility.

4.13.1 The Web Browser Example

```
class WebBrowser {
public:
    void clearCache();
    void clearHistory();
    void removeCookies();

    // Should clearEverything() be a member function?
    void clearEverything(); // Calls clearCache, clearHistory, removeCookies
};
```

```
// Attempt 1 (member function):
void WebBrowser::clearEverything() {
    clearCache();
    clearHistory();
    removeCookies();
}

// Attempt 2 (non-member non-friend function):
void clearBrowser(WebBrowser& wb) {
    wb.clearCache();
    wb.clearHistory();
    wb.removeCookies();
}
```

Which is better? The non-member non-friend function. Here's why.

4.13.2 The Encapsulation Argument

Encapsulation means that something is hidden from view. The more things are hidden, the greater the flexibility to change them. The more things that are encapsulated, the greater our ability to change them without affecting other code.

The number of functions that can access the private members of a class is a measure of how **un-encapsulated** those members are. The more functions that can access private data, the less encapsulated that data is.

- A **member function** can access all private members.
- A **non-member non-friend function** cannot access any private members.

Therefore, choosing a non-member non-friend function over a member function **increases encapsulation** (all else being equal). The non-member function `clearBrowser` can only call public functions on `WebBrowser`. It adds no new access to private data.

Note: this reasoning only applies when you're choosing between a member function and a non-member **non-friend** function. Friends have the same access as members, so choosing a friend over a member doesn't improve encapsulation.

4.13.3 Namespace-Based Organization

The natural home for non-member functions associated with a class is the **same namespace** as the class. This is the C++ way of saying “these functions are related to this class” without granting them private access.

```
// WebBrowser.h
namespace WebBrowserStuff {

class WebBrowser {
public:
    void clearCache();
    void clearHistory();
    void removeCookies();
    // ...
};

// Convenience function in the same namespace
void clearBrowser(WebBrowser& wb);

} // namespace WebBrowserStuff
```

4.13.4 Splitting Functionality Across Headers

Unlike classes, namespaces can be split across multiple header files. This is enormously useful for managing large interfaces. The standard library does exactly this: `std::vector` is in `<vector>`, `std::sort` is in `<algorithm>`, etc. – all in namespace `std`, but spread across many headers.

```
// webbrowser.h -- core WebBrowser class
namespace WebBrowserStuff {

class WebBrowser { /* ... */ };

// Core non-member functions that nearly everyone needs
void clearBrowser(WebBrowser& wb);

} // namespace WebBrowserStuff

// webbrowserbookmarks.h -- bookmark-related convenience functions
namespace WebBrowserStuff {

void addBookmark(WebBrowser& wb, const std::string& url);
void removeBookmark(WebBrowser& wb, const std::string& url);
std::vector<std::string> getBookmarks(const WebBrowser& wb);

} // namespace WebBrowserStuff

// webbrowsercookies.h -- cookie-related convenience functions
namespace WebBrowserStuff {

void importCookies(WebBrowser& wb, const std::string& file);
void exportCookies(const WebBrowser& wb, const std::string& file);
Cookie getCookie(const WebBrowser& wb, const std::string& domain);

} // namespace WebBrowserStuff
```

Clients include only the headers they need, reducing compilation dependencies. This is **impossible** with member functions – all member functions must be declared in the class definition, which lives in a single header. A client who wants to use `addBookmark` must include the entire class definition, including declarations for cookie-related and cache-related functions they don’t need.

4.13.5 Extensibility by Clients

Because namespaces are open, clients can add their own convenience functions:

```
// In the client's own header:
namespace WebBrowserStuff {

// Client-defined convenience function
void clearBrowserAndNotifyUser(WebBrowser& wb, const User& user) {
    clearBrowser(wb);
    user.notify("Browser cleared");
}

} // namespace WebBrowserStuff
```

A client cannot add member functions to a class they don't control. But they can add non-member functions to a namespace. This is one of the key extensibility benefits.

4.13.6 When Member Functions ARE Appropriate

This item does not say “never use member functions.” Member functions are appropriate when the function:

1. Needs access to private data (and no public interface can provide that access), or
2. Is a virtual function (virtual dispatch requires membership), or
3. Is an operator that must be a member (operator=, operator[], operator->, operator()), or
4. Affects the object's internal invariants in a way that can only be done with private access.

```
class String {
public:
    // These MUST be members:
    String& operator=(const String& rhs); // Assignment operator must be a member
    char& operator[](size_t index);      // Subscript operator must be a member
    size_t size() const;                 // Needs access to internal length_ member

    // This SHOULD be a non-member:
    // friend String operator+(const String& lhs, const String& rhs);
    // See Item 24 for why.

private:
    char* data_;
    size_t length_;
};

// Non-member convenience functions:
bool isAllUpperCase(const String& s) {
    for (size_t i = 0; i < s.size(); ++i) {
        if (!std::isupper(s[i])) return false;
    }
    return true;
}

// isAllUpperCase uses only the public interface. Making it a member would
// decrease encapsulation for no benefit.
```

4.13.7 Real-World Example: Algorithm-Style Functions

```

class Matrix {
public:
    size_t rows() const;
    size_t cols() const;
    double& operator()(size_t r, size_t c);
    const double& operator()(size_t r, size_t c) const;

private:
    std::vector<double> data_;
    size_t rows_, cols_;
};

// These should be non-member non-friend functions:
// They operate entirely through the public interface.

Matrix transpose(const Matrix& m) {
    Matrix result(m.cols(), m.rows());
    for (size_t r = 0; r < m.rows(); ++r)
        for (size_t c = 0; c < m.cols(); ++c)
            result(c, r) = m(r, c);
    return result;
}

Matrix multiply(const Matrix& a, const Matrix& b) {
    assert(a.cols() == b.rows());
    Matrix result(a.rows(), b.cols());
    for (size_t i = 0; i < a.rows(); ++i)
        for (size_t j = 0; j < b.cols(); ++j) {
            double sum = 0;
            for (size_t k = 0; k < a.cols(); ++k)
                sum += a(i, k) * b(k, j);
            result(i, j) = sum;
        }
    return result;
}

bool isSymmetric(const Matrix& m) {
    if (m.rows() != m.cols()) return false;
    for (size_t r = 0; r < m.rows(); ++r)
        for (size_t c = r + 1; c < m.cols(); ++c)
            if (m(r, c) != m(c, r)) return false;
    return true;
}

// None of these functions need private access.
// Making them non-member non-friend improves encapsulation.
// They can live in different headers for reduced coupling.
// Clients can add their own algorithms (e.g., determinant, inverse)
// without modifying the Matrix class.

```

4.13.8 Things to Remember

- Prefer non-member non-friend functions to member functions. Doing so increases encapsulation, packaging flexibility, and functional extensibility.

4.14 Item 24: Declare Non-Member Functions When Type Conversions Should Apply to All Parameters

4.15 Visual Summary

ITEM 24: DECLARE NON-MEMBER FUNCTIONS WHEN TYPE CONVERSIONS SHOULD APPLY TO

1. Binary operator as member → left operand must already be class type.
2. Implicit conversion only helps right operand.
3. Non-member operator → both operands can be converted symmetrically.
4. Friend only if private access is truly required.
5. Meaning: symmetric operations usually belong outside the class.

This item addresses a specific but important situation: when you want implicit type conversions to work for **all** arguments of a function, including the object on which a member function would be called (i.e., `*this`).

4.15.1 The Rational Number Multiplication Problem

```
class Rational {
public:
    Rational(int numerator = 0, int denominator = 1); // Not explicit -- allows
                                                    // implicit int-to-Rational

    int numerator() const { return n_; }
    int denominator() const { return d_; }

    // Attempt: operator* as a member function
    const Rational operator*(const Rational& rhs) const;

private:
    int n_, d_;
};
```

This seems to work:

```
Rational oneHalf(1, 2);
Rational result;

result = oneHalf * 2; // OK! Same as: oneHalf.operator*(Rational(2))
                      // The int 2 is implicitly converted to Rational(2, 1)
                      // via the non-explicit constructor.

result = 2 * oneHalf; // ERROR! Same as: 2.operator*(oneHalf)
                      // int doesn't have an operator* that takes a Rational!
                      // The compiler also tries:
                      //   operator*(2, oneHalf)
                      // but no such non-member function exists.
```

The asymmetry is the problem. When `operator*` is a member function, the left-hand operand must be a `Rational` (it becomes `*this`). Implicit conversions are never applied to `*this` – only to function arguments. So `2 * oneHalf` fails because `2` is not a `Rational` and can't be implicitly converted in the `*this` position.

4.15.2 The Solution: Make `operator*` a Non-Member Function

```

class Rational {
public:
    Rational(int numerator = 0, int denominator = 1); // Implicit conversion allowed
    int numerator() const { return n_; }
    int denominator() const { return d_; }

    // No operator* member function!

private:
    int n_, d_;
};

// Non-member operator*
const Rational operator*(const Rational& lhs, const Rational& rhs) {
    return Rational(lhs.numerator() * rhs.numerator(),
                    lhs.denominator() * rhs.denominator());
}

```

Now both forms work:

```

Rational oneHalf(1, 2);
Rational result;

result = oneHalf * 2; // OK! operator*(oneHalf, Rational(2))
                     // 2 is implicitly converted to Rational(2, 1)

result = 2 * oneHalf; // OK! operator*(Rational(2), oneHalf)
                     // 2 is implicitly converted to Rational(2, 1)

result = 2 * 3; // Still uses built-in int multiplication
               // (no Rational involved, so no conversion)

```

4.15.3 Should operator* Be a Friend?

Notice that the non-member operator* above only uses numerator() and denominator() – both public member functions. It doesn't need private access. Therefore, **it should not be a friend**.

```

// BAD: unnecessary friendship
class Rational {
    friend const Rational operator*(const Rational& lhs, const Rational& rhs);
    // ...
};

// This works, but the friendship is gratuitous.
// operator* can do everything it needs through the public interface.

// GOOD: non-member non-friend (as shown above)
// Maximizes encapsulation (Item 23) while supporting type conversions.

```

The takeaway from Items 23 and 24 combined: if a function doesn't need private access, it shouldn't be a member or a friend. The only reason to make it a friend would be if it genuinely needs access to private data.

4.15.4 What About Explicit Constructors?

If the Rational constructor were explicit, implicit conversions would be disabled:

```

class Rational {
public:
    explicit Rational(int numerator = 0, int denominator = 1);
    // ...
};

Rational oneHalf(1, 2);
Rational result = oneHalf * 2; // ERROR! Can't implicitly convert 2 to Rational
Rational result = oneHalf * Rational(2); // OK -- explicit conversion

```

Whether to use `explicit` depends on your design goals. For a numeric type like `Rational`, implicit conversion from `int` is usually desirable (it mirrors how `int` implicitly converts to `double`). For other types, `explicit` is usually safer.

4.15.5 Real-World Example: A Vector2D Class

```

class Vector2D {
public:
    Vector2D(double x = 0, double y = 0) : x_(x), y_(y) {}

    double x() const { return x_; }
    double y() const { return y_; }

    // operator+= is a member -- it modifies *this
    Vector2D& operator+=(const Vector2D& rhs) {
        x_ += rhs.x_;
        y_ += rhs.y_;
        return *this;
    }

    // operator*= (scalar) is a member -- it modifies *this
    Vector2D& operator*=(double scalar) {
        x_ *= scalar;
        y_ *= scalar;
        return *this;
    }

private:
    double x_, y_;
};

// operator+ is a non-member -- both sides should support conversion
Vector2D operator+(const Vector2D& lhs, const Vector2D& rhs) {
    return Vector2D(lhs.x() + rhs.x(), lhs.y() + rhs.y());
}

// operator* (scalar) is a non-member -- we want both:
//   vector * scalar
//   scalar * vector
Vector2D operator*(const Vector2D& v, double scalar) {
    return Vector2D(v.x() * scalar, v.y() * scalar);
}

Vector2D operator*(double scalar, const Vector2D& v) {
    return v * scalar; // Delegate to the other overload
}

// Usage:
Vector2D v(3, 4);
Vector2D w = v * 2.0; // OK

```

```
Vector2D x = 2.0 * v;    // OK -- would fail if operator* were a member
Vector2D y = v + w;     // OK
```

4.15.6 Real-World Example: Comparison Operators

```
class Temperature {
public:
    Temperature(double kelvin) : kelvin_(kelvin) {} // Implicit -- intentional

    double kelvin() const { return kelvin_; }
    double celsius() const { return kelvin_ - 273.15; }
    double fahrenheit() const { return kelvin_ * 9.0/5.0 - 459.67; }

private:
    double kelvin_;
};

// Non-member comparisons -- both sides can undergo implicit conversion
bool operator==(const Temperature& lhs, const Temperature& rhs) {
    return lhs.kelvin() == rhs.kelvin();
}

bool operator<(const Temperature& lhs, const Temperature& rhs) {
    return lhs.kelvin() < rhs.kelvin();
}

// etc. for !=, >, <=, >=

// Usage:
Temperature boiling(373.15);

if (boiling == 373.15) { /* ... */ } // OK: 373.15 converts to Temperature
if (373.15 == boiling) { /* ... */ } // OK: works because operator== is non-member
// If operator== were a member function, the second form would fail.
```

4.15.7 Koenig Lookup (Argument-Dependent Lookup, ADL)

When you call a non-member function, the compiler searches for the function not only in the usual scopes but also in the **namespaces of the argument types**. This is called Argument-Dependent Lookup (ADL), or Koenig Lookup after Andrew Koenig who proposed it.

```
namespace Geometry {

class Point {
public:
    Point(double x, double y) : x_(x), y_(y) {}
    double x() const { return x_; }
    double y() const { return y_; }
private:
    double x_, y_;
};

// Non-member function in the same namespace as Point
double distance(const Point& a, const Point& b) {
    double dx = a.x() - b.x();
    double dy = a.y() - b.y();
    return std::sqrt(dx*dx + dy*dy);
}
```

```
// Non-member operator
Point operator+(const Point& a, const Point& b) {
    return Point(a.x() + b.x(), a.y() + b.y());
}

} // namespace Geometry

// Client code -- no "using" declaration needed!
int main() {
    Geometry::Point p1(1, 2);
    Geometry::Point p2(4, 6);

    double d = distance(p1, p2); // Found via ADL! The compiler searches
                                // Geometry:: because p1 and p2 are in Geometry.

    Geometry::Point p3 = p1 + p2; // Also found via ADL.
}
```

ADL is the reason why `std::cout << "hello"` works – `operator<<` is a non-member function in namespace `std`, and it's found via ADL because `std::cout` is in namespace `std`.

4.15.8 Things to Remember

- If you need type conversions on all parameters to a function (including the one that would otherwise be pointed to by this), the function must be a non-member.

4.16 Item 25: Consider Support for a Non-Throwing Swap

4.17 Visual Summary

ITEM 25: CONSIDER SUPPORT FOR A NON-THROWING SWAP
1. Class owns expensive/resource state → generic swap may copy too much. 2. Provide efficient swap → exchange handles/pointers directly. 3. Mark/specialize as non-throwing → algorithms and copy-and-swap can rely on it. 4. Use <code>using std::swap</code> then unqualified swap for ADL. 5. Meaning: good swap is a small hook with large exception-safety payoff.

`swap` is one of the most important functions in C++. It is central to exception-safe programming (see Item 29), to the copy-and-swap idiom for assignment operators, and to guarding against self-assignment (see Item 11). This item explains how to write an efficient, non-throwing swap for your types.

4.17.1 The Default `std::swap`

The default `std::swap` implementation uses a temporary and three copies:

```

namespace std {

template<typename T>
void swap(T& a, T& b) {
    T temp(a);    // Copy a
    a = b;        // Copy b into a
    b = temp;     // Copy temp into b
}

} // namespace std

```

For types that are expensive to copy, this is wasteful. Many types can be swapped far more efficiently – for example, by just swapping internal pointers.

4.17.2 The Pimpl Idiom: Where Default Swap Hurts

```

// A Widget that uses the Pimpl (Pointer to Implementation) idiom
class WidgetImpl {
public:
    // ... lots of data ...
private:
    int a, b, c;
    std::vector<double> v;
    std::map<std::string, std::string> m;
    // ... potentially huge ...
};

class Widget {
public:
    Widget(const Widget& rhs);
    Widget& operator=(const Widget& rhs) {
        // Copy all of rhs's WidgetImpl data
        *pImpl = *(rhs.pImpl);
        return *this;
    }
private:
    WidgetImpl* pImpl; // Pointer to the implementation
};

```

Swapping two `Widget` objects using the default `std::swap` copies three `Widget` objects, which means copying three `WidgetImpl` objects – extremely expensive. But all we really need to do is swap the `pImpl` pointers.

4.17.3 Step 1: Write a Member swap Function

```

class Widget {
public:
    void swap(Widget& other) {
        using std::swap;           // Make std::swap available as a fallback
        swap(pImpl, other.pImpl); // Just swap the pointers -- O(1)!
    }

    // ...

private:
    WidgetImpl* pImpl;
};

```

4.17.4 Step 2: Specialize std::swap for Your Non-Template Class

```

namespace std {

// Total specialization of std::swap for Widget
template<>
void swap<Widget>(Widget& a, Widget& b) {
    a.swap(b); // Delegate to the member function
}

} // namespace std

```

This is legal – the C++ standard allows you to **totally specialize** templates in std for user-defined types.

4.17.5 Step 3: For Class Templates, Use a Non-Member swap in Your Namespace

If Widget is itself a class template, you **cannot** partially specialize std::swap:

```

// Widget is now a template
template<typename T>
class Widget {
public:
    void swap(Widget<T>& other) {
        using std::swap;
        swap(pImpl, other.pImpl);
    }

private:
    WidgetImpl<T>* pImpl;
};

// BAD: You CANNOT partially specialize a function template!
namespace std {
template<typename T>
void swap<Widget<T>>(Widget<T>& a, Widget<T>& b) { // ILLEGAL!
    a.swap(b);
}
}

// BAD: You could OVERLOAD std::swap, but adding new functions to std is
// undefined behavior (only specializations are allowed):
namespace std {
template<typename T>
void swap(Widget<T>& a, Widget<T>& b) { // Technically undefined behavior!
    a.swap(b);
}
}

```

```
}
```

The solution: declare a non-member swap in **your own namespace**:

```
namespace WidgetStuff {

template<typename T>
class Widget { /* ... as above ... */ };

// Non-member swap in the same namespace as Widget
template<typename T>
void swap(Widget<T>& a, Widget<T>& b) {
    a.swap(b);
}

} // namespace WidgetStuff
```

ADL (Koenig Lookup) ensures that when client code calls swap on two Widget<T> objects, the compiler finds WidgetStuff::swap because it looks in the namespace of the argument types.

4.17.6 The Correct Way to Call swap

When writing generic code that uses swap, you need to ensure both the std::swap default and any type-specific swap are considered:

```
template<typename T>
void doSomething(T& obj1, T& obj2) {
    using std::swap;           // Make std::swap visible as a fallback
    swap(obj1, obj2);          // Let the compiler choose the best swap:
                                // 1. If there's a T-specific swap in T's namespace, ADL finds it.
                                // 2. If there's a std::swap<T> specialization, it's considered.
                                // 3. Otherwise, the default std::swap is used.
}

// BAD: Qualifying the call prevents ADL from finding type-specific swap:
template<typename T>
void doSomething(T& obj1, T& obj2) {
    std::swap(obj1, obj2);     // ALWAYS uses std::swap!
                                // Will NOT find WidgetStuff::swap<Widget<T>>
                                // even though it's more efficient.
}
```

4.17.7 Complete Example: Putting It All Together

```
// For a non-template class:

class NetworkBuffer {
public:
    NetworkBuffer() : data_(nullptr), size_(0) {}
    NetworkBuffer(size_t size) : data_(new char[size]), size_(size) {}

    NetworkBuffer(const NetworkBuffer& rhs)
        : data_(new char[rhs.size_]), size_(rhs.size_) {
        std::memcpy(data_, rhs.data_, size_);
    }

    // Copy-and-swap idiom for exception-safe assignment
    NetworkBuffer& operator=(NetworkBuffer rhs) { // Note: pass by value
        swap(rhs);           // Swap our guts with the copy's guts
        return *this;         // The copy's destructor cleans up our old data
    }
}
```



```

~NetworkBuffer() { delete[] data_; }

// Member swap -- efficient, non-throwing
void swap(NetworkBuffer& other) noexcept {
    using std::swap;
    swap(data_, other.data_);    // Swap pointers -- O(1), noexcept
    swap(size_, other.size_);    // Swap sizes -- O(1), noexcept
}

private:
    char* data_;
    size_t size_;
};

// Non-member swap in the same namespace
void swap(NetworkBuffer& a, NetworkBuffer& b) noexcept {
    a.swap(b);
}

// Also specialize std::swap
namespace std {
template<>
void swap<NetworkBuffer>(NetworkBuffer& a, NetworkBuffer& b) noexcept {
    a.swap(b);
}
}

```

4.17.8 The noexcept Guarantee

Highly efficient swaps are almost always non-throwing. This is important because:

1. The copy-and-swap idiom depends on a non-throwing swap for strong exception safety.
2. Many standard library operations (e.g., `std::sort`, `std::vector` reallocation) work more efficiently when swap doesn't throw.

```

class Widget {
public:
    // The member swap should be noexcept
    void swap(Widget& other) noexcept {
        using std::swap;
        swap(pImpl, other.pImpl); // Pointer swap never throws
    }
};

// Built-in types (pointers, ints, etc.) can always be swapped without throwing.
// If your class's swap only swaps built-in types (like pointers), it should be noexcept.

```

4.17.9 Summary of the swap Protocol

Scenario	What to do
Non-template class	1. Write a public swap member function (noexcept). 2. Write a non-member swap in your namespace that calls the member. 3. Specialize <code>std::swap</code> to call the member.
Class template	1. Write a public swap member function (noexcept). 2. Write a non-member swap in your namespace that calls the member. 3. Do NOT partially specialize or overload <code>std::swap</code> .
Calling swap in generic code	Always do <code>using std::swap;</code> followed by an unqualified call to <code>swap</code> .

4.17.10 Things to Remember

- Provide a swap member function when `std::swap` would be inefficient for your type. Make sure your swap doesn't throw exceptions.
- If you offer a member swap, also offer a non-member swap that calls the member. For classes (not templates), specialize `std::swap`, too.
- When calling swap, employ a using declaration for `std::swap`, then call swap without namespace qualification.
- It's fine to totally specialize std templates for user-defined types, but never try to add something completely new to std.

4.18 Item 26: Postpone Variable Definitions as Long as Possible

4.19 Visual Summary

ITEM 26: POSTPONE VARIABLE DEFINITIONS AS LONG AS POSSIBLE
1. Define variable early -> constructor/destructor cost paid even if unused. 2. Early definition before validation -> exceptions/returns waste work. 3. Define at first meaningful value -> avoids default construction plus assignment. 4. Loops -> choose inside or outside based on cost and needed lifetime. 5. Meaning: give variables the smallest useful lifetime and best initial value.

Whenever you define a variable of a type with a constructor and destructor, you incur the cost of construction when control reaches the variable's definition, and the cost of destruction when the variable leaves scope. This cost is wasted if the variable is never used – and that happens more often than you might think.

4.19.1 The Obvious Case: Variables Before Early Returns

```
// BAD: encrypted is constructed even if the password is too short
std::string encryptPassword(const std::string& password) {
    using namespace std;
    string encrypted; // Constructed here -- default constructor

    if (password.length() < MinimumPasswordLength) {
        throw logic_error("Password is too short");
        // If we throw, 'encrypted' was constructed and destroyed for nothing.
    }

    encrypted = password; // Assignment operator (second cost!)
    encrypt(encrypted);
    return encrypted;
}
```

```
// BETTER: postpone until after the check
std::string encryptPassword(const std::string& password) {
    using namespace std;

    if (password.length() < MinimumPasswordLength) {
        throw logic_error("Password is too short");
    }

    string encrypted; // Not constructed if we threw above
    encrypted = password; // But still: default construction + assignment
    encrypt(encrypted);
    return encrypted;
}
```

```
// BEST: postpone AND initialize directly -- skip the default construction
std::string encryptPassword(const std::string& password) {
    using namespace std;

    if (password.length() < MinimumPasswordLength) {
        throw logic_error("Password is too short");
    }

    string encrypted(password); // Copy constructor -- one operation instead of two!
    encrypt(encrypted);
    return encrypted;
}
```

The final version avoids both the unnecessary default construction and the assignment. It directly initializes `encrypted` with `password` using the copy constructor.

4.19.2 The General Rule

You should postpone a variable's definition until: 1. You can give it an initial value, AND 2. You're certain the variable will actually be used.

```

// BAD: defining variables long before they're needed
void processData(const std::vector<int>& data) {
    int sum = 0;           // Defined here...
    double average = 0.0;  // ...and here...
    std::string report;    // ...and here...

    if (data.empty()) {
        return; // sum, average, and report were never used!
    }

    // ... 50 lines of validation code ...

    for (int x : data) {
        sum += x;           // First use of sum -- 60 lines after definition!
    }
    average = static_cast<double>(sum) / data.size(); // First use of average
    report = generateReport(average);                // First use of report
}

// GOOD: define each variable at the point of first use
void processData(const std::vector<int>& data) {
    if (data.empty()) {
        return;
    }

    // ... 50 lines of validation code ...

    int sum = 0;           // Right where it's needed
    for (int x : data) {
        sum += x;
    }

    double average = static_cast<double>(sum) / data.size(); // Right here
    std::string report = generateReport(average);            // Right here
}

```

4.19.3 Variables in Loops

What about variables used only inside a loop? There are two approaches:

```

// Approach A: Define outside the loop
Widget w;
for (int i = 0; i < n; ++i) {
    w = some_value_dependent_on_i;
    // ... use w ...
}
// Cost: 1 constructor + 1 destructor + n assignments

// Approach B: Define inside the loop
for (int i = 0; i < n; ++i) {
    Widget w(some_value_dependent_on_i);
    // ... use w ...
}
// Cost: n constructors + n destructors

```

The costs are:

Approach	Cost
A (outside)	1 construction + 1 destruction + n assignments
B (inside)	n constructions + n destructions

Approach A is more efficient **if** an assignment is cheaper than a constructor-destructor pair. Otherwise, Approach B is better.

The recommendation: default to Approach B (define inside the loop) unless: 1. You know that the assignment is significantly cheaper than a construction-destruction pair, **AND** 2. You are dealing with a performance-sensitive part of your code.

Approach B is preferred because: - It limits the variable's scope to the loop body (better readability, fewer bugs). - The variable can't be accidentally used after the loop. - It's easier to reason about correctness.

```
// GOOD (default choice): define inside the loop
for (int i = 0; i < n; ++i) {
    std::string s = computeString(i); // Fresh string each iteration
    processString(s);
    // s is destroyed here -- can't leak into the next iteration or beyond the loop
}

// ACCEPTABLE (performance-critical path with expensive construction):
std::string s;
for (int i = 0; i < n; ++i) {
    s = computeString(i); // Reuse s's memory allocation
    processString(s);
}
// But now s is visible after the loop -- wider scope, more potential for bugs.
```

4.19.4 Real-World Example: Database Query Processing

```
// BAD: premature definitions
void processQuery(Database& db, const std::string& queryStr) {
    Connection conn = db.getConnection(); // Expensive! Opens a connection.
    PreparedStatement stmt = conn.prepare(queryStr); // Expensive! Parses SQL.
    ResultSet results; // Default-constructed.

    if (!db.isAvailable()) {
        throw DatabaseException("Database unavailable");
        // conn, stmt, and results were all constructed for nothing!
        // The connection was opened and must now be closed in the destructor.
    }

    if (!isValidQuery(queryStr)) {
        throw QueryException("Invalid query");
        // conn was opened for nothing!
    }

    results = stmt.execute(); // Assignment, not initialization
    processResults(results);
}

// GOOD: postpone everything
void processQuery(Database& db, const std::string& queryStr) {
    if (!db.isAvailable()) {
        throw DatabaseException("Database unavailable");
        // No resources acquired yet
    }
}
```

```

if (!isValidQuery(queryStr)) {
    throw QueryException("Invalid query");
    // Still no resources acquired
}

Connection conn = db.getConnection();           // NOW open the connection
PreparedStatement stmt = conn.prepare(queryStr); // NOW parse the SQL
ResultSet results = stmt.execute();             // Direct initialization!
processResults(results);
}

```

4.19.5 Real-World Example: File Processing with Multiple Error Paths

```

// BAD: all variables at the top
bool convertFile(const std::string& inputPath, const std::string& outputPath) {
    std::ifstream input;
    std::ofstream output;
    std::string line;
    std::vector<std::string> processedLines;
    size_t lineCount = 0;
    bool success = false;

    input.open(inputPath);
    if (!input.is_open()) return false;

    output.open(outputPath);
    if (!output.is_open()) return false; // input constructed, output default-constructed
                                        // then opened -- wasteful

    while (std::getline(input, line)) {
        processedLines.push_back(transformLine(line));
        ++lineCount;
    }

    for (const auto& pl : processedLines) {
        output << pl << "\n";
    }

    success = true;
    return success;
}

// GOOD: each variable defined at the point of first use
bool convertFile(const std::string& inputPath, const std::string& outputPath) {
    std::ifstream input(inputPath);           // Open immediately via constructor
    if (!input.is_open()) return false;

    std::ofstream output(outputPath);         // Only open if input succeeded
    if (!output.is_open()) return false;

    std::vector<std::string> processedLines;
    std::string line;
    while (std::getline(input, line)) {       // 'line' reused in the loop (Approach A)
        processedLines.push_back(transformLine(line));
    }

    for (const auto& pl : processedLines) {
        output << pl << "\n";
    }
}

```

```

    return true;                                // No need for a 'success' variable
}

```

4.19.6 The Relationship to const

Postponing definitions also enables more uses of const:

```

// BAD: can't make x const because it's defined before the value is known
int x;
// ... lots of code ...
x = computeValue();
// x is non-const even though it never changes after this point.

// GOOD: define at the point of initialization -- now it can be const
// ... lots of code ...
const int x = computeValue();
// x is const, which communicates intent and enables optimizations.

```

4.19.7 Things to Remember

- Postpone variable definitions as long as possible. It increases program clarity and improves program efficiency.

4.20 Summary

Chapter 4 covers the core design decisions in C++ software:

Item	Key Principle
18	Design interfaces that are easy to use correctly and hard to misuse
19	Treat class design as type design – ask the right questions
20	Pass by reference-to-const by default; avoid the slicing problem
21	Return objects by value when you must; don't return dangling references
22	Make data members private for encapsulation and flexibility
23	Prefer non-member non-friend functions for better encapsulation
24	Use non-member functions when type conversions should apply to all parameters
25	Implement efficient, non-throwing swap for your types
26	Define variables at the latest possible point, initialized with their real values

These principles work together: good type design (Item 19) leads to interfaces that are hard to misuse (Item 18), with private data members (Item 22) accessed through well-designed functions that are members only when necessary (Items 23, 24), passed by reference (Item 20), returned by value when needed (Item 21), with efficient swap support (Item 25) and minimal variable lifetimes (Item 26).

5 Chapter 5: Implementations

5.1 Chapter Flow

CHAPTER 5: IMPLEMENTATIONS

Item 27 -> treat casts as visible risk markers.
Item 28 -> avoid leaking handles to internals.
Item 29 -> define object state guarantees after exceptions.
Item 30 -> inline only when size, stability, and performance justify it.
Item 31 -> hide implementation details to reduce rebuild coupling.

Most of the time, coming up with appropriate definitions for your classes and declarations for your functions is the lion's share of the battle. Once you have those right, the corresponding implementations are largely straightforward. Still, there are things to watch out for. Defining variables too soon can cause a drag on performance. Overuse of casts can lead to code that is slow, hard to maintain, and infected with subtle bugs. Returning handles to an object's internals can defeat encapsulation and leave clients with dangling handles. Failure to consider the impact of exceptions can lead to leaked resources and corrupted data structures. Overzealous inlining can cause code bloat. And excessive coupling can result in unacceptably long build times.

Each of these problems is addressed in the Items that follow.

5.2 Item 27: Minimize casting

5.3 Visual Summary

ITEM 27: MINIMIZE CASTING

1. Cast appears -> compiler protections are being bypassed.
2. C-style cast -> hides const/static/reinterpret behavior.
3. static_cast/dynamic_cast/const_cast/reinterpret_cast -> expose exact intent.
4. Design smell? -> prefer virtual functions, overloads, or better types.
5. Meaning: casts are sometimes necessary, but each one is a local risk marker.

The rules of C++ are designed to guarantee that type errors are impossible. In theory, if your program compiles cleanly, it is not trying to perform any unsafe or nonsensical operations on any objects. This guarantee is valuable, and you should not lightly abandon it. Unfortunately, casts subvert the type system. That can lead to trouble of all kinds, some easy to recognize, some insidiously subtle.

5.3.1 The Old C-Style Casts

C-style casts look like this:


```
// C-style cast
(T) expression           // cast expression to be of type T

// Function-style cast (same semantics, different syntax)
T(expression)           // cast expression to be of type T
```

There is no difference in meaning between these forms. It is purely a matter of where you put the parentheses. These are called old-style casts.

Problem with old-style casts: They are blunt instruments. You cannot tell at a glance what kind of conversion is taking place — whether the cast is removing `const`, reinterpreting a pointer type, performing a safe numeric conversion, or something else entirely. They are also essentially impossible to search for in code, since the syntax `(int)x` or `int(x)` blends in with surrounding code.

5.3.2 The Four C++ Casts

C++ offers four new cast forms, each with a specific, narrow purpose:

5.3.2.1 1. `const_cast`: Casting Away Constness `const_cast` is used to add or remove `const` (or `volatile`) from an object. It is the only C++ cast that can do this.

```
class TextBlock {
public:
    // Non-const operator[] wants to reuse the const version to avoid
    // code duplication (see Item 3).
    const char& operator[](std::size_t position) const {
        // ... bounds checking, logging, data integrity verification ...
        return text[position];
    }

    char& operator[](std::size_t position) {
        // Cast *this to const to call the const version, then cast
        // away const from the return value. This is safe because the
        // caller must have a non-const object (otherwise this non-const
        // overload would not have been called).
        return const_cast<char&>(
            static_cast<const TextBlock&>(*this)[position]
        );
    }

private:
    std::string text;
};
```

A real-world scenario where `const_cast` is necessary: interfacing with legacy C APIs that do not use `const` correctly:

```
// Legacy C library header
// void legacy_print(char* str); // should be const char*, but isn't

void printMessage(const std::string& msg) {
    // We know legacy_print does not modify str, but the API is not
    // const-correct. const_cast is the appropriate escape hatch.
    legacy_print(const_cast<char*>(msg.c_str()));
}
```

Warning: Using `const_cast` to modify an object that was originally declared `const` is undefined behavior:

```
const int x = 42;
int* p = const_cast<int*>(&x);
*p = 99; // UNDEFINED BEHAVIOR! x was declared const.
```

5.3.2.2 2. `dynamic_cast`: Safe Downcasting in Inheritance Hierarchies `dynamic_cast` is used to perform safe downcasts in polymorphic class hierarchies (classes that have at least one virtual function). It checks at runtime whether the cast is valid.

```
class Shape {
public:
    virtual ~Shape() = default;
    virtual void draw() const = 0;
};

class Circle : public Shape {
public:
    void draw() const override { /* ... */ }
    double radius() const { return radius_; }
private:
    double radius_ = 1.0;
};

class Rectangle : public Shape {
public:
    void draw() const override { /* ... */ }
    double width() const { return width_; }
    double height() const { return height_; }
private:
    double width_ = 1.0;
    double height_ = 1.0;
};

// Pointer form: returns nullptr on failure
void processShape(Shape* shape) {
    if (Circle* circle = dynamic_cast<Circle*>(shape)) {
        // Safe: we verified that shape really is a Circle
        std::cout << "Circle with radius " << circle->radius() << "\n";
    }
    else if (Rectangle* rect = dynamic_cast<Rectangle*>(shape)) {
        std::cout << "Rectangle " << rect->width()
            << " x " << rect->height() << "\n";
    }
}

// Reference form: throws std::bad_cast on failure
void mustBeCircle(Shape& shape) {
    try {
        Circle& circle = dynamic_cast<Circle&>(shape);
        std::cout << "Confirmed circle, radius = "
            << circle.radius() << "\n";
    }
```

```

    }
    catch (const std::bad_cast& e) {
        std::cerr << "Not a circle: " << e.what() << "\n";
    }
}

```

Performance concern: `dynamic_cast` can be slow. Many implementations use string comparisons of class names (derived from `typeid`), and in deep hierarchies this may require traversing the entire inheritance chain. In performance-sensitive code, consider alternatives:

```

// ALTERNATIVE 1: Use virtual functions instead of dynamic_cast.
// This is almost always the right answer.
class Shape {
public:
    virtual ~Shape() = default;
    virtual void draw() const = 0;
    virtual void processSpecifics() const {} // default: do nothing
};

class Circle : public Shape {
public:
    void draw() const override { /* ... */ }
    void processSpecifics() const override {
        std::cout << "Circle with radius " << radius_ << "\n";
    }
private:
    double radius_ = 1.0;
};

// Now you can process shapes without any casting at all:
void processShape(Shape& shape) {
    shape.draw();
    shape.processSpecifics(); // polymorphism does the dispatching
}

// ALTERNATIVE 2: Store the derived pointer directly if you know the type.
// Use containers of typed pointers rather than casting from a base.
class Window {
    std::vector<std::unique_ptr<Circle>> circles_;
    std::vector<std::unique_ptr<Rectangle>> rectangles_;
    // No need for dynamic_cast; you already know the types.
};

```

Cascading `dynamic_cast` is a code smell:

```

// BAD: This is brittle and violates the open/closed principle.
// Every time you add a new Shape subclass, you must update this function.
void handleShape(Shape* shape) {
    if (auto* c = dynamic_cast<Circle*>(shape)) {
        // handle circle
    } else if (auto* r = dynamic_cast<Rectangle*>(shape)) {
        // handle rectangle
    } else if (auto* t = dynamic_cast<Triangle*>(shape)) {
        // handle triangle
    }
    // ... and so on for every subclass
}

// GOOD: Use virtual functions. The Shape hierarchy handles itself.
void handleShape(Shape* shape) {
    shape->handle(); // Each subclass implements its own behavior.
}

```

5.3.2.3 3. static_cast: Explicit Type Conversions `static_cast` is the workhorse cast. It performs conversions that are reasonable but not guaranteed safe at runtime. It can do implicit conversions (int to double, pointer-to-derived to pointer-to-base), and it can do the reverse of many implicit conversions (but not remove const).

```

// Numeric conversions
int totalItems = 97;
int numberOfBuckets = 7;

// Without cast: integer division truncates to 13
double avgBad = totalItems / numberOfBuckets;

// With static_cast: floating-point division gives 13.857...
double avgGood = static_cast<double>(totalItems) / numberOfBuckets;

// Enum conversions
enum class Color { Red, Green, Blue };
int colorValue = static_cast<int>(Color::Green); // 1

// Pointer conversions in a known hierarchy (no runtime check)
class Base { public: virtual ~Base() = default; };
class Derived : public Base { public: void specificMethod() {} };

Base* bp = new Derived;
// You (the programmer) assert that bp really points to a Derived.
// No runtime check is performed. If you are wrong, behavior is undefined.
Derived* dp = static_cast<Derived*>(bp);
dp->specificMethod(); // OK only if bp truly points to a Derived.
delete bp;

// void* conversions (common in C callback interfaces)
void timerCallback(void* userData) {
    auto* connection = static_cast<Connection*>(userData);
    connection->onTimeout();
}

// Calling an explicit constructor
class Widget {
public:
    explicit Widget(int size);
};

```

```
void doSomething(const Widget& w);

doSomething(static_cast<Widget>(15)); // Calls Widget(int) explicitly
```

Beware: static_cast within an inheritance hierarchy can silently produce the wrong pointer value.

This is one of the most subtle and important points about casting. When you have multiple inheritance or even single inheritance with virtual functions, the base class subobject may be at a different offset within the derived object. A cast actually adjusts the pointer value:

```
class Window {
public:
    virtual ~Window() = default;
    int windowId = 1;
};

class ScrollBar {
public:
    virtual ~ScrollBar() = default;
    int scrollPos = 0;
};

class FancyWindow : public Window, public ScrollBar {
public:
    int fancyLevel = 5;
};

FancyWindow fw;

// These two pointers may have DIFFERENT numeric values!
Window* wp = &fw; // Points to the Window subobject
ScrollBar* sp = &fw; // Points to the ScrollBar subobject

// The compiler knows the layout and adjusts the pointer correctly.
// But if you bypass the type system with reinterpret_cast, you get garbage.
```

Another critical subtlety — casting *this in a derived class:

```
class Window {
public:
    virtual void onResize() {
        // ... base class resize logic ...
        std::cout << "Window::onResize for " << this << "\n";
    }
};

class SpecialWindow : public Window {
public:
    virtual void onResize() override {
        // WRONG! This creates a TEMPORARY COPY of the Window part of *this,
        // then calls onResize on that temporary. The original object is
        // NOT affected by anything Window::onResize does to its members.
        static_cast<Window>(*this).onResize(); // BUG!

        // CORRECT: Call the base class version on *this.
        Window::onResize();

        // ... SpecialWindow-specific resize logic ...
    }
};
```

This is a real pitfall. The cast `static_cast<Window>(*this)` creates a temporary copy. Any side effects of

Window::onResize() (such as modifying data members) happen on the temporary, not on the actual object. The fix is simply to use the qualified call Window::onResize().

5.3.2.4 4. reinterpret_cast: Low-Level Bit Reinterpretation reinterpret_cast performs low-level, implementation-dependent casts. The result is almost always unportable. The most common legitimate use is casting between pointer types and integer types, or between unrelated pointer types when interfacing with hardware or very low-level system code.

```
// Legitimate use 1: Interfacing with hardware memory-mapped registers
volatile uint32_t* timerRegister =
    reinterpret_cast<volatile uint32_t*>(0x40000C00);

// Legitimate use 2: Implementing type-punning for serialization
// (Note: in modern C++, std::bit_cast from <bit> is preferred for this.)
float f = 3.14f;
uint32_t bits = *reinterpret_cast<uint32_t*>(&f);
// bits now contains the IEEE 754 representation of 3.14f.
// WARNING: This technically violates strict aliasing. Use std::bit_cast
// in C++20 or memcpy in earlier standards for well-defined behavior.

// Legitimate use 3: Hash functions that need to examine pointer bits
struct Hasher {
    size_t operator()(const void* ptr) const {
        return reinterpret_cast<uintptr_t>(ptr) >> 4;
    }
};
```

reinterpret_cast should almost never appear in application-level code. Its presence is a strong signal that something unusual and potentially non-portable is happening.

5.3.3 A Comprehensive Example: Why Casts Are Dangerous

```
#include <iostream>
#include <vector>
#include <memory>

class Animal {
public:
    virtual ~Animal() = default;
    virtual std::string speak() const = 0;
    int age_ = 0;
};

class Dog : public Animal {
public:
    std::string speak() const override { return "Woof!"; }
    std::string favoriteToy_ = "ball";
};

class Cat : public Animal {
public:
    std::string speak() const override { return "Meow!"; }
    bool isIndoor_ = true;
};

void demonstrateCastingDangers() {
    std::vector<std::unique_ptr<Animal>> zoo;
    zoo.push_back(std::make_unique<Dog>());
    zoo.push_back(std::make_unique<Cat>());
}
```

```

for (auto& animal : zoo) {
    // BAD: Casting every animal to Dog*. When animal is a Cat,
    // static_cast succeeds (no runtime check!) and you get UB.
    Dog* dog = static_cast<Dog*>(animal.get());
    std::cout << dog->favoriteToy_ << "\n";    // UB when animal is a Cat

    // GOOD: Use dynamic_cast which checks at runtime.
    if (Dog* dog2 = dynamic_cast<Dog*>(animal.get())) {
        std::cout << dog2->favoriteToy_ << "\n";    // Safe
    }

    // BEST: Use virtual functions --- no casting at all.
    std::cout << animal->speak() << "\n";
}
}

```

5.3.4 Summary of When Each Cast Is Appropriate

Cast	Purpose	Runtime check?	Can remove const?
const_cast	Add/remove const or volatile	No	Yes (only cast that can)
static_cast	“Reasonable” conversions, force explicit conversions	No	No
dynamic_cast	Safe downcast in polymorphic hierarchies	Yes	No
reinterpret_cast	Low-level bit reinterpretation	No	No

5.3.5 Things to Remember

- **Avoid casts whenever practical.** Code that requires casts often indicates a design that could be improved. This is especially true for `dynamic_cast` in performance-sensitive code.
- **When casting is necessary, prefer C++-style casts** (`static_cast`, `const_cast`, `dynamic_cast`, `reinterpret_cast`) over old-style casts. They are easier to identify in code (both for humans and for tools like `grep`), and their more narrowly specified purpose makes it possible for compilers to diagnose usage errors.
- **Beware the hidden copy when casting *this.** Casting `*this` to a base class value type creates a temporary copy; side effects on that copy do not affect the real object.
- **Consider alternatives to `dynamic_cast`,** especially cascading `dynamic_cast`s. Virtual function calls and container-of-typed-pointer designs are almost always better.

5.4 Item 28: Avoid returning “handles” to object internals

5.5 Visual Summary

ITEM 28: AVOID RETURNING "HANDLES" TO OBJECT INTERNALS

1. Method returns pointer/reference/iterator to private internals.
2. Caller can mutate private state or keep handle after object changes/dies.
3. Const handle helps mutation but not dangling lifetime.
4. Return value/proxy/operation instead when possible.
5. Meaning: do not let encapsulation escape through a handle.

A handle is anything that provides access to an object's internals: a reference, a pointer, or an iterator. Returning handles to object internals undermines encapsulation, can allow const member functions to enable modification of object state, and creates the risk of dangling handles.

5.5.1 Problem 1: Encapsulation Violation

```
struct Point {
    int x, y;
};

class Rectangle {
public:
    Rectangle(const Point& topLeft, const Point& bottomRight)
        : topLeft_(topLeft), bottomRight_(bottomRight) {}

    // DANGEROUS: Returns a non-const reference to internal data.
    // Client code can modify the internals directly, bypassing any
    // invariant-checking logic that Rectangle might want to enforce.
    Point& upperLeft() const { return topLeft_; }
    Point& lowerRight() const { return bottomRight_; }

private:
    Point topLeft_;
    Point bottomRight_;
};

// Client code:
Rectangle rect(Point{0, 0}, Point{100, 50});

// This compiles and runs! A const-qualified member function is allowing
// modification of the object's internal state through the returned reference.
rect.upperLeft().x = 999; // Modifies the "private" data!
```

The member functions `upperLeft()` and `lowerRight()` are declared `const`, promising not to modify the `Rectangle`. But they return non-const references to private data members, allowing clients to modify the data anyway.

The core issue: **a data member is only as encapsulated as the most accessible function returning a reference to it.** Here, `topLeft_` and `bottomRight_` are declared private but are effectively public because of the returned references.

5.5.2 Problem 2: Const Member Functions Enabling Mutation

Even if you meant for the class to be fully const-correct:


```

class StringWrapper {
public:
    explicit StringWrapper(const std::string& s) : data_(s) {}

    // The const on the member function only protects the pointer/reference
    // itself (i.e., which object data_ refers to), not the object it
    // points to. Returning a non-const reference to the internal string
    // allows callers to modify the string through a const StringWrapper.
    std::string& get() const { return data_; }

private:
    std::string data_;
};

const StringWrapper sw("hello");
sw.get() = "hacked";    // Compiles! Const-correctness is defeated.

```

5.5.3 Solution: Return const References (But Beware Dangling)

```

class Rectangle {
public:
    Rectangle(const Point& topLeft, const Point& bottomRight)
        : topLeft_(topLeft), bottomRight_(bottomRight) {}

    // BETTER: Return const references. Callers can read but not modify.
    const Point& upperLeft() const { return topLeft_; }
    const Point& lowerRight() const { return bottomRight_; }

private:
    Point topLeft_;
    Point bottomRight_;
};

// Now this will not compile:
// rect.upperLeft().x = 999;    // Error: assignment to member of const reference

```

This addresses the mutation problem but introduces a new one: **dangling handles**.

5.5.4 Problem 3: Dangling Handles

A dangling handle is a reference (or pointer or iterator) that refers to an object that no longer exists. This is one of the most pernicious bugs in C++.

```

class GUIObject { /* ... */ };

// Suppose this function returns a Rectangle by value (a temporary):
const Rectangle boundingBox(const GUIObject& obj);

// Client code:
GUIObject button;

// pUpperLeft points into the temporary Rectangle returned by boundingBox().
// At the semicolon, the temporary is destroyed, and pUpperLeft dangles.
const Point* pUpperLeft = &(boundingBox(button).upperLeft());

// UNDEFINED BEHAVIOR: dereferencing a dangling pointer.
std::cout << pUpperLeft->x << "\n";

```

The temporary Rectangle returned by boundingBox() is destroyed at the end of the full expression (the semicolon). The pointer pUpperLeft now points to memory that has been reclaimed. Any use of it is undefined behavior.

5.5.5 Real-World Dangling Handle Scenarios

Scenario 1: Returning references from containers of smart pointers

```

class WidgetCache {
public:
    // DANGEROUS: If the shared_ptr is reset or the Widget is removed from
    // the cache, the reference dangles.
    const Widget& getWidget(int id) const {
        auto it = cache_.find(id);
        if (it == cache_.end()) throw std::runtime_error("not found");
        return *(it->second); // Reference into the object owned by shared_ptr
    }

    void removeWidget(int id) {
        cache_.erase(id); // The Widget is destroyed here...
    }

private:
    std::map<int, std::shared_ptr<Widget>> cache_;
};

// Usage:
WidgetCache cache;
// cache.addWidget(1, std::make_shared<Widget>(/* ... */));

const Widget& w = cache.getWidget(1); // Reference is valid here.
cache.removeWidget(1);                // Widget is destroyed!
w.doSomething();                      // UNDEFINED BEHAVIOR: dangling reference.

// SAFER ALTERNATIVE: Return a shared_ptr so the caller shares ownership.
// std::shared_ptr<Widget> getWidget(int id) const {
//     auto it = cache_.find(id);
//     if (it == cache_.end()) return nullptr;
//     return it->second; // Caller holds a shared_ptr; Widget stays alive.
// }

```

Scenario 2: Returning iterators or pointers to internal containers

```

class MessageQueue {
public:
    // DANGEROUS: The returned pointer is invalidated if the internal
    // vector reallocates (e.g., on the next push).
    const std::string* front() const {
        if (messages_.empty()) return nullptr;
        return &messages_.front();
    }

    void push(const std::string& msg) {
        messages_.push_back(msg); // May reallocate, invalidating pointers!
    }

private:
    std::vector<std::string> messages_;
};

// Usage:
MessageQueue q;
q.push("hello");
const std::string* msg = q.front(); // Valid pointer.
q.push("world"); // Vector may reallocate!
std::cout << *msg << "\n"; // POTENTIALLY UNDEFINED BEHAVIOR.

// SAFER ALTERNATIVE: Return by value.
// std::string front() const {
//     if (messages_.empty()) throw std::runtime_error("empty");
//     return messages_.front(); // Returns a copy; no dangling possible.
// }

```

Scenario 3: Storing references returned from temporary expressions

```

class Config {
public:
    const std::string& getHostname() const { return hostname_; }
private:
    std::string hostname_ = "localhost";
};

// A factory function that returns a Config by value:
Config loadConfig();

// DANGEROUS:
const std::string& host = loadConfig().getHostname();
// The temporary Config is destroyed at the semicolon.
// 'host' is now a dangling reference.

// SAFE:
Config config = loadConfig(); // Keep the Config alive.
const std::string& host2 = config.getHostname(); // Now it is fine.

// ALSO SAFE:
std::string host3 = loadConfig().getHostname(); // Copy the string.

```

5.5.6 When Returning Handles Is Acceptable

There are cases where returning handles is appropriate:

```

// operator[] for containers MUST return a reference to be useful.
// This is an accepted, well-understood convention.
template <typename T>
class MyVector {
public:
    T& operator[](size_t index) { return data_[index]; }
    const T& operator[](size_t index) const { return data_[index]; }
private:
    T* data_;
    size_t size_;
};

// std::string::c_str() returns a pointer to internal data. This is
// acceptable because the documentation clearly states the pointer is
// invalidated by any non-const operation on the string, and the
// convention is universally understood.

```

Even `operator[]` and similar functions constitute exceptions to the rule, not refutations of it. The general guideline stands: avoid returning handles to internals whenever you can.

5.5.7 Things to Remember

- **Avoid returning handles** (references, pointers, iterators) **to object internals**. Not returning handles increases encapsulation, helps `const` member functions act `const`, and minimizes the creation of dangling handles.
- **If you must return a handle, return a `const` handle** to prevent callers from modifying the object's internals through the handle.
- **Even `const` handles can dangle**. A handle to data inside a temporary object becomes invalid when the temporary is destroyed. Be especially careful with function return values.
- **When handles are part of the interface contract** (like `operator[]`), document the lifetime guarantees clearly.

5.6 Item 29: Strive for exception-safe code

5.7 Visual Summary

ITEM 29: STRIVE FOR EXCEPTION-SAFE CODE
1. Operation starts → exception may interrupt at any point. 2. Basic guarantee → invariants hold and no resources leak. 3. Strong guarantee → operation either succeeds fully or changes nothing. 4. No-throw guarantee → operation never emits exceptions. 5. RAII + copy-and-swap + commit/rollback create clear failure boundaries. 6. Meaning: exception safety is about object state after failure.

Exception safety is not about whether your code throws exceptions. It is about how your code behaves when exceptions are thrown — possibly by code you call. Exception-safe functions offer one of three guarantees, and functions that offer no guarantee at all are not acceptable in well-written C++.

5.7.1 The Three Exception Safety Guarantees

1. The Basic Guarantee: If an exception is thrown, the program remains in a valid state. No resources are leaked, and all objects remain in a self-consistent state (i.e., all class invariants are satisfied). However,

the exact state of the program may not be predictable. For example, after an exception in a “change the background image” function, the old image might be displayed, or some default image, or something else — but the object is not corrupt.

2. The Strong Guarantee: If an exception is thrown, the state of the program is unchanged. Calls to functions offering the strong guarantee are atomic: they either succeed completely or have no effect at all. This is a “commit or rollback” model.

3. The Nothrow Guarantee: The function never throws exceptions. All operations on built-in types (ints, pointers, etc.) are nothrow. This is the strongest guarantee. Functions marked `noexcept` promise this guarantee.

5.7.2 A Motivating Example

```
// A class for GUI menus with a changeable background image.
class PrettyMenu {
public:
    void changeBackground(std::istream& imgSrc);

private:
    std::mutex mutex_;
    Image* bgImage_;           // Raw pointer: current background image
    int imageChanges_;         // Number of times image has been changed
};

// VERSION 1: Not exception-safe at all.
void PrettyMenu::changeBackground(std::istream& imgSrc) {
    mutex_.lock();             // Acquire mutex

    delete bgImage_;          // Destroy old image
    ++imageChanges_;           // Increment change counter
    bgImage_ = new Image(imgSrc); // Install new image

    mutex_.unlock();           // Release mutex
}
```

What goes wrong with Version 1:

1. **Resource leak:** If `new Image(imgSrc)` throws, `mutex_` is never unlocked. The mutex is leaked (permanently locked).
2. **Corrupted state:** If `new Image(imgSrc)` throws, `bgImage_` points to a deleted object (dangling pointer), and `imageChanges_` has already been incremented even though the image was never actually changed.

5.7.3 Fixing for the Basic Guarantee

```

// VERSION 2: Offers the basic guarantee using RAII and careful ordering.
void PrettyMenu::changeBackground(std::istream& imgSrc) {
    // Use lock_guard for RAII-based mutex management.
    // The mutex will be released when the function exits, whether
    // normally or via an exception.
    std::lock_guard<std::mutex> lock(mutex_);

    // Allocate the new image BEFORE deleting the old one.
    // If new throws, the old image is still intact.
    Image* newImage = new Image(imgSrc);

    delete bgImage_;           // Delete old image (only after new one succeeded)
    bgImage_ = newImage;       // Install new image (no-throw: pointer assignment)
    ++imageChanges_;           // Increment counter (no-throw: integer increment)
}

```

This is better: the mutex cannot leak, and `bgImage_` always points to a valid image. But it only offers the basic guarantee, not the strong guarantee. If `new Image` throws, the state is valid but the caller cannot know whether the image has changed or not.

5.7.4 Achieving the Strong Guarantee with Copy-and-Swap

The copy-and-swap idiom is the classic technique for achieving the strong guarantee:

```

// VERSION 3: Offers the strong guarantee using copy-and-swap.

// Step 1: Move the data that might change into a separate implementation struct.
struct PMImpl {
    std::shared_ptr<Image> bgImage;
    int imageChanges = 0;
};

class PrettyMenu {
public:
    void changeBackground(std::istream& imgSrc);

private:
    std::mutex mutex_;
    std::shared_ptr<PMImpl> pImpl_; // pImpl idiom (see Item 31)
};

void PrettyMenu::changeBackground(std::istream& imgSrc) {
    std::lock_guard<std::mutex> lock(mutex_);

    // STEP 1: Make a copy of the current state.
    auto pNew = std::make_shared<PMImpl>(*pImpl_);

    // STEP 2: Modify the copy. If this throws, the original is untouched.
    pNew->bgImage.reset(new Image(imgSrc)); // May throw
    ++pNew->imageChanges;                  // Won't throw

    // STEP 3: Swap the copy into place. swap for shared_ptr is noexcept.
    std::swap(pImpl_, pNew);

    // If we reach here, the change succeeded atomically.
    // If step 2 threw, we never reached step 3, so the original state
    // is completely unchanged --- the strong guarantee.
}

```

5.7.5 Copy-and-Swap as a General Pattern

```
// A strongly exception-safe assignment operator using copy-and-swap.
class String {
public:
    String(const char* s = "") : data_(new char[strlen(s) + 1]) {
        strcpy(data_, s);
    }

    ~String() { delete[] data_; }

    // Copy constructor: makes an independent copy.
    String(const String& rhs) : data_(new char[strlen(rhs.data_) + 1]) {
        strcpy(data_, rhs.data_);
    }

    // Copy-and-swap assignment operator.
    // Takes the parameter BY VALUE, which invokes the copy constructor.
    // Then we swap the copy's internals with ours.
    // This is exception-safe because:
    //   - The copy is made before any state is modified.
    //   - swap is noexcept (just swaps two pointers).
    //   - If the copy constructor throws, *this is unchanged.
    String& operator=(String rhs) { // Note: pass by value!
        swap(rhs);
        return *this;
    }

    void swap(String& rhs) noexcept {
        std::swap(data_, rhs.data_);
    }

private:
    char* data_;
};
```

5.7.6 The Nothrow Guarantee and noexcept

```
// Functions that should be noexcept:

// 1. Move constructors and move assignment operators
class Buffer {
public:
    Buffer(Buffer&& other) noexcept
        : data_(other.data_), size_(other.size_) {
        other.data_ = nullptr;
        other.size_ = 0;
    }

    Buffer& operator=(Buffer&& other) noexcept {
        if (this != &other) {
            delete[] data_;
            data_ = other.data_;
            size_ = other.size_;
            other.data_ = nullptr;
            other.size_ = 0;
        }
        return *this;
    }
}
```

```

// 2. Destructors (implicitly noexcept in C++11 and later)
~Buffer() noexcept {
    delete[] data_;
}

// 3. Swap functions
void swap(Buffer& other) noexcept {
    std::swap(data_, other.data_);
    std::swap(size_, other.size_);
}

private:
    char* data_ = nullptr;
    size_t size_ = 0;
};

// Why noexcept matters for performance:
// std::vector::push_back will use move semantics only if the move
// constructor is noexcept. Otherwise it falls back to copying, because
// a throwing move would leave the vector in an inconsistent state.

```

5.7.7 A Complex Real-World Example: Transaction Processing

```

class Database {
public:
    // Strong guarantee: either the entire transaction succeeds,
    // or the database is unchanged.
    void executeTransaction(const std::vector<Operation>& ops) {
        // Step 1: Create a snapshot (copy) of affected data.
        Snapshot snapshot = createSnapshot(ops);

        // Step 2: Apply operations to the snapshot.
        // If any operation throws, the real data is untouched.
        for (const auto& op : ops) {
            applyToSnapshot(snapshot, op);    // May throw
        }

        // Step 3: Commit the snapshot (swap into place).
        // This step must be noexcept.
        commitSnapshot(std::move(snapshot)); // noexcept
    }

private:
    Snapshot createSnapshot(const std::vector<Operation>& ops);
    void applyToSnapshot(Snapshot& snap, const Operation& op);
    void commitSnapshot(Snapshot&& snap) noexcept;
};

```

5.7.8 RAI: The Foundation of Exception Safety

RAII (Resource Acquisition Is Initialization) is the single most important technique for writing exception-safe code. Every resource should be managed by an object whose destructor releases it.


```

// BAD: Manual resource management is not exception-safe.
void processFile(const std::string& filename) {
    FILE* fp = fopen(filename.c_str(), "r");
    if (!fp) throw std::runtime_error("Cannot open file");

    char* buffer = new char[4096];

    // If readData throws, both fp and buffer are leaked.
    readData(fp, buffer);

    delete[] buffer;
    fclose(fp);
}

// GOOD: RAII manages all resources.
void processFile(const std::string& filename) {
    // ifstream closes itself on destruction (RAII).
    std::ifstream file(filename);
    if (!file) throw std::runtime_error("Cannot open file");

    // vector manages its own memory (RAII).
    std::vector<char> buffer(4096);

    // If readData throws, both file and buffer are cleaned up
    // automatically by their destructors.
    readData(file, buffer);
}

```

5.7.9 When the Strong Guarantee Is Not Practical

The strong guarantee cannot always be achieved efficiently. Consider a function that operates on two objects:

```

void transferFunds(Account& from, Account& to, double amount) {
    from.withdraw(amount);    // Strong guarantee on 'from'
    to.deposit(amount);       // Strong guarantee on 'to'
}

```

Even though both `withdraw` and `deposit` individually offer the strong guarantee, `transferFunds` as a whole does not. If `deposit` throws after `withdraw` has succeeded, rolling back the withdrawal requires calling `from.deposit(amount)`, which itself could throw.

In such cases, the basic guarantee is often the practical choice. The strong guarantee would require either: - A copy-and-swap of both `Account` objects (possibly expensive), or - A transactional log with undo/redo capability (complex).

5.7.10 Things to Remember

- **Exception-safe functions leak no resources and allow no data structures to become corrupted when exceptions are thrown**, even when those functions call other functions that might throw.
- **The strong guarantee can often be implemented via copy-and-swap**, but the strong guarantee is not practical for all functions (especially those that modify multiple independent objects).
- **A function can usually offer a guarantee no stronger than the weakest guarantee of the functions it calls**. If your function calls a function offering only the basic guarantee, the best your function can generally offer is the basic guarantee.
- **Use RAII to manage resources**. `lock_guard`, `unique_ptr`, `shared_ptr`, `fstream`, and similar types ensure cleanup happens automatically.
- **Mark functions `noexcept` when they truly cannot throw**, especially move operations, swap functions, and destructors.

5.8 Item 30: Understand the ins and outs of inlining

5.9 Visual Summary

ITEM 30: UNDERSTAND THE INS AND OUTS OF INLINING

1. inline request -> compiler may expand function at call site.
2. Benefit -> remove call overhead and expose optimization.
3. Cost -> larger binaries, more compile dependencies, ABI churn.
4. Virtual/function pointer/complex code -> often cannot inline usefully.
5. Meaning: inline tiny stable hot functions, measure the rest.

Inline functions — what a wonderful idea! They look like functions, they act like functions, they are vastly better than macros (see Item 2), and you can call them without incurring the overhead of a function call. What is not to love?

Actually, quite a lot. The basic idea behind inlining is to replace each call of a function with its code body, and this has implications you need to understand before you wield this tool.

5.9.1 What Inlining Actually Does

When a function is inlined, the compiler replaces every call site with a copy of the function body. This eliminates:

- The function call overhead (pushing arguments, jumping, returning)
- The opportunity for instruction cache misses at the call target

But it also means:

- The generated code is larger (one copy per call site)
- Larger code means more instruction cache pressure
- More instruction cache misses can slow down the program

The net effect of inlining can be positive or negative. For small functions, inlining almost always wins. For large functions, inlining almost always loses.

5.9.2 Implicit Inlining: Functions Defined Inside a Class

Functions defined inside a class definition are implicitly declared inline:

```
class Person {
public:
    // Implicitly inline: defined inside the class body.
    int age() const { return age_; }

    // Implicitly inline.
    const std::string& name() const { return name_; }

    // This is also implicitly inline, but it is too complex to be
    // a good candidate for inlining. The compiler may choose NOT
    // to inline it despite the implicit request.
    std::string toString() const {
        std::ostringstream oss;
        oss << name_ << " (age " << age_ << ")";
        return oss.str();
    }

private:
    std::string name_;
    int age_;
};
```

Friend functions defined inside a class are also implicitly inline:

```

class Widget {
public:
    // This friend function is defined inside the class, so it is
    // implicitly inline.
    friend bool operator==(const Widget& lhs, const Widget& rhs) {
        return lhs.id_ == rhs.id_;
    }

private:
    int id_;
};

```

5.9.3 Explicit Inlining: The inline Keyword

```

// Explicit inline function.
inline int square(int x) { return x * x; }

// Explicit inline template function.
// (Note: templates are often defined in headers anyway, but they
// are NOT automatically inline just because they are templates.)
template <typename T>
inline T clamp(T value, T lo, T hi) {
    return (value < lo) ? lo : (value > hi) ? hi : value;
}

```

Important: The inline keyword is a request, not a command. The compiler is free to ignore it. Most modern compilers make their own inlining decisions based on heuristics (function size, call frequency, optimization level) and largely ignore the inline keyword for optimization purposes. The inline keyword's primary practical effect in modern C++ is to relax the one-definition rule (ODR), allowing the function to be defined in a header included by multiple translation units.

5.9.4 What the Compiler Cannot Inline

1. Virtual function calls (through a base pointer/reference):

```

class Shape {
public:
    virtual double area() const = 0;
};

class Circle : public Shape {
public:
    // Even though this is defined in the class (implicitly inline),
    // calls through a Shape pointer/reference cannot be inlined because
    // the compiler does not know (at compile time) which function to call.
    double area() const override { return 3.14159 * r_ * r_; }
private:
    double r_ = 1.0;
};

void printArea(const Shape& s) {
    // Virtual call --- cannot be inlined (in general).
    std::cout << s.area() << "\n";
}

void printCircleArea(const Circle& c) {
    // Non-virtual call --- CAN be inlined because the compiler knows
    // the exact type is Circle.
    std::cout << c.area() << "\n";
}

```

2. Calls through function pointers:

```
inline int add(int a, int b) { return a + b; }

int (*funcPtr)(int, int) = add;

int result1 = add(3, 4);           // Likely inlined
int result2 = funcPtr(3, 4);       // Likely NOT inlined: called via pointer
```

3. Recursive functions (cannot be fully inlined):

```
// The compiler may inline one or two levels of recursion, but it
// cannot inline all of them (there are infinitely many).
inline unsigned factorial(unsigned n) {
    return (n <= 1) ? 1 : n * factorial(n - 1);
}
```

5.9.5 Constructors and Destructors: Deceptively Complex

Constructors and destructors are often poor candidates for inlining, even if they appear short, because the compiler inserts substantial hidden code:

```
class Base {
public:
    Base() { std::cout << "Base constructed\n"; }
    ~Base() { std::cout << "Base destroyed\n"; }
private:
    std::string name_;
    std::vector<int> data_;
};

class Derived : public Base {
public:
    // This constructor LOOKS empty, but the compiler generates code to:
    // 1. Call Base::Base()
    // 2. Construct value1_ (call std::string constructor)
    // 3. Construct value2_ (call std::string constructor)
    // 4. If step 2 or 3 throws, destroy already-constructed members
    //    and call Base::~~Base()
    Derived() {} // Implicitly inline, but the generated code is large!

    ~Derived() {} // Similarly, the destructor must destroy value2_,
                  // then value1_, then call Base::~~Base().

private:
    std::string value1_;
    std::string value2_;
};
```

The compiler-generated code for `Derived::Derived()` might look something like:

```
// Pseudocode: what the compiler actually generates for Derived::Derived()
Derived::Derived() {
    Base::Base(); // Construct base class

    try {
        value1_.std::string::string(); // Construct first member
    } catch (...) {
        Base::~~Base(); // Destroy base if member construction fails
        throw;
    }

    try {
        value2_.std::string::string(); // Construct second member
    } catch (...) {
        value1_.std::string::~~string(); // Destroy first member
        Base::~~Base(); // Destroy base
        throw;
    }
}
```

Inlining this at every call site would generate a lot of code.

5.9.6 Inlining and Libraries: Binary Compatibility

Inlined functions are baked into the caller's object code. If you change the body of an inline function in a library, every client that uses the function must recompile. With a non-inline function, clients need only relink.

```
// header: mathlib.h (version 1)
inline double computeTax(double income) {
    return income * 0.25; // 25% flat tax
}

// If you change this to:
inline double computeTax(double income) {
    return income * 0.30; // 30% flat tax
}

// Every source file that #includes mathlib.h must be RECOMPILED.
// With a non-inline function, you would only need to recompile
// mathlib.cpp and relink.
```

This is a serious concern for library authors. Inline functions in public headers become part of your binary interface.

5.9.7 Inlining and Debugging

Most debuggers cannot set breakpoints inside inlined functions. The inlined code exists at the call site, not as a separate function, so stepping “into” an inlined function does not work the way you expect. During development and debugging, it can be helpful to minimize inlining (compile with `-O0` or equivalent).

5.9.8 Guidelines for When to Inline

```

// GOOD candidates for inlining:
// Small, frequently called functions with simple bodies.
class Point {
public:
    double x() const { return x_; }    // Trivial accessor
    double y() const { return y_; }    // Trivial accessor

    void setX(double x) { x_ = x; }    // Trivial mutator
    void setY(double y) { y_ = y; }    // Trivial mutator

    // Small computation
    double distanceTo(const Point& other) const {
        double dx = x_ - other.x_;
        double dy = y_ - other.y_;
        return std::sqrt(dx * dx + dy * dy);
    }

private:
    double x_ = 0.0;
    double y_ = 0.0;
};

// BAD candidates for inlining:
// Large functions, functions that are rarely called, virtual functions
// called through base pointers, constructors/destructors of complex classes.

class DatabaseConnection {
public:
    // Do NOT inline: complex function with error handling, logging,
    // resource management, etc.
    void executeQuery(const std::string& sql) {
        validateConnection();
        logQuery(sql);
        auto result = sendToServer(sql);
        parseResponse(result);
        updateStatistics();
    }
};

```

5.9.9 constexpr and Inlining (C++11 and later)

constexpr functions are implicitly inline (in the ODR sense), and when evaluated at compile time, they are “inlined” in the most extreme way possible: the function call is replaced with a constant:

```
constexpr int fibonacci(int n) {
    if (n <= 1) return n;
    return fibonacci(n - 1) + fibonacci(n - 2);
}

// At compile time, this is evaluated to 55. No function call at runtime.
constexpr int fib10 = fibonacci(10);

// At runtime, this may or may not be inlined, just like any other function.
int n;
std::cin >> n;
int result = fibonacci(n); // Runtime call
```

5.9.10 Things to Remember

- **Limit most inlining to small, frequently called functions.** This facilitates debugging and binary upgradability, minimizes potential code bloat, and maximizes the chances of greater program speed.
- **Do not declare function templates inline just because they appear in header files.** Templates in headers are there because the compiler needs to see the definition to instantiate them, not because they should be inlined. Only add `inline` if the function is genuinely a good inlining candidate.
- **The `inline` keyword is a request, not a command.** The compiler decides whether to actually inline a function based on its own heuristics. Many compilers issue warnings (at the right warning level) when they fail to inline a function you asked to be inlined.
- **Constructors and destructors are often poor inlining candidates**, because the compiler-generated code for them is often much longer than it appears in source.

5.10 Item 31: Minimize compilation dependencies between files

5.11 Visual Summary

ITEM 31: MINIMIZE COMPILETIME DEPENDENCIES BETWEEN FILES
1. Header includes concrete details -> every change recompiles users. 2. Forward declarations -> users see names without full definitions. 3. Pimpl/interface classes -> hide representation in implementation file. 4. Tradeoff -> more indirection for lower build coupling. 5. Meaning: separate interface knowledge from implementation knowledge.

C++ does not do a great job of separating interfaces from implementations. A class definition includes not just the interface (public member functions) but also a substantial amount of implementation detail (private members, private functions). This means that if you change a private member of a class, every file that `#includes` that class's header must be recompiled — even if no client code uses the private member.

In a large project, this can lead to devastating build times. A single change to a private data member in a core header can trigger recompilation of hundreds of source files.

5.11.1 The Problem

```

// person.h
#include <string>           // Needed for std::string data member
#include <memory>           // Needed for std::shared_ptr
#include "date.h"          // Needed for Date data member
#include "address.h"       // Needed for Address data member

class Person {
public:
    Person(const std::string& name, const Date& birthday,
           const Address& addr);

    std::string name() const;
    std::string birthDate() const;
    std::string address() const;

private:
    std::string name_;      // Implementation detail!
    Date birthDate_;        // Implementation detail!
    Address address_;       // Implementation detail!
};

```

The dependency chain:

Any file that #includes person.h also transitively includes <string>, <memory>, date.h, and address.h. If date.h or address.h changes, every file that includes person.h must be recompiled — even if the change was to a private implementation detail of Date or Address.

If Person is a widely used class, this can mean that changing a private member of Address triggers recompilation of your entire project.

5.11.2 Solution 1: The plmpl (Pointer to Implementation) Idiom

The key insight: **you can replace data members with a pointer to a struct that contains them.** The class definition then depends only on a forward declaration of the implementation struct, not on the full definitions of the member types.

```

// =====
// person.h --- the public header (interface)
// =====
#include <string>
#include <memory>

// Forward declarations: no #include needed for Date or Address!
class Date;
class Address;

class Person {
public:
    Person(const std::string& name, const Date& birthday,
           const Address& addr);
    ~Person(); // Must be declared (see below)

    // Move and copy operations must also be declared here if you want
    // them, because the compiler cannot generate them in headers that
    // do not see the full definition of PersonImpl.
    Person(const Person& rhs);
    Person& operator=(const Person& rhs);
    Person(Person&& rhs) noexcept;
    Person& operator=(Person&& rhs) noexcept;

    std::string name() const;

```



```

    std::string birthDate() const;
    std::string address() const;

private:
    // The only data member: a pointer to the implementation.
    // This is the "pImpl" (pointer-to-implementation).
    struct Impl; // Forward declaration of nested struct
    std::unique_ptr<Impl> pImpl_; // Pointer to implementation
};

// =====
// person.cpp --- the implementation file
// =====
#include "person.h"
#include "date.h" // Now these includes are only in the .cpp file.
#include "address.h" // Changes to date.h or address.h only trigger
                    // recompilation of person.cpp, NOT of all the
                    // files that include person.h.

// Define the implementation struct.
struct Person::Impl {
    std::string name;
    Date birthDate;
    Address address;

    Impl(const std::string& n, const Date& bd, const Address& a)
        : name(n), birthDate(bd), address(a) {}
};

// Constructor: create the Impl on the heap.
Person::Person(const std::string& name, const Date& birthday,
               const Address& addr)
    : pImpl_(std::make_unique<Impl>(name, birthday, addr)) {}

// Destructor: must be defined in the .cpp file where Impl is complete.
// If it were defined in the header (or defaulted in the header), the
// compiler would try to generate ~unique_ptr<Impl>, which needs the
// full definition of Impl. This would fail or produce undefined behavior.
Person::~Person() = default;

// Copy constructor: deep copy the Impl.
Person::Person(const Person& rhs)
    : pImpl_(std::make_unique<Impl>(*rhs.pImpl_)) {}

// Copy assignment: deep copy the Impl.
Person& Person::operator=(const Person& rhs) {
    if (this != &rhs) {
        *pImpl_ = *rhs.pImpl_;
    }
    return *this;
}

// Move constructor.
Person::Person(Person&& rhs) noexcept = default;

// Move assignment.
Person& Person::operator=(Person&& rhs) noexcept = default;

// Member functions: delegate to Impl.
std::string Person::name() const { return pImpl_>name; }

```

```
std::string Person::birthDate() const { return pImpl_>birthDate.toString(); }
std::string Person::address() const { return pImpl_>address.toString(); }
```

What we achieved: Files that #include "person.h" no longer depend on date.h or address.h. If we change Date or Address (or even add new private data members to Person), only person.cpp needs to be recompiled.

5.11.3 Solution 2: Abstract Base Classes (Interface Classes)

An alternative to pImpl is to define the interface as an abstract base class with pure virtual functions, and provide the implementation in a derived class visible only in the .cpp file:

```
// =====
// person.h --- the public header (interface class)
// =====
#include <string>
#include <memory>

class Date;      // Forward declaration
class Address;   // Forward declaration

// Person is an abstract base class (interface).
// It has no data members, so it depends on nothing except the
// types used in its public interface.
class Person {
public:
    virtual ~Person() = default;

    virtual std::string name() const = 0;
    virtual std::string birthDate() const = 0;
    virtual std::string address() const = 0;

    // Factory function: clients call this to create Person objects.
    // They never see the concrete class.
    static std::unique_ptr<Person> create(
        const std::string& name,
        const Date& birthday,
        const Address& addr
    );
};

// =====
// person.cpp --- the implementation file
// =====
#include "person.h"
#include "date.h"
#include "address.h"

// RealPerson is the concrete implementation. It is NOT visible
// in person.h, so clients have no dependency on it.
class RealPerson : public Person {
public:
    RealPerson(const std::string& name, const Date& bd, const Address& a)
        : name_(name), birthDate_(bd), address_(a) {}

    std::string name() const override { return name_; }
    std::string birthDate() const override { return birthDate_.toString(); }
    std::string address() const override { return address_.toString(); }

private:
    std::string name_;
```

```

    Date birthDate_;
    Address address_;
};

// Factory function implementation.
std::unique_ptr<Person> Person::create(
    const std::string& name, const Date& birthday, const Address& addr)
{
    return std::make_unique<RealPerson>(name, birthday, addr);
}

```

Usage:

```

// client.cpp
#include "person.h"
// No need to include date.h or address.h here if we use forward declarations
// or if Date and Address are only used to pass to the factory.

#include "date.h" // Needed here only because we construct Date/Address objects
#include "address.h"

void processPersons() {
    auto p = Person::create(
        "Alice",
        Date(1990, 3, 15),
        Address("123 Main St", "Springfield", "IL")
    );

    std::cout << p->name() << "\n";
    std::cout << p->birthDate() << "\n";
    std::cout << p->address() << "\n";
}

```

5.11.4 plmpl vs. Interface Classes: Trade-offs

Aspect	plmpl	Interface Class
Runtime cost	One indirection (pointer dereference) per member access	Virtual function call per member access (vtable lookup)
Memory	Extra heap allocation for Impl	Extra vptr per object
Extensibility	Internal; clients cannot extend	Clients can derive new implementations
Binary compatibility	Excellent: adding private members does not break ABI	Excellent: adding new pure virtual functions breaks ABI, but adding non-pure virtual functions may not
Boilerplate	Must forward all public functions to Impl	Must implement all pure virtual functions in derived class

5.11.5 Forward Declarations: The Foundation of Dependency Reduction

The key principle is: **depend on declarations, not definitions**. You can use forward declarations instead of `#include` whenever you only need to know that a type exists, not what it contains:

```
// You CAN use a forward declaration when:
class Widget;    // Forward declaration is enough for:

// Widget* ptr;           // Pointers to Widget
// Widget& ref;           // References to Widget (in declarations)
// Widget func();         // Functions that return Widget by value (declaration only)
// void func(Widget w);    // Functions that take Widget by value (declaration only)
// std::unique_ptr<Widget>; // Smart pointers to Widget (with caveats)

// You CANNOT use a forward declaration when:
// - You need to know the size of Widget (to allocate it on the stack)
// - You need to call Widget's member functions
// - You need to access Widget's data members
// - You need to inherit from Widget
```

5.11.6 A Comprehensive Real-World Example

Consider a graphics engine with complex interdependencies:

```
// =====
// BEFORE: Tight coupling --- one change rebuilds everything
// =====

// renderer.h
#include "texture.h"    // Full definition of Texture
#include "shader.h"     // Full definition of Shader
#include "mesh.h"       // Full definition of Mesh
#include "camera.h"     // Full definition of Camera
#include "light.h"      // Full definition of Light

class Renderer {
public:
    void render(const Camera& cam, const std::vector<Light>& lights);
    void loadTexture(const std::string& path);
    void setShader(const Shader& shader);
    void addMesh(const Mesh& mesh);

private:
    Texture currentTexture_;    // Requires full definition of Texture
    Shader currentShader_;     // Requires full definition of Shader
    std::vector<Mesh> meshes_;  // Requires full definition of Mesh
    Camera mainCamera_;        // Requires full definition of Camera
    std::vector<Light> lights_; // Requires full definition of Light
};

// Any change to Texture, Shader, Mesh, Camera, or Light forces
// recompilation of renderer.h and EVERY file that includes it.
```

```

// =====
// AFTER: Loose coupling with pImpl
// =====

// renderer.h
#include <string>
#include <memory>
#include <vector>

// Forward declarations --- no #includes for our types!
class Texture;
class Shader;
class Mesh;
class Camera;
class Light;

class Renderer {
public:
    Renderer();
    ~Renderer();
    Renderer(Renderer&&) noexcept;
    Renderer& operator=(Renderer&&) noexcept;

    void render(const Camera& cam, const std::vector<Light*>& lights);
    void loadTexture(const std::string& path);
    void setShader(const Shader& shader);
    void addMesh(const Mesh& mesh);

private:
    struct Impl;
    std::unique_ptr<Impl> pImpl_;
};

// renderer.cpp
#include "renderer.h"
#include "texture.h"      // Includes are now localized to the .cpp file.
#include "shader.h"
#include "mesh.h"
#include "camera.h"
#include "light.h"

struct Renderer::Impl {
    Texture currentTexture;
    Shader currentShader;
    std::vector<Mesh> meshes;
    Camera mainCamera;
    std::vector<Light> lights;
};

Renderer::Renderer() : pImpl_(std::make_unique<Impl>()) {}
Renderer::~~Renderer() = default;
Renderer::Renderer(Renderer&&) noexcept = default;
Renderer& Renderer::operator=(Renderer&&) noexcept = default;

void Renderer::render(const Camera& cam, const std::vector<Light*>& lights) {
    pImpl_>mainCamera = cam;
    // ... rendering logic using pImpl_>currentShader, pImpl_>meshes, etc.
}

void Renderer::loadTexture(const std::string& path) {

```

```

    pImpl->currentTexture.loadFromFile(path);
}

void Renderer::setShader(const Shader& shader) {
    pImpl->currentShader = shader;
}

void Renderer::addMesh(const Mesh& mesh) {
    pImpl->meshes.push_back(mesh);
}

```

Now, changing Texture, Shader, Mesh, Camera, or Light only requires recompiling `renderer.cpp` — not the hundreds of files that include `renderer.h`.

5.11.7 Practical Tips for Reducing Compilation Dependencies

1. Prefer forward declarations to `#include` in headers:

```

// BAD: header includes everything
// widget.h
#include "gadget.h"           // Only needed for Gadget* parameter

class Widget {
public:
    void useGadget(Gadget* g); // Only uses pointer; forward decl suffices
};

// GOOD: forward declare in header, include in source
// widget.h
class Gadget; // Forward declaration

class Widget {
public:
    void useGadget(Gadget* g);
};

// widget.cpp
#include "widget.h"
#include "gadget.h" // Include only where the full definition is needed

void Widget::useGadget(Gadget* g) {
    g->activate(); // Need full definition here, which we have via #include
}

```

2. Use `<iosfwd>` instead of `<iostream>` in headers:

```

// BAD: <iostream> is a heavy header
#include <iostream>

class Logger {
public:
    void log(std::ostream& os, const std::string& msg);
};

// GOOD: <iosfwd> is a lightweight header with only forward declarations
#include <iosfwd>
#include <string>

class Logger {
public:
    void log(std::ostream& os, const std::string& msg);
};

// logger.cpp
#include "logger.h"
#include <iostream>    // Full definition needed only in the implementation

```

3. Provide separate “fwd” headers for your own types:

```

// geometry_fwd.h --- lightweight forward declarations
class Point;
class Line;
class Circle;
class Rectangle;
class Polygon;

// geometry.h --- full definitions
#include "geometry_fwd.h"

class Point {
    double x_, y_;
public:
    Point(double x = 0, double y = 0);
    // ...
};

class Line {
    Point start_, end_;
public:
    Line(const Point& s, const Point& e);
    // ...
};
// ... etc.

// client.h --- only needs forward declarations
#include "geometry_fwd.h"    // Lightweight: no dependency on geometry details

class Renderer {
public:
    void drawPoint(const Point& p);
    void drawLine(const Line& l);
};

// client.cpp --- needs full definitions for implementation
#include "client.h"
#include "geometry.h"    // Full definitions needed here

```

```
void Renderer::drawPoint(const Point& p) {
    // ... uses Point's members
}
```

4. Use `unique_ptr` rather than `shared_ptr` for plmpl:

```
// Prefer unique_ptr for pImpl. It has no overhead beyond a raw pointer
// and clearly expresses sole ownership.

class Widget {
public:
    Widget();
    ~Widget(); // Must be declared in header, defined in .cpp where Impl is complete
    Widget(Widget&&) noexcept;
    Widget& operator=(Widget&&) noexcept;

private:
    struct Impl;
    std::unique_ptr<Impl> pImpl_; // Preferred
};

// Note: shared_ptr does not require the destructor trick because
// shared_ptr stores its deleter at construction time. But shared_ptr
// has higher overhead (reference counting, separate control block),
// and shared ownership semantics are usually not what you want for pImpl.
```

5. Include-what-you-use (IWYU) discipline:

```
// Every header should include exactly the headers it needs and no more.
// Every source file should include the headers for every type it uses
// directly, not relying on transitive includes.

// BAD: relying on transitive include
// foo.h includes <vector>
#include "foo.h"
std::vector<int> v; // Works by accident (through foo.h's include)

// GOOD: include what you use
#include "foo.h"
#include <vector> // Explicitly include what you use
std::vector<int> v; // Works by design
```

5.11.8 Things to Remember

- **The general idea behind minimizing compilation dependencies is to depend on declarations instead of definitions.** Two approaches based on this idea are the plmpl idiom and interface classes.
- **Library header files should exist in full and declaration-only forms.** The Standard Library provides `<iosfwd>` as a model. Your own libraries should provide similar `_fwd.h` headers.
- **The plmpl idiom replaces data members with a pointer to an implementation struct.** This moves `#include` dependencies from the header to the source file, making recompilation cheaper when implementation details change.
- **Interface classes (abstract base classes with factory functions) achieve the same decoupling.** Clients program to the abstract interface and never see the concrete implementation class.
- **Both plmpl and interface classes have a small runtime cost** (pointer indirection and heap allocation for plmpl; virtual function dispatch for interface classes), but the improvement in build times for large projects more than compensates.

6 Chapter 6: Inheritance and Object-Oriented Design

6.1 Chapter Flow

CHAPTER 6: INHERITANCE AND OBJECT-ORIENTED DESIGN

Item 32 -> public inheritance means substitutable is-a.
Items 33-37 -> understand name hiding, virtual contracts, non-virtual invariants, defaults.
Items 38-39 -> prefer composition; use private inheritance only for implementation needs.
Item 40 -> use multiple inheritance only when ambiguity and diamond costs are controlled.

Inheritance and object-oriented design in C++ is deceptively rich. The language offers public, protected, and private inheritance; single and multiple inheritance; virtual and non-virtual functions; pure virtual, simple virtual, and non-virtual member functions; and interactions between inheritance and other language features such as default parameter values and names in enclosing scopes. This chapter explains what these features really mean – not what they look like, but what they actually *express* in a well-designed C++ program.

6.2 Item 32: Make sure public inheritance models “is-a”

6.3 Visual Summary

ITEM 32: MAKE SURE PUBLIC INHERITANCE MODELS "IS-A"

1. public Derived : Base -> every Derived must be usable as a Base.
2. If base promises behavior -> derived must honor it for all callers.
3. Square-rectangle style mismatch -> mathematical relation is not enough.
4. If not true is-a -> use composition or private inheritance.
5. Meaning: public inheritance is a substitutability contract.

6.3.1 The Fundamental Rule

Public inheritance means “**is-a**.” Every object of the derived class *is* an object of the base class. The base class represents a more general concept; the derived class represents a more specialized concept. Anywhere the base class can appear, the derived class can appear in its place. This is a direct expression of the **Liskov Substitution Principle (LSP)**.

```
class Person { ... };
class Student : public Person { ... };
```

Every student is a person, but not every person is a student. This is precisely what C++ expresses via public inheritance. Any function expecting a Person (or Person&, or Person*) will also accept a Student (or Student&, or Student*).

```
void eat(const Person& p);      // anyone can eat
void study(const Student& s);   // only students study

Person p;
Student s;

eat(p);      // fine -- p is a Person
eat(s);      // fine -- s is a Person (Student is-a Person)
study(s);    // fine -- s is a Student
study(p);    // ERROR -- p is not a Student
```

6.3.2 The Classic Pitfall: Penguins and Birds

Consider this seemingly reasonable hierarchy:

```
// BAD -- violates Liskov Substitution Principle
class Bird {
public:
    virtual void fly() {
        std::cout << "Flying through the air!\n";
    }
};

class Penguin : public Bird {
    // Penguins can't fly, but Bird says they can!
};
```

A penguin is a bird, but penguins cannot fly. The hierarchy above says that all birds can fly, which is simply wrong. There are several ways to handle this.

Approach 1: Refine the hierarchy

```
// GOOD -- more accurate hierarchy
class Bird {
public:
    virtual ~Bird() = default;
    // ... general bird things (lay eggs, have feathers, etc.)
};

class FlyingBird : public Bird {
public:
    virtual void fly() {
        std::cout << "Flying through the air!\n";
    }
};

class NonFlyingBird : public Bird {
    // no fly() member at all
};

class Penguin : public NonFlyingBird {
    // penguins are non-flying birds
};

class Eagle : public FlyingBird {
```

```

    // eagles can fly
};

```

Approach 2: Runtime error (less desirable but sometimes pragmatic)

```

// ACCEPTABLE but not ideal -- runtime enforcement
class Bird {
public:
    virtual ~Bird() = default;
    virtual void fly() {
        std::cout << "Flying through the air!\n";
    }
};

class Penguin : public Bird {
public:
    void fly() override {
        throw std::runtime_error("Penguins can't fly!");
    }
};

```

This says: “Penguins can try to fly, but it’s an error for them to actually do it.” The problem is moved from compile time to runtime, which is inferior but sometimes necessary when the hierarchy cannot be restructured.

6.3.3 The Rectangle/Square Problem

This is another classic violation:

```

// BAD -- Square is NOT a behavioral substitute for Rectangle
class Rectangle {
public:
    virtual void setHeight(int h) { height_ = h; }
    virtual void setWidth(int w) { width_ = w; }
    int getHeight() const { return height_; }
    int getWidth() const { return width_; }
    int area() const { return height_ * width_; }

private:
    int height_;
    int width_;
};

class Square : public Rectangle {
public:
    // Must maintain the invariant: height == width
    void setHeight(int h) override {
        Rectangle::setHeight(h);
        Rectangle::setWidth(h);    // force width = height
    }
    void setWidth(int w) override {
        Rectangle::setHeight(w);    // force height = width
        Rectangle::setWidth(w);
    }
};

```

Now consider any code that uses a Rectangle&:

```
void makeBigger(Rectangle& r) {
    int oldHeight = r.getHeight();
    r.setWidth(r.getWidth() + 10);
    assert(r.getHeight() == oldHeight); // FAILS for Square!
}
```

The postcondition that `setWidth` does not change the height is violated by `Square`. Mathematically a square is a rectangle, but *behaviorally* a `Square` object cannot be substituted for a `Rectangle` object. This is a Liskov Substitution Principle violation.

// GOOD -- separate types, or use a single Shape with appropriate constraints

```
class Shape {
public:
    virtual ~Shape() = default;
    virtual int area() const = 0;
};

class Rectangle : public Shape {
public:
    Rectangle(int w, int h) : width_(w), height_(h) {}
    void setHeight(int h) { height_ = h; }
    void setWidth(int w) { width_ = w; }
    int area() const override { return width_ * height_; }
private:
    int width_, height_;
};

class Square : public Shape {
public:
    explicit Square(int side) : side_(side) {}
    void setSide(int s) { side_ = s; }
    int area() const override { return side_ * side_; }
private:
    int side_;
};
```

6.3.4 Real-World Example: Vehicle Hierarchy

// GOOD -- well-modeled public inheritance hierarchy

```
class Vehicle {
public:
    virtual ~Vehicle() = default;
    virtual void start() = 0;
    virtual void stop() = 0;
    virtual double fuelRemaining() const = 0;
    virtual std::string description() const = 0;
};

class Car : public Vehicle {
public:
    void start() override {
        ignition_ = true;
        std::cout << "Car engine started\n";
    }
    void stop() override {
        ignition_ = false;
        std::cout << "Car engine stopped\n";
    }
    double fuelRemaining() const override { return fuelGallons_; }
    std::string description() const override { return "Sedan"; }
```

```

    // Car-specific
    void openTrunk() { /* ... */ }

private:
    bool ignition_ = false;
    double fuelGallons_ = 15.0;
};

class Truck : public Vehicle {
public:
    void start() override {
        dieselPreheat();
        ignition_ = true;
        std::cout << "Truck diesel engine started\n";
    }
    void stop() override {
        ignition_ = false;
        std::cout << "Truck engine stopped\n";
    }
    double fuelRemaining() const override { return dieselGallons_; }
    std::string description() const override { return "18-Wheeler"; }

    // Truck-specific
    void attachTrailer() { /* ... */ }

private:
    void dieselPreheat() { /* ... */ }
    bool ignition_ = false;
    double dieselGallons_ = 150.0;
};

// Any function dealing with Vehicle works seamlessly with both
void refuelIfNeeded(Vehicle& v) {
    if (v.fuelRemaining() < 5.0) {
        std::cout << v.description() << " needs refueling!\n";
    }
}

```

6.3.5 Things to Remember

- Public inheritance means “is-a.” Everything that applies to base class objects must also apply to derived class objects, because every derived class object *is* a base class object.
- The Liskov Substitution Principle states: if D is publicly derived from B, then any property provable about objects of type B should also be provable about objects of type D.
- Intuitions from the real world (penguins are birds, squares are rectangles) do not always translate correctly into class hierarchies. What matters is *behavioral* substitutability, not taxonomic relationships.

6.4 Item 33: Avoid hiding inherited names

6.5 Visual Summary

ITEM 33: AVOID HIDING INHERITED NAMES

1. Derived declares function named `f` → base overloads named `f` become hidden.
2. Caller expects overload set → only derived names are visible.
3. using `Base::f` → reintroduces base overloads.
4. Forwarding functions → expose selected base functions only.
5. Meaning: name lookup is by name first, not by full signature first.

6.5.1 How Name Hiding Works in C++

C++ name hiding in inheritance follows the same principle as name hiding in nested scopes: names in an inner scope hide names in an outer scope. The types and parameter lists are irrelevant – it is purely about the *name*.

```
// Demonstration of scope-based name hiding
int x = 5;           // global x

void someFunc() {
    double x = 3.14; // local x HIDES global x
    std::cout << x;  // uses the local double x, not global int x
}
```

The same thing happens with inheritance:

```
// BAD -- derived class hides base class overloads
class Base {
public:
    virtual void mf1() { std::cout << "Base::mf1()\n"; }
    virtual void mf1(int x) { std::cout << "Base::mf1(int)\n"; }

    virtual void mf2() { std::cout << "Base::mf2()\n"; }

    void mf3() { std::cout << "Base::mf3()\n"; }
    void mf3(double d) { std::cout << "Base::mf3(double)\n"; }
};

class Derived : public Base {
public:
    void mf1() override { // hides ALL Base::mf1 overloads!
        std::cout << "Derived::mf1()\n";
    }

    void mf3() { // hides ALL Base::mf3 overloads!
        std::cout << "Derived::mf3()\n";
    }
};

Derived d;
d.mf1(); // OK: calls Derived::mf1()
d.mf1(42); // ERROR! Base::mf1(int) is hidden!
d.mf2(); // OK: calls Base::mf2()
d.mf3(); // OK: calls Derived::mf3()
d.mf3(3.14); // ERROR! Base::mf3(double) is hidden!
```

6.5.2 The Fix: Using Declarations

The using declaration brings the hidden base class names into the derived class scope:

```
// GOOD -- using declarations unhide base class names
class Derived : public Base {
public:
    using Base::mf1;    // make all Base::mf1 overloads visible
    using Base::mf3;    // make all Base::mf3 overloads visible

    void mf1() override {
        std::cout << "Derived::mf1()\n";
    }

    void mf3() {
        std::cout << "Derived::mf3()\n";
    }
};

Derived d;
d.mf1();           // OK: calls Derived::mf1()
d.mf1(42);         // OK: calls Base::mf1(int) -- no longer hidden!
d.mf2();           // OK: calls Base::mf2()
d.mf3();           // OK: calls Derived::mf3()
d.mf3(3.14);       // OK: calls Base::mf3(double) -- no longer hidden!
```

6.5.3 Private Inheritance and Forwarding Functions

Under private inheritance, you might not want all inherited overloads to be visible. In that case, use a forwarding function instead of a using declaration:

```
// GOOD -- forwarding function selectively exposes base functionality
class Base {
public:
    virtual void mf1() { std::cout << "Base::mf1()\n"; }
    virtual void mf1(int x) { std::cout << "Base::mf1(" << x << ")\n"; }
    virtual void mf1(double d, int x) { std::cout << "Base::mf1(double,int)\n"; }
};

class Derived : private Base {
public:
    // Only expose the no-arg version
    virtual void mf1() {
        Base::mf1();    // forwarding function
    }
    // Base::mf1(int) and Base::mf1(double, int) remain inaccessible
};

Derived d;
d.mf1();           // OK: calls Derived::mf1() which forwards to Base::mf1()
d.mf1(42);         // ERROR: hidden, as intended
```

6.5.4 A More Realistic Example: Widget Hierarchy

```

// BAD -- name hiding breaks the interface
class Widget {
public:
    virtual ~Widget() = default;

    virtual void draw() {
        std::cout << "Widget::draw() -- default rendering\n";
    }
    virtual void draw(const Rect& clipRect) {
        std::cout << "Widget::draw(Rect) -- clipped rendering\n";
    }
    virtual void draw(const Rect& clipRect, int opacity) {
        std::cout << "Widget::draw(Rect, int) -- clipped + opacity\n";
    }

    void resize(int w, int h) { width_ = w; height_ = h; }
    void resize(const Size& s) { width_ = s.w; height_ = s.h; }

protected:
    int width_ = 100, height_ = 100;
};

class Button : public Widget {
public:
    // Override only the no-arg version, but ACCIDENTALLY hides the others!
    void draw() override {
        std::cout << "Button::draw() -- button-specific rendering\n";
    }
};

Button b;
b.draw(); // OK
b.draw(Rect{0, 0, 50, 50}); // ERROR! Hidden!
b.draw(Rect{0, 0, 50, 50}, 128); // ERROR! Hidden!

// GOOD -- using declaration preserves the full interface
class Button : public Widget {
public:
    using Widget::draw; // unhide all draw overloads

    void draw() override {
        std::cout << "Button::draw() -- button-specific rendering\n";
    }
};

Button b;
b.draw(); // OK: Button::draw()
b.draw(Rect{0, 0, 50, 50}); // OK: Widget::draw(Rect)
b.draw(Rect{0, 0, 50, 50}, 128); // OK: Widget::draw(Rect, int)

```

6.5.5 Template Base Classes and Name Hiding

Name hiding also interacts with template base classes. The compiler does not look into dependent base classes:


```

// BAD -- name not found in dependent base class
template <typename T>
class LoggingBase {
public:
    void log(const std::string& msg) {
        std::cout << "[LOG] " << msg << "\n";
    }
};

template <typename T>
class Processor : public LoggingBase<T> {
public:
    void process() {
        log("Processing..."); // ERROR! Compiler doesn't look in LoggingBase<T>
    }
};

// GOOD -- three ways to fix it
template <typename T>
class Processor : public LoggingBase<T> {
public:
    using LoggingBase<T>::log; // Approach 1: using declaration

    void process() {
        log("Processing..."); // works with Approach 1
        this->log("Processing..."); // Approach 2: qualify with this->
        LoggingBase<T>::log("Proc..."); // Approach 3: fully qualified (suppresses virtual)
    }
};

```

6.5.6 Things to Remember

- Names in derived classes hide names in base classes. Under public inheritance, this is never desirable because it violates the “is-a” relationship.
- To make hidden names visible again, employ using declarations or forwarding functions.
- A using declaration brings *all* overloads of a given name into the derived class scope. Use forwarding functions when you want to expose only specific overloads.

6.6 Item 34: Differentiate between inheritance of interface and inheritance of implementation

6.7 Visual Summary

ITEM 34: DIFFERENTIATE BETWEEN INHERITANCE OF INTERFACE AND INHERITANCE OF

1. Pure virtual -> inherit interface only; derived must implement.
2. Impure virtual -> inherit interface plus overridable default implementation.
3. Non-virtual -> inherit interface plus mandatory implementation/invariant.
4. Choose deliberately based on what derived classes may change.
5. Meaning: each function form communicates a different inheritance contract.

6.7.1 The Three Kinds of Member Functions

When you design a base class with member functions, you must decide among three choices for each function:

1. **Pure virtual functions:** derived classes inherit the *interface only*. Each derived class *must* provide its own implementation.
2. **Simple (impure) virtual functions:** derived classes inherit *both interface and a default implementation*. They may override but are not required to.
3. **Non-virtual functions:** derived classes inherit *both interface and a mandatory implementation*. They should not override it (see Item 36).

```
class Shape {
public:
    virtual ~Shape() = default;

    // Pure virtual: interface only
    virtual void draw() const = 0;

    // Simple virtual: interface + default implementation
    virtual void resize(double factor) {
        // Default: scale uniformly
        scale_ *= factor;
    }

    // Non-virtual: interface + mandatory implementation
    int objectID() const {
        return id_;
    }

private:
    double scale_ = 1.0;
    int id_ = nextID();
    static int nextID() { static int n = 0; return n++; }
};
```

6.7.2 Pure Virtual Functions: Interface Inheritance

A pure virtual function tells derived classes: “You *must* provide this function, but I have no opinion about how you implement it.”

```

class Shape {
public:
    virtual ~Shape() = default;
    virtual double area() const = 0;
    virtual void draw() const = 0;
    virtual std::string name() const = 0;
};

class Circle : public Shape {
public:
    explicit Circle(double r) : radius_(r) {}

    double area() const override {
        return 3.14159265 * radius_ * radius_;
    }
    void draw() const override {
        std::cout << "Drawing circle with radius " << radius_ << "\n";
    }
    std::string name() const override { return "Circle"; }

private:
    double radius_;
};

class Triangle : public Shape {
public:
    Triangle(double b, double h) : base_(b), height_(h) {}

    double area() const override {
        return 0.5 * base_ * height_;
    }
    void draw() const override {
        std::cout << "Drawing triangle " << base_ << "x" << height_ << "\n";
    }
    std::string name() const override { return "Triangle"; }

private:
    double base_, height_;
};

```

Pure virtual functions *can* have implementations – they just can't be called through the normal virtual dispatch mechanism. They must be called with full qualification:

```

class Shape {
public:
    virtual ~Shape() = default;
    virtual void draw() const = 0;
};

// Definition of a pure virtual function
void Shape::draw() const {
    std::cout << "Shape::draw() -- default wireframe drawing\n";
}

class Circle : public Shape {
public:
    void draw() const override {
        // Can explicitly call the pure virtual's implementation
        Shape::draw(); // use the default wireframe
        std::cout << " ...now filling in the circle\n";
    }
}

```

```
};
```

6.7.3 The Danger of Simple Virtual Functions

Simple virtual functions provide both an interface and a default implementation, but this creates a risk: a derived class may accidentally inherit a default that is inappropriate, simply by failing to override.

```
// BAD -- dangerous default that a new derived class might accidentally inherit
class Airport { /* ... */ };

class Airplane {
public:
    virtual ~Airplane() = default;

    virtual void fly(const Airport& from, const Airport& to) {
        // default flight plan: fly in a straight line at 30,000 ft
        std::cout << "Flying default route at 30,000 ft\n";
    }
};

class ModelA : public Airplane {
    // inherits default fly() -- works fine for Model A
};

class ModelB : public Airplane {
    // inherits default fly() -- works fine for Model B too
};

// Later, a new model is added...
class ModelC : public Airplane {
    // Forgot to override fly()!
    // ModelC needs a different flight plan, but silently gets the default.
    // This could be a disaster.
};
```

Solution 1: Pure virtual + protected default (Meyers' preferred approach)

```
// GOOD -- separate interface from default implementation
class Airplane {
public:
    virtual ~Airplane() = default;
    virtual void fly(const Airport& from, const Airport& to) = 0;

protected:
    void defaultFly(const Airport& from, const Airport& to) {
        std::cout << "Flying default route at 30,000 ft\n";
    }
};

class ModelA : public Airplane {
public:
    void fly(const Airport& from, const Airport& to) override {
        defaultFly(from, to); // explicitly opt in to default
    }
};

class ModelC : public Airplane {
public:
    void fly(const Airport& from, const Airport& to) override {
        // ModelC MUST provide its own implementation -- compiler enforces it
        std::cout << "Flying ModelC-specific route at 25,000 ft\n";
    }
};
```

```

    }
};

```

Solution 2: Pure virtual with a definition

```

// GOOD -- use the pure virtual function's own body as the default
class Airplane {
public:
    virtual ~Airplane() = default;
    virtual void fly(const Airport& from, const Airport& to) = 0;
};

// Provide the default as the pure virtual's body
void Airplane::fly(const Airport& from, const Airport& to) {
    std::cout << "Flying default route at 30,000 ft\n";
}

class ModelA : public Airplane {
public:
    void fly(const Airport& from, const Airport& to) override {
        Airplane::fly(from, to); // explicitly invoke default
    }
};

class ModelC : public Airplane {
public:
    void fly(const Airport& from, const Airport& to) override {
        std::cout << "ModelC-specific route\n"; // forced to provide own impl
    }
};

```

6.7.4 Non-Virtual Functions: Invariant over Specialization

Non-virtual functions represent behavior that is *invariant* across the hierarchy – behavior that should never change regardless of how specialized a derived class becomes.

```

class Transaction {
public:
    virtual ~Transaction() = default;

    // Non-virtual: every transaction has a unique ID, period.
    std::string transactionID() const { return id_; }

    // Non-virtual: the audit trail format is standardized.
    std::string auditRecord() const {
        return "[" + id_ + "]" + description();
    }

    // Pure virtual: each transaction type describes itself differently.
    virtual std::string description() const = 0;

private:
    std::string id_ = generateUUID();
};

class Purchase : public Transaction {
public:
    std::string description() const override {
        return "Purchase of " + item_ + " for $" + std::to_string(amount_);
    }
private:

```

```

    std::string item_ = "Widget";
    double amount_ = 9.99;
};

class Refund : public Transaction {
public:
    std::string description() const override {
        return "Refund of $" + std::to_string(amount_);
    }
private:
    double amount_ = 4.50;
};

```

6.7.5 Summary Table

Declaration	What is inherited	Derived class must override?
Pure virtual	Interface only	Yes
Simple virtual	Interface + default impl	No (may override)
Non-virtual	Interface + mandatory impl	No (should not override)

6.7.6 Things to Remember

- Pure virtual functions specify *interface inheritance only*. Derived classes must provide their own implementation.
- Simple (impure) virtual functions specify *interface inheritance plus default implementation inheritance*. Derived classes may override or accept the default.
- Non-virtual functions specify *interface inheritance plus mandatory implementation inheritance*. Derived classes should not redefine them (see Item 36).
- The danger of simple virtual functions is that a derived class can accidentally inherit an inappropriate default. Consider using pure virtual + a separate protected default to avoid this.

6.8 Item 35: Consider alternatives to virtual functions

6.9 Visual Summary

ITEM 35: CONSIDER ALTERNATIVES TO VIRTUAL FUNCTIONS

1. Need customizable behavior -> virtual function is one option.
2. NVI -> public non-virtual wrapper enforces invariants around private virtual hook.
3. Strategy/function object -> behavior supplied by composition.
4. Templates -> compile-time customization without runtime dispatch.
5. Meaning: dynamic polymorphism is useful, but not the only variation point.

6.9.1 The Problem with a Straightforward Virtual Function Design

Consider a simple health computation for game characters:

```
// Straightforward but rigid approach
class GameCharacter {
public:
    virtual ~GameCharacter() = default;
    virtual int healthValue() const {
        // default algorithm for computing health
        return baseHealth_ - damage_;
    }
private:
    int baseHealth_ = 100;
    int damage_ = 0;
};
```

This works, but there are several alternatives that offer greater flexibility. Meyers presents four alternatives, each with different tradeoffs.

6.9.2 Alternative 1: The Template Method Pattern via the Non-Virtual Interface (NVI) Idiom

The NVI idiom wraps virtual functions with non-virtual public functions. The non-virtual wrapper does “before” and “after” work, while the virtual function is private or protected.

```
// GOOD -- NVI idiom (Template Method Pattern)
class GameCharacter {
public:
    virtual ~GameCharacter() = default;

    // Non-virtual interface: the public entry point
    int healthValue() const {
        // "before" work: lock mutex, log, validate invariants, etc.
        logHealthQuery();

        int result = doHealthValue(); // delegate to virtual

        // "after" work: unlock mutex, verify postconditions, etc.
        assert(result >= 0 && result <= maxHealth_);
        return result;
    }
private:
    virtual int doHealthValue() const {
        // default health calculation
        return baseHealth_ - damage_;
    }

    void logHealthQuery() const {
        std::cout << "[LOG] Health queried for character\n";
    }

    int baseHealth_ = 100;
    int damage_ = 0;
    int maxHealth_ = 100;
};

class Warrior : public GameCharacter {
private:
    int doHealthValue() const override {
        // Warriors get bonus health from armor
        return GameCharacter::doHealthValue() + armorBonus_;
    }
    int armorBonus_ = 20;
};
```

```

class Mage : public GameCharacter {
private:
    int doHealthValue() const override {
        // Mages have lower base health but get mana shield
        return GameCharacter::doHealthValue() + manaShield_;
    }
    int manaShield_ = 15;
};

```

Key advantages of NVI: - Pre-conditions and post-conditions are always enforced. - Logging, locking, and instrumentation happen in one place. - Derived classes customize *what* is done, while the base class controls *when* and *how* it's done.

6.9.3 Alternative 2: The Strategy Pattern via Function Pointers

Decouple the health calculation entirely from the class hierarchy by using a function pointer:

```

// GOOD -- Strategy Pattern via function pointers
class GameCharacter; // forward declaration

// Health calculation is a free function (a "strategy")
int defaultHealthCalc(const GameCharacter& gc);
int conservativeHealthCalc(const GameCharacter& gc);
int aggressiveHealthCalc(const GameCharacter& gc);

class GameCharacter {
public:
    using HealthCalcFunc = int (*)(const GameCharacter&);

    explicit GameCharacter(HealthCalcFunc hcf = defaultHealthCalc)
        : healthFunc_(hcf) {}

    int healthValue() const {
        return healthFunc_(*this);
    }

    // Can change strategy at runtime!
    void setHealthCalculator(HealthCalcFunc hcf) {
        healthFunc_ = hcf;
    }

    int getBaseHealth() const { return baseHealth_; }
    int getDamage() const { return damage_; }

private:
    HealthCalcFunc healthFunc_;
    int baseHealth_ = 100;
    int damage_ = 0;
};

int defaultHealthCalc(const GameCharacter& gc) {
    return gc.getBaseHealth() - gc.getDamage();
}

int conservativeHealthCalc(const GameCharacter& gc) {
    return static_cast<int>((gc.getBaseHealth() - gc.getDamage()) * 0.8);
}

int aggressiveHealthCalc(const GameCharacter& gc) {
    return static_cast<int>((gc.getBaseHealth() - gc.getDamage()) * 1.2);
}

```



```

}

// Usage
GameCharacter warrior(aggressiveHealthCalc);
GameCharacter healer(conservativeHealthCalc);

// Switch strategies at runtime
warrior.setHealthCalculator(conservativeHealthCalc);

```

Key advantages: - Different instances of the *same* class can have different health strategies. - Strategies can be swapped at runtime. - Health calculation is fully decoupled from the class hierarchy.

Key disadvantage: - The function pointer has no access to private/protected members. You may need to weaken encapsulation (provide public accessors or declare friends).

6.9.4 Alternative 3: The Strategy Pattern via `std::function`

`std::function` generalizes function pointers to accept any callable: regular functions, lambdas, functors, bound member functions, etc.

```

// GOOD -- Strategy Pattern via std::function (most flexible)
#include <functional>

class GameCharacter {
public:
    using HealthCalcFunc = std::function<int(const GameCharacter&)>;

    explicit GameCharacter(HealthCalcFunc hcf = defaultHealthCalc)
        : healthFunc_(std::move(hcf)) {}

    int healthValue() const {
        return healthFunc_(*this);
    }

    void setHealthCalculator(HealthCalcFunc hcf) {
        healthFunc_ = std::move(hcf);
    }

    int getBaseHealth() const { return baseHealth_; }
    int getDamage() const { return damage_; }

private:
    HealthCalcFunc healthFunc_;
    int baseHealth_ = 100;
    int damage_ = 0;

    static int defaultHealthCalc(const GameCharacter& gc) {
        return gc.baseHealth_ - gc.damage_;
    }
};

// Now we can use ANYTHING callable:

// 1. Regular function
int heroCalc(const GameCharacter& gc) {
    return gc.getBaseHealth() * 2 - gc.getDamage();
}

// 2. Lambda
auto timidCalc = [](const GameCharacter& gc) -> int {
    return std::max(0, gc.getBaseHealth() / 2 - gc.getDamage());
};

```

```

};

// 3. Functor
struct LevelAdjustedCalc {
    int level;
    int operator()(const GameCharacter& gc) const {
        return gc.getBaseHealth() + level * 10 - gc.getDamage();
    }
};

// 4. Bound member function of another class
class GameLevel {
public:
    int environmentalHealthAdjust(const GameCharacter& gc) const {
        // Poison swamp reduces health
        return gc.getBaseHealth() - gc.getDamage() - poisonDamage_;
    }
private:
    int poisonDamage_ = 15;
};

// Usage:
GameCharacter c1(heroCalc); // function pointer
GameCharacter c2(timidCalc); // lambda
GameCharacter c3(LevelAdjustedCalc{5}); // functor

GameLevel swamp;
GameCharacter c4(std::bind(&GameLevel::environmentalHealthAdjust,
                        &swamp, std::placeholders::_1)); // bound member
// Or with a lambda (preferred over std::bind in modern C++):
GameCharacter c5([&swamp](const GameCharacter& gc) {
    return swamp.environmentalHealthAdjust(gc);
});

```

6.9.5 Alternative 4: The Classic Strategy Pattern

Extract the strategy into its own class hierarchy:

```

// GOOD -- Classic Strategy Pattern with its own hierarchy
class GameCharacter; // forward declaration

class HealthCalcStrategy {
public:
    virtual ~HealthCalcStrategy() = default;
    virtual int calc(const GameCharacter& gc) const = 0;
};

class DefaultHealthCalc : public HealthCalcStrategy {
public:
    int calc(const GameCharacter& gc) const override;
};

class SlowRegenHealthCalc : public HealthCalcStrategy {
public:
    int calc(const GameCharacter& gc) const override {
        // Regenerates slowly over time
        return gc.getBaseHealth() - gc.getDamage() + regenRate_ * turnCount_;
    }
private:
    int regenRate_ = 2;
};

```

```

    int turnCount_ = 0;
};

class PoisonedHealthCalc : public HealthCalcStrategy {
public:
    int calc(const GameCharacter& gc) const override {
        return gc.getBaseHealth() - gc.getDamage() - poisonPerTurn_ * turns_;
    }
private:
    int poisonPerTurn_ = 5;
    int turns_ = 3;
};

class GameCharacter {
public:
    explicit GameCharacter(std::shared_ptr<HealthCalcStrategy> strategy
                          = std::make_shared<DefaultHealthCalc>())
        : strategy_(std::move(strategy)) {}

    int healthValue() const {
        return strategy_>calc(*this);
    }

    void setStrategy(std::shared_ptr<HealthCalcStrategy> s) {
        strategy_ = std::move(s);
    }

    int getBaseHealth() const { return baseHealth_; }
    int getDamage() const { return damage_; }

private:
    std::shared_ptr<HealthCalcStrategy> strategy_;
    int baseHealth_ = 100;
    int damage_ = 0;
};

int DefaultHealthCalc::calc(const GameCharacter& gc) const {
    return gc.getBaseHealth() - gc.getDamage();
}

```

This is the most elaborate approach but offers the greatest extensibility: new strategies can be added as new classes without modifying existing code. Strategies can carry their own state, be configured independently, and be shared among multiple characters.

6.9.6 Comparison of Approaches

Approach	Flexibility	Complexity	Encapsulation Impact
Virtual functions	Low	Low	None
NVI (Template Method)	Low-Medium	Low	None
Function pointers	Medium	Low	May need public accessors
<code>std::function</code>	High	Medium	May need public accessors
Classic Strategy	High	High	May need public accessors

6.9.7 Things to Remember

- Alternatives to virtual functions include the NVI idiom and various forms of the Strategy design pattern. The NVI idiom is the Template Method design pattern; it wraps public non-virtual member functions around less accessible virtual functions.

- Moving functionality from a member function to a function outside the class means the non-member function has no special access to non-public members.
- `std::function` objects act like generalized function pointers. They accept any callable entity compatible with the target signature.
- The classic Strategy pattern replaces virtual functions in the primary hierarchy with a separate hierarchy of strategy objects.

6.10 Item 36: Never redefine an inherited non-virtual function

6.11 Visual Summary

ITEM 36: NEVER REDEFINE AN INHERITED NON-VIRTUAL FUNCTION

1. Base non-virtual function → base says implementation is invariant.
2. Derived redefines same name/signature → static type decides which body runs.
3. Base pointer calls Base version; Derived object may call Derived version.
4. Meaning: redefining non-virtual functions breaks substitutability and surprises callers.

6.11.1 The Problem

Non-virtual functions are statically bound. The function called depends on the *declared type* of the pointer or reference, not the *actual type* of the object.

```
// BAD -- redefining a non-virtual function
class Base {
public:
    void doWork() {
        std::cout << "Base::doWork()\n";
    }
};

class Derived : public Base {
public:
    void doWork() {          // hides Base::doWork -- NOT an override!
        std::cout << "Derived::doWork()\n";
    }
};

Derived d;
Base* bp = &d;
Derived* dp = &d;

bp->doWork();    // calls Base::doWork()    -- static binding!
dp->doWork();    // calls Derived::doWork() -- static binding!
```

Both `bp` and `dp` point to the **same object** `d`, yet they call **different functions**. This is deeply confusing and almost always a bug.

6.11.2 Why This is Always Wrong

The argument is both theoretical and practical:

Theoretical argument (from Item 32 and Item 34):

1. Public inheritance means “is-a” (Item 32).

2. Non-virtual functions establish an invariant over specialization (Item 34).
3. If Derived redefines a non-virtual function, either:
 - Derived is not really “is-a” Base (contradicts point 1), or
 - The function is not really invariant over specialization and should have been virtual (contradicts point 2).

Either way, the design is flawed.

Practical argument:

```
// BAD -- causes bizarre behavior in real code
class Document {
public:
    std::string fileExtension() { return ".doc"; }
};

class Spreadsheet : public Document {
public:
    std::string fileExtension() { return ".xls"; } // redefines non-virtual!
};

void saveToFile(Document& doc) {
    std::string name = "report" + doc.fileExtension();
    // ALWAYS saves as "report.doc" even for Spreadsheets!
    std::cout << "Saving as: " << name << "\n";
}

Spreadsheet ss;
saveToFile(ss); // Saves as "report.doc" -- WRONG!
```

6.11.3 The Fix

If the function should differ by type, make it virtual. If it should not differ, do not redefine it.

```
// GOOD -- make it virtual if behavior should vary
class Document {
public:
    virtual ~Document() = default;
    virtual std::string fileExtension() const { return ".doc"; }
};

class Spreadsheet : public Document {
public:
    std::string fileExtension() const override { return ".xls"; }
};

class Presentation : public Document {
public:
    std::string fileExtension() const override { return ".ppt"; }
};

void saveToFile(Document& doc) {
    std::string name = "report" + doc.fileExtension();
    std::cout << "Saving as: " << name << "\n"; // Now correct for all types
}
```

6.11.4 Another Example: Logger Hierarchy

```
// BAD -- non-virtual function redefined
class Logger {
public:
    void log(const std::string& msg) {
        std::cout << "[INFO] " << msg << "\n";
    }
};

class ErrorLogger : public Logger {
public:
    void log(const std::string& msg) { // hides Logger::log!
        std::cerr << "[ERROR] " << msg << "\n";
    }
};

void processAndLog(Logger& logger, const std::string& action) {
    // Always calls Logger::log, even if passed an ErrorLogger!
    logger.log("Processing: " + action);
}
```

```
// GOOD -- use virtual functions
class Logger {
public:
    virtual ~Logger() = default;
    virtual void log(const std::string& msg) {
        std::cout << "[INFO] " << msg << "\n";
    }
};

class ErrorLogger : public Logger {
public:
    void log(const std::string& msg) override {
        std::cerr << "[ERROR] " << msg << "\n";
    }
};

class FileLogger : public Logger {
public:
    explicit FileLogger(const std::string& filename)
        : file_(filename) {}

    void log(const std::string& msg) override {
        file_ << msg << "\n";
    }
private:
    std::ofstream file_;
};
```

6.11.5 Things to Remember

- Never redefine an inherited non-virtual function. Non-virtual functions are statically bound; virtual functions are dynamically bound. Redefining a non-virtual function causes the called function to depend on the declared type of the pointer/reference, not the actual object type.
- If you need different behavior in a derived class, the function should be virtual. If the behavior truly must be invariant, do not redefine it in derived classes.

6.12 Item 37: Never redefine a function's inherited default parameter value

6.13 Visual Summary

ITEM 37: NEVER REDEFINE A FUNCTION'S INHERITED DEFAULT PARAMETER VALUE

1. Virtual dispatch chooses function body at runtime.
2. Default argument is bound from static type at compile time.
3. Derived changes default → body and default value can come from different classes.
4. Use NVI or overloads to keep defaults centralized.
5. Meaning: virtual functions are dynamic; default parameters are not.

6.13.1 The Core Issue

Virtual functions are dynamically bound, but **default parameter values are statically bound**. This mismatch leads to bizarre behavior:

```
// BAD -- redefining inherited default parameter value
class Shape {
public:
    virtual ~Shape() = default;

    enum class Color { Red, Green, Blue };

    virtual void draw(Color color = Color::Red) const = 0;
};

class Circle : public Shape {
public:
    // Redefines the default parameter! BAD!
    void draw(Color color = Color::Green) const override {
        std::cout << "Circle::draw with color "
                  << static_cast<int>(color) << "\n";
    }
};

class Rectangle : public Shape {
public:
    // Also redefines the default! BAD!
    void draw(Color color = Color::Blue) const override {
        std::cout << "Rectangle::draw with color "
                  << static_cast<int>(color) << "\n";
    }
};

// The horror:
Shape* pc = new Circle;
Shape* pr = new Rectangle;

pc->draw(); // Calls Circle::draw(Color::Red)!
           // Virtual dispatch picks Circle::draw (dynamic binding)
           // BUT default parameter comes from Shape (static binding)
           // User sees Red, not Green!

pr->draw(); // Calls Rectangle::draw(Color::Red)!
           // Same issue: dynamic body, static default
```

```
Circle c;
c.draw();    // Calls Circle::draw(Color::Green)
             // When called via Circle*, uses Circle's default!
```

The same object, the same function call, but different results depending on the *declared type* of the pointer. This is exactly the kind of insanity you want to avoid.

6.13.2 Why C++ Works This Way

Default parameters are statically bound for runtime efficiency. If default parameter values were dynamically bound, the runtime would need a mechanism to determine the appropriate default at each call site, which would be slower and more complex than the current approach.

6.13.3 The Fix: NVI Idiom

The NVI idiom elegantly sidesteps this problem. Make the public function non-virtual (with the default parameter), and delegate to a private virtual function (without a default parameter).

```
// GOOD -- NVI idiom avoids the default parameter problem entirely
class Shape {
public:
    virtual ~Shape() = default;

    enum class Color { Red, Green, Blue };

    // Non-virtual: owns the default parameter
    void draw(Color color = Color::Red) const {
        doDraw(color); // pass explicit argument to virtual
    }

private:
    // Virtual: no default parameter to worry about
    virtual void doDraw(Color color) const = 0;
};

class Circle : public Shape {
private:
    void doDraw(Color color) const override {
        std::cout << "Circle::draw with color "
                  << static_cast<int>(color) << "\n";
    }
};

class Rectangle : public Shape {
private:
    void doDraw(Color color) const override {
        std::cout << "Rectangle::draw with color "
                  << static_cast<int>(color) << "\n";
    }
};

// Now it works correctly:
Shape* pc = new Circle;
Shape* pr = new Rectangle;

pc->draw();           // Circle::draw(Color::Red) -- always consistent!
pr->draw();           // Rectangle::draw(Color::Red) -- always consistent!
pc->draw(Shape::Color::Blue); // Circle::draw(Color::Blue)
```

6.13.4 A More Elaborate Example


```
// BAD -- complex hierarchy with inconsistent defaults
class Widget {
public:
    virtual ~Widget() = default;
    virtual void render(int opacity = 255, bool shadow = true) const {
        std::cout << "Widget::render opacity=" << opacity
                    << " shadow=" << shadow << "\n";
    }
};

class Button : public Widget {
public:
    // Changes opacity default -- BAD!
    void render(int opacity = 200, bool shadow = true) const override {
        std::cout << "Button::render opacity=" << opacity
                    << " shadow=" << shadow << "\n";
    }
};

class FlatButton : public Button {
public:
    // Changes shadow default -- BAD!
    void render(int opacity = 200, bool shadow = false) const override {
        std::cout << "FlatButton::render opacity=" << opacity
                    << " shadow=" << shadow << "\n";
    }
};

Widget* w = new FlatButton;
w->render(); // FlatButton::render(255, true) -- Widget's defaults!
            // User expected opacity=200, shadow=false
```

```
// GOOD -- NVI with clean defaults
class Widget {
public:
    virtual ~Widget() = default;

    void render(int opacity = 255, bool shadow = true) const {
        doRender(opacity, shadow);
    }

private:
    virtual void doRender(int opacity, bool shadow) const {
        std::cout << "Widget::render opacity=" << opacity
                    << " shadow=" << shadow << "\n";
    }
};

class Button : public Widget {
private:
    void doRender(int opacity, bool shadow) const override {
        std::cout << "Button::render opacity=" << opacity
                    << " shadow=" << shadow << "\n";
    }
};

class FlatButton : public Button {
private:
    void doRender(int opacity, bool shadow) const override {
        std::cout << "FlatButton::render opacity=" << opacity
```

```

        << " shadow=" << shadow << "\n";
    }
};

Widget* w = new FlatButton;
w->render(); // FlatButton::doRender(255, true) -- consistent!

```

6.13.5 What if the Derived Class Genuinely Needs a Different Default?

If you truly need different defaults when calling through different types, you probably have a design problem. But if you must, overload instead of changing the default:

```

// ACCEPTABLE -- overloads instead of changing defaults
class Shape {
public:
    virtual ~Shape() = default;

    enum class Color { Red, Green, Blue };

    void draw() const { doDraw(defaultColor()); }
    void draw(Color c) const { doDraw(c); }

protected:
    virtual Color defaultColor() const { return Color::Red; }

private:
    virtual void doDraw(Color color) const = 0;
};

class OceanShape : public Shape {
protected:
    Color defaultColor() const override { return Color::Blue; }
private:
    void doDraw(Color color) const override {
        std::cout << "OceanShape with color " << static_cast<int>(color) << "\n";
    }
};

```

6.13.6 Things to Remember

- Never redefine an inherited default parameter value, because default parameter values are statically bound, while virtual functions – the only functions you should be overriding – are dynamically bound.
- Use the NVI idiom (Item 35) to have the non-virtual public function specify the default, delegating to a private virtual with no default parameter.

6.14 Item 38: Model “has-a” or “is-implemented-in-terms-of” through composition

6.15 Visual Summary

ITEM 38: MODEL "HAS-A" OR "IS-IMPLEMENTED-IN-TERMS-OF" THROUGH COMPOSITION

1. Class contains another object -> either has-a or implemented-in-terms-of.
2. Application domain -> Car has Engine.
3. Implementation domain -> Set implemented with list/vector/tree.
4. Composition keeps interfaces separate and coupling lower than inheritance.
5. Meaning: prefer containment when the relationship is not substitutable is-a.

6.15.1 Composition vs. Inheritance

Composition (also called layering, containment, aggregation, or embedding) is the relationship where one class *contains* an object of another class as a data member. It models two different relationships depending on the domain:

1. **"Has-a"**: in the application domain (objects modeling real-world things).
2. **"Is-implemented-in-terms-of"**: in the implementation domain (objects that are implementation artifacts).

6.15.2 "Has-a" Relationship

```
// GOOD -- Person "has-a" name, address, and phone numbers
class Address {
public:
    std::string street;
    std::string city;
    std::string state;
    std::string zip;
};

class PhoneNumber {
public:
    std::string areaCode;
    std::string number;
    std::string extension;
};

class Person {
public:
    const std::string& name() const { return name_; }
    const Address& address() const { return address_; }

private:
    std::string name_;           // Person has-a name
    Address address_;           // Person has-a address
    std::vector<PhoneNumber> phones_; // Person has phone numbers
};
```

A Person is *not* a string, an Address, or a PhoneNumber. A person *has* those things. This is obvious, and nobody would use public inheritance here.

6.15.3 "Is-Implemented-In-Terms-Of" Relationship

This is less obvious and is the more common source of confusion. Consider implementing a Set using a `std::list`:

```
// BAD -- Set is NOT a list!
template <typename T>
class Set : public std::list<T> {
    // A list can contain duplicates; a set cannot.
    // A list has push_front, push_back, etc. -- inappropriate for a set.
    // This violates "is-a": a Set is NOT a list.
};
```

A set is not a list. A list allows duplicates and preserves insertion order; a set does not. Public inheritance is wrong here.

```
// GOOD -- Set is-implemented-in-terms-of list
template <typename T>
class Set {
public:
    bool contains(const T& item) const {
        return std::find(rep_.begin(), rep_.end(), item) != rep_.end();
    }

    void insert(const T& item) {
        if (!contains(item)) {
            rep_.push_back(item);
        }
    }

    void remove(const T& item) {
        auto it = std::find(rep_.begin(), rep_.end(), item);
        if (it != rep_.end()) {
            rep_.erase(it);
        }
    }

    std::size_t size() const { return rep_.size(); }
    bool empty() const { return rep_.empty(); }

    // Iterators for range-based for
    auto begin() const { return rep_.begin(); }
    auto end() const { return rep_.end(); }

private:
    std::list<T> rep_; // Set is-implemented-in-terms-of list
};
```

The Set *uses* a list for its implementation, but it is not a *kind of* list. Users of Set never see the list; it is a hidden implementation detail.

6.15.4 Real-World Example: Connection Pool

```

// GOOD -- ConnectionPool uses a queue internally
class Connection {
public:
    explicit Connection(const std::string& host) : host_(host) {
        std::cout << "Connected to " << host_ << "\n";
    }
    void execute(const std::string& query) {
        std::cout << "Executing on " << host_ << ": " << query << "\n";
    }
    void reset() {
        std::cout << "Resetting connection to " << host_ << "\n";
    }
private:
    std::string host_;
};

// BAD -- trying to inherit from deque
// class ConnectionPool : public std::deque<Connection> { ... };
// A pool is NOT a deque!

// GOOD -- composition (is-implemented-in-terms-of)
class ConnectionPool {
public:
    explicit ConnectionPool(const std::string& host, int poolSize)
        : host_(host)
    {
        for (int i = 0; i < poolSize; ++i) {
            pool_.push(std::make_unique<Connection>(host));
        }
    }

    std::unique_ptr<Connection> acquire() {
        if (pool_.empty()) {
            return std::make_unique<Connection>(host_); // grow if needed
        }
        auto conn = std::move(pool_.front());
        pool_.pop();
        return conn;
    }

    void release(std::unique_ptr<Connection> conn) {
        conn->reset();
        pool_.push(std::move(conn));
    }

    std::size_t available() const { return pool_.size(); }

private:
    std::string host_;
    std::queue<std::unique_ptr<Connection>> pool_; // implemented in terms of queue
};

```

6.15.5 Real-World Example: A Timer-Based Notification System

```

// BAD -- a NotificationScheduler is NOT a priority_queue
// class NotificationScheduler : public std::priority_queue<...> { };

// GOOD
struct Notification {
    std::chrono::system_clock::time_point when;
    std::string message;
    std::string recipient;

    bool operator<(const Notification& rhs) const {
        // Earlier notifications have higher priority (min-heap behavior)
        return when > rhs.when;
    }
};

class NotificationScheduler {
public:
    void schedule(Notification n) {
        queue_.push(std::move(n));
    }

    bool hasReady() const {
        if (queue_.empty()) return false;
        return queue_.top().when <= std::chrono::system_clock::now();
    }

    Notification getNext() {
        Notification n = queue_.top();
        queue_.pop();
        return n;
    }

    bool empty() const { return queue_.empty(); }

private:
    std::priority_queue<Notification> queue_; // implemented in terms of pq
};

```

6.15.6 Things to Remember

- Composition has meaning very different from that of public inheritance. It means either “has-a” (in the application domain) or “is-implemented-in-terms-of” (in the implementation domain).
- In the application domain, composition means “has-a.” A Person has-a name, an address, phone numbers.
- In the implementation domain, composition means “is-implemented-in-terms-of.” A Set can be implemented in terms of a list, but a Set is not a list.

6.16 Item 39: Use private inheritance judiciously

6.17 Visual Summary

ITEM 39: USE PRIVATE INHERITANCE JUDICIOUSLY

1. private inheritance -> implemented-in-terms-of, not is-a.
2. Composition usually expresses this with less coupling.
3. Use private inheritance when you need protected access or virtual override hooks.
4. Empty base optimization can be a special storage reason.
5. Meaning: private inheritance is an implementation tool, not a public model.

6.17.1 What Private Inheritance Means

Private inheritance means “is-implemented-in-terms-of.” It has nothing to do with “is-a.” If Derived privately inherits from Base:

- All public and protected members of Base become private in Derived.
- There is no implicit conversion from Derived* to Base*.
- Compilers will not convert a Derived object to a Base object.

```
class Timer {
public:
    explicit Timer(int intervalMs) : interval_(intervalMs) {}
    virtual void onTick() {
        std::cout << "Timer tick\n";
    }
    void start() {
        // simulate periodic ticking
        for (int i = 0; i < 5; ++i) {
            onTick();
        }
    }
private:
    int interval_;
};

// Private inheritance: Widget is-implemented-in-terms-of Timer
class Widget : private Timer {
public:
    Widget() : Timer(500) {}

    void startMonitoring() {
        start(); // can call Timer::start() from within Widget
    }

private:
    void onTick() override {
        std::cout << "Widget: checking for updates...\n";
    }
};

Widget w;
w.startMonitoring(); // OK
// Timer* tp = &w;   // ERROR -- private inheritance prevents conversion
// w.start();         // ERROR -- start() is private in Widget
```

6.17.2 Composition is Usually Preferable

Private inheritance achieves the same thing as composition, and composition is generally preferable because:

- Composition allows you to control what is exposed.
- Composition is easier to understand.
- Composition does not create implicit coupling.

```
// GOOD -- prefer composition over private inheritance
class Widget {
public:
    void startMonitoring() {
        timer_.start();
    }

private:
    // Inner class that overrides onTick
    class WidgetTimer : public Timer {
    public:
        WidgetTimer() : Timer(500) {}
    private:
        void onTick() override {
            std::cout << "Widget: checking for updates...\n";
        }
    };

    WidgetTimer timer_; // composition: Widget has-a WidgetTimer
};
```

This composition approach has two additional advantages over private inheritance: 1. **Prevents derived classes from overriding onTick.** If Widget privately inherits from Timer, classes derived from Widget can still override onTick. With composition, the WidgetTimer class is private, so derived classes cannot interfere. 2. **Minimizes compilation dependencies.** With composition, Timer can be forward-declared and the WidgetTimer can be defined in the .cpp file (using the Pimpl idiom), breaking the compile-time dependency.

6.17.3 When Private Inheritance is the Right Choice

Case 1: The Empty Base Optimization (EBO)

When the base class has no data members (it is “empty”), private inheritance can save space:

```
class Empty {
    // No data members, but may have typedefs, enums, static members,
    // or non-virtual functions
    using DataType = int;
    static int count();
    void doSomething() {}
};

// With composition:
class Widget1 {
    int data_;
    Empty e_; // typically takes 1 byte + padding = 4 or 8 bytes wasted
};
// sizeof(Widget1) > sizeof(int)

// With private inheritance:
class Widget2 : private Empty {
    int data_;
};
// sizeof(Widget2) == sizeof(int) -- EBO kicks in!
```

The Empty Base Optimization (EBO) means that an empty base class need not occupy any space. This matters when you are dealing with policy classes, traits, or allocator classes that contain no data.


```

// GOOD -- EBO with allocator (real-world usage)
template <typename T, typename Allocator = std::allocator<T>>
class SmallVector : private Allocator {
    // Allocator is typically empty. Private inheritance + EBO means
    // SmallVector is no bigger than it needs to be.
public:
    using Allocator::allocate;    // selectively expose if needed
    using Allocator::deallocate;

    // ... vector implementation
private:
    T* data_;
    std::size_t size_;
    std::size_t capacity_;
};

```

Case 2: You need access to protected members

*// Private inheritance is needed when you must access protected members
// or override virtual functions, AND you want to hide the base interface.*

```

class DatabaseDriver {
protected:
    virtual void onConnect() {
        std::cout << "Default connection setup\n";
    }
    virtual void onDisconnect() {
        std::cout << "Default disconnection cleanup\n";
    }

    void rawQuery(const std::string& sql) {
        std::cout << "Executing raw SQL: " << sql << "\n";
    }
};

// Cannot use composition here because we need to override
// protected virtual functions and call protected members.
class DatabaseConnection : private DatabaseDriver {
public:
    void connect() {
        onConnect(); // access protected member
    }
    void disconnect() {
        onDisconnect(); // access protected member
    }
    void executeQuery(const std::string& sql) {
        rawQuery(sql); // access protected member
    }

private:
    void onConnect() override {
        std::cout << "Custom connection: setting timeout, charset...\n";
    }
    void onDisconnect() override {
        std::cout << "Custom disconnect: flushing logs...\n";
    }
};

```

6.17.4 A Practical Comparison

```

// Scenario: Implement a widget that needs to react to timer events
// and also needs an allocator

// Approach 1: All composition (possibly wastes space)
class WidgetV1 {
    class TimerImpl : public Timer {
        void onTick() override { /* ... */ }
    };
    TimerImpl timer_;
    std::allocator<int> alloc_; // wastes space -- allocator is empty
    int data_;
};

// Approach 2: Private inheritance for EBO, composition for timer
class WidgetV2 : private std::allocator<int> {
    class TimerImpl : public Timer {
        void onTick() override { /* ... */ }
    };
    TimerImpl timer_;
    int data_;
    // std::allocator<int> takes zero space via EBO
};

```

6.17.5 Things to Remember

- Private inheritance means “is-implemented-in-terms-of.” It is usually inferior to composition, but it makes sense when a derived class needs access to protected base class members or needs to redefine inherited virtual functions.
- Unlike composition, private inheritance enables the empty base optimization (EBO). This can be important for library developers who work hard to minimize object sizes.
- Use private inheritance judiciously. Use it only when composition truly cannot do the job.

6.18 Item 40: Use multiple inheritance judiciously

6.19 Visual Summary

ITEM 40: USE MULTIPLE INHERITANCE JUDICIOUSLY
1. Multiple bases -> more interfaces/implementation enter one object. 2. Risks -> ambiguity, diamond duplication, constructor complexity. 3. Virtual inheritance -> solves shared base duplication with cost/complexity. 4. Good use -> public interface + private implementation helper, or mixins. 5. Meaning: MI is powerful when relationships are precise and conflicts managed.

6.19.1 The Basics of Multiple Inheritance

Multiple inheritance (MI) means a class inherits from more than one base class. It introduces several complications that single inheritance does not have.

6.19.2 Problem 1: Ambiguity from Multiple Bases

```

// BAD -- ambiguity
class BorrowableItem {
public:
    void checkOut() {
        std::cout << "Checking out from library\n";
    }
};

class ElectronicGadget {
public:
    void checkOut() {
        std::cout << "Running diagnostic check\n";
    }
};

class MP3Player : public BorrowableItem, public ElectronicGadget {
    // Inherits TWO checkOut() functions
};

MP3Player mp;
// mp.checkOut(); // ERROR! Ambiguous: which checkOut()?

```

You must disambiguate explicitly:

```

mp.BorrowableItem::checkOut(); // OK
mp.ElectronicGadget::checkOut(); // OK

```

Note that the ambiguity exists even if one of the functions would be inaccessible (e.g., private in one base). C++ resolves ambiguity *before* checking accessibility.

6.19.3 Problem 2: The Diamond Inheritance Problem

The diamond problem occurs when a class inherits from two classes that share a common base:

```

// The diamond problem
class File {
public:
    std::string filename;
    int size;
};

class InputFile : public File {
public:
    void read() { std::cout << "Reading " << filename << "\n"; }
};

class OutputFile : public File {
public:
    void write() { std::cout << "Writing " << filename << "\n"; }
};

class IOFile : public InputFile, public OutputFile {
    // IOFile has TWO copies of File!
    // IOFile::InputFile::filename and IOFile::OutputFile::filename
};

IOFile f;
// f.filename; // ERROR! Ambiguous -- which filename?
f.InputFile::filename = "input.txt"; // one copy
f.OutputFile::filename = "output.txt"; // different copy!

```

6.19.4 The Solution: Virtual Inheritance

Virtual inheritance ensures that only one copy of the common base class exists:

```
// GOOD -- virtual inheritance solves the diamond
class File {
public:
    std::string filename;
    int size = 0;
};

class InputFile : virtual public File {
public:
    void read() { std::cout << "Reading " << filename << "\n"; }
};

class OutputFile : virtual public File {
public:
    void write() { std::cout << "Writing " << filename << "\n"; }
};

class IOFile : public InputFile, public OutputFile {
    // Only ONE copy of File thanks to virtual inheritance
};

IOFile f;
f.filename = "data.txt"; // unambiguous -- only one filename
f.read();               // "Reading data.txt"
f.write();               // "Writing data.txt"
```

6.19.5 The Costs of Virtual Inheritance

Virtual inheritance is not free. It imposes costs in:

1. **Size:** Objects with virtual bases are larger (they contain vpointers or equivalent to navigate to the virtual base subobject).
2. **Speed:** Accessing members of virtual base classes is slower (indirection through vpointers).
3. **Initialization:** The most derived class must initialize the virtual base, even if it is several levels up the hierarchy.

```
class Animal {
public:
    Animal(const std::string& name) : name_(name) {}
    std::string name_;
};

class Mammal : virtual public Animal {
public:
    // Must pass Animal's constructor arguments
    Mammal(const std::string& name) : Animal(name) {}
};

class WingedAnimal : virtual public Animal {
public:
    WingedAnimal(const std::string& name) : Animal(name) {}
};

class Bat : public Mammal, public WingedAnimal {
public:
    // Bat (the most derived class) MUST initialize Animal directly
    Bat(const std::string& name)
        : Animal(name) // required! Virtual base init
    {}
};
```

```

        , Mammal(name)           // Mammal's Animal init is IGNORED
        , WingedAnimal(name)     // WingedAnimal's Animal init is IGNORED
    {}
};

```

6.19.6 Advice on Virtual Inheritance

1. Do not use virtual inheritance unless you truly need it.
2. If you must use virtual inheritance, try to avoid putting data in virtual base classes. This sidesteps the initialization complexity. Virtual base classes with no data (like interfaces) have minimal overhead.

```

// GOOD -- virtual base class with no data (like a Java/C# interface)
class IPrintable {
public:
    virtual ~IPrintable() = default;
    virtual void print() const = 0;
    // No data members!
};

class ISerializable {
public:
    virtual ~ISerializable() = default;
    virtual std::string serialize() const = 0;
    // No data members!
};

```

6.19.7 The Legitimate Use Case: Interface + Implementation

The most defensible use of multiple inheritance combines a public interface (abstract base class) with a private implementation:

```

// GOOD -- practical MI: public interface inheritance + private implementation

// Pure interface
class IDatabase {
public:
    virtual ~IDatabase() = default;
    virtual void connect(const std::string& connStr) = 0;
    virtual void disconnect() = 0;
    virtual void execute(const std::string& query) = 0;
    virtual bool isConnected() const = 0;
};

// Reusable implementation detail
class ConnectionManager {
public:
    void openConnection(const std::string& connStr) {
        connStr_ = connStr;
        connected_ = true;
        std::cout << "Connection opened to: " << connStr_ << "\n";
    }
    void closeConnection() {
        connected_ = false;
        std::cout << "Connection closed\n";
    }
    bool connected() const { return connected_; }
    const std::string& connectionString() const { return connStr_; }
private:
    std::string connStr_;
    bool connected_ = false;
};

```

```

};

// Concrete class: inherits interface publicly, implementation privately
class PostgresDB : public IDatabase, private ConnectionManager {
public:
    void connect(const std::string& connStr) override {
        openConnection(connStr); // from ConnectionManager
    }
    void disconnect() override {
        closeConnection(); // from ConnectionManager
    }
    void execute(const std::string& query) override {
        if (!connected()) {
            throw std::runtime_error("Not connected");
        }
        std::cout << "Postgres executing: " << query << "\n";
    }
    bool isConnected() const override {
        return connected(); // from ConnectionManager
    }
};

// Client code uses only the interface
void runMigration(IDatabase& db) {
    db.connect("host=localhost dbname=mydb");
    db.execute("CREATE TABLE users (id SERIAL PRIMARY KEY, name TEXT)");
    db.execute("INSERT INTO users (name) VALUES ('Alice')");
    db.disconnect();
}

```

6.19.8 Full Example: Observer Pattern with Multiple Interfaces

```

// GOOD -- MI to implement multiple interfaces (common and legitimate pattern)

class IObserver {
public:
    virtual ~IObserver() = default;
    virtual void onEvent(const std::string& event) = 0;
};

class ILogger {
public:
    virtual ~ILogger() = default;
    virtual void log(const std::string& msg) = 0;
};

class IConfigurable {
public:
    virtual ~IConfigurable() = default;
    virtual void configure(const std::map<std::string, std::string>& opts) = 0;
};

// A monitoring agent implements all three interfaces
class MonitoringAgent : public IObserver,
                        public ILogger,
                        public IConfigurable {
public:
    void onEvent(const std::string& event) override {
        log("Event received: " + event);
    }
}

```

```

        events_.push_back(event);
    }

    void log(const std::string& msg) override {
        std::cout << "[Monitor] " << msg << "\n";
    }

    void configure(const std::map<std::string, std::string>& opts) override {
        auto it = opts.find("verbose");
        if (it != opts.end()) {
            verbose_ = (it->second == "true");
        }
    }

private:
    std::vector<std::string> events_;
    bool verbose_ = false;
};

// Each subsystem works with only the interface it needs:
void attachObserver(IObserver& obs) {
    obs.onEvent("system_start");
}

void setupLogging(ILogger& logger) {
    logger.log("Logging initialized");
}

void loadConfig(IConfigurable& conf) {
    conf.configure({{"verbose", "true"}});
}

// One object, many roles:
MonitoringAgent agent;
attachObserver(agent);
setupLogging(agent);
loadConfig(agent);

```

6.19.9 The Diamond Problem in Practice with Virtual Inheritance

A more complete real-world example showing the diamond pattern properly handled:

```

// Virtual base: shared interface/state
class StreamBase {
public:
    virtual ~StreamBase() = default;

    void setBufferSize(std::size_t sz) { bufferSize_ = sz; }
    std::size_t bufferSize() const { return bufferSize_; }

    virtual void flush() = 0;

protected:
    std::size_t bufferSize_ = 4096;
    std::string name_ = "unnamed";
};

class InputStream : virtual public StreamBase {
public:
    virtual std::string read(std::size_t bytes) = 0;
}

```

```

    void flush() override {
        std::cout << "Flushing input buffer for " << name_ << "\n";
    }
};

class OutputStream : virtual public StreamBase {
public:
    virtual void write(const std::string& data) = 0;

    void flush() override {
        std::cout << "Flushing output buffer for " << name_ << "\n";
    }
};

class IOStream : public InputStream, public OutputStream {
public:
    IOStream(const std::string& name) {
        name_ = name; // only one name_ thanks to virtual inheritance
    }

    std::string read(std::size_t bytes) override {
        std::cout << "Reading " << bytes << " bytes from " << name_ << "\n";
        return "data";
    }

    void write(const std::string& data) override {
        std::cout << "Writing " << data.size()
            << " bytes to " << name_ << "\n";
    }

    // Must resolve the flush() ambiguity from InputStream and OutputStream
    void flush() override {
        InputStream::flush();
        OutputStream::flush();
    }
};

IOStream io("socket://localhost:8080");
io.setBufferSize(8192); // unambiguous -- one StreamBase
io.write("Hello");
io.read(1024);
io.flush();

```

6.19.10 Decision Framework for Multiple Inheritance

1. **Is it multiple *interface* inheritance?** (All or most bases are pure abstract classes with no data.) This is almost always fine and is the most common legitimate use of MI.
2. **Is it one interface + one implementation base?** This is the classic “public interface, private implementation” pattern and is usually fine.
3. **Is it a diamond pattern?** Use virtual inheritance, but be aware of the costs. Prefer keeping virtual bases data-free.
4. **Are you inheriting from multiple concrete classes with data?** Strongly reconsider your design. Composition is almost certainly the better approach.

6.19.11 Things to Remember

- Multiple inheritance is more complex than single inheritance. It can lead to ambiguity issues and the diamond inheritance problem.

- Virtual inheritance imposes costs in size, speed, and complexity of initialization. It is most practical when virtual base classes have no data.
- Multiple inheritance does have legitimate uses. One scenario involves combining public inheritance from an interface class with private inheritance from a class that helps with implementation.
- When faced with MI complexity, prefer composition when possible. Use MI primarily for implementing multiple pure interfaces.

6.20 Summary

The nine items in this chapter cover the fundamental principles of inheritance and OO design in C++:

Item	Core Principle
32	Public inheritance = “is-a” (Liskov Substitution Principle)
33	Use using declarations to prevent name hiding
34	Pure virtual = interface; simple virtual = interface + default; non-virtual = invariant
35	NVI, function pointers, <code>std::function</code> , and Strategy pattern as alternatives to virtual
36	Non-virtual functions are statically bound – never redefine them
37	Default parameters are statically bound – never redefine them in overrides
38	Composition = “has-a” or “is-implemented-in-terms-of”
39	Private inheritance = “is-implemented-in-terms-of” (prefer composition, use for EBO)
40	MI is complex but legitimate for multiple interface inheritance

The overarching theme: understand what each C++ construct *means* in terms of design, and use each construct to express exactly the design relationship you intend. Do not use a language feature simply because it is available; use it because it precisely communicates your architectural intent.

7 Chapter 7: Templates and Generic Programming

7.1 Chapter Flow

CHAPTER 7: TEMPLATES AND GENERIC PROGRAMMING
Item 41 -> template interfaces are implicit expression requirements. Items 42–43 -> mark dependent types/names explicitly for the compiler. Items 44–46 -> avoid template bloat and enable compatible conversions. Items 47–48 -> use traits/TMP for compile-time knowledge when benefit exceeds complexity.

Templates are the foundation of generic programming in C++. They move part of the type-checking work from runtime to compile time, enable code reuse without sacrificing type safety, and open the door to an entirely separate computation model – template metaprogramming – that executes during compilation. This chapter covers Items 41–48 of Scott Meyers’ *Effective C++* (Third Edition), spanning implicit interfaces, the typename disambiguator, accessing names in templated base classes, code bloat, member function templates, friend functions in templates, traits classes, and template metaprogramming.

7.2 Item 41: Understand implicit interfaces and compile-time polymorphism

7.3 Visual Summary

ITEM 41: UNDERSTAND IMPLICIT INTERFACES AND COMPILE-TIME POLYMORPHISM

1. Runtime polymorphism -> explicit base interface + virtual dispatch.
2. Template polymorphism -> implicit expression requirements on T.
3. Compiler instantiates code -> checks whether T supports required operations.
4. No vtable needed -> decisions happen at compile time.
5. Meaning: templates define interfaces by what code does with the type.

7.3.1 The two worlds of polymorphism

Object-oriented programming revolves around **explicit interfaces** and **runtime polymorphism**:

// Explicit interface: the class declaration spells out every operation.

```
class Widget {  
public:  
    Widget();  
    virtual ~Widget();  
    virtual std::size_t size() const;  
    virtual void normalize();  
    void swap(Widget& other);  
};
```

*// Runtime polymorphism: which size() runs is determined at runtime
// via the vtable.*

```
void doProcessing(Widget& w) {  
    if (w.size() > 10 && w != someNastyWidget) {  
        Widget temp(w);  
        temp.normalize();  
        temp.swap(w);  
    }  
}
```

The interface of Widget is explicit – you can open the header file and see every member function signature, every typedef, every data member. Polymorphism happens at runtime through virtual functions and the vtable mechanism.

Templates flip both of these on their head:

```

template <typename T>
void doProcessing(T& w) {
    if (w.size() > 10 && w != someNastyWidget) {
        T temp(w);
        temp.normalize();
        temp.swap(w);
    }
}

```

Now T must support an **implicit interface**: it must have a `size()` member function that returns something comparable to `int`, it must support operator `!=`, it must be copy-constructible, and it must have a `normalize()` and `swap()` member function. But none of this is stated in any single class declaration – it is implied by how T is used in the template body.

The polymorphism here is **compile-time**: which `size()` to call, which operator `!=` to invoke, etc., is resolved when the template is instantiated. The mechanism is template instantiation, not vtable dispatch.

7.3.2 Implicit interfaces in detail

An implicit interface is defined by the set of valid expressions that appear in the template, not by specific function signatures. Consider:

```

template <typename T>
void process(T& obj) {
    // Implicit interface requirement: T must support .begin() and .end()
    // returning iterators, and the iterators must be dereferenceable.
    for (auto it = obj.begin(); it != obj.end(); ++it) {
        std::cout << *it << "\n";
    }
}

```

Any type that satisfies these constraints works:

```

std::vector<int> v = {1, 2, 3};
process(v); // OK: vector has begin(), end(), iterators support * and !=

std::list<std::string> l = {"hello", "world"};
process(l); // OK: list also satisfies the implicit interface

std::map<int, double> m = {{1, 1.1}, {2, 2.2}};
process(m); // OK: map's iterators dereference to std::pair, which supports <<
            // only if operator<< is defined for pair -- this would fail at
            // compile time, illustrating the implicit interface check.

```

7.3.3 Constraints are on expressions, not types

A crucial insight: the implicit interface does not require that `size()` returns an integral type. It requires that the expression `w.size() > 10` is valid and yields something convertible to `bool`. This is far more flexible:

```

class Gadget {
public:
    // size() returns a custom type, not size_t
    SizeProxy size() const;
    void normalize();
    void swap(Gadget& other);
};

// This is fine as long as SizeProxy supports operator>(int)
// and the result is convertible to bool.
class SizeProxy {
public:
    bool operator>(int rhs) const { return value_ > rhs; }
private:
    int value_;
};

```

7.3.4 Compile-time polymorphism with CRTP

The Curiously Recurring Template Pattern (CRTP) is a powerful form of compile-time polymorphism:

```

template <typename Derived>
class Shape {
public:
    double area() const {
        // Compile-time dispatch: calls the derived class's implementation
        return static_cast<const Derived*>(this)->area_impl();
    }

    void print() const {
        std::cout << "Area: " << area() << "\n";
    }
};

class Circle : public Shape<Circle> {
    double radius_;
public:
    explicit Circle(double r) : radius_(r) {}
    double area_impl() const { return 3.14159265 * radius_ * radius_; }
};

class Rectangle : public Shape<Rectangle> {
    double w_, h_;
public:
    Rectangle(double w, double h) : w_(w), h_(h) {}
    double area_impl() const { return w_ * h_; }
};

template <typename ShapeType>
void displayArea(const Shape<ShapeType>& s) {
    s.print(); // Compile-time polymorphism, no vtable
}

int main() {
    Circle c(5.0);
    Rectangle r(3.0, 4.0);
    displayArea(c); // "Area: 78.5398"
    displayArea(r); // "Area: 12"
}

```

No virtual functions, no vtable overhead – the dispatch is resolved entirely at compile time.

7.3.5 Combining both kinds of polymorphism

In real systems you often use both. Runtime polymorphism lets you store heterogeneous objects in containers; compile-time polymorphism gives you zero-overhead abstraction:

```
// Runtime polymorphism for a plugin system
class Renderer {
public:
    virtual ~Renderer() = default;
    virtual void render(const Scene& scene) = 0;
};

// Compile-time polymorphism for a high-performance math library
template <typename T>
T dot(const std::vector<T>& a, const std::vector<T>& b) {
    T result = T();
    for (std::size_t i = 0; i < a.size(); ++i)
        result += a[i] * b[i];
    return result;
}
```

7.3.6 Things to Remember

- Both classes and templates support interfaces and polymorphism.
- For classes, interfaces are explicit and centered on function signatures. Polymorphism occurs at runtime through virtual functions.
- For template parameters, interfaces are implicit and based on valid expressions. Polymorphism occurs at compile time through template instantiation and function overloading resolution.
- Implicit interface constraints are on expressions, not on types. An expression like `w.size() > 10` does not mandate a particular return type for `size()` – only that the complete expression is valid and yields something convertible to `bool`.

7.4 Item 42: Understand the two meanings of typename

7.5 Visual Summary

ITEM 42: UNDERSTAND THE TWO MEANINGS OF TYPENAME

1. Template parameter list → typename means type parameter.
2. Nested dependent name → compiler cannot know if name is type or value.
3. Use `typename T::value_type` → tell compiler it is a type.
4. Base class lists and `mem-init` lists have special rules.
5. Meaning: `typename` sometimes declares a type parameter, sometimes disambiguates a dependent type.

7.5.1 Meaning 1: declaring template type parameters

In a template parameter list, `typename` and `class` are interchangeable:

```
template <typename T>    // typename
class Widget { };

template <class T>       // class -- means exactly the same thing
class Widget { };
```

There is no semantic difference. `typename` was introduced later, and some programmers prefer it because it makes clear that `T` need not be a class type – it could be `int`, `double`, or any other type. But both are valid, and the choice is purely stylistic.

7.5.2 Meaning 2: disambiguating dependent names

This second meaning is where subtlety and real-world bugs live.

7.5.2.1 Dependent and non-dependent names Inside a template, names that depend on a template parameter are called **dependent names**. Names that do not depend on a template parameter are **non-dependent names**:

```
template <typename T>
void print(const T& container) {
    // "std::cout" is a non-dependent name -- it doesn't depend on T.
    // "container.size()" involves a dependent name -- size() depends on T.

    // T::const_iterator is a dependent name -- it depends on T.
    typename T::const_iterator it = container.begin();

    for (; it != container.end(); ++it)
        std::cout << *it << "\n";
}
```

7.5.2.2 The parsing ambiguity When the compiler sees `T::const_iterator`, it does not know whether `const_iterator` is a type or a value. `T` is a template parameter, and until instantiation, the compiler cannot look inside `T` to find out. Consider:

```
template <typename T>
void foo() {
    T::something * ptr; // Ambiguity!
    // Is this a declaration of a pointer named "ptr" of type T::something?
    // Or is this a multiplication of T::something by ptr?
}
```

The C++ standard resolves this ambiguity with a rule: **a dependent qualified name is assumed to be a non-type (a value) unless preceded by `typename`.**

```
template <typename T>
void foo() {
    typename T::something * ptr; // Now unambiguous: ptr is a pointer to T::something
}
```

7.5.2.3 Real-world examples of when typename is required Accessing nested typedefs from template parameters:

```
template <typename Container>
void printAll(const Container& c) {
    // ERROR without typename: compiler assumes value_type is a value
    // typename Container::value_type element;

    typename Container::value_type element; // Correct
    typename Container::const_iterator it; // Correct

    for (it = c.begin(); it != c.end(); ++it) {
        element = *it;
        std::cout << element << " ";
    }
    std::cout << "\n";
}
```

Using nested types in function return types:

```
template <typename Container>
typename Container::iterator findFirst(Container& c,
                                       const typename Container::value_type& val) {
    return std::find(c.begin(), c.end(), val);
}
```

With std::iterator_traits:

```
template <typename Iterator>
void printIteratorValue(Iterator it) {
    // iterator_traits<Iterator>::value_type is a dependent type
    typename std::iterator_traits<Iterator>::value_type val = *it;
    std::cout << val << "\n";
}
```

Inside class templates with dependent base classes:

```
template <typename T>
class Derived : public Base<T> {
public:
    void doWork() {
        // Base<T>::NestedType is a dependent name
        typename Base<T>::NestedType obj;
        // ...
    }
};
```

7.5.2.4 Where typename must NOT appear There are two contexts where typename is not allowed even though a dependent name appears:

1. Base class lists:

```
template <typename T>
class Derived : public Base<T>::Nested { // typename NOT allowed here
    // ...
};
```

2. Member initialization lists:

```
template <typename T>
class Derived : public Base<T>::Nested {
public:
    explicit Derived(int x)
        : Base<T>::Nested(x) { // typename NOT allowed here
    }
};
```

The rationale: in both of these positions, the name can only be a type, so no disambiguation is needed.

7.5.2.5 Simplifying with typedef and using The verbosity of typename can be tamed with type aliases:

```
template <typename Iterator>
void workWithIterator(Iterator begin, Iterator end) {
    // Verbose:
    // typename std::iterator_traits<Iterator>::value_type temp = *begin;

    // Better: create a local typedef
    typedef typename std::iterator_traits<Iterator>::value_type value_type;
    value_type temp = *begin;

    // Or with C++11 using:
    using difference_type = typename std::iterator_traits<Iterator>::difference_type;
    difference_type dist = std::distance(begin, end);

    std::cout << "First: " << temp << ", distance: " << dist << "\n";
}
```

7.5.2.6 Compiler-specific behavior and portability Some compilers (notably older versions of MSVC) historically did not enforce the typename requirement strictly, accepting code without typename where it was technically required. This means code that compiles on one compiler may fail on another. Always use typename where the standard demands it.

7.5.2.7 C++20 relaxation C++20 relaxed the requirement for typename in many contexts where the compiler can unambiguously determine that a dependent name must be a type (e.g., in `static_cast`, new expressions, trailing return types, default arguments of type parameters). However, understanding the original rule remains important for reading legacy code and for contexts where the relaxation does not apply.


```
// C++20: typename not required here because static_cast target must be a type
template <typename T>
void convert(T val) {
    auto result = static_cast<T::value_type>(val); // OK in C++20
}
```

7.5.3 Things to Remember

- When declaring template parameters, `class` and `typename` are interchangeable.
- Use `typename` to identify nested dependent type names, except in base class lists or as a base class identifier in a member initialization list.
- A nested dependent name is assumed to be a non-type unless you precede it with `typename`.
- Use `typedef` or `using` to create convenient aliases for long `typename`-qualified dependent names.

7.6 Item 43: Know how to access names in templated base classes

7.7 Visual Summary

ITEM 43: KNOW HOW TO ACCESS NAMES IN TEMPLATIZED BASE CLASSES

1. Template derived from dependent base → base contents depend on T.
2. Unqualified lookup does not assume dependent base members exist.
3. Use `this->name`, `Base<T>::name`, or using `Base<T>::name`.
4. This makes dependency explicit to compiler and reader.
5. Meaning: dependent base names must be made visible deliberately.

7.7.1 The problem: names in dependent base classes are invisible

Consider a messaging system with compile-time selection of the transport:

```
class CompanyA {
public:
    void sendCleartext(const std::string& msg) { /* ... */ }
    void sendEncrypted(const std::string& msg) { /* ... */ }
};

class CompanyB {
public:
    void sendCleartext(const std::string& msg) { /* ... */ }
    void sendEncrypted(const std::string& msg) { /* ... */ }
};

template <typename Company>
class MsgSender {
public:
    void sendClear(const std::string& info) {
        Company c;
        c.sendCleartext(info);
    }

    void sendSecret(const std::string& info) {
        Company c;
        c.sendEncrypted(info);
    }
};
```

Now a derived class that logs every message:

```
template <typename Company>
class LoggingMsgSender : public MsgSender<Company> {
public:
    void sendClearMsg(const std::string& info) {
        // write "before sending" to log
        sendClear(info); // ERROR! Won't compile!
        // write "after sending" to log
    }
};
```

This fails because the compiler refuses to look inside `MsgSender<Company>` for `sendClear`. Why? Because `Company` is a template parameter, and there could be a **total specialization** of `MsgSender` that does not have `sendClear`:

```
// A company for which encrypted-only communication is mandated.
class CompanyZ {
public:
    void sendEncrypted(const std::string& msg) { /* ... */ }
    // No sendCleartext!
};

// Total specialization: MsgSender<CompanyZ> has no sendClear()
template <>
class MsgSender<CompanyZ> {
public:
    void sendSecret(const std::string& info) {
        CompanyZ c;
        c.sendEncrypted(info);
    }
    // No sendClear() here!
};
```

Because `MsgSender<CompanyZ>` does not have `sendClear`, the compiler is right to refuse to assume it exists in the general case. The standard says: **names in dependent base classes are not examined during unqualified lookup**.

7.7.2 Solution 1: `this->`

Prefix the call with `this->` to make it a dependent expression, deferring lookup to instantiation time:

```
template <typename Company>
class LoggingMsgSender : public MsgSender<Company> {
public:
    void sendClearMsg(const std::string& info) {
        logToFile("Before sending");
        this->sendClear(info); // OK: defers lookup to instantiation time
        logToFile("After sending");
    }
};
```

This is the most common and recommended approach.

7.7.3 Solution 2: using declaration

Bring the base class name into the derived class's scope:

```

template <typename Company>
class LoggingMsgSender : public MsgSender<Company> {
public:
    using MsgSender<Company>::sendClear; // Make sendClear visible

    void sendClearMsg(const std::string& info) {
        logToFile("Before sending");
        sendClear(info); // OK: using declaration made it visible
        logToFile("After sending");
    }
};

```

7.7.4 Solution 3: Explicit qualification

Qualify the call with the base class name:

```

template <typename Company>
class LoggingMsgSender : public MsgSender<Company> {
public:
    void sendClearMsg(const std::string& info) {
        logToFile("Before sending");
        MsgSender<Company>::sendClear(info); // OK but problematic
        logToFile("After sending");
    }
};

```

This works but has a significant drawback: **it suppresses virtual dispatch**. If `sendClear` were virtual, this would always call the base class version, never an overridden version. For this reason, `this->` or `using` declarations are generally preferred.

7.7.5 A comprehensive real-world example

Consider a policy-based design for database operations:

```

// Policy for SQL generation
template <typename Dialect>
class SQLGenerator {
public:
    std::string generateSelect(const std::string& table,
                              const std::vector<std::string>& cols) {
        return Dialect::selectPrefix() + buildColumnList(cols) + " FROM " + table;
    }

protected:
    std::string buildColumnList(const std::vector<std::string>& cols) {
        std::string result;
        for (size_t i = 0; i < cols.size(); ++i) {
            if (i > 0) result += ", ";
            result += Dialect::quoteIdentifier(cols[i]);
        }
        return result;
    }

    std::string escapeString(const std::string& s) {
        return Dialect::escapeImpl(s);
    }
};

// Extended generator that adds WHERE clause support
template <typename Dialect>
class FilteredSQLGenerator : public SQLGenerator<Dialect> {

```

```

public:
    // Must use this-> or using declarations to access base class members

    using SQLGenerator<Dialect>::generateSelect;
    using SQLGenerator<Dialect>::escapeString;

    std::string generateFilteredSelect(
        const std::string& table,
        const std::vector<std::string>& cols,
        const std::string& whereClause) {
        // Without the using declarations above, these calls would fail:
        std::string base = generateSelect(table, cols);
        return base + " WHERE " + escapeString(whereClause);
    }
};

// Even deeper inheritance -- same rules apply at every level
template <typename Dialect>
class PaginatedSQLGenerator : public FilteredSQLGenerator<Dialect> {
public:
    std::string generatePaginatedSelect(
        const std::string& table,
        const std::vector<std::string>& cols,
        const std::string& whereClause,
        int limit, int offset) {
        // this-> needed because FilteredSQLGenerator<Dialect> is a dependent base
        std::string query = this->generateFilteredSelect(table, cols, whereClause);
        return query + " LIMIT " + std::to_string(limit)
            + " OFFSET " + std::to_string(offset);
    }
};

```

7.7.6 Accessing dependent base class types

The problem extends to types as well. Accessing a typedef or nested type from a dependent base class requires both typename (Item 42) and one of the three solutions above:

```

template <typename T>
class Base {
public:
    using value_type = T;
    using container_type = std::vector<T>;
};

template <typename T>
class Derived : public Base<T> {
public:
    // WRONG: Base<T>::value_type is not found
    // value_type getData();

    // CORRECT: use typename + full qualification
    typename Base<T>::value_type getData() {
        typename Base<T>::container_type storage;
        storage.push_back(T());
        return storage.front();
    }

    // ALTERNATIVE: bring the type in with using
    using typename Base<T>::value_type;
    // Now value_type can be used unqualified in this class

```

```
};
```

7.7.7 Multiple dependent base classes

When inheriting from multiple templated bases, you need to resolve ambiguity for each:

```
template <typename T>
class Serializable {
public:
    std::string serialize() const { /* ... */ return ""; }
};

template <typename T>
class Printable {
public:
    void print() const { /* ... */ }
};

template <typename T>
class Document : public Serializable<T>, public Printable<T> {
public:
    void save() {
        std::string data = this->serialize(); // From Serializable<T>
        this->print();                       // From Printable<T>
        // store data...
    }
};
```

7.7.8 Things to Remember

- In derived class templates, refer to names in base class templates via a `this->` prefix, via using declarations, or via an explicit base class qualification.
- The compiler does not search dependent base classes during unqualified name lookup because a specialization of the base class template might not contain the name.
- Prefer `this->` or using declarations over explicit qualification, because explicit qualification inhibits virtual dispatch.

7.8 Item 44: Factor parameter-independent code out of templates

7.9 Visual Summary

ITEM 44: FACTOR PARAMETER-INDEPENDENT CODE OUT OF TEMPLATES

1. Template body repeated for many T/N values -> code bloat risk.
2. Logic independent of template parameter -> move to non-template base/helper.
3. Parameter-dependent shell remains small.
4. Tradeoff -> preserve type safety while sharing compiled code.
5. Meaning: templates should not duplicate code that does not depend on the template.

7.9.1 The code bloat problem

Templates generate code for each set of template arguments. If the generated code contains logic that does not actually depend on the template parameters, you get **code bloat** – multiple identical (or nearly identical)

copies of functions in your binary.

```
// Naive implementation: generates separate code for every N
template <typename T, std::size_t N>
class SquareMatrix {
public:
    void invert() {
        // Complex matrix inversion algorithm
        // This code is identical for SquareMatrix<double, 5> and
        // SquareMatrix<double, 10> except for the value of N.
        // But the compiler generates it twice.
    }

private:
    T data_[N * N];
};

// Each of these generates a SEPARATE copy of invert():
SquareMatrix<double, 5> m5;
SquareMatrix<double, 10> m10;
SquareMatrix<double, 5> m5b; // Same as m5 -- no new code generated
SquareMatrix<float, 5> mf5;  // Different T -- new code is expected
```

`SquareMatrix<double, 5>::invert()` and `SquareMatrix<double, 10>::invert()` will be separate functions even though their logic is fundamentally the same – only the dimension differs.

7.9.2 Solution: factor parameter-independent code into a base class

Move the code that does not depend on the non-type parameter into a base class that takes the dimension as a runtime parameter:

```
// Base class: knows dimension at runtime, not compile time
template <typename T>
class SquareMatrixBase {
protected:
    SquareMatrixBase(std::size_t n, T* pMem)
        : size_(n), pData_(pMem) {}

    void invert() {
        // Single copy of inversion code, parameterized by size_
        // Works for any dimension
        doInversion(pData_, size_);
    }

    void setDataPtr(T* ptr) { pData_ = ptr; }

private:
    std::size_t size_;
    T* pData_;

    void doInversion(T* data, std::size_t n) {
        // Actual inversion logic -- only one copy exists per T
        // ...
    }
};

// Derived class: adds the compile-time dimension and storage
template <typename T, std::size_t N>
class SquareMatrix : private SquareMatrixBase<T> {
public:
    SquareMatrix()
        : SquareMatrixBase<T>(N, data_) {}
};
```

```

    // Thin inline wrapper -- compiled per N, but trivially small
    void invert() {
        SquareMatrixBase<T>::invert();
    }

private:
    T data_[N * N];
};

```

Now `SquareMatrix<double, 5>::invert()` and `SquareMatrix<double, 10>::invert()` both call the same `SquareMatrixBase<double>::invert()`. The per-N code is just a tiny inline forwarding function.

7.9.3 Storage strategies

The base class needs access to the matrix data. There are several strategies:

Strategy 1: Pointer from derived (shown above)

```

template <typename T>
class SquareMatrixBase {
protected:
    SquareMatrixBase(std::size_t n, T* pMem) : size_(n), pData_(pMem) {}
    // ...
private:
    std::size_t size_;
    T* pData_;
};

template <typename T, std::size_t N>
class SquareMatrix : private SquareMatrixBase<T> {
    T data_[N * N]; // Storage is in the derived class
public:
    SquareMatrix() : SquareMatrixBase<T>(N, data_) {}
};

```

Strategy 2: Heap allocation in the base class

```

template <typename T>
class SquareMatrixBase {
protected:
    explicit SquareMatrixBase(std::size_t n)
        : size_(n), pData_(new T[n * n]()) {}

    ~SquareMatrixBase() { delete[] pData_; }

    void invert() { /* uses pData_ and size_ */ }

private:
    std::size_t size_;
    T* pData_;
};

template <typename T, std::size_t N>
class SquareMatrix : private SquareMatrixBase<T> {
public:
    SquareMatrix() : SquareMatrixBase<T>(N) {}
    using SquareMatrixBase<T>::invert;
};

```

7.9.4 Non-type parameters are the most common bloat source

Non-type template parameters (like `std::size_t N`) are the most obvious source of bloat because the code is often identical across different values of `N`. But type parameters can also cause bloat:

```
// These three instantiations generate three copies of all member functions,  
// but on most platforms int*, long*, and Widget* are all the same size.  
template <typename T>  
class SmartPtr {  
public:  
    T* get() const { return ptr_; }  
    void reset(T* p) { delete ptr_; ptr_ = p; }  
    // 20 more member functions...  
private:  
    T* ptr_;  
};  
  
SmartPtr<int> p1;  
SmartPtr<long> p2;  
SmartPtr<Widget> p3;
```

The fix: implement the core logic in terms of `void*` and have the typed template provide thin inline wrappers:

```
// Untyped base -- one copy of the implementation  
class SmartPtrBase {  
protected:  
    SmartPtrBase(void* p) : ptr_(p) {}  
    void* get() const { return ptr_; }  
    void setPtr(void* p) { ptr_ = p; }  
private:  
    void* ptr_;  
};  
  
// Typed wrapper -- only inline casts, no real code duplication  
template <typename T>  
class SmartPtr : private SmartPtrBase {  
public:  
    explicit SmartPtr(T* p = nullptr) : SmartPtrBase(p) {}  
  
    T* get() const {  
        return static_cast<T*>(SmartPtrBase::get());  
    }  
  
    void reset(T* p) {  
        delete static_cast<T*>(SmartPtrBase::get());  
        SmartPtrBase::setPtr(p);  
    }  
};
```

This pattern is used in real-world standard library implementations. For example, many implementations of `std::vector` share a single implementation for all pointer types.

7.9.5 Measuring bloat

Before optimizing, measure:


```
# On Linux/Mac, list symbol sizes in an object file
nm --size-sort --print-size your_binary | c++filt | grep "SquareMatrix"

# Or use bloaty for a higher-level view
bloaty your_binary -d compileunits
```

7.9.6 The trade-off

Factoring out parameter-independent code reduces binary size but can: - Reduce compile-time optimization opportunities (the compiler knows the dimension at compile time and can unroll loops, vectorize, etc.) - Add indirection through base class pointers - Make the code harder to read

For hot inner loops (matrix math, signal processing), keeping the non-type parameter may be worth the bloat because the compiler can aggressively optimize. For less performance-critical code, factoring it out is usually the right call.

```
// When performance is critical, keep the compile-time parameter:
template <typename T, std::size_t N>
class SmallMatrix {
public:
    // Compiler can fully unroll this loop for small N
    void multiply(const SmallMatrix& rhs, SmallMatrix& result) const {
        for (std::size_t i = 0; i < N; ++i)
            for (std::size_t j = 0; j < N; ++j) {
                T sum = T();
                for (std::size_t k = 0; k < N; ++k)
                    sum += data_[i * N + k] * rhs.data_[k * N + j];
                result.data_[i * N + j] = sum;
            }
    }
private:
    T data_[N * N];
};
```

7.9.7 Things to Remember

- Templates generate multiple classes and multiple functions, so any template code not dependent on a template parameter causes bloat.
- Bloat due to non-type template parameters can often be eliminated by replacing template parameters with function parameters or class data members.
- Bloat due to type parameters can be reduced by sharing implementations for instantiation types that have the same binary representations (e.g., all pointer types can share a single `void*`-based implementation).

7.10 Item 45: Use member function templates to accept “all compatible types”

7.11 Visual Summary

ITEM 45: USE MEMBER FUNCTION TEMPLATES TO ACCEPT "ALL COMPATIBLE TYPES"

1. Template class `SmartPtr<T>` → compatible `SmartPtr<U>` conversions may be useful.
2. Normal copy ctor handles only exact same `T`.
3. Member function template accepts `U` when `U*` converts to `T*`.
4. Still keep normal copy operations for exact type behavior.
5. Meaning: member templates model families of compatible types.

7.11.1 The problem: implicit conversions don't work across template instantiations

In a class hierarchy, raw pointers support implicit conversions:

```
class Base { };
class Derived : public Base { };
class AnotherDerived : public Base { };
```

```
Base* pb = new Derived;           // OK: implicit upcast
Base* pb2 = new AnotherDerived;   // OK
```

But smart pointer templates do not automatically support these conversions:

```
template <typename T>
class SmartPtr {
public:
    explicit SmartPtr(T* rawPtr) : ptr_(rawPtr) {}
    T& operator*() const { return *ptr_; }
    T* operator->() const { return ptr_; }
private:
    T* ptr_;
};

SmartPtr<Base> pb = SmartPtr<Derived>(new Derived); // ERROR!
// SmartPtr<Derived> and SmartPtr<Base> are completely unrelated types.
```

You cannot enumerate all possible conversions – new derived classes can be added at any time. You need a **generalized** copy constructor and assignment operator.

7.11.2 Solution: member function templates (generalized copy operations)

```
template <typename T>
class SmartPtr {
public:
    explicit SmartPtr(T* rawPtr = nullptr) : ptr_(rawPtr) {}

    // Generalized copy constructor: accepts SmartPtr of any compatible type
    template <typename U>
    SmartPtr(const SmartPtr<U>& other)
        : ptr_(other.get()) { // Compiles only if U* converts to T*
    }

    // Generalized assignment operator
    template <typename U>
    SmartPtr& operator=(const SmartPtr<U>& other) {
        ptr_ = other.get(); // Compiles only if U* converts to T*
        return *this;
    }
}
```

```

T* get() const { return ptr_; }
T& operator*() const { return *ptr_; }
T* operator->() const { return ptr_; }

private:
    T* ptr_;
};

```

The key line is `ptr_(other.get())`. This initializes a `T*` from a `U*`. If `U*` is not implicitly convertible to `T*`, the code will not compile. This means:

```

SmartPtr<Base> pb = SmartPtr<Derived>(new Derived);           // OK: Derived* -> Base*
SmartPtr<Derived> pd = SmartPtr<Base>(new Base);             // ERROR: Base* -> Derived*
SmartPtr<const Base> pcb = SmartPtr<Derived>(new Derived);   // OK: Derived* -> const Base*

```

The type system enforces the same conversion rules as raw pointers, but automatically.

7.11.3 How `std::shared_ptr` does it

`std::shared_ptr` uses exactly this technique:

```

// Simplified sketch of std::shared_ptr
template <typename T>
class shared_ptr {
public:
    // Normal constructor
    explicit shared_ptr(T* ptr = nullptr);

    // Generalized copy constructor
    template <typename Y>
    shared_ptr(const shared_ptr<Y>& r);

    // Generalized move constructor
    template <typename Y>
    shared_ptr(shared_ptr<Y>&& r) noexcept;

    // Generalized copy assignment
    template <typename Y>
    shared_ptr& operator=(const shared_ptr<Y>& r);

    // Generalized move assignment
    template <typename Y>
    shared_ptr& operator=(shared_ptr<Y>&& r) noexcept;

    // Construct from unique_ptr (taking ownership)
    template <typename Y, typename Deleter>
    shared_ptr(std::unique_ptr<Y, Deleter>&& r);

    // ...
};

```

This allows all of the following:

```

class Base { public: virtual ~Base() = default; };
class Derived : public Base { };

std::shared_ptr<Derived> pd = std::make_shared<Derived>();
std::shared_ptr<Base> pb = pd;           // Generalized copy ctor
std::shared_ptr<const Base> pcb = pd;    // Also works
std::shared_ptr<Base> pb2 = std::move(pd); // Generalized move ctor

std::unique_ptr<Derived> ud = std::make_unique<Derived>();
std::shared_ptr<Base> pb3 = std::move(ud); // From unique_ptr

```

7.11.4 Critical: member templates do NOT suppress compiler-generated functions

A member template that looks like a copy constructor is NOT a copy constructor. The compiler will still generate the default copy constructor and copy assignment operator:

```

template <typename T>
class SmartPtr {
public:
    // Generalized copy constructor (member template)
    template <typename U>
    SmartPtr(const SmartPtr<U>& other) : ptr_(other.get()) {
        std::cout << "generalized copy ctor\n";
    }

    // The compiler STILL generates:
    // SmartPtr(const SmartPtr<T>& other); -- default copy ctor
    // SmartPtr& operator=(const SmartPtr<T>& other); -- default copy assignment
};

SmartPtr<int> a(new int(42));
SmartPtr<int> b(a); // Calls the COMPILER-GENERATED copy ctor, NOT the template!

```

If the compiler-generated versions do the wrong thing (and for resource-managing classes they usually do), you must declare them explicitly:

```

template <typename T>
class SmartPtr {
public:
    // Normal copy constructor -- must be declared explicitly
    SmartPtr(const SmartPtr& other) : ptr_(other.ptr_) {
        // proper reference counting, deep copy, etc.
    }

    // Normal copy assignment -- must be declared explicitly
    SmartPtr& operator=(const SmartPtr& other) {
        if (this != &other) {
            // proper cleanup and copy
            ptr_ = other.ptr_;
        }
        return *this;
    }

    // Generalized copy constructor
    template <typename U>
    SmartPtr(const SmartPtr<U>& other) : ptr_(other.get()) {
        // proper reference counting, deep copy, etc.
    }

    // Generalized copy assignment
    template <typename U>

```

```

SmartPtr& operator=(const SmartPtr<U>& other) {
    ptr_ = other.get();
    return *this;
}

T* get() const { return ptr_; }

private:
    T* ptr_;
};

```

7.11.5 A complete real-world example: a reference-counted smart pointer

```

#include <atomic>
#include <iostream>
#include <utility>

struct RefCount {
    std::atomic<int> count{1};
};

template <typename T>
class SharedPtr {
    template <typename U> friend class SharedPtr; // All SharedPtr<U> are friends

public:
    explicit SharedPtr(T* p = nullptr)
        : ptr_(p), refCount_(p ? new RefCount : nullptr) {}

    // Copy constructor (same type)
    SharedPtr(const SharedPtr& other)
        : ptr_(other.ptr_), refCount_(other.refCount_) {
        incrementRef();
    }

    // Generalized copy constructor (different compatible type)
    template <typename U>
    SharedPtr(const SharedPtr<U>& other)
        : ptr_(other.ptr_), // Compiles only if U* -> T*
          refCount_(other.refCount_) {
        incrementRef();
    }

    // Move constructor (same type)
    SharedPtr(SharedPtr&& other) noexcept
        : ptr_(other.ptr_), refCount_(other.refCount_) {
        other.ptr_ = nullptr;
        other.refCount_ = nullptr;
    }

    // Generalized move constructor
    template <typename U>
    SharedPtr(SharedPtr<U>&& other) noexcept
        : ptr_(other.ptr_), refCount_(other.refCount_) {
        other.ptr_ = nullptr;
        other.refCount_ = nullptr;
    }

    ~SharedPtr() { decrementRef(); }

```

```

// Copy assignment (same type)
SharedPtr& operator=(const SharedPtr& other) {
    SharedPtr tmp(other);
    swap(tmp);
    return *this;
}

// Generalized copy assignment
template <typename U>
SharedPtr& operator=(const SharedPtr<U>& other) {
    SharedPtr tmp(other);
    swap(tmp);
    return *this;
}

T* get() const { return ptr_; }
T& operator*() const { return *ptr_; }
T* operator->() const { return ptr_; }

int useCount() const {
    return refCount_ ? refCount_>count.load() : 0;
}

private:
void incrementRef() {
    if (refCount_) refCount_>count.fetch_add(1);
}

void decrementRef() {
    if (refCount_ && refCount_>count.fetch_sub(1) == 1) {
        delete ptr_;
        delete refCount_;
    }
}

void swap(SharedPtr& other) noexcept {
    std::swap(ptr_, other.ptr_);
    std::swap(refCount_, other.refCount_);
}

T* ptr_;
RefCount* refCount_;
};

// Usage:
struct Animal { virtual ~Animal() = default; virtual void speak() = 0; };
struct Dog : Animal { void speak() override { std::cout << "Woof!\n"; } };
struct Cat : Animal { void speak() override { std::cout << "Meow!\n"; } };

int main() {
    SharedPtr<Dog> dog(new Dog);
    SharedPtr<Animal> animal = dog;           // Generalized copy ctor
    animal->speak();                          // "Woof!"
    std::cout << "Use count: " << dog.useCount() << "\n"; // 2

    SharedPtr<Cat> cat(new Cat);
    animal = cat;                             // Generalized copy assignment
    animal->speak();                          // "Meow!"
    std::cout << "Use count: " << cat.useCount() << "\n"; // 2
}

```

```
}
```

7.11.6 Things to Remember

- Use member function templates to generate functions that accept all compatible types.
- If you declare member templates for generalized copy construction or generalized assignment, you still need to declare the normal copy constructor and copy assignment operator. The compiler will generate them even when member templates exist.
- The conversion safety comes from the underlying pointer assignment – if U^* cannot convert to T^* , the template instantiation will fail at compile time.

7.12 Item 46: Define non-member functions inside templates when type conversions are desired

7.13 Visual Summary

ITEM 46: DEFINE NON-MEMBER FUNCTIONS INSIDE TEMPLATES WHEN TYPE CONVERSIONS

1. Class template + non-member operator $\rightarrow$ conversions on both operands desired.
2. Function template argument deduction does not use implicit conversions.
3. Define friend non-member inside class template $\rightarrow$ concrete function generated with class.
4. Then ordinary overload resolution can use conversions.
5. Meaning: inside-template friend functions combine symmetry with usable conversions.

7.13.1 The problem: templates and implicit type conversions don't mix

Recall from Item 24 that mixed-mode arithmetic (e.g., `Rational * int`) requires non-member functions so that implicit conversions can apply to all arguments. When you templatize `Rational`, this breaks:

```
template <typename T>
class Rational {
public:
    Rational(const T& numerator = 0, const T& denominator = 1)
        : num_(numerator), den_(denominator) {}

    const T& numerator() const { return num_; }
    const T& denominator() const { return den_; }

private:
    T num_, den_;
};

// Non-member operator*
template <typename T>
const Rational<T> operator*(const Rational<T>& lhs, const Rational<T>& rhs) {
    return Rational<T>(lhs.numerator() * rhs.numerator(),
        lhs.denominator() * rhs.denominator());
}

Rational<int> oneHalf(1, 2);
Rational<int> result = oneHalf * 2; // ERROR!
```

Why does this fail? During template argument deduction, implicit conversions are **not** considered. The compiler sees `oneHalf * 2` and tries to deduce `T` for `operator*`: - From `oneHalf` (type `Rational<int>`), it deduces `T = int`. - From `2` (type `int`), it cannot deduce `T` because `int` is not `Rational<something>`.

Template argument deduction fails, and the compiler never even considers the implicit conversion from `int` to `Rational<int>`.

7.13.2 Solution: declare the function as a friend inside the class template

When a class template is instantiated, the declarations of its friend functions become known. Since they are not themselves templates (they are specific functions associated with a specific instantiation), implicit conversions apply to their arguments:

```
template <typename T>
class Rational {
public:
    Rational(const T& numerator = 0, const T& denominator = 1)
        : num_(numerator), den_(denominator) {}

    const T& numerator() const { return num_; }
    const T& denominator() const { return den_; }

    // Friend declaration AND definition inside the class
    friend const Rational operator*(const Rational& lhs, const Rational& rhs) {
        return Rational(lhs.numerator() * rhs.numerator(),
                        lhs.denominator() * rhs.denominator());
    }

private:
    T num_, den_;
};
```

Now:

```
Rational<int> oneHalf(1, 2);
Rational<int> result = oneHalf * 2; // OK!
// 1) oneHalf is Rational<int>, so the compiler instantiates Rational<int>
// 2) This makes the friend function operator*(const Rational<int>&, const Rational<int>&)
//    a known, non-template function
// 3) For non-template functions, implicit conversions ARE considered
// 4) 2 is converted to Rational<int>(2) via the converting constructor

Rational<int> result2 = 2 * oneHalf; // Also OK -- conversion on first arg
```

7.13.3 Why the friend must be defined inside the class

If you only declare the friend inside the class but define it outside, you get a linker error:


```

template <typename T>
class Rational {
public:
    // ...
    friend const Rational operator*(const Rational& lhs, const Rational& rhs);
    // Just a declaration -- the linker won't find the definition
};

// This is a function TEMPLATE, not the friend function!
template <typename T>
const Rational<T> operator*(const Rational<T>& lhs, const Rational<T>& rhs) {
    return Rational<T>(lhs.numerator() * rhs.numerator(),
                       lhs.denominator() * rhs.denominator());
}

```

The friend declaration creates a non-template function `operator*(const Rational<int>&, const Rational<int>&)`. The definition outside the class is a function template. They are different entities. The linker finds the declaration but no matching definition.

Defining the friend inside the class body solves this because the definition is right there.

7.13.4 Using a helper to keep the friend function short

If the implementation is long, you can delegate to a helper function template:

```

// Helper: a function template (not subject to implicit conversion, but
// that's OK -- it's only called from the friend function)
template <typename T>
const Rational<T> doMultiply(const Rational<T>& lhs, const Rational<T>& rhs) {
    return Rational<T>(lhs.numerator() * rhs.numerator(),
                       lhs.denominator() * rhs.denominator());
}

template <typename T>
class Rational {
public:
    Rational(const T& numerator = 0, const T& denominator = 1)
        : num_(numerator), den_(denominator) {}

    const T& numerator() const { return num_; }
    const T& denominator() const { return den_; }

    // Short friend: just delegates
    friend const Rational operator*(const Rational& lhs, const Rational& rhs) {
        return doMultiply(lhs, rhs);
        // Here, lhs and rhs are already Rational<T>, so template argument
        // deduction for doMultiply succeeds.
    }

private:
    T num_, den_;
};

```

7.13.5 A more complete example: arithmetic on a Vector class

```

#include <iostream>
#include <cmath>

template <typename T>
class Vec3 {
public:
    Vec3(T x = T(), T y = T(), T z = T()) : x_(x), y_(y), z_(z) {}

    T x() const { return x_; }
    T y() const { return y_; }
    T z() const { return z_; }

    T length() const { return std::sqrt(x_*x_ + y_*y_ + z_*z_); }

    // All arithmetic operators need type conversions on both sides,
    // so they are friends defined inside the class.

    friend Vec3 operator+(const Vec3& a, const Vec3& b) {
        return Vec3(a.x_ + b.x_, a.y_ + b.y_, a.z_ + b.z_);
    }

    friend Vec3 operator-(const Vec3& a, const Vec3& b) {
        return Vec3(a.x_ - b.x_, a.y_ - b.y_, a.z_ - b.z_);
    }

    // Scalar multiplication: Vec3 * scalar and scalar * Vec3
    friend Vec3 operator*(const Vec3& v, const T& s) {
        return Vec3(v.x_ * s, v.y_ * s, v.z_ * s);
    }

    friend Vec3 operator*(const T& s, const Vec3& v) {
        return v * s;
    }

    // Dot product
    friend T dot(const Vec3& a, const Vec3& b) {
        return a.x_ * b.x_ + a.y_ * b.y_ + a.z_ * b.z_;
    }

    // Cross product
    friend Vec3 cross(const Vec3& a, const Vec3& b) {
        return Vec3(a.y_ * b.z_ - a.z_ * b.y_,
                    a.z_ * b.x_ - a.x_ * b.z_,
                    a.x_ * b.y_ - a.y_ * b.x_);
    }

    friend std::ostream& operator<<(std::ostream& os, const Vec3& v) {
        return os << "(" << v.x_ << ", " << v.y_ << ", " << v.z_ << ")";
    }

private:
    T x_, y_, z_;
};

int main() {
    Vec3<double> a(1.0, 2.0, 3.0);
    Vec3<double> b(4.0, 5.0, 6.0);

    std::cout << "a + b = " << (a + b) << "\n";           // (5, 7, 9)

```

```

std::cout << "a * 2 = " << (a * 2) << "\n";           // (2, 4, 6) -- int 2 converts to double
std::cout << "3 * b = " << (3 * b) << "\n";           // (12, 15, 18) -- int 3 converts to double
std::cout << "dot(a,b) = " << dot(a, b) << "\n";       // 32
std::cout << "cross(a,b) = " << cross(a, b) << "\n";    // (-3, 6, -3)
}

```

7.13.6 Comparison operators with friend in templates

```

template <typename T>
class Money {
public:
    explicit Money(T amount) : amount_(amount) {}

    T amount() const { return amount_; }

    friend bool operator==(const Money& a, const Money& b) {
        return a.amount_ == b.amount_;
    }

    friend bool operator<(const Money& a, const Money& b) {
        return a.amount_ < b.amount_;
    }

    friend bool operator!=(const Money& a, const Money& b) { return !(a == b); }
    friend bool operator>(const Money& a, const Money& b) { return b < a; }
    friend bool operator<=(const Money& a, const Money& b) { return !(b < a); }
    friend bool operator>=(const Money& a, const Money& b) { return !(a < b); }

private:
    T amount_;
};

```

7.13.7 Things to Remember

- When writing a class template that offers functions related to the template that support implicit type conversions on all parameters, define those functions as friends inside the class template.
- Template argument deduction does not consider implicit conversions. A friend function declared inside a class template instantiation is a non-template function, and implicit conversions apply to its arguments normally.
- If the friend function body is long, have it call a helper function template defined outside the class.

7.14 Item 47: Use traits classes for information about types

7.15 Visual Summary

ITEM 47: USE TRAITS CLASSES FOR INFORMATION ABOUT TYPES

1. Generic algorithm needs type-specific facts.
2. Traits class maps type → compile-time properties/tags.
3. Algorithm dispatches by tag or constexpr condition.
4. Specialize traits for new types without rewriting algorithm.
5. Meaning: traits move type knowledge into reusable compile-time metadata.

7.15.1 The need for type information at compile time

Consider writing an advance function that moves an iterator forward by n positions:

```
template <typename IterT, typename DistT>
void advance(IterT& iter, DistT d) {
    iter += d; // Only works for random access iterators!
}
```

For input iterators, you must use ++iter in a loop. For bidirectional iterators, you can go backward with --iter. For random access iterators, you can use iter += d. The optimal implementation depends on the **iterator category**, which is a compile-time property.

7.15.2 Iterator categories (the five kinds)

The C++ standard defines five iterator categories, forming a hierarchy:

```
input iterator      output iterator
   \                /
   forward iterator
       |
   bidirectional iterator
       |
   random access iterator
```

Each category has a tag type:

```
struct input_iterator_tag { };
struct output_iterator_tag { };
struct forward_iterator_tag : input_iterator_tag { };
struct bidirectional_iterator_tag : forward_iterator_tag { };
struct random_access_iterator_tag : bidirectional_iterator_tag { };
```

7.15.3 Traits: a convention for compile-time type information

A traits class is a template that provides information about a type. The standard library's iterator_traits works like this:

```

template <typename IterT>
struct iterator_traits {
    typedef typename IterT::iterator_category iterator_category;
    typedef typename IterT::value_type       value_type;
    typedef typename IterT::difference_type   difference_type;
    typedef typename IterT::pointer           pointer;
    typedef typename IterT::reference         reference;
};

```

For user-defined iterators, you embed the information in the class:

```

template <typename T>
class LinkedListIterator {
public:
    // Nested typedefs that iterator_traits will pick up
    using iterator_category = std::bidirectional_iterator_tag;
    using value_type       = T;
    using difference_type   = std::ptrdiff_t;
    using pointer           = T*;
    using reference         = T&;

    // Iterator operations...
    LinkedListIterator& operator++() { node_ = node_>next; return *this; }
    LinkedListIterator& operator--() { node_ = node_>prev; return *this; }
    T& operator*() { return node_>data; }
    // ...

private:
    struct Node { T data; Node* next; Node* prev; };
    Node* node_;
};

```

7.15.4 Handling raw pointers with partial specialization

Raw pointers are iterators too, but they have no nested typedefs. `iterator_traits` uses partial specialization:

```

// Partial specialization for pointer types
template <typename T>
struct iterator_traits<T*> {
    typedef random_access_iterator_tag iterator_category;
    typedef T                          value_type;
    typedef std::ptrdiff_t             difference_type;
    typedef T*                         pointer;
    typedef T&                         reference;
};

// Partial specialization for const pointer types
template <typename T>
struct iterator_traits<const T*> {
    typedef random_access_iterator_tag iterator_category;
    typedef T                          value_type;
    typedef std::ptrdiff_t             difference_type;
    typedef const T*                   pointer;
    typedef const T&                   reference;
};

```

Now `iterator_traits<int*>::iterator_category` is `random_access_iterator_tag`, which is correct – raw pointers support random access.

7.15.5 Using traits: tag dispatch

You cannot use if statements with types – if (typeid(...) == ...) evaluates at runtime and would require all branches to compile. Instead, use **tag dispatch** (also known as compile-time dispatch):

```
// Implementation for input iterators: increment one at a time
template <typename IterT, typename DistT>
void doAdvance(IterT& iter, DistT d, std::input_iterator_tag) {
    if (d < 0) {
        throw std::out_of_range("Negative distance on input iterator");
    }
    while (d--> 0) ++iter;
}

// Implementation for bidirectional iterators: can go backward
template <typename IterT, typename DistT>
void doAdvance(IterT& iter, DistT d, std::bidirectional_iterator_tag) {
    if (d >= 0) { while (d--> 0) ++iter; }
    else { while (d++< 0) --iter; }
}

// Implementation for random access iterators: O(1)
template <typename IterT, typename DistT>
void doAdvance(IterT& iter, DistT d, std::random_access_iterator_tag) {
    iter += d;
}

// The public interface: uses traits to dispatch
template <typename IterT, typename DistT>
void advance(IterT& iter, DistT d) {
    doAdvance(iter, d,
              typename std::iterator_traits<IterT>::iterator_category());
    // ~~~~~~
    // Creates a temporary tag object; overload resolution picks
    // the right doAdvance at compile time.
}
```

The tag inheritance hierarchy means that `forward_iterator_tag` inherits from `input_iterator_tag`, so a forward iterator will match the input iterator overload (which is correct – forward iterators can only advance forward).

7.15.6 Building your own traits class: a complete example

Suppose you have a serialization library and need different strategies for different types:

```

#include <iostream>
#include <string>
#include <vector>
#include <type_traits>

// Step 1: Define tag types for serialization categories
struct PrimitiveSerialization {};
struct StringSerialization {};
struct ContainerSerialization {};
struct CustomSerialization {};

// Step 2: Define the traits template (default: custom)
template <typename T, typename Enable = void>
struct serialization_traits {
    using category = CustomSerialization;
};

// Step 3: Specialize for primitive types using SFINAE
template <typename T>
struct serialization_traits<T, std::enable_if_t<std::is_arithmetic_v<T>>> {
    using category = PrimitiveSerialization;
};

// Step 4: Specialize for strings
template <>
struct serialization_traits<std::string> {
    using category = StringSerialization;
};

// Step 5: Specialize for containers (anything with begin/end and value_type)
template <typename T>
struct serialization_traits<std::vector<T>> {
    using category = ContainerSerialization;
};

// Step 6: Implement the overloaded serialization functions
template <typename T>
void doSerialize(const T& value, std::ostream& os, PrimitiveSerialization) {
    os.write(reinterpret_cast<const char*>(&value), sizeof(T));
}

template <typename T>
void doSerialize(const T& value, std::ostream& os, StringSerialization) {
    auto size = value.size();
    os.write(reinterpret_cast<const char*>(&size), sizeof(size));
    os.write(value.data(), size);
}

template <typename T>
void doSerialize(const T& container, std::ostream& os, ContainerSerialization) {
    auto size = container.size();
    os.write(reinterpret_cast<const char*>(&size), sizeof(size));
    for (const auto& elem : container) {
        serialize(elem, os); // Recursive: each element uses its own traits
    }
}

// Step 7: Public interface with tag dispatch
template <typename T>
void serialize(const T& value, std::ostream& os) {

```

```
doSerialize(value, os,
            typename serialization_traits<T>::category());
}
```

7.15.7 C++17 and beyond: `if constexpr` as an alternative to tag dispatch

In C++17, `if constexpr` provides a more direct way to branch on compile-time conditions:

```
template <typename IterT, typename DistT>
void advance(IterT& iter, DistT d) {
    using category = typename std::iterator_traits<IterT>::iterator_category;

    if constexpr (std::is_base_of_v<std::random_access_iterator_tag, category>) {
        iter += d; // O(1) for random access
    } else if constexpr (std::is_base_of_v<std::bidirectional_iterator_tag, category>) {
        if (d >= 0) { while (d-->) ++iter; }
        else      { while (d++>) --iter; }
    } else {
        if (d < 0) throw std::out_of_range("Negative distance");
        while (d-->) ++iter;
    }
}
```

With `if constexpr`, branches that do not match are discarded at compile time, so they do not need to be valid for the given type. This is a significant advantage over regular `if` statements.

7.15.8 SFINAE: Substitution Failure Is Not An Error

SFINAE is the mechanism that enables `enable_if` and many advanced traits techniques. When the compiler substitutes template arguments and the result is invalid, it does not produce a hard error – it simply removes that overload from consideration:

```
// Only enabled for integral types
template <typename T>
std::enable_if_t<std::is_integral_v<T>, T>
safeDivide(T a, T b) {
    if (b == 0) throw std::domain_error("Division by zero");
    return a / b;
}

// Only enabled for floating-point types
template <typename T>
std::enable_if_t<std::is_floating_point_v<T>, T>
safeDivide(T a, T b) {
    if (b == T(0)) return std::numeric_limits<T>::infinity();
    return a / b;
}

int main() {
    std::cout << safeDivide(10, 3) << "\n"; // Calls integral version: 3
    std::cout << safeDivide(10.0, 3.0) << "\n"; // Calls floating version: 3.33333
    // safeDivide("hello", "world"); // No matching overload -- SFINAE
}
```

7.15.9 Type traits in the standard library

The `<type_traits>` header provides a rich set of compile-time type queries:


```

#include <type_traits>

// Primary type categories
static_assert(std::is_integral_v<int>);
static_assert(std::is_floating_point_v<double>);
static_assert(std::is_pointer_v<int*>);
static_assert(std::is_reference_v<int&>);
static_assert(std::is_class_v<std::string>);
static_assert(std::is_enum_v<std::byte>);

// Type properties
static_assert(std::is_const_v<const int>);
static_assert(std::is_trivially_copyable_v<int>);
static_assert(std::is_polymorphic_v<std::ostream>);

// Type relationships
static_assert(std::is_base_of_v<std::ios_base, std::ostream>);
static_assert(std::is_convertible_v<int, double>);
static_assert(std::is_same_v<int, int>);

// Type transformations
using T1 = std::remove_const_t<const int>;           // int
using T2 = std::remove_reference_t<int&>;           // int
using T3 = std::add_pointer_t<int>;                 // int*
using T4 = std::decay_t<const int&>;                 // int
using T5 = std::conditional_t<true, int, double>;    // int

```

7.15.10 Implementing a simple type trait

```

// is_same: check if two types are the same
template <typename T, typename U>
struct my_is_same {
    static constexpr bool value = false;
};

template <typename T>
struct my_is_same<T, T> { // Partial specialization when both types match
    static constexpr bool value = true;
};

// is_pointer
template <typename T>
struct my_is_pointer {
    static constexpr bool value = false;
};

template <typename T>
struct my_is_pointer<T*> {
    static constexpr bool value = true;
};

// has_type_member: detect if a type has a nested ::type typedef (using SFINAE)
template <typename T, typename = void>
struct has_type_member : std::false_type {};

template <typename T>
struct has_type_member<T, std::void_t<typename T::type>> : std::true_type {};

// Usage:

```

```
struct WithType { using type = int; };
struct WithoutType { int value; };

static_assert(has_type_member<WithType>::value);
static_assert(!has_type_member<WithoutType>::value);
```

7.15.11 Things to Remember

- Traits classes make information about types available during compilation. They are implemented using templates and template specializations.
- In conjunction with overloading, traits classes make it possible to perform compile-time `if...else` tests on types (tag dispatch).
- The standard library provides `iterator_traits`, `char_traits`, `numeric_limits`, `allocator_traits`, and the extensive `<type_traits>` header as pre-built traits facilities.
- SFINAE (Substitution Failure Is Not An Error) is the mechanism that makes many traits techniques possible, including `std::enable_if`.
- In C++17, `if constexpr` provides a more readable alternative to tag dispatch for compile-time branching.

7.16 Item 48: Be aware of template metaprogramming

7.17 Visual Summary

ITEM 48: BE AWARE OF TEMPLATE METAPROGRAMMING
1. Template instantiation can compute types/values at compile time. 2. Benefits → remove runtime branches, generate optimized code, enforce constraints. 3. Costs → compile time, diagnostics, complexity. 4. Use for real generic power, not cleverness. 5. Meaning: TMP is compile-time programming with a high readability price.

7.17.1 What is template metaprogramming?

Template metaprogramming (TMP) is the process of writing programs that execute during compilation. The C++ template system is Turing-complete – it can compute anything that a general-purpose computer can compute (given enough resources). TMP programs are written in C++ template syntax but run at compile time, producing constants, types, or code as their output.

TMP has two great strengths: 1. It makes some things easy that would otherwise be hard or impossible. 2. It shifts work from runtime to compile time, leading to smaller executables, shorter runtimes, and earlier error detection.

The main downsides: longer compile times, difficult-to-read code, and notoriously inscrutable error messages.

7.17.2 The classic example: compile-time factorial

```

// TMP factorial: computes n! at compile time
template <unsigned N>
struct Factorial {
    static constexpr unsigned value = N * Factorial<N - 1>::value;
};

// Base case: 0! = 1
template <>
struct Factorial<0> {
    static constexpr unsigned value = 1;
};

// Usage:
static_assert(Factorial<5>::value == 120);
static_assert(Factorial<0>::value == 1);
static_assert(Factorial<10>::value == 3628800);

// The value 120 is computed at compile time -- no runtime computation.
int main() {
    // This array declaration proves the value is a compile-time constant:
    int arr[Factorial<5>::value]; // int arr[120]; -- legal only if value is constexpr
    std::cout << "5! = " << Factorial<5>::value << "\n";
}

```

The recursion unfolds at compile time: - Factorial<5>::value = 5 * Factorial<4>::value - Factorial<4>::value = 4 * Factorial<3>::value - Factorial<3>::value = 3 * Factorial<2>::value - Factorial<2>::value = 2 * Factorial<1>::value - Factorial<1>::value = 1 * Factorial<0>::value - Factorial<0>::value = 1 (base case)

7.17.3 Compile-time Fibonacci

```

template <unsigned N>
struct Fibonacci {
    static constexpr unsigned long long value =
        Fibonacci<N - 1>::value + Fibonacci<N - 2>::value;
};

template <>
struct Fibonacci<0> {
    static constexpr unsigned long long value = 0;
};

template <>
struct Fibonacci<1> {
    static constexpr unsigned long long value = 1;
};

static_assert(Fibonacci<0>::value == 0);
static_assert(Fibonacci<1>::value == 1);
static_assert(Fibonacci<10>::value == 55);
static_assert(Fibonacci<20>::value == 6765);
static_assert(Fibonacci<46>::value == 1836311903);

```

Note: this naive recursive approach has exponential compile-time complexity (each Fibonacci<N> instantiates Fibonacci<N-1> and Fibonacci<N-2>, and memoization depends on the compiler). A linear version:

```

template <unsigned N, unsigned long long Prev = 0, unsigned long long Curr = 1>
struct FibLinear {
    static constexpr unsigned long long value =
        FibLinear<N - 1, Curr, Prev + Curr>::value;
};

template <unsigned long long Prev, unsigned long long Curr>
struct FibLinear<0, Prev, Curr> {
    static constexpr unsigned long long value = Prev;
};

static_assert(FibLinear<10>::value == 55);
static_assert(FibLinear<50>::value == 12586269025ULL);

```

7.17.4 Compile-time greatest common divisor

```

template <unsigned A, unsigned B>
struct GCD {
    static constexpr unsigned value = GCD<B, A % B>::value;
};

template <unsigned A>
struct GCD<A, 0> {
    static constexpr unsigned value = A;
};

static_assert(GCD<12, 8>::value == 4);
static_assert(GCD<100, 75>::value == 25);
static_assert(GCD<17, 13>::value == 1); // Coprime

```

7.17.5 Type-level computation: type lists

TMP can manipulate types themselves, not just values:

```

#include <type_traits>
#include <iostream>
#include <string>

// A type list is a compile-time list of types
template <typename... Ts>
struct TypeList {};

// Length of a type list
template <typename List>
struct Length;

template <typename... Ts>
struct Length<TypeList<Ts...>> {
    static constexpr std::size_t value = sizeof...(Ts);
};

// Access the Nth type in a type list
template <typename List, std::size_t N>
struct TypeAt;

template <typename Head, typename... Tail>
struct TypeAt<TypeList<Head, Tail...>, 0> {
    using type = Head;
};

```

```

template <typename Head, typename... Tail, std::size_t N>
struct TypeAt<TypeList<Head, Tail...>, N> {
    using type = typename TypeAt<TypeList<Tail...>, N - 1>::type;
};

// Append a type to a type list
template <typename List, typename T>
struct Append;

template <typename... Ts, typename T>
struct Append<TypeList<Ts...>, T> {
    using type = TypeList<Ts..., T>;
};

// Prepend a type to a type list
template <typename List, typename T>
struct Prepend;

template <typename... Ts, typename T>
struct Prepend<TypeList<Ts...>, T> {
    using type = TypeList<T, Ts...>;
};

// Check if a type is in the list
template <typename List, typename T>
struct Contains;

template <typename T>
struct Contains<TypeList<>>, T> {
    static constexpr bool value = false;
};

template <typename Head, typename... Tail, typename T>
struct Contains<TypeList<Head, Tail...>, T> {
    static constexpr bool value =
        std::is_same_v<Head, T> || Contains<TypeList<Tail...>, T>::value;
};

// Remove duplicates from a type list
template <typename List>
struct Unique;

template <>
struct Unique<TypeList<>> {
    using type = TypeList<>;
};

template <typename Head, typename... Tail>
struct Unique<TypeList<Head, Tail...>> {
private:
    using UniqueTail = typename Unique<TypeList<Tail...>>::type;
public:
    using type = std::conditional_t<
        Contains<UniqueTail, Head>::value,
        UniqueTail,
        typename Prepend<UniqueTail, Head>::type
    >;
};

```

```
// Usage:
using MyTypes = TypeList<int, double, std::string, float>;

static_assert(Length<MyTypes>::value == 4);
static_assert(std::is_same_v<typename TypeAt<MyTypes, 0>::type, int>);
static_assert(std::is_same_v<typename TypeAt<MyTypes, 2>::type, std::string>);
static_assert(Contains<MyTypes, double>::value);
static_assert(!Contains<MyTypes, char>::value);

using WithDups = TypeList<int, double, int, float, double>;
using NoDups = typename Unique<WithDups>::type;
static_assert(Length<NoDups>::value == 3); // int, double, float
```

7.17.6 TMP for compile-time dimensional analysis

A powerful real-world application of TMP is checking physical units at compile time:

```
#include <iostream>
#include <ratio>

// Represent physical dimensions as compile-time integers
// (mass, length, time)
template <int Mass, int Length, int Time>
struct Dimension {
    static constexpr int mass = Mass;
    static constexpr int length = Length;
    static constexpr int time = Time;
};

// A quantity with a value and a dimension
template <typename Dim>
class Quantity {
public:
    explicit Quantity(double val) : value_(val) {}
    double value() const { return value_; }

    // Addition: only quantities of the same dimension can be added
    Quantity operator+(const Quantity& rhs) const {
        return Quantity(value_ + rhs.value_);
    }

    Quantity operator-(const Quantity& rhs) const {
        return Quantity(value_ - rhs.value_);
    }

    // Scalar multiplication
    Quantity operator*(double scalar) const {
        return Quantity(value_ * scalar);
    }

    // Multiplication of quantities: dimensions add
    template <typename OtherDim>
    auto operator*(const Quantity<OtherDim>& rhs) const {
        using ResultDim = Dimension<
            Dim::mass + OtherDim::mass,
            Dim::length + OtherDim::length,
            Dim::time + OtherDim::time
        >;
        return Quantity<ResultDim>(value_ * rhs.value());
    }
}
```

```

// Division of quantities: dimensions subtract
template <typename OtherDim>
auto operator/(const Quantity<OtherDim>& rhs) const {
    using ResultDim = Dimension<
        Dim::mass - OtherDim::mass,
        Dim::length - OtherDim::length,
        Dim::time - OtherDim::time
    >;
    return Quantity<ResultDim>(value_ / rhs.value());
}

private:
    double value_;
};

// Define common physical dimensions
using Scalar      = Dimension<0, 0, 0>;
using Mass        = Dimension<1, 0, 0>;    // kg
using Length      = Dimension<0, 1, 0>;    // m
using Time        = Dimension<0, 0, 1>;    // s
using Velocity     = Dimension<0, 1, -1>;   // m/s
using Accel       = Dimension<0, 1, -2>;   // m/s^2
using Force       = Dimension<1, 1, -2>;   // kg*m/s^2 = Newton
using Energy      = Dimension<1, 2, -2>;   // kg*m^2/s^2 = Joule

int main() {
    Quantity<Mass> m(10.0);    // 10 kg
    Quantity<Accel> a(9.8);    // 9.8 m/s^2
    auto force = m * a;        // Quantity<Force> -- 98 N (computed at compile time)

    Quantity<Length> d(100.0); // 100 m
    auto energy = force * d;    // Quantity<Energy> -- 9800 J

    Quantity<Time> t(5.0);
    auto vel = d / t;          // Quantity<Velocity> -- 20 m/s

    std::cout << "Force: " << force.value() << " N\n";
    std::cout << "Energy: " << energy.value() << " J\n";
    std::cout << "Velocity: " << vel.value() << " m/s\n";

    // Compile-time error: cannot add mass and velocity!
    // auto bad = m + vel; // ERROR: Quantity<Mass> + Quantity<Velocity> -- no match
}

```

The dimension checking happens entirely at compile time. The generated code is as efficient as raw double arithmetic – zero runtime overhead.

7.17.7 TMP for compile-time loop unrolling

```

// Compile-time dot product with loop unrolling
template <int N>
struct DotProduct {
    template <typename T>
    static T compute(const T* a, const T* b) {
        return a[N-1] * b[N-1] + DotProduct<N-1>::compute(a, b);
    }
};

template <>
struct DotProduct<1> {
    template <typename T>
    static T compute(const T* a, const T* b) {
        return a[0] * b[0];
    }
};

// Usage:
double a[] = {1.0, 2.0, 3.0, 4.0};
double b[] = {5.0, 6.0, 7.0, 8.0};
double result = DotProduct<4>::compute(a, b);
// Compiler generates: a[3]*b[3] + a[2]*b[2] + a[1]*b[1] + a[0]*b[0]
// No loop overhead -- completely unrolled at compile time.

```

7.17.8 Compile-time power function

```

// Compute base^exp at compile time with fast exponentiation
template <unsigned Base, unsigned Exp>
struct Power {
    static constexpr unsigned long long value =
        (Exp % 2 == 0)
        ? Power<Base, Exp / 2>::value * Power<Base, Exp / 2>::value
        : Base * Power<Base, Exp - 1>::value;
};

template <unsigned Base>
struct Power<Base, 0> {
    static constexpr unsigned long long value = 1;
};

static_assert(Power<2, 10>::value == 1024);
static_assert(Power<3, 5>::value == 243);
static_assert(Power<10, 6>::value == 1000000);

```

7.17.9 TMP for compile-time string processing (C++17)


```

// Compile-time string hash (FNV-1a)
constexpr std::size_t fnv1a_hash(const char* str, std::size_t len) {
    std::size_t hash = 14695981039346656037ULL;
    for (std::size_t i = 0; i < len; ++i) {
        hash ^= static_cast<std::size_t>(str[i]);
        hash *= 1099511628211ULL;
    }
    return hash;
}

// A compile-time string wrapper
template <std::size_t N>
struct FixedString {
    char data[N];
    constexpr FixedString(const char (&str)[N]) {
        for (std::size_t i = 0; i < N; ++i) data[i] = str[i];
    }
    constexpr std::size_t hash() const { return fnv1a_hash(data, N - 1); }
};

// Usage in a compile-time switch-like construct:
constexpr auto h = FixedString("hello").hash();
static_assert(h != 0); // Non-zero hash computed at compile time

```

7.17.10 constexpr: the modern alternative to classic TMP

C++11 introduced constexpr, which provides a much more readable way to do compile-time computation. C++14 and C++17 further relaxed its restrictions:

```

// constexpr factorial -- reads like normal code!
constexpr unsigned long long factorial(unsigned n) {
    unsigned long long result = 1;
    for (unsigned i = 2; i <= n; ++i)
        result *= i;
    return result;
}

static_assert(factorial(5) == 120);
static_assert(factorial(20) == 2432902008176640000ULL);

// constexpr Fibonacci
constexpr unsigned long long fibonacci(unsigned n) {
    if (n <= 1) return n;
    unsigned long long prev = 0, curr = 1;
    for (unsigned i = 2; i <= n; ++i) {
        unsigned long long next = prev + curr;
        prev = curr;
        curr = next;
    }
    return curr;
}

static_assert(fibonacci(10) == 55);
static_assert(fibonacci(50) == 12586269025ULL);

// constexpr GCD
constexpr unsigned gcd(unsigned a, unsigned b) {
    while (b != 0) {
        unsigned t = b;
        b = a % b;
    }
}

```

```

        a = t;
    }
    return a;
}

static_assert(gcd(12, 8) == 4);

```

While constexpr replaces many uses of classic TMP for value computation, TMP remains essential for **type computation** – selecting types, transforming type lists, and generating code based on type properties.

7.17.11 Real-world uses of TMP

1. **Ensuring correctness at compile time** – dimensional analysis (shown above), strong typedefs, and policy checking.
2. **Optimizing performance** – expression templates in linear algebra libraries (Eigen, Blaze) eliminate temporary objects:

```

// Without expression templates:
Matrix a, b, c, d;
Matrix result = a + b + c + d;
// Creates 3 temporaries: (a+b), ((a+b)+c), (((a+b)+c)+d)

// With expression templates (TMP):
// The expression a + b + c + d creates a lightweight expression object.
// Only when assigned to result does a single pass evaluate the sum.
// Zero temporaries, one loop -- as fast as hand-written code.

```

3. **Static interface checking** – C++20 concepts are the modern way, but pre-C++20 code uses TMP:

```

// Pre-C++20: SFINAE-based concept checking
template <typename T,
         typename = std::enable_if_t<
             std::is_default_constructible_v<T> &&
             std::is_copy_assignable_v<T>>>
class Container {
    // Only instantiates if T is default-constructible and copy-assignable
};

// C++20: concepts (built on top of TMP infrastructure)
template <typename T>
concept Storable = std::is_default_constructible_v<T> &&
                  std::is_copy_assignable_v<T>;

template <Storable T>
class Container {
    // Clean, readable constraint
};

```

4. **Compile-time state machines** – verifying protocol compliance at compile time:

```

// States
struct Disconnected {};
struct Connected {};
struct Authenticated {};

// Connection class with compile-time state tracking
template <typename State>
class Connection {
public:
    // Only callable in Disconnected state
    Connection<Connected> connect(const std::string& host)
        requires std::is_same_v<State, Disconnected>
    {
        // ... perform connection ...
        return Connection<Connected>{};
    }

    // Only callable in Connected state
    Connection<Authenticated> authenticate(const std::string& token)
        requires std::is_same_v<State, Connected>
    {
        // ... perform auth ...
        return Connection<Authenticated>{};
    }

    // Only callable in Authenticated state
    void sendData(const std::string& data)
        requires std::is_same_v<State, Authenticated>
    {
        // ... send data ...
    }
};

// Usage:
// Connection<Disconnected> conn;
// auto connected = conn.connect("example.com");
// auto authed = connected.authenticate("token123");
// authed.sendData("hello");
//
// conn.sendData("hello"); // COMPILE ERROR: wrong state!
// connected.sendData("x"); // COMPILE ERROR: not authenticated!

```

7.17.12 Things to Remember

- Template metaprogramming can shift work from runtime to compile time, enabling earlier error detection and higher runtime performance.
- TMP can be used to generate custom code based on combinations of policy choices, and it can be used to avoid generating code inappropriate for particular types.
- Classic TMP uses recursive template instantiation for loops and template specialization for conditionals and base cases.
- constexpr functions (C++11/14/17/20) provide a more readable alternative to classic TMP for value computations, but TMP remains essential for type-level computation.
- Real-world applications of TMP include dimensional analysis, expression templates, compile-time interface checking, and static state machines.
- The trade-offs of TMP include longer compile times, harder-to-read code, and more difficult debugging – use it when the benefits (correctness, performance) justify the costs.

8 Chapter 8: Customizing new and delete

8.1 Chapter Flow

CHAPTER 8: CUSTOMIZING NEW AND DELETE
Item 49 -> new-handler defines allocation failure retry policy. Item 50 -> replace allocation only for measured, specific reasons. Item 51 -> custom new/delete must obey standard conventions. Item 52 -> placement new needs matching placement delete for constructor failure.

Items 49-52: Mastering C++ memory allocation — from new-handlers and custom allocators to placement new/delete and the conventions that keep everything safe.

8.2 Item 49: Understand the Behavior of the new-handler

8.3 Visual Summary

ITEM 49: UNDERSTAND THE BEHAVIOR OF THE NEW-HANDLER
1. operator new fails to allocate -> checks current new-handler. 2. Handler may free memory, install another handler, uninstall itself, throw, or abort. 3. If handler returns -> operator new retries allocation. 4. No handler -> std::bad_alloc is thrown. 5. Meaning: new-handler is a retry policy hook for allocation failure.

8.3.1 Core Concept

When `operator new` cannot satisfy a memory allocation request, it does **not** immediately throw `std::bad_alloc`. Instead, it first calls a **new-handler** function — a client-installable error-handling function. Understanding this mechanism is essential because it gives you a last-chance hook to free memory, log diagnostics, or gracefully shut down before allocation failure propagates.

8.3.2 How `set_new_handler` Works

```

#include <new>
#include <cstdlib>
#include <iostream>

// The new_handler type is simply a pointer to a function taking and returning nothing
// namespace std {
//     typedef void (*new_handler)();
//     new_handler set_new_handler(new_handler p) noexcept;
// }

void outOfMemory() {
    std::cerr << "FATAL: Unable to allocate memory. Shutting down.\n";
    std::abort();
}

int main() {
    // Install our custom handler; set_new_handler returns the previously installed one
    std::new_handler oldHandler = std::set_new_handler(outOfMemory);

    // Now if any new expression fails, outOfMemory() is called before bad_alloc is thrown
    try {
        // Attempt a ludicrously large allocation
        int* p = new int[1000000000000L];
        // If outOfMemory() didn't abort, we'd never get here
        delete[] p;
    } catch (const std::bad_alloc& e) {
        std::cerr << "Caught bad_alloc: " << e.what() << "\n";
    }

    // Restore the old handler
    std::set_new_handler(oldHandler);
}

```

8.3.3 The new-handler Contract

A well-designed new-handler must do one of the following things. If it does not, operator new will call it in an infinite loop:

1. **Make more memory available** — so that the next allocation attempt inside operator new might succeed.
2. **Install a different new-handler** — if the current handler cannot make more memory available, perhaps it knows of another handler that can.
3. **Deinstall the new-handler** — pass nullptr to set_new_handler. With no handler installed, operator new will throw std::bad_alloc.
4. **Throw an exception** — of type std::bad_alloc or derived from it.
5. **Not return** — call std::abort() or std::exit().

```

// STRATEGY 1: Make more memory available
// Pre-allocate a reserve block that can be freed under pressure

class EmergencyMemoryPool {
public:
    EmergencyMemoryPool(size_t reserveSize = 1024 * 1024 * 64) // 64 MB reserve
        : reserve_(static_cast<char*>(std::malloc(reserveSize)))
        , reserveSize_(reserveSize)
    {
        if (!reserve_) {
            throw std::runtime_error("Cannot allocate emergency reserve");
        }
    }

    ~EmergencyMemoryPool() {
        std::free(reserve_);
    }

    static void handler() {
        // Release the emergency reserve to make memory available
        if (instance_ && instance_>reserve_) {
            std::cerr << "WARNING: Memory pressure detected. "
                << "Releasing " << instance_>reserveSize_ << " byte reserve.\n";
            std::free(instance_>reserve_);
            instance_>reserve_ = nullptr;

            // Deinstall this handler – we've done all we can
            // Next failure will throw bad_alloc
            std::set_new_handler(nullptr);
            return;
        }

        // No reserve left – throw
        throw std::bad_alloc();
    }

    static EmergencyMemoryPool* instance_;

private:
    char* reserve_;
    size_t reserveSize_;
};

EmergencyMemoryPool* EmergencyMemoryPool::instance_ = nullptr;

void setupEmergencyPool() {
    static EmergencyMemoryPool pool;
    EmergencyMemoryPool::instance_ = &pool;
    std::set_new_handler(EmergencyMemoryPool::handler);
}

```

```
// STRATEGY 2: Install a different handler (chain of responsibility)

void lastResortHandler() {
    std::cerr << "All memory recovery strategies exhausted. Aborting.\n";
    std::abort();
}

void tryCompactHandler() {
    std::cerr << "Attempting memory compaction...\n";
    // ... attempt to compact or coalesce free blocks ...

    // If compaction didn't help, escalate to the last resort
    std::set_new_handler(lastResortHandler);
}

void tryCacheFlushHandler() {
    std::cerr << "Flushing caches to free memory...\n";
    // ... flush application caches ...

    // If cache flush didn't free enough, try compaction next
    std::set_new_handler(tryCompactHandler);
}

// Install the first link in the chain:
// std::set_new_handler(tryCacheFlushHandler);
```

8.3.4 Per-Class new-handlers

C++ does not directly provide per-class new-handlers, but you can implement them yourself by overriding operator new in your class:

```
#include <new>
#include <mutex>

class Widget {
public:
    static std::new_handler set_new_handler(std::new_handler p) noexcept {
        std::new_handler old = currentHandler_;
        currentHandler_ = p;
        return old;
    }

    static void* operator new(std::size_t size) {
        // Install Widget's handler, save the global one
        NewHandlerHolder h(std::set_new_handler(currentHandler_));
        // Now allocate – if it fails, Widget's handler is called
        return ::operator new(size);
    }

    static void operator delete(void* p) noexcept {
        ::operator delete(p);
    }

private:
    // RAII class to restore the global handler on scope exit
    class NewHandlerHolder {
    public:
        explicit NewHandlerHolder(std::new_handler h) noexcept : handler_(h) {}
        ~NewHandlerHolder() { std::set_new_handler(handler_); }
    };
};
```

```

        NewHandlerHolder(const NewHandlerHolder&) = delete;
        NewHandlerHolder& operator=(const NewHandlerHolder&) = delete;
    private:
        std::new_handler handler_;
    };

    static std::new_handler currentHandler_;
};

std::new_handler Widget::currentHandler_ = nullptr;

```

8.3.5 Templatizing the Per-Class new-handler (CRTP Mixin)

The above pattern can be generalized using the Curiously Recurring Template Pattern (CRTP) so that any class can gain per-class new-handler support:

```

template<typename T>
class NewHandlerSupport {
public:
    static std::new_handler set_new_handler(std::new_handler p) noexcept {
        std::new_handler old = currentHandler_;
        currentHandler_ = p;
        return old;
    }

    static void* operator new(std::size_t size) {
        // Install T's handler, saving the global one for later restoration
        NewHandlerHolder h(std::set_new_handler(currentHandler_));
        return ::operator new(size);
    }

    static void operator delete(void* p) noexcept {
        ::operator delete(p);
    }

private:
    class NewHandlerHolder {
    public:
        explicit NewHandlerHolder(std::new_handler h) noexcept : handler_(h) {}
        ~NewHandlerHolder() { std::set_new_handler(handler_); }
        NewHandlerHolder(const NewHandlerHolder&) = delete;
        NewHandlerHolder& operator=(const NewHandlerHolder&) = delete;
    private:
        std::new_handler handler_;
    };

    static std::new_handler currentHandler_;
};

template<typename T>
std::new_handler NewHandlerSupport<T>::currentHandler_ = nullptr;

// Usage – just inherit from the mixin:
class Widget : public NewHandlerSupport<Widget> {
    // Widget gets its own static currentHandler_ due to the template instantiation
    // No additional code needed for per-class new-handler support
};

class Gadget : public NewHandlerSupport<Gadget> {
    // Gadget gets its own independent currentHandler_

```



```
};

// Now each class can have its own handler:
void widgetOutOfMem() { std::cerr << "Widget allocation failed!\n"; std::abort(); }
void gadgetOutOfMem() { std::cerr << "Gadget allocation failed!\n"; std::abort(); }

void setup() {
    Widget::set_new_handler(widgetOutOfMem);
    Gadget::set_new_handler(gadgetOutOfMem);
}
```

8.3.6 nothrow new

C++ also provides a nothrow form of new that returns nullptr instead of throwing:

```
#include <new>

// nothrow form – returns nullptr on failure instead of throwing
Widget* pw = new (std::nothrow) Widget;
if (pw == nullptr) {
    // Allocation failed – handle gracefully
    std::cerr << "Widget allocation failed.\n";
}
```

Critical caveat: nothrow new only guarantees that operator new itself won't throw. The **constructor** of the object being created can still throw. So new (std::nothrow) Widget is **not** a guarantee of no exceptions:

```
class Widget {
public:
    Widget() {
        // This constructor allocates memory internally using regular (throwing) new
        data_ = new int[1000000]; // THIS CAN STILL THROW std::bad_alloc!
    }
private:
    int* data_;
};

// BAD: False sense of safety
Widget* pw = new (std::nothrow) Widget;
// The nothrow only applies to the raw memory allocation for the Widget object itself.
// If Widget's constructor internally uses throwing new, you still get bad_alloc.
// You've gained essentially nothing.
```

```
// GOOD: If you truly need no-throw allocation, ensure the constructor is also nothrow
class SafeWidget {
public:
    SafeWidget() noexcept {
        data_ = static_cast<int*>(std::malloc(1000000 * sizeof(int)));
        // Use malloc (returns nullptr on failure) instead of new
    }

    ~SafeWidget() {
        std::free(data_);
    }

    bool isValid() const noexcept { return data_ != nullptr; }

private:
    int* data_ = nullptr;
};
```

8.3.7 Modern C++ Update (C++11/14/17/20)

In modern C++, `set_new_handler` remains relevant, but direct use of `new` is discouraged in favor of smart pointers:

```
// C++14 and later – prefer make_unique/make_shared  
// These still go through operator new internally, so the new-handler mechanism applies  
auto widget = std::make_unique<Widget>();    // throws bad_alloc on failure  
auto shared = std::make_shared<Widget>();    // throws bad_alloc on failure  
  
// If you need nothrow semantics with smart pointers, you must do it manually:  
Widget* raw = new (std::nothrow) Widget;  
std::unique_ptr<Widget> safeWidget(raw);      // Wrap in unique_ptr after nothrow new  
if (!safeWidget) {  
    // Handle failure  
}
```

8.3.8 Things to Remember

- `set_new_handler` lets you specify a function to be called when memory allocation requests cannot be satisfied.
- Nothrow `new` is of limited value because it only prevents `operator new` from throwing, not the object's constructor.
- A new-handler must either make more memory available, install a different handler, deinstall itself, throw `bad_alloc` (or a derived type), or not return at all.
- You can implement per-class new-handlers by overriding `operator new` in your class, saving and restoring the global handler using `RAII`.
- The CRTP mixin `NewHandlerSupport<T>` lets any class gain per-class new-handler support with zero boilerplate.

8.4 Item 50: Understand When It Makes Sense to Replace `new` and `delete`

8.5 Visual Summary

ITEM 50: UNDERSTAND WHEN IT MAKES SENSE TO REPLACE <code>new</code> AND <code>delete</code>
1. Default allocator is general-purpose. 2. Replace <code>new/delete</code> only for clear goals: speed, memory overhead, debugging, locality, alignment. 3. Wrong replacement → broken conventions, poor threading, worse performance. 4. Measure before and after. 5. Meaning: custom allocation is an engineering tradeoff, not a default optimization.

8.5.1 Core Concept

The default implementations of `operator new` and `operator delete` provided by the C++ runtime are designed for **general-purpose** use. They must handle allocation patterns ranging from a single `int` to millions of heterogeneous objects. This generality comes at a cost. There are several legitimate reasons to replace them with custom versions.

8.5.2 Reason 1: Detecting Usage Errors

Memory bugs — writing past the end of an allocated block, writing before the beginning, double-deletes, forgetting to delete — are among the most pernicious in C++. A custom allocator can catch them early:

```
#include <cstdint>
#include <cstdlib>
#include <cstring>
#include <iostream>
#include <new>

// Debugging allocator that adds signature bytes before and after each allocation
// to detect buffer overruns and underruns.

static const uint32_t FRONT_SIGNATURE = 0xDEADBEEF;
static const uint32_t BACK_SIGNATURE = 0xCAFEBAFE;
static const unsigned char FILL_BYTE = 0xCD; // Uninitialized memory pattern
static const unsigned char FREED_BYTE = 0xDD; // Freed memory pattern

struct AllocationHeader {
    uint32_t frontSignature;
    std::size_t requestedSize;
    const char* file;
    int line;
    // Padding to maintain alignment (see alignment discussion below)
};

void* operator new(std::size_t size) {
    // Calculate total size: header + requested bytes + back signature
    const std::size_t totalSize = sizeof(AllocationHeader) + size + sizeof(uint32_t);

    void* rawMemory = std::malloc(totalSize);
    if (!rawMemory) {
        throw std::bad_alloc();
    }

    // Write header
    auto* header = static_cast<AllocationHeader*>(rawMemory);
    header->frontSignature = FRONT_SIGNATURE;
    header->requestedSize = size;
    header->file = nullptr;
    header->line = 0;

    // Fill user memory with a pattern to detect reads of uninitialized memory
    void* userMemory = header + 1;
    std::memset(userMemory, FILL_BYTE, size);

    // Write back signature
    auto* backSig = reinterpret_cast<uint32_t*>(
        static_cast<char*>(userMemory) + size
    );
    *backSig = BACK_SIGNATURE;

    return userMemory;
}

void operator delete(void* p) noexcept {
    if (!p) return;

    auto* header = static_cast<AllocationHeader*>(p) - 1;
```

```

// Check front signature – detects underruns
if (header->frontSignature != FRONT_SIGNATURE) {
    std::cerr << "HEAP CORRUPTION: front signature destroyed at " << p << "\n";
    std::abort();
}

// Check back signature – detects overruns
auto* backSig = reinterpret_cast<uint32_t*>(
    static_cast<char*>(p) + header->requestedSize
);
if (*backSig != BACK_SIGNATURE) {
    std::cerr << "HEAP CORRUPTION: buffer overrun detected at " << p
        << " (allocated " << header->requestedSize << " bytes)\n";
    std::abort();
}

// Scribble freed memory so use-after-free is more likely to crash immediately
std::memset(header, FREED_BYTE, sizeof(AllocationHeader) + header->requestedSize + sizeof(
std::free(header);
}

// Sized deallocation (C++14)
void operator delete(void* p, std::size_t size) noexcept {
    // Delegate to the unsized version; we track size in the header anyway
    ::operator delete(p);
}

```

8.5.3 Reason 2: Improving Performance (Speed)

The default allocator must handle all sizes and patterns. If you know your allocation pattern, you can beat it significantly:

```

// Fixed-size allocator (memory pool) for a specific class
// Eliminates per-allocation overhead and fragmentation for uniform-sized objects.

template<typename T>
class FixedSizePool {
public:
    explicit FixedSizePool(std::size_t blockCount = 1024)
        : blockSize_(sizeof(T) < sizeof(FreeBlock*) ? sizeof(FreeBlock*) : sizeof(T))
        , blockCount_(blockCount)
    {
        expandPool();
    }

    ~FixedSizePool() {
        for (auto* chunk : chunks_) {
            ::operator delete(chunk);
        }
    }

    void* allocate() {
        if (!freeList_) {
            expandPool(); // All blocks used – allocate a new chunk
        }
        FreeBlock* block = freeList_;
        freeList_ = block->next;
        return block;
    }
}

```

```

void deallocate(void* p) noexcept {
    if (!p) return;
    auto* block = static_cast<FreeBlock*>(p);
    block->next = freeList_;
    freeList_ = block;
}

private:
    struct FreeBlock {
        FreeBlock* next;
    };

    void expandPool() {
        // Allocate a contiguous chunk of blockCount_ blocks
        std::size_t chunkSize = blockSize_ * blockCount_;
        char* chunk = static_cast<char*>(::operator new(chunkSize));
        chunks_.push_back(chunk);

        // Thread all blocks in the chunk onto the free list
        for (std::size_t i = 0; i < blockCount_; ++i) {
            auto* block = reinterpret_cast<FreeBlock*>(chunk + i * blockSize_);
            block->next = freeList_;
            freeList_ = block;
        }

        std::size_t blockSize_;
        std::size_t blockCount_;
        FreeBlock* freeList_ = nullptr;
        std::vector<char*> chunks_;
    };

    // Using the pool in a class:
    class Bullet {
    public:
        static void* operator new(std::size_t size) {
            if (size != sizeof(Bullet)) {
                // If a derived class is larger, fall back to global new
                return ::operator new(size);
            }
            return pool_.allocate();
        }

        static void operator delete(void* p, std::size_t size) noexcept {
            if (size != sizeof(Bullet)) {
                ::operator delete(p);
                return;
            }
            pool_.deallocate(p);
        }

        // ... Bullet members ...
    private:
        double x_, y_, z_;
        double vx_, vy_, vz_;
        float damage_;
        int ownerId_;
    };

```

```

    static FixedSizePool<Bullet> pool_;
};

FixedSizePool<Bullet> Bullet::pool_(4096); // Pre-allocate 4096 bullet slots

```

8.5.4 Reason 3: Improving Performance (Memory Usage / Reducing Fragmentation)

The general-purpose allocator typically adds bookkeeping overhead per allocation (often 8-16 bytes on 64-bit systems). For small objects, this overhead can be a massive percentage of the total memory used:

```

// Consider allocating millions of 4-byte objects:
// With default allocator: each 4-byte object may consume 32+ bytes (header + alignment padding)
// That's 8x overhead!

// BAD: Allocating small objects individually
std::vector<int*> pointers;
for (int i = 0; i < 1000000; ++i) {
    pointers.push_back(new int(i));
    // Each new int likely costs 32 bytes total on a 64-bit system
    // Total: ~32 MB for 4 MB of actual data
}

// GOOD: Use a pool allocator or just use a vector
std::vector<int> values;
values.reserve(1000000);
for (int i = 0; i < 1000000; ++i) {
    values.push_back(i);
    // All data contiguous, no per-element overhead
    // Total: ~4 MB
}

```

8.5.5 Reason 4: Improving Locality of Reference

Objects allocated close in time are often used close in time. The default allocator may scatter them across pages. A custom allocator can cluster related objects:

```

// Arena allocator for scene graph nodes – all nodes for one frame
// are allocated from a contiguous arena, giving excellent cache locality.

class ArenaAllocator {
public:
    explicit ArenaAllocator(std::size_t arenaSize = 1024 * 1024)
        : arenaSize_(arenaSize)
        , current_(nullptr)
        , end_(nullptr)
    {}

    ~ArenaAllocator() {
        for (auto* block : blocks_) {
            std::free(block);
        }
    }

    void* allocate(std::size_t size, std::size_t alignment = alignof(std::max_align_t)) {
        // Align the current pointer
        std::size_t space = static_cast<std::size_t>(end_ - current_);
        void* aligned = current_;
        if (!std::align(alignment, size, aligned, space)) {
            // Not enough room in current block – allocate a new one
            allocateBlock(std::max(arenaSize_, size + alignment));
        }
    }
};

```

```

        space = static_cast<std::size_t>(end_ - current_);
        aligned = current_;
        if (!std::align(alignment, size, aligned, space)) {
            throw std::bad_alloc();
        }
    }
    current_ = static_cast<char*>(aligned) + size;
    return aligned;
}

// Reset the arena – "free" everything at once (O(1))
void reset() noexcept {
    for (auto* block : blocks_) {
        std::free(block);
    }
    blocks_.clear();
    current_ = nullptr;
    end_ = nullptr;
}

// No individual deallocation – everything freed together via reset()
// This is the key advantage: allocation is a pointer bump, deallocation is free

private:
    void allocateBlock(std::size_t size) {
        auto* block = static_cast<char*>(std::malloc(size));
        if (!block) throw std::bad_alloc();
        blocks_.push_back(block);
        current_ = block;
        end_ = block + size;
    }

    std::size_t arenaSize_;
    char* current_;
    char* end_;
    std::vector<char*> blocks_;
};

// Usage in a game engine:
class SceneNode {
public:
    static void* operator new(std::size_t size) {
        return currentArena_>allocate(size);
    }

    // delete is a no-op – memory is freed when the arena is reset
    static void operator delete(void*) noexcept {}

    static void setArena(ArenaAllocator* arena) { currentArena_ = arena; }

private:
    static thread_local ArenaAllocator* currentArena_;
    // ... node data ...
};

```

8.5.6 Reason 5: Gathering Allocation Statistics

Before optimizing, you need data. A custom allocator can collect detailed statistics:

```

#include <map>
#include <mutex>
#include <iostream>

struct AllocationStats {
    std::size_t totalAllocations = 0;
    std::size_t totalDeallocations = 0;
    std::size_t currentBytesAllocated = 0;
    std::size_t peakBytesAllocated = 0;
    std::size_t totalBytesAllocated = 0;
    std::map<std::size_t, std::size_t> sizeHistogram; // size -> count
    std::mutex mutex;

    void recordAllocation(std::size_t size) {
        std::lock_guard<std::mutex> lock(mutex);
        ++totalAllocations;
        currentBytesAllocated += size;
        totalBytesAllocated += size;
        peakBytesAllocated = std::max(peakBytesAllocated, currentBytesAllocated);
        ++sizeHistogram[size];
    }

    void recordDeallocation(std::size_t size) {
        std::lock_guard<std::mutex> lock(mutex);
        ++totalDeallocations;
        currentBytesAllocated -= size;
    }

    void report() const {
        std::cout << "=== Allocation Statistics ===\n";
        std::cout << "Total allocations:  " << totalAllocations << "\n";
        std::cout << "Total deallocations: " << totalDeallocations << "\n";
        std::cout << "Leaked allocations:  " << (totalAllocations - totalDeallocations) << "\n";
        std::cout << "Peak memory usage:   " << peakBytesAllocated << " bytes\n";
        std::cout << "Total memory used:   " << totalBytesAllocated << " bytes\n";
        std::cout << "\nSize distribution:\n";
        for (const auto& [size, count] : sizeHistogram) {
            std::cout << "  " << size << " bytes: " << count << " allocations\n";
        }
    }
};

static AllocationStats globalStats;

void* operator new(std::size_t size) {
    // Store the size at the beginning of the block so we can track deallocation
    void* p = std::malloc(size + sizeof(std::size_t));
    if (!p) throw std::bad_alloc();
    *static_cast<std::size_t*>(p) = size;
    globalStats.recordAllocation(size);
    return static_cast<char*>(p) + sizeof(std::size_t);
}

void operator delete(void* p) noexcept {
    if (!p) return;
    void* real = static_cast<char*>(p) - sizeof(std::size_t);
    std::size_t size = *static_cast<std::size_t*>(real);
    globalStats.recordDeallocation(size);
    std::free(real);
}

```



```
// Call globalStats.report() at program exit via atexit() or a destructor
```

8.5.7 Reason 6: Alignment Requirements

The default operator `new` is guaranteed to return memory aligned for any standard type (`alignof(std::max_align_t)`, typically 8 or 16 bytes). But some hardware requires stricter alignment:

```
// SSE/AVX require 16-byte or 32-byte alignment
// GPU buffers may require 256-byte alignment

// BAD: Default new may not provide sufficient alignment
float* data = new float[1024]; // Only guaranteed 8- or 16-byte aligned
// __m256* avxData = reinterpret_cast<__m256*>(data); // May crash on misaligned access!

// GOOD: Custom allocator with explicit alignment (pre-C++17)
void* alignedNew(std::size_t size, std::size_t alignment) {
    // Allocate extra space for alignment and a pointer to the real start
    std::size_t totalSize = size + alignment + sizeof(void*);
    void* rawMemory = std::malloc(totalSize);
    if (!rawMemory) throw std::bad_alloc();

    // Align the user pointer
    char* aligned = reinterpret_cast<char*>(
        (reinterpret_cast<std::uintptr_t>(rawMemory) + sizeof(void*) + alignment - 1)
        & ~(alignment - 1)
    );

    // Store the real pointer just before the aligned pointer
    reinterpret_cast<void**>(aligned)[-1] = rawMemory;
    return aligned;
}

void alignedDelete(void* p) noexcept {
    if (p) {
        std::free(reinterpret_cast<void**>(p)[-1]);
    }
}

// BETTER (C++17): Use aligned new directly
struct alignas(32) AVXData {
    float values[8];
};

// C++17 guarantees that new respects alignas for over-aligned types
AVXData* data = new AVXData[1024]; // Guaranteed 32-byte aligned in C++17

// Or use the explicit aligned allocation functions:
void* p = ::operator new(sizeof(AVXData), std::align_val_t{32});
::operator delete(p, std::align_val_t{32});
```

8.5.8 When NOT to Replace new and delete

Sometimes the default allocator is perfectly fine. Do not replace it just because you can:

```
// BAD: Premature optimization – replacing new/delete without profiling
// The default allocator in modern C++ runtimes (glibc, tcmalloc, jemalloc)
// is already highly optimized for most workloads.

// GOOD: Profile first, then decide
// Use tools like:
// - Valgrind (memcheck, massif)
// - AddressSanitizer / LeakSanitizer
// - HeapTrack
// - Custom profiling via LD_PRELOAD
// Only replace new/delete when you have data showing it's a bottleneck.
```

8.5.9 Summary: Legitimate Reasons to Replace

Reason	Description
Detect usage errors	Overflow/underflow detection, double-delete detection, leak detection
Improve speed	Pool allocators for fixed-size objects, arena allocators for batch allocation
Reduce memory overhead	Eliminate per-allocation bookkeeping for small objects
Reduce fragmentation	Custom strategies for known allocation patterns
Improve locality	Cluster related objects in memory for cache performance
Alignment	Guarantee alignment beyond the default (e.g., SIMD, DMA)
Statistics	Gather data about allocation patterns to inform optimization
Security	Overwrite freed memory to prevent information leakage

8.5.10 Things to Remember

- There are many valid reasons to write custom versions of `new` and `delete`, including improving performance, debugging heap usage errors, and collecting heap usage information.
- The default allocator is a general-purpose compromise. If you understand your allocation patterns, you can often do significantly better.
- Profile before replacing. Modern allocators (tcmalloc, jemalloc, mimalloc) are very good, and replacing the global operator `new/delete` has program-wide consequences.
- When replacing operator `new`, always also replace operator `delete`, and vice versa. They must be a matched pair.

8.6 Item 51: Adhere to Convention When Writing `new` and `delete`

8.7 Visual Summary

ITEM 51: ADHERE TO CONVENTION WHEN WRITING NEW AND DELETE

1. Custom operator new starts -> handle size 0 as a valid request.
2. Allocation fails -> call new-handler in a retry loop or throw bad_alloc.
3. Class-specific allocator -> wrong size falls back to global new/delete.
4. operator delete receives nullptr -> do nothing.
5. Meaning: custom allocation must still obey the standard allocation contract.

8.7.1 Core Concept

If you decide to write custom operator new and operator delete, you must follow the conventions established by the C++ standard. Violating these conventions leads to undefined behavior, subtle bugs, and code that breaks when others (or the standard library) rely on the expected semantics.

8.7.2 Convention 1: operator new Must Return a Legitimate Pointer Even for Zero-Byte Requests

The C++ standard says that operator new(0) must succeed and return a legitimate, unique pointer. This is typically handled by treating zero-byte requests as one-byte requests:

```
// CORRECT: Handling zero-byte requests
void* operator new(std::size_t size) {
    if (size == 0) {
        size = 1; // Treat zero-byte requests as one-byte requests
    }

    while (true) {
        // Attempt allocation
        void* p = std::malloc(size);
        if (p) return p;

        // Allocation failed – get the current new-handler
        std::new_handler handler = std::get_new_handler();
        if (handler) {
            handler();
            // Handler should either make memory available, throw, or abort
            // Loop back and try again
        } else {
            throw std::bad_alloc();
        }
    }
}
```

8.7.3 Convention 2: operator new Must Have an Infinite Loop Calling the new-handler

This is the most commonly violated convention. operator new must keep trying to allocate, calling the new-handler each time it fails, until either the allocation succeeds or the handler throws/aborts/deinstalls itself:

```

// BAD: Only tries once
void* operator new(std::size_t size) {
    void* p = std::malloc(size);
    if (!p) throw std::bad_alloc(); // Wrong! Must call new-handler first
    return p;
}

// BAD: Calls handler but doesn't loop
void* operator new(std::size_t size) {
    void* p = std::malloc(size);
    if (!p) {
        std::new_handler handler = std::get_new_handler();
        if (handler) handler();
        p = std::malloc(size); // One retry only – wrong!
        if (!p) throw std::bad_alloc();
    }
    return p;
}

// GOOD: Correct infinite-loop convention
void* operator new(std::size_t size) {
    if (size == 0) size = 1;

    while (true) {
        void* p = std::malloc(size);
        if (p) return p;

        // Allocation failed – invoke the new-handler
        std::new_handler handler = std::get_new_handler();
        if (!handler) throw std::bad_alloc();
        handler();
        // The handler must have done one of the five things (see Item 49)
        // or we loop forever. That's by design – it's the handler's contract.
    }
}

```

8.7.4 Convention 3: Class-Specific operator new Must Handle “Wrong-Size” Requests

When a class provides operator new, it is designed for objects of exactly that class’s size. But if a derived class inherits the operator new without overriding it, the size parameter will be **larger** than expected:

```

class Base {
public:
    static void* operator new(std::size_t size) {
        // BAD: Assumes size == sizeof(Base)
        return pool_.allocate(); // Pool is sized for Base objects only!
    }

    static void operator delete(void* p, std::size_t size) noexcept {
        pool_.deallocate(p); // Also wrong for derived classes
    }

private:
    int x_, y_;
    static FixedSizePool<Base> pool_;
};

class Derived : public Base {
    int z_; // Derived is larger than Base
    double w_;
};

```

```

};

// When you do: Derived* d = new Derived;
// Base::operator new is called with size == sizeof(Derived), not sizeof(Base)!
// The pool allocator returns a block sized for Base – buffer overflow!

// GOOD: Check the size and fall back to global new for wrong sizes
class Base {
public:
    static void* operator new(std::size_t size) {
        if (size != sizeof(Base)) {
            // Derived class – use global operator new
            return ::operator new(size);
        }
        // Exact match – use our pool
        return pool_.allocate();
    }

    static void operator delete(void* p, std::size_t size) noexcept {
        if (!p) return; // C++ guarantees delete(nullptr) is safe
        if (size != sizeof(Base)) {
            ::operator delete(p);
            return;
        }
        pool_.deallocate(p);
    }

private:
    int x_, y_;
    static FixedSizePool<Base> pool_;
};

```

Note: The size check `size != sizeof(Base)` also implicitly handles the zero-byte case on most platforms, because `sizeof(Base) > 0` for any class with data members. However, for an empty class, `sizeof(Base)` is typically 1, and the zero-byte-to-one-byte promotion means you'd get `size == 1 == sizeof(Base)`, which is correct.

8.7.5 Convention 4: operator delete Must Handle nullptr

```

// GOOD: operator delete must tolerate null pointers
void operator delete(void* p) noexcept {
    if (!p) return; // Do nothing for nullptr – standard requires this
    // ... actual deallocation logic ...
    std::free(p);
}

```

8.7.6 Convention 5: operator new[] and operator delete[] Must Be Consistent

Array forms have their own set of subtleties:

```

class Widget {
public:
    // For operator new[], the size parameter includes extra space for
    // the array element count (typically sizeof(size_t) bytes prepended)
    static void* operator new[](std::size_t size) {
        // size = N * sizeof(Widget) + overhead for element count
        // You generally CANNOT deduce the number of elements from size
        return ::operator new(size);
    }

    static void operator delete[](void* p) noexcept {
        ::operator delete(p);
    }
};

```

8.7.7 Putting It All Together: A Correct Class-Level operator new/delete

```

#include <new>
#include <cstdlib>

class GameObject {
public:
    // Per-class operator new with all conventions followed
    static void* operator new(std::size_t size) {
        // Convention 1: Handle zero-byte requests (unlikely for a real class, but correct)
        if (size == 0) size = 1;

        // Convention 3: Handle wrong-size requests from derived classes
        if (size != sizeof(GameObject)) {
            return ::operator new(size);
        }

        // Convention 2: Infinite loop with new-handler
        while (true) {
            void* p = allocateFromPool(size);
            if (p) return p;

            // Allocation failed – try the new-handler
            std::new_handler handler = std::get_new_handler();
            if (!handler) throw std::bad_alloc();
            handler();
        }
    }

    // Per-class operator delete
    static void operator delete(void* p, std::size_t size) noexcept {
        // Convention 4: Handle nullptr
        if (!p) return;

        // Convention 3: Handle wrong-size (derived class) requests
        if (size != sizeof(GameObject)) {
            ::operator delete(p);
            return;
        }

        returnToPool(p);
    }

    virtual ~GameObject() = default;
};

```

```
private:
    static void* allocateFromPool(std::size_t size);
    static void returnToPool(void* p) noexcept;

    // ... class members ...
};
```

8.7.8 C++17 Aligned Allocation Conventions

C++17 introduced aligned versions of `operator new` and `operator delete`. If you replace one, you should replace all matching forms:

// C++17: The full set of operator new/delete for a class

```
class AlignedWidget {
public:
    // Regular forms
    static void* operator new(std::size_t size);
    static void operator delete(void* p) noexcept;
    static void operator delete(void* p, std::size_t size) noexcept; // C++14 sized delete

    // Array forms
    static void* operator new[](std::size_t size);
    static void operator delete[](void* p) noexcept;
    static void operator delete[](void* p, std::size_t size) noexcept;

    // Aligned forms (C++17) – called when alignof(AlignedWidget) > __STDCPP_DEFAULT_NEW_ALIGNMENT_
    static void* operator new(std::size_t size, std::align_val_t al);
    static void operator delete(void* p, std::align_val_t al) noexcept;
    static void operator delete(void* p, std::size_t size, std::align_val_t al) noexcept;

    // Aligned array forms (C++17)
    static void* operator new[](std::size_t size, std::align_val_t al);
    static void operator delete[](void* p, std::align_val_t al) noexcept;
    static void operator delete[](void* p, std::size_t size, std::align_val_t al) noexcept;
};
```

8.7.9 Complete Example: Thread-Safe Pool Allocator Following All Conventions

```
#include <new>
#include <cstdlib>
#include <mutex>
#include <vector>

class Particle {
public:
    Particle(float x, float y, float z, float life)
        : x_(x), y_(y), z_(z), life_(life) {}

    static void* operator new(std::size_t size) {
        if (size != sizeof(Particle)) {
            return ::operator new(size); // Wrong size – delegate
        }

        while (true) {
            {
                std::lock_guard<std::mutex> lock(poolMutex_);
                if (freeList_) {
                    void* p = freeList_;
```

```

        freeList_ = *static_cast<void**>(freeList_);
        return p;
    }
    // Free list empty – expand the pool
    expandPool();
    if (freeList_) {
        void* p = freeList_;
        freeList_ = *static_cast<void**>(freeList_);
        return p;
    }
    // Pool expansion failed – try the new-handler
    std::new_handler handler = std::get_new_handler();
    if (!handler) throw std::bad_alloc();
    handler();
}

static void operator delete(void* p, std::size_t size) noexcept {
    if (!p) return;
    if (size != sizeof(Particle)) {
        ::operator delete(p);
        return;
    }
    std::lock_guard<std::mutex> lock(poolMutex_);
    *static_cast<void**>(p) = freeList_;
    freeList_ = p;
}

static void operator delete(void* p) noexcept {
    // Fallback for when size is not available
    operator delete(p, sizeof(Particle));
}

private:
    float x_, y_, z_, life_;

    static constexpr std::size_t BLOCK_SIZE =
        sizeof(Particle) < sizeof(void*) ? sizeof(void*) : sizeof(Particle);
    static constexpr std::size_t POOL_GROWTH = 1024;

    static void expandPool() {
        // Must be called with poolMutex_ held
        char* chunk = static_cast<char*>(std::malloc(BLOCK_SIZE * POOL_GROWTH));
        if (!chunk) return; // Caller will check freeList_ or invoke new-handler
        chunks_.push_back(chunk);
        for (std::size_t i = 0; i < POOL_GROWTH; ++i) {
            void* block = chunk + i * BLOCK_SIZE;
            *static_cast<void**>(block) = freeList_;
            freeList_ = block;
        }
    }

    static std::mutex poolMutex_;
    static void* freeList_;
    static std::vector<char*> chunks_;
};

std::mutex Particle::poolMutex_;
void* Particle::freeList_ = nullptr;

```



```
std::vector<char*> Particle::chunks_;
```

8.7.10 Things to Remember

- `operator new` should contain an infinite loop trying to allocate memory, should call the new-handler if it can't satisfy a memory request, and should handle zero-byte requests. Class-specific versions should handle requests for blocks larger than expected (from derived classes).
- `operator delete` should do nothing if passed a null pointer. Class-specific versions should handle blocks that are larger than expected.
- Always pair `operator new` with a matching `operator delete`. If you provide a class-level `operator new`, provide the corresponding `operator delete` in the same class.
- For class-level `operator new`, always check the size parameter against `sizeof(YourClass)` and fall back to `::operator new` for mismatches — this correctly handles derived classes.

8.8 Item 52: Write Placement delete If You Write Placement new

8.9 Visual Summary

ITEM 52: WRITE PLACEMENT DELETE IF YOU WRITE PLACEMENT NEW

1. Placement new called → memory allocation succeeds, constructor then runs.
2. Constructor throws → compiler looks for matching placement delete.
3. No matching placement delete → allocated storage may leak.
4. Provide delete with same extra parameters as placement new.
5. Meaning: placement new needs matching cleanup for construction failure.

8.9.1 Core Concept

This item addresses a subtle but critical requirement: **if you write a placement version of `operator new`, you must also write the corresponding placement version of `operator delete`**. Failing to do so causes memory leaks when constructors throw exceptions. Understanding why requires understanding the relationship between new expressions and `operator new` functions.

8.9.2 The new Expression vs. `operator new`

A new expression like `Widget* pw = new Widget;` does **two** things:

1. Calls `operator new` to allocate raw memory
2. Calls the `Widget` constructor on that raw memory

If step 2 (the constructor) throws an exception, the memory from step 1 must be freed. The C++ runtime automatically does this by calling the matching `operator delete`. But which `operator delete` is “matching”?

[illegible]

```
Widget* pw = new Widget;           operator new(size_t)           operator delete(void*)
Widget* pw = new (arena) W;  operator new(size_t, Arena&)  operator delete(void*, Arena&)
Widget* pw = new (nothrow) W; operator new(size_t, nothrow_t) operator delete(void*, nothrow_t)
```

The rule: **the runtime calls the operator delete whose extra parameters match the extra parameters of the operator new that was used.**

8.9.3 What Happens Without a Matching Placement delete

```
// BAD: Placement new without matching placement delete – MEMORY LEAK

class Widget {
public:
    // Placement new that logs allocations
    static void* operator new(std::size_t size, std::ostream& logStream) {
        logStream << "Allocating " << size << " bytes for Widget\n";
        return ::operator new(size);
    }

    // No matching placement delete!
    // static void operator delete(void* p, std::ostream& logStream) noexcept { ... }

    Widget() {
        // Suppose this throws...
        throw std::runtime_error("Widget construction failed");
    }
};

void createWidget() {
    try {
        // Step 1: operator new(size_t, ostream&) is called – memory allocated
        // Step 2: Widget() constructor is called – it throws
        // Step 3: Runtime looks for operator delete(void*, ostream&) – NOT FOUND
        // Step 4: Runtime does NOTHING – memory is leaked!
        Widget* pw = new (std::cerr) Widget;
    } catch (...) {
        // The memory allocated in step 1 has been leaked
        // There is no way to recover it
    }
}
```

```
// GOOD: Matching placement delete

class Widget {
public:
    static void* operator new(std::size_t size, std::ostream& logStream) {
        logStream << "Allocating " << size << " bytes for Widget\n";
        return ::operator new(size);
    }

    // Matching placement delete – called by runtime if constructor throws
    static void operator delete(void* p, std::ostream& logStream) noexcept {
        logStream << "Deallocating Widget memory (constructor threw)\n";
        ::operator delete(p);
    }

    // Normal delete – called by delete expressions (e.g., delete pw;)
    static void operator delete(void* p) noexcept {
        ::operator delete(p);
    }
};
```

8.9.4 Important Distinction: Placement delete Is NEVER Called Directly

Placement operator `delete` is **only** called by the runtime when a constructor throws during a placement new expression. Client code never directly invokes it:

```
Widget* pw = new (std::cerr) Widget; // Uses placement new

// When you're done with the object, you call normal delete:
delete pw; // Calls Widget::operator delete(void*), NOT the placement version!

// There is no "placement delete expression" syntax in C++.
// You cannot write: delete (std::cerr) pw; // Syntax error!
```

8.9.5 Real-World Example: Debug Allocation with Source Location

```
#include <new>
#include <cstdio>
#include <cstring>

class TrackedObject {
public:
    // Placement new that records source location
    static void* operator new(std::size_t size, const char* file, int line) {
        std::printf("[ALLOC] %zu bytes at %s:%d\n", size, file, line);

        // Store the source location alongside the allocation
        void* p = ::operator new(size);
        recordAllocation(p, size, file, line);
        return p;
    }

    // MATCHING placement delete – critical for exception safety
    static void operator delete(void* p, const char* file, int line) noexcept {
        std::printf("[DEALLOC-EXCEPTION] at %s:%d\n", file, line);
        removeAllocationRecord(p);
        ::operator delete(p);
    }
};
```

```

// Normal delete – used by delete expressions
static void operator delete(void* p) noexcept {
    if (p) {
        reportDeallocation(p);
        removeAllocationRecord(p);
    }
    ::operator delete(p);
}

virtual ~TrackedObject() = default;

private:
    static void recordAllocation(void*, std::size_t, const char*, int);
    static void removeAllocationRecord(void*);
    static void reportDeallocation(void*);
};

// Macro to capture source location automatically
#define NEW_TRACKED new (__FILE__, __LINE__)

// Usage:
// TrackedObject* obj = NEW_TRACKED TrackedObject;
// delete obj; // Calls normal operator delete

```

8.9.6 The Name-Hiding Problem

A class-specific operator `new` **hides** both the global operator `new` and all inherited versions. This is a separate but related trap:

```

// BAD: Class-specific operator new hides the normal form

class Base {
public:
    // Only provides a placement form – hides the normal operator new!
    static void* operator new(std::size_t size, const MemoryPool& pool) {
        return pool.allocate(size);
    }

    static void operator delete(void* p, const MemoryPool& pool) noexcept {
        pool.deallocate(p);
    }

    static void operator delete(void* p) noexcept {
        ::operator delete(p);
    }
};

// This breaks:
// Base* pb = new Base; // Error! Normal operator new is hidden!
// Only this works:
// Base* pb = new (myPool) Base;

```

// GOOD: Provide all standard forms alongside custom forms

```
class Base {
public:
    // Normal new/delete
    static void* operator new(std::size_t size) {
        return ::operator new(size);
    }

    static void operator delete(void* p) noexcept {
        ::operator delete(p);
    }

    // Nothrow new/delete
    static void* operator new(std::size_t size, const std::nothrow_t& nt) noexcept {
        return ::operator new(size, nt);
    }

    static void operator delete(void* p, const std::nothrow_t&) noexcept {
        ::operator delete(p);
    }

    // Custom placement new/delete
    static void* operator new(std::size_t size, const MemoryPool& pool) {
        return pool.allocate(size);
    }

    static void operator delete(void* p, const MemoryPool& pool) noexcept {
        pool.deallocate(p);
    }
};
```

8.9.7 A Convenient Base Class to Avoid Hiding

You can create a base class that declares all the standard forms, then inherit from it:

```
class StandardNewDeleteForms {
public:
    // Normal new/delete
    static void* operator new(std::size_t size) {
        return ::operator new(size);
    }

    static void operator delete(void* p) noexcept {
        ::operator delete(p);
    }

    // Nothrow new/delete
    static void* operator new(std::size_t size, const std::nothrow_t& nt) noexcept {
        return ::operator new(size, nt);
    }

    static void operator delete(void* p, const std::nothrow_t&) noexcept {
        ::operator delete(p);
    }

    // Standard placement new/delete (construct in pre-allocated memory)
    static void* operator new(std::size_t, void* ptr) noexcept {
        return ptr; // Standard placement new – just returns the pointer
    }

    static void operator delete(void*, void*) noexcept {
        // Standard placement delete – does nothing
    }
};
```

```

    }
};

class Widget : public StandardNewDeleteForms {
public:
    // Bring the base class versions into scope so they aren't hidden
    using StandardNewDeleteForms::operator new;
    using StandardNewDeleteForms::operator delete;

    // Now add custom placement forms – they won't hide the standard ones
    static void* operator new(std::size_t size, std::ostream& log) {
        log << "Allocating Widget (" << size << " bytes)\n";
        return ::operator new(size);
    }

    static void operator delete(void* p, std::ostream& log) noexcept {
        log << "Deallocating Widget (constructor threw)\n";
        ::operator delete(p);
    }
};

// All of these work:
// Widget* w1 = new Widget;                // Normal
// Widget* w2 = new (std::nothrow) Widget;   // Nothrow
// Widget* w3 = new (std::cerr) Widget;      // Custom placement
// char buffer[sizeof(Widget)];
// Widget* w4 = new (buffer) Widget;         // Standard placement

```

8.9.8 Real Placement new (Constructing in Pre-Allocated Memory)

The term “placement new” originally referred specifically to constructing an object in memory you’ve already allocated. This is the standard library’s `operator new(size_t, void*)`:

```

#include <new>
#include <cstdlib>
#include <memory>

// Constructing objects in pre-allocated memory (the "real" placement new)

class Entity {
public:
    Entity(int id, const std::string& name)
        : id_(id), name_(name) {
        std::cout << "Entity " << id_ << " (" << name_ << ") constructed\n";
    }

    ~Entity() {
        std::cout << "Entity " << id_ << " (" << name_ << ") destroyed\n";
    }

private:
    int id_;
    std::string name_;
};

void demonstratePlacementNew() {
    // 1. Allocate raw memory (no constructor called)
    void* rawMemory = ::operator new(sizeof(Entity));

    // 2. Construct an Entity in that raw memory using placement new

```

```

Entity* entity = new (rawMemory) Entity(42, "Player");

// 3. When done, you must manually call the destructor
entity->~Entity(); // Explicit destructor call – normally never done except with placement new

// 4. Then free the raw memory
::operator delete(rawMemory);
}

// A more realistic example: pre-allocated buffer of objects
class EntityPool {
public:
    explicit EntityPool(std::size_t maxEntities)
        : maxEntities_(maxEntities)
        , count_(0)
    {
        // Allocate raw memory for maxEntities Entity objects
        buffer_ = static_cast<Entity*> (::operator new(maxEntities * sizeof(Entity)));
    }

    ~EntityPool() {
        // Destroy all constructed entities
        for (std::size_t i = 0; i < count_; ++i) {
            buffer_[i].~Entity();
        }
        // Free the raw memory
        ::operator delete(buffer_);
    }

    template<typename... Args>
    Entity* construct(Args&&... args) {
        if (count_ >= maxEntities_) {
            throw std::runtime_error("EntityPool full");
        }
        // Placement new: construct in pre-allocated memory
        Entity* e = new (&buffer_[count_]) Entity(std::forward<Args>(args)...);
        ++count_;
        return e;
    }

private:
    Entity* buffer_;
    std::size_t maxEntities_;
    std::size_t count_;
};

```

8.9.9 Interaction with Smart Pointers

When using placement new with smart pointers, you need custom deleters:

```

// Custom deleter for objects constructed via placement new in a pool
class PoolDeleter {
public:
    explicit PoolDeleter(MemoryPool& pool) : pool_(&pool) {}

    void operator()(Widget* p) const noexcept {
        if (p) {
            p->~Widget();           // Call destructor
            pool_>deallocate(p);    // Return memory to pool (not free/delete)
        }
    }

private:
    MemoryPool* pool_;
};

// Usage:
MemoryPool pool(1024);
void* mem = pool.allocate(sizeof(Widget));
Widget* raw = new (mem) Widget(args);
std::unique_ptr<Widget, PoolDeleter> pw(raw, PoolDeleter(pool));
// pw's destructor will properly clean up using the custom deleter

```

8.9.10 Complete Example: Logging Allocator with Full Placement new/delete Support

```

#include <new>
#include <iostream>
#include <fstream>

class LoggedAllocation {
public:
    // --- Standard forms (must not be hidden) ---

    static void* operator new(std::size_t size) {
        return ::operator new(size);
    }

    static void operator delete(void* p) noexcept {
        ::operator delete(p);
    }

    static void* operator new(std::size_t size, const std::nothrow_t& nt) noexcept {
        return ::operator new(size, nt);
    }

    static void operator delete(void* p, const std::nothrow_t&) noexcept {
        ::operator delete(p);
    }

    // --- Custom placement form: log to a stream ---

    static void* operator new(std::size_t size, std::ostream& log) {
        void* p = ::operator new(size);
        log << "[ALLOC] " << size << " bytes at " << p << "\n";
        return p;
    }

    // Matching placement delete (called only when constructor throws)
    static void operator delete(void* p, std::ostream& log) noexcept {

```



```

        log << "[DEALLOC-EXCEPTION] at " << p << "\n";
        ::operator delete(p);
    }

    // --- Custom placement form: log to a file by name ---

    static void* operator new(std::size_t size, const char* logFile) {
        void* p = ::operator new(size);
        std::ofstream log(logFile, std::ios::app);
        log << "[ALLOC] " << size << " bytes at " << p << "\n";
        return p;
    }

    // Matching placement delete
    static void operator delete(void* p, const char* logFile) noexcept {
        std::ofstream log(logFile, std::ios::app);
        log << "[DEALLOC-EXCEPTION] at " << p << "\n";
        ::operator delete(p);
    }

    // --- Array forms ---

    static void* operator new[](std::size_t size) {
        return ::operator new[](size);
    }

    static void operator delete[](void* p) noexcept {
        ::operator delete[](p);
    }

    static void* operator new[](std::size_t size, std::ostream& log) {
        void* p = ::operator new[](size);
        log << "[ALLOC[]] " << size << " bytes at " << p << "\n";
        return p;
    }

    static void operator delete[](void* p, std::ostream& log) noexcept {
        log << "[DEALLOC[]-EXCEPTION] at " << p << "\n";
        ::operator delete[](p);
    }
};

// Usage:
// LoggedAllocation* la1 = new LoggedAllocation;           // Normal
// LoggedAllocation* la2 = new (std::nothrow) LoggedAllocation; // Nothrow
// LoggedAllocation* la3 = new (std::cerr) LoggedAllocation;  // Logged to cerr
// LoggedAllocation* la4 = new ("alloc.log") LoggedAllocation; // Logged to file
// delete la1; // Always uses normal operator delete

```

8.9.11 Things to Remember

- When you write a placement version of `operator new`, be sure to write the corresponding placement version of `operator delete`. If you don't, your program will have subtle, intermittent memory leaks whenever the constructed object's constructor throws an exception.
- When you declare placement versions of `new` and `delete` in a class, be sure not to unintentionally hide the normal (non-placement) versions. Use using declarations or inherit from a base class that provides all standard forms.
- Placement `operator delete` is **only** called by the runtime during exception handling (when a constructor throws in a placement `new` expression). It is **never** called by a `delete` expression — `delete pw`

always calls the normal operator `delete(void*)`.

- The standard placement `new` (constructing in pre-allocated memory) requires explicit destructor calls and manual memory management. Wrap such objects in smart pointers with custom deleters for safety.

8.10 Summary of Chapter 8

Item	Core Message
49	<code>set_new_handler</code> gives you a last-chance hook before allocation failure. The handler must follow a strict five-option contract. <code>nothrow</code> <code>new</code> is less useful than it appears because constructors can still throw.
50	Replace <code>new/delete</code> for debugging, performance, statistics, alignment, or locality — but profile first and know the conventions.
51	Follow the standard's conventions: infinite retry loop with <code>new_handler</code> , handle zero-byte requests, handle wrong-size requests from derived classes, and handle <code>nullptr</code> in <code>delete</code> .
52	Every placement operator <code>new</code> needs a matching placement operator <code>delete</code> or you get memory leaks when constructors throw. Use <code>using</code> declarations to avoid hiding the standard forms.

9 Chapter 9: Miscellany

9.1 Chapter Flow

CHAPTER 9: MISCELLANY
Item 53 -> warnings are early bug reports; keep code warning-clean. Item 54 -> standard library first for common containers/algorithms/utilities. Item 55 -> Boost can fill advanced gaps, but manage dependency cost.

Items 53-55: Closing advice on compiler warnings, the standard library landscape (including TR1), and the Boost ecosystem — updated with modern C++11/14/17/20 context.

9.2 Item 53: Pay Attention to Compiler Warnings

9.3 Visual Summary

ITEM 53: PAY ATTENTION TO COMPILER WARNINGS

1. Compiler warning appears -> compiler found a suspicious construct.
2. Ignoring warnings -> portability and correctness risk accumulates.
3. Fix source cause or document intentional suppression narrowly.
4. Different compilers warn about different bugs.
5. Meaning: warning-clean code lets real regressions stand out.

9.3.1 Core Concept

Compiler warnings are not optional noise to be silenced. They represent the compiler's best effort to tell you about code that is **technically legal** but **almost certainly wrong**. Experienced C++ programmers treat warnings as errors. Many of the most insidious bugs in C++ programs are things the compiler warned about and the programmer ignored.

9.3.2 Warning Level Settings

Before examining specific warnings, know how to maximize your compiler's help:

GCC / Clang: Maximum useful warnings

```
g++ -Wall -Wextra -Wpedantic -Werror -Wshadow -Wconversion -Wsign-conversion \
    -Wnon-virtual-dtor -Wold-style-cast -Wcast-align -Wunused \
    -Woverloaded-virtual -Wmisleading-indentation -Wduplicated-cond \
    -Wduplicated-branches -Wlogical-op -Wnull-dereference \
    -Wdouble-promotion -Wformat=2 -Wimplicit-fallthrough \
    -std=c++17 source.cpp
```

MSVC: Maximum useful warnings

```
cl /W4 /WX /permissive- /analyze source.cpp
```

CMake: Setting warning levels properly

In CMakeLists.txt:

target_compile_options(mylib PRIVATE

\$<\$<CXX_COMPILER_ID:GNU,Clang>:-Wall -Wextra -Wpedantic -Werror>

\$<\$<CXX_COMPILER_ID:MSVC>:/W4 /WX>

#)

9.3.3 Warning 1: Hiding Inherited Virtual Functions

This is the most famous and dangerous warning — the one Meyers specifically highlights in the book. Redefining a non-virtual function in a derived class **hides** the base class version rather than overriding it:

// BAD: Accidentally hiding a virtual function due to different parameter types

```
class Base {
public:
    virtual void display(int value) const {
        std::cout << "Base::display(int): " << value << "\n";
    }
    virtual ~Base() = default;
};

class Derived : public Base {
public:
    // WARNING: 'Derived::display' hides 'Base::display'
    // This has a DIFFERENT parameter type – it's hiding, not overriding!
    virtual void display(double value) const {
        std::cout << "Derived::display(double): " << value << "\n";
    }
};

void demonstrate() {
    Derived d;
    Base* pb = &d;

    pb->display(42);    // Calls Base::display(int) – NOT Derived::display(double)!
    d.display(42);     // Calls Derived::display(double) – implicit int-to-double conversion
    // d.display(42) behaves differently from pb->display(42) – this is almost always a bug
}
```

// GOOD: Use override (C++11) to catch this at compile time

```
class Derived : public Base {
public:
    // COMPILE ERROR: 'display' marked 'override' but does not override any base class method
    void display(double value) const override { // <-- override catches the mistake
        std::cout << "Derived::display(double): " << value << "\n";
    }
};

// CORRECT version:
class Derived : public Base {
public:
    void display(int value) const override { // Correct parameter type
        std::cout << "Derived::display(int): " << value << "\n";
    }
};
```

9.3.4 Warning 2: Implicit Conversions and Narrowing

```

// BAD: Implicit narrowing conversion
void processData(int count) {
    // WARNING: conversion from 'size_t' (aka 'unsigned long') to 'int', possible loss of data
    std::vector<double> data(100);
    int size = data.size(); // size_t -> int: potential data loss on large vectors

    // WARNING: comparing signed and unsigned integers
    for (int i = 0; i < data.size(); ++i) { // int vs. size_t comparison
        data[i] = i * 1.5;
    }
}

// WARNING: implicit conversion loses integer precision
void sendPacket(uint16_t port) {
    int userPort = 80443; // Larger than uint16_t can hold
    sendPacket(userPort); // Silently truncated! Port becomes garbage
}

// GOOD: Be explicit about conversions
void processData(int count) {
    std::vector<double> data(100);
    auto size = data.size(); // auto deduces size_t – no conversion

    for (std::size_t i = 0; i < data.size(); ++i) {
        data[i] = static_cast<double>(i) * 1.5;
    }

    // Or better yet, use range-based for:
    for (auto& val : data) {
        val = 0.0;
    }
}

void sendPacket(uint16_t port);

void setupNetwork() {
    int userPort = 80443;
    if (userPort < 0 || userPort > 65535) {
        throw std::out_of_range("Port number out of range");
    }
    sendPacket(static_cast<uint16_t>(userPort));
}

```

9.3.5 Warning 3: Unused Variables and Parameters

```

// BAD: Unused variables – often indicate logic errors
int computeResult(int input) {
    int intermediateResult = input * 42; // WARNING: unused variable
    int finalResult = input + 1;
    return finalResult;
    // Was intermediateResult supposed to be used? Probably a bug.
}

// BAD: Unused parameters
class EventHandler {
public:
    // WARNING: unused parameter 'event'
    virtual void onMouseMove(int x, int y, const MouseEvent& event) {
        drawCursor(x, y);
        // 'event' is unused – but it's part of the interface
    }
};

// GOOD: Make intent explicit
int computeResult(int input) {
    int intermediateResult = input * 42;
    int finalResult = intermediateResult + 1; // Actually use it!
    return finalResult;
}

// GOOD: Suppress unused parameter warnings explicitly
class EventHandler {
public:
    // Option 1: Comment out or omit the parameter name
    virtual void onMouseMove(int x, int y, const MouseEvent& /*event*/) {
        drawCursor(x, y);
    }

    // Option 2 (C++17): Use [[maybe_unused]]
    virtual void onClick([[maybe_unused]] int x, [[maybe_unused]] int y) {
        // Parameters intentionally unused in base class
    }
};

```

9.3.6 Warning 4: Uninitialized Variables

```

// BAD: Using uninitialized variables – undefined behavior
int calculateScore(bool bonusRound) {
    int score; // WARNING: 'score' may be used uninitialized
    if (bonusRound) {
        score = 100;
    }
    return score; // If bonusRound is false, score is uninitialized – UB!
}

// BAD: Partially initialized struct
struct Config {
    int width;
    int height;
    bool fullscreen;
    int refreshRate;
};

Config loadConfig() {

```

```

Config cfg;
cfg.width = 1920;
cfg.height = 1080;
// WARNING: 'cfg.fullscreen' and 'cfg.refreshRate' not initialized
return cfg;
}

```

```

// GOOD: Always initialize
int calculateScore(bool bonusRound) {
    int score = 0; // Default value
    if (bonusRound) {
        score = 100;
    }
    return score;
}

```

```

// GOOD: Use designated initializers (C++20) or value-initialization
struct Config {
    int width = 0;
    int height = 0;
    bool fullscreen = false;
    int refreshRate = 60;
};

```

```

Config loadConfig() {
    Config cfg{
        .width = 1920,
        .height = 1080,
        // fullscreen and refreshRate get their default values
    };
    return cfg;
}

```

9.3.7 Warning 5: Order of Member Initialization

```

// BAD: Member initializer list order doesn't match declaration order
class TextRenderer {
public:
    // WARNING: member initializer for 'fontSize_' will be executed after
    //           member initializer for 'lineHeight_' because of declaration order
    TextRenderer(int fontSize)
        : fontSize_(fontSize) // This is written first but executed SECOND
        , lineHeight_(fontSize_ * 1.5) // This is written second but executed FIRST!
    {
        // lineHeight_ is initialized before fontSize_, so fontSize_ is garbage
        // when lineHeight_ is computed. lineHeight_ gets a garbage value.
    }

private:
    double lineHeight_; // Declared first – initialized first!
    int fontSize_;      // Declared second – initialized second!
};

```

```

// GOOD: Match initializer order to declaration order
class TextRenderer {
public:
    TextRenderer(int fontSize)
        : lineHeight_(fontSize * 1.5) // Use constructor parameter, not member
        , fontSize_(fontSize)
    {}

    // BETTER: Reorder declarations to match logical initialization order
    // Or rewrite so the order doesn't matter:

private:
    int fontSize_; // Now declared first
    double lineHeight_; // Now declared second
};

class TextRenderer {
public:
    TextRenderer(int fontSize)
        : fontSize_(fontSize)
        , lineHeight_(fontSize_ * 1.5) // Safe - fontSize_ is initialized first
    {}

private:
    int fontSize_;
    double lineHeight_;
};

```

9.3.8 Warning 6: Missing Return Statements

```

// BAD: Not all control paths return a value
// WARNING: control reaches end of non-void function
int classify(int value) {
    if (value > 0) return 1;
    if (value < 0) return -1;
    // What about value == 0? Missing return! Undefined behavior!
}

// BAD: Switch without default
enum class Color { Red, Green, Blue };

const char* colorName(Color c) {
    switch (c) {
        case Color::Red: return "Red";
        case Color::Green: return "Green";
        // WARNING: not all enum values handled, and no default
        // If Color::Blue is passed, UB!
    }
}

```



```

// GOOD: All paths return
int classify(int value) {
    if (value > 0) return 1;
    if (value < 0) return -1;
    return 0;
}

// GOOD: Handle all cases
const char* colorName(Color c) {
    switch (c) {
        case Color::Red:    return "Red";
        case Color::Green:  return "Green";
        case Color::Blue:   return "Blue";
    }
    // Some compilers still warn without this, even though all enum values are covered:
    return "Unknown";
}

```

9.3.9 Warning 7: Dangling References and Lifetime Issues

```

// BAD: Returning reference to local
// WARNING: reference to local variable returned
const std::string& greet(const std::string& name) {
    std::string greeting = "Hello, " + name + "!";
    return greeting; // greeting is destroyed when the function returns!
}

// BAD: Dangling reference from temporary (C++20 compilers catch more of these)
std::string_view getName() {
    std::string name = "Alice";
    return name; // WARNING: returning string_view of local string
}

// GOOD: Return by value
std::string greet(const std::string& name) {
    return "Hello, " + name + "!"; // Return by value – move semantics makes this efficient
}

// GOOD: Ensure the string outlives the string_view
class User {
public:
    std::string_view getName() const { return name_; } // OK – member outlives the call
private:
    std::string name_;
};

```

9.3.10 Warning 8: Implicit Fallthrough in Switch

```
// BAD: Accidental fallthrough
void handleEvent(int eventType) {
    switch (eventType) {
        case 1:
            startAnimation();
            // WARNING: implicit fallthrough
        case 2:
            playSound();
            break;
        case 3:
            logEvent();
            break;
    }
}
```

```
// GOOD: Use [[fallthrough]] (C++17) when intentional, break otherwise
void handleEvent(int eventType) {
    switch (eventType) {
        case 1:
            startAnimation();
            [[fallthrough]]; // Explicit: "yes, I intend to fall through"
        case 2:
            playSound();
            break;
        case 3:
            logEvent();
            break;
        default:
            break;
    }
}
```

9.3.11 Warning 9: Shadowed Variables

```
// BAD: Variable shadowing
int count = 10; // Global

void process() {
    int count = 5; // WARNING: declaration shadows a global variable
    for (int i = 0; i < count; ++i) {
        int count = i * 2; // WARNING: declaration shadows a local variable
        std::cout << count << "\n"; // Which count? The innermost one.
    }
}
```

```
// GOOD: Use distinct names
int globalCount = 10;

void process() {
    int localCount = 5;
    for (int i = 0; i < localCount; ++i) {
        int doubled = i * 2;
        std::cout << doubled << "\n";
    }
}
```

9.3.12 Warning 10: Non-Virtual Destructor in Base Class

```
// BAD: Base class with virtual functions but non-virtual destructor
// WARNING: class 'Base' has virtual functions but a non-virtual destructor
class Base {
public:
    virtual void doWork() = 0;
    ~Base() {} // Non-virtual destructor!
};

class Derived : public Base {
public:
    void doWork() override { /* ... */ }
    ~Derived() { delete[] data_; } // This won't be called via Base*!
private:
    int* data_ = new int[100];
};

void leak() {
    Base* pb = new Derived;
    delete pb; // Undefined behavior! Only ~Base() runs, ~Derived() is skipped
               // data_ is leaked
}
```

```
// GOOD: Virtual destructor in polymorphic base classes
class Base {
public:
    virtual void doWork() = 0;
    virtual ~Base() = default; // Virtual destructor
};
```

9.3.13 The -Werror Philosophy

```
// In your build system, treat warnings as errors:
// This prevents "warning debt" from accumulating.

// CMake:
// target_compile_options(myproject PRIVATE
//     $<$<CXX_COMPILER_ID:GNU,Clang>:-Werror>
//     $<$<CXX_COMPILER_ID:MSVC>:/WX>
// )

// EXCEPTION: Third-party headers may produce warnings you can't fix.
// Use system includes to suppress them:
// target_include_directories(myproject SYSTEM PRIVATE ${THIRD_PARTY_INCLUDE_DIR})

// Or selectively suppress specific warnings:
#pragma GCC diagnostic push
#pragma GCC diagnostic ignored "-Wunused-parameter"
#include <third_party_header.h>
#pragma GCC diagnostic pop
```

9.3.14 Modern C++ Attributes That Help

```
// C++11: [[noreturn]] – function never returns
[[noreturn]] void fatalError(const std::string& msg) {
    std::cerr << "FATAL: " << msg << "\n";
    std::abort();
}

// C++14: [[deprecated]] – mark APIs for removal
[[deprecated("Use newFunction() instead")]]
void oldFunction() { /* ... */ }

// C++17: [[nodiscard]] – warn if return value is ignored
[[nodiscard]] std::error_code saveFile(const std::string& path) {
    // ...
    return {};
}

void usage() {
    saveFile("data.txt"); // WARNING: ignoring return value with 'nodiscard' attribute
    // GOOD:
    auto ec = saveFile("data.txt");
    if (ec) handleError(ec);

    // Or explicitly discard:
    (void)saveFile("data.txt"); // Intentionally ignoring
}

// C++20: [[likely]] / [[unlikely]] – branch prediction hints
int processInput(int value) {
    if (value > 0) [[likely]] {
        return value * 2;
    } else [[unlikely]] {
        throw std::invalid_argument("Negative value");
    }
}
```

9.3.15 Things to Remember

- Take compiler warnings seriously. Strive for warning-free code at the maximum warning level supported by your compilers.
- Don't become dependent on compiler warnings to catch errors. What one compiler warns about, another may not. Moving to a new compiler may eliminate warnings you've been relying on.
- Use `override` (C++11) to catch virtual function signature mismatches at compile time rather than relying on warnings.
- Treat warnings as errors (`-Werror` / `/WX`) in your build system to prevent warning debt from accumulating.
- Modern C++ attributes (`[[nodiscard]]`, `[[deprecated]]`, `[[maybe_unused]]`, `[[fallthrough]]`) replace many situations where you'd previously rely on warnings or suppress them.

9.4 Item 54: Familiarize Yourself with the Standard Library, Including TR1

9.5 Visual Summary

ITEM 54: FAMILIARIZE YOURSELF WITH THE STANDARD LIBRARY, INCLUDING TR1

1. Need common data structure/algorithm/utility -> check standard library first.
2. Containers, algorithms, iterators, function objects, smart pointers solve common patterns.
3. Library code is portable, tested, and idiomatic.
4. Custom code remains for domain-specific behavior.
5. Meaning: standard components reduce bugs and make intent recognizable.

9.5.1 Core Concept

When Meyers wrote this item, TR1 (Technical Report 1) was a preview of features that would eventually be incorporated into C++11. Today, everything from TR1 and much more is part of the standard library. This item is a comprehensive tour of the standard library as it exists through C++20, noting which components originated in TR1 and which were added later.

9.5.2 Historical Context: TR1 to C++20

TR1 (2005) previewed these components under `std::tr1::`. All of them are now in `std::`:

TR1 Component	Modern Standard	Header
<code>shared_ptr</code> , <code>weak_ptr</code>	C++11	<code><memory></code>
<code>function</code>	C++11	<code><functional></code>
<code>bind</code>	C++11 (largely replaced by lambdas)	<code><functional></code>
<code>tuple</code>	C++11	<code><tuple></code>
<code>array</code>	C++11	<code><array></code>
<code>unordered_set</code> , <code>unordered_map</code>	C++11	<code><unordered_set></code> , <code><unordered_map></code>
<code>regex</code>	C++11	<code><regex></code>
<code>type_traits</code>	C++11 (greatly expanded)	<code><type_traits></code>
<code>random</code>	C++11	<code><random></code>
<code>reference_wrapper</code> , <code>ref</code> , <code>cref</code>	C++11	<code><functional></code>
<code>result_of</code>	C++11 (deprecated C++17, removed C++20)	<code><type_traits></code>

9.5.3 Smart Pointers

TR1 introduced `shared_ptr` and `weak_ptr`. C++11 added `unique_ptr` (replacing the deprecated `auto_ptr`). C++14 added `make_unique`.

```
#include <memory>

// BAD: Raw owning pointers
class ResourceManager {
public:
    ResourceManager() {
        db_ = new DatabaseConnection("localhost:5432");
        cache_ = new Cache(1024);
    }

    ~ResourceManager() {
        delete cache_;
        delete db_; // If Cache destructor throws, db_ leaks
    }

private:
    DatabaseConnection* db_;
    Cache* cache_;
};

// GOOD: Smart pointers (C++14)
class ResourceManager {
public:
    ResourceManager()
        : db_(std::make_unique<DatabaseConnection>("localhost:5432"))
        , cache_(std::make_unique<Cache>(1024))
    {
        // If Cache constructor throws, db_ is automatically cleaned up
    }

    // No destructor needed – smart pointers handle cleanup
    // Implicitly deleted copy constructor/assignment (unique_ptr is move-only)

private:
    std::unique_ptr<DatabaseConnection> db_;
    std::unique_ptr<Cache> cache_;
};
```

```
// shared_ptr: Shared ownership with reference counting
class TextureCache {
public:
    std::shared_ptr<Texture> getTexture(const std::string& path) {
        auto it = cache_.find(path);
        if (it != cache_.end()) {
            // Try to lock the weak_ptr – texture may have been evicted
            if (auto sp = it->second.lock()) {
                return sp; // Cache hit
            }
        }
        // Cache miss – load and cache
        auto texture = std::make_shared<Texture>(path);
        cache_[path] = texture; // Store as weak_ptr
        return texture;
    }

private:
```

```

    // weak_ptr avoids preventing texture destruction when no one is using it
    std::unordered_map<std::string, std::weak_ptr<Texture>> cache_;
};

// unique_ptr with custom deleter
auto fileDeleter = [](FILE* f) { if (f) std::fclose(f); };
std::unique_ptr<FILE, decltype(fileDeleter)> file(std::fopen("data.txt", "r"), fileDeleter);

```

9.5.4 Containers

The standard library provides a rich set of containers. Here is the complete landscape:

```

#include <vector>
#include <deque>
#include <list>
#include <forward_list>    // C++11
#include <array>           // C++11 (from TR1)
#include <set>
#include <map>
#include <unordered_set>   // C++11 (from TR1)
#include <unordered_map>   // C++11 (from TR1)
#include <stack>
#include <queue>
#include <span>           // C++20

// --- Sequence Containers ---

// std::array – fixed-size array with STL interface (TR1 -> C++11)
std::array<int, 5> scores = {95, 87, 92, 78, 88};
// Advantages over C arrays: knows its size, can be copied, works with algorithms
auto it = std::find(scores.begin(), scores.end(), 92);
std::sort(scores.begin(), scores.end());

// std::vector – dynamic array, the default container
std::vector<std::string> names;
names.reserve(1000); // Pre-allocate to avoid reallocations
names.emplace_back("Alice"); // C++11: construct in-place

// std::deque – double-ended queue
std::deque<int> dq;
dq.push_front(1); // O(1)
dq.push_back(2);  // O(1)

// std::list – doubly-linked list
std::list<int> lst = {3, 1, 4, 1, 5};
lst.sort();           // Member function sort (can't use std::sort – no random access)
lst.unique();         // Remove consecutive duplicates
lst.merge(otherList); // Merge two sorted lists

// std::forward_list – singly-linked list (C++11)
std::forward_list<int> fl = {1, 2, 3};
fl.push_front(0);
// Minimal overhead – no size() member, insert_after instead of insert

// --- Associative Containers ---

// std::set / std::multiset – sorted unique/non-unique keys (red-black tree)
std::set<std::string> uniqueWords;
uniqueWords.insert("hello");
uniqueWords.insert("world");

```

```

uniqueWords.insert("hello"); // Ignored – already present
// uniqueWords.size() == 2

// std::map / std::multimap – sorted key-value pairs (red-black tree)
std::map<std::string, int> wordCount;
wordCount["hello"]++;
wordCount["world"]++;
wordCount["hello"]++;
// wordCount["hello"] == 2

// C++17: Structured bindings for map iteration
for (const auto& [word, count] : wordCount) {
    std::cout << word << ": " << count << "\n";
}

// C++17: try_emplace and insert_or_assign
wordCount.try_emplace("new_word", 1); // Only inserts if key doesn't exist
wordCount.insert_or_assign("hello", 100); // Inserts or overwrites

// C++17: extract and merge nodes between maps
auto node = wordCount.extract("hello");
node.key() = "greeting"; // Change the key without reallocating the value!
wordCount.insert(std::move(node));

// --- Unordered (Hash) Containers (TR1 -> C++11) ---

// std::unordered_set / std::unordered_multiset – hash-based unique/non-unique keys
std::unordered_set<std::string> fastLookup;
fastLookup.insert("alpha");
fastLookup.count("alpha"); // O(1) average
fastLookup.contains("alpha"); // C++20 – cleaner than count()

// std::unordered_map / std::unordered_multimap – hash-based key-value pairs
std::unordered_map<std::string, std::vector<int>> index;
index["chapter1"] = {1, 5, 12, 47};

// Custom hash for user-defined types:
struct Point { int x, y; };

struct PointHash {
    std::size_t operator()(const Point& p) const noexcept {
        auto h1 = std::hash<int>{}(p.x);
        auto h2 = std::hash<int>{}(p.y);
        return h1 ^ (h2 << 1);
    }
};

struct PointEqual {
    bool operator()(const Point& a, const Point& b) const noexcept {
        return a.x == b.x && a.y == b.y;
    }
};

std::unordered_set<Point, PointHash, PointEqual> pointSet;

// --- Container Adaptors ---

// std::stack – LIFO (default: backed by deque)
std::stack<int> callStack;
callStack.push(42);

```



```

int top = callStack.top();
callStack.pop();

// std::queue – FIFO (default: backed by deque)
std::queue<std::string> messageQueue;
messageQueue.push("first");
std::string msg = messageQueue.front();
messageQueue.pop();

// std::priority_queue – max-heap (default: backed by vector)
std::priority_queue<int> maxHeap;
maxHeap.push(3);
maxHeap.push(1);
maxHeap.push(4);
int highest = maxHeap.top(); // 4

// Min-heap:
std::priority_queue<int, std::vector<int>, std::greater<int>> minHeap;

// --- C++20: std::span ---
void processData(std::span<const int> data) {
    // Non-owning view of contiguous data – works with vector, array, C arrays
    for (int val : data) {
        std::cout << val << " ";
    }
}

std::vector<int> v = {1, 2, 3, 4, 5};
int arr[] = {10, 20, 30};
processData(v); // Works
processData(arr); // Also works
processData(std::span(v).subspan(1, 3)); // View of elements [1..3]

```

9.5.5 Algorithms

The `<algorithm>` header is one of the most powerful parts of the standard library:

```

#include <algorithm>
#include <numeric> // accumulate, iota, etc.
#include <execution> // C++17 parallel algorithms

std::vector<int> data = {5, 3, 8, 1, 9, 2, 7, 4, 6};

// --- Sorting and Ordering ---
std::sort(data.begin(), data.end()); // Ascending
std::sort(data.begin(), data.end(), std::greater<int>{}); // Descending
std::stable_sort(data.begin(), data.end()); // Preserves relative order of elements
std::partial_sort(data.begin(), data.begin() + 3, data.end()); // Top 3 elements sorted
std::nth_element(data.begin(), data.begin() + 4, data.end()); // Median (5th element in sorted order)

// --- Searching ---
auto it = std::find(data.begin(), data.end(), 7);
auto it2 = std::find_if(data.begin(), data.end(), [](int x) { return x > 5; });
bool found = std::binary_search(data.begin(), data.end(), 7); // Requires sorted range
auto [lo, hi] = std::equal_range(data.begin(), data.end(), 5); // Range of equal elements

// --- Modifying ---
std::transform(data.begin(), data.end(), data.begin(),
    [](int x) { return x * 2; }); // Double each element

```

```

std::replace_if(data.begin(), data.end(),
                [](int x) { return x < 0; }, 0); // Replace negatives with 0

data.erase(
    std::remove_if(data.begin(), data.end(), [](int x) { return x % 2 == 0; }),
    data.end()
); // Erase-remove idiom: delete all even numbers

// C++20: std::erase_if simplifies this
std::erase_if(data, [](int x) { return x % 2 == 0; });

// --- Aggregation ---
int sum = std::accumulate(data.begin(), data.end(), 0);
int product = std::accumulate(data.begin(), data.end(), 1, std::multiplies<int>{});
auto [minIt, maxIt] = std::minmax_element(data.begin(), data.end());

// --- Set Operations (on sorted ranges) ---
std::vector<int> a = {1, 2, 3, 4, 5};
std::vector<int> b = {3, 4, 5, 6, 7};
std::vector<int> result;

std::set_intersection(a.begin(), a.end(), b.begin(), b.end(),
                     std::back_inserter(result)); // {3, 4, 5}

result.clear();
std::set_union(a.begin(), a.end(), b.begin(), b.end(),
              std::back_inserter(result)); // {1, 2, 3, 4, 5, 6, 7}

result.clear();
std::set_difference(a.begin(), a.end(), b.begin(), b.end(),
                   std::back_inserter(result)); // {1, 2}

// --- Permutations ---
std::vector<int> perm = {1, 2, 3};
do {
    // Process each permutation: {1,2,3}, {1,3,2}, {2,1,3}, ...
} while (std::next_permutation(perm.begin(), perm.end()));

// --- C++17 Parallel Algorithms ---
std::vector<double> bigData(10000000);
std::sort(std::execution::par, bigData.begin(), bigData.end()); // Parallel sort
auto total = std::reduce(std::execution::par, bigData.begin(), bigData.end());
std::for_each(std::execution::par_unseq, bigData.begin(), bigData.end(),
              [](double& x) { x = std::sin(x); });

// --- C++20 Ranges ---
#include <ranges>

// Composable, lazy algorithm pipelines
auto evens = data | std::views::filter([](int x) { return x % 2 == 0; });
auto squares = data | std::views::transform([](int x) { return x * x; });
auto firstThreeSquaredEvens = data
    | std::views::filter([](int x) { return x % 2 == 0; })
    | std::views::transform([](int x) { return x * x; })
    | std::views::take(3);

for (int val : firstThreeSquaredEvens) {
    std::cout << val << " ";
}

```

```
// Ranges-based algorithms (take ranges directly, not iterator pairs)
std::ranges::sort(data);
auto found_it = std::ranges::find(data, 42);
bool all_positive = std::ranges::all_of(data, [](int x) { return x > 0; });
```

9.5.6 Iterators

```
#include <iterator>

// Iterator categories (from weakest to strongest):
// InputIterator      -> read-once, forward-only (e.g., istream_iterator)
// OutputIterator     -> write-once, forward-only (e.g., ostream_iterator)
// ForwardIterator    -> read/write, multi-pass forward (e.g., forward_list::iterator)
// BidirectionalIterator -> + backward movement (e.g., list::iterator, set::iterator)
// RandomAccessIterator -> + O(1) jumps (e.g., vector::iterator, deque::iterator)
// ContiguousIterator  -> + contiguous memory (C++20) (e.g., vector::iterator, array::iterator)

// Stream iterators – treat I/O as ranges
std::vector<int> readFromInput() {
    std::vector<int> result;
    std::copy(std::istream_iterator<int>(std::cin),
              std::istream_iterator<int>(),
              std::back_inserter(result));
    return result;
}

void writeToOutput(const std::vector<int>& data) {
    std::copy(data.begin(), data.end(),
              std::ostream_iterator<int>(std::cout, ", "));
}

// Insert iterators
std::vector<int> src = {1, 2, 3};
std::vector<int> dst;
std::copy(src.begin(), src.end(), std::back_inserter(dst)); // push_back
std::copy(src.begin(), src.end(), std::front_inserter(dst)); // push_front (deque, list)
std::copy(src.begin(), src.end(), std::inserter(dst, dst.begin())); // insert at position

// Reverse iterators
std::vector<int> v = {1, 2, 3, 4, 5};
for (auto rit = v.rbegin(); rit != v.rend(); ++rit) {
    std::cout << *rit << " "; // 5 4 3 2 1
}

// Move iterators (C++11) – convert copy operations to moves
std::vector<std::string> source = {"hello", "world"};
std::vector<std::string> target;
std::move(source.begin(), source.end(), std::back_inserter(target));
// source strings are now in moved-from state
```

9.5.7 Function Objects and Callables

```

#include <functional>

// --- std::function (TR1 -> C++11) ---
// Type-erased callable wrapper – can hold functions, lambdas, functors, member pointers

std::function<int(int, int)> operation;

operation = [](int a, int b) { return a + b; };
std::cout << operation(3, 4) << "\n"; // 7

operation = std::multiplies<int>{};
std::cout << operation(3, 4) << "\n"; // 12

int divide(int a, int b) { return a / b; }
operation = divide;
std::cout << operation(12, 4) << "\n"; // 3

// BAD: Using std::function when a template parameter would suffice
// std::function has overhead: heap allocation, virtual dispatch
void sortWithComparator(std::vector<int>& v, std::function<bool(int, int)> comp) {
    std::sort(v.begin(), v.end(), comp);
}

// GOOD: Use template parameter for zero-overhead
template<typename Comparator>
void sortWithComparator(std::vector<int>& v, Comparator comp) {
    std::sort(v.begin(), v.end(), comp);
}

// GOOD: Use std::function when you NEED type erasure (e.g., storing in a container)
std::map<std::string, std::function<double(double, double)>> operations = {
    {"add", [](double a, double b) { return a + b; }},
    {"sub", [](double a, double b) { return a - b; }},
    {"mul", [](double a, double b) { return a * b; }},
    {"div", [](double a, double b) { return a / b; }},
};

double result = operations["add"](3.0, 4.0); // 7.0

// --- std::bind (TR1 -> C++11) – largely superseded by lambdas ---

// BAD: std::bind is harder to read
auto bound = std::bind(divide, std::placeholders::_1, 2);
bound(10); // divide(10, 2) = 5

// GOOD: Lambda is clearer
auto divideByTwo = [](int x) { return divide(x, 2); };
divideByTwo(10); // 5

// --- std::reference_wrapper (TR1 -> C++11) ---
// Allows references to be stored in containers and used with std::bind/function

void increment(int& x) { ++x; }

int value = 10;
auto ref = std::ref(value); // Creates reference_wrapper<int>

// Store references in a vector (vector<int&> is illegal, but vector<reference_wrapper<int>> works)
std::vector<std::reference_wrapper<int>> refs;

```

```

int a = 1, b = 2, c = 3;
refs.push_back(std::ref(a));
refs.push_back(std::ref(b));
refs.push_back(std::ref(c));
for (auto& r : refs) {
    r.get() *= 10; // Modifies original variables
}
// a == 10, b == 20, c == 30

// --- std::invoke (C++17) – unified call syntax ---
struct Foo {
    int value = 42;
    int getValue() const { return value; }
};

Foo foo;
int v1 = std::invoke(&Foo::getValue, foo); // Member function call
int v2 = std::invoke(&Foo::value, foo);    // Member access
int v3 = std::invoke([](int x) { return x + 1; }, 41); // Lambda

```

9.5.8 Tuples

```

#include <tuple>

// std::tuple (TR1 -> C++11)
std::tuple<std::string, int, double> person("Alice", 30, 5.6);

// Access
std::string name = std::get<0>(person);
int age = std::get<1>(person);

// C++14: std::get by type (if types are unique)
std::string name2 = std::get<std::string>(person);

// C++17: Structured bindings – the modern way
auto [name3, age2, height] = person;

// C++17: std::apply – call a function with tuple elements as arguments
int add(int a, int b, int c) { return a + b + c; }
auto args = std::make_tuple(1, 2, 3);
int sum = std::apply(add, args); // add(1, 2, 3) = 6

// Returning multiple values
std::tuple<bool, std::string, int> parseConfig(const std::string& line) {
    // ... parsing logic ...
    return {true, "key", 42};
}

auto [success, key, value] = parseConfig("key=42");

// std::tie for comparison operators (pre-C++20)
struct Employee {
    std::string name;
    int department;
    double salary;

    bool operator<(const Employee& rhs) const {
        return std::tie(department, name, salary)
            < std::tie(rhs.department, rhs.name, rhs.salary);
    }
};

```

```

    }
};

// C++20: Use the spaceship operator instead
struct EmployeeModern {
    std::string name;
    int department;
    double salary;

    auto operator<=>(const EmployeeModern&) const = default;
};

```

9.5.9 Type Traits

```

#include <type_traits>

// Type traits (TR1 -> C++11, massively expanded through C++20)
// Compile-time introspection and transformation of types

// --- Primary type categories ---
static_assert(std::is_integral_v<int>);           // true
static_assert(std::is_floating_point_v<double>); // true
static_assert(std::is_pointer_v<int*>);          // true
static_assert(std::is_reference_v<int&>);        // true
static_assert(std::is_array_v<int[10]>);         // true
static_assert(std::is_class_v<std::string>);     // true
static_assert(std::is_void_v<void>);             // true

// --- Composite type categories ---
static_assert(std::is_arithmetic_v<int>);        // true
static_assert(std::is_fundamental_v<double>);    // true
static_assert(std::is_scalar_v<int*>);           // true
static_assert(std::is_object_v<std::string>);    // true

// --- Type properties ---
static_assert(std::is_const_v<const int>);       // true
static_assert(std::is_volatile_v<volatile int>); // true
static_assert(std::is_trivially_copyable_v<int>); // true
static_assert(std::is_standard_layout_v<int>);   // true
static_assert(std::is_trivial_v<int>);           // true

// --- Type relationships ---
static_assert(std::is_same_v<int, int>);         // true
static_assert(std::is_convertible_v<int, double>); // true

// --- Type transformations ---
using NoConst = std::remove_const_t<const int>; // int
using NoRef = std::remove_reference_t<int&>;    // int
using Decayed = std::decay_t<const int&>;       // int
using Ptr = std::add_pointer_t<int>;            // int*
using Common = std::common_type_t<int, double>; // double

// --- Practical usage: SFINAE and if constexpr ---

// BAD: No type checking – compiles but may produce garbage for non-arithmetic types
template<typename T>
T average_bad(const std::vector<T>& v) {
    T sum = std::accumulate(v.begin(), v.end(), T{});
    return sum / v.size();
}

```

```

}

// GOOD: Constrained with type traits (C++17 if constexpr)
template<typename T>
auto average(const std::vector<T>& v) {
    static_assert(std::is_arithmetic_v<T>, "average() requires arithmetic types");
    if constexpr (std::is_integral_v<T>) {
        // Integer types: return double to preserve precision
        double sum = std::accumulate(v.begin(), v.end(), 0.0);
        return sum / v.size();
    } else {
        T sum = std::accumulate(v.begin(), v.end(), T{});
        return sum / static_cast<T>(v.size());
    }
}

// BEST: C++20 Concepts
template<typename T>
concept Arithmetic = std::is_arithmetic_v<T>;

template<Arithmetic T>
auto average_best(const std::vector<T>& v) {
    double sum = std::accumulate(v.begin(), v.end(), 0.0);
    return sum / v.size();
}

```

9.5.10 Regular Expressions

```

#include <regex>

// std::regex (TR1 -> C++11)
// NOTE: std::regex has historically had poor performance in some implementations.
// Consider third-party alternatives (RE2, CTRE) for performance-critical code.

// Basic matching
std::string email = "user@example.com";
std::regex emailPattern(R"(\w+@\w+\.\w+)");
bool isValid = std::regex_match(email, emailPattern);

// Search (find first match)
std::string text = "Call 555-1234 or 555-5678";
std::regex phonePattern(R"((\d{3})-(\d{4}))");
std::smatch match;
if (std::regex_search(text, match, phonePattern)) {
    std::cout << "Full match: " << match[0] << "\n"; // 555-1234
    std::cout << "Area code: " << match[1] << "\n"; // 555
    std::cout << "Number: " << match[2] << "\n"; // 1234
}

// Iterate all matches
auto matchBegin = std::sregex_iterator(text.begin(), text.end(), phonePattern);
auto matchEnd = std::sregex_iterator();
for (auto it = matchBegin; it != matchEnd; ++it) {
    std::cout << "Found: " << (*it)[0] << "\n";
}

// Output: Found: 555-1234
//         Found: 555-5678

// Replace

```

```

std::string result = std::regex_replace(text, phonePattern, "XXX-XXXX");
// "Call XXX-XXXX or XXX-XXXX"

// Splitting strings using regex
std::string csv = "field1,field2,,field4";
std::regex comma(",");
std::vector<std::string> fields(
    std::sregex_token_iterator(csv.begin(), csv.end(), comma, -1),
    std::sregex_token_iterator()
);
// fields = {"field1", "field2", "", "field4"}

// Practical example: log parser
struct LogEntry {
    std::string timestamp;
    std::string level;
    std::string message;
};

std::optional<LogEntry> parseLogLine(const std::string& line) {
    static const std::regex logPattern(
        R"^(^\\(\\d{4}-\\d{2}-\\d{2} \\d{2}:\\d{2}:\\d{2})\\) \\[(\\w+)\\] (\\.+)$)"
    );

    std::smatch m;
    if (std::regex_match(line, m, logPattern)) {
        return LogEntry{m[1], m[2], m[3]};
    }
    return std::nullopt;
}

```

9.5.11 Random Number Generation

```

#include <random>

// BAD: Using rand() and srand() – biased, predictable, not thread-safe
int badRandom(int min, int max) {
    return min + rand() % (max - min + 1); // Non-uniform distribution!
}

// GOOD: Modern <random> library (TR1 -> C++11)
class RandomEngine {
public:
    RandomEngine() : gen_(std::random_device{}()) {}

    // Uniform integer distribution
    int uniformInt(int min, int max) {
        std::uniform_int_distribution<int> dist(min, max);
        return dist(gen_);
    }

    // Uniform real distribution
    double uniformReal(double min, double max) {
        std::uniform_real_distribution<double> dist(min, max);
        return dist(gen_);
    }

    // Normal (Gaussian) distribution
    double normal(double mean, double stddev) {

```



```

        std::normal_distribution<double> dist(mean, stddev);
        return dist(gen_);
    }

    // Bernoulli (coin flip with probability p)
    bool bernoulli(double probability) {
        std::bernoulli_distribution dist(probability);
        return dist(gen_);
    }

    // Poisson distribution (for modeling random events over time)
    int poisson(double meanRate) {
        std::poisson_distribution<int> dist(meanRate);
        return dist(gen_);
    }

    // Discrete distribution (weighted random selection)
    int weightedChoice(std::initializer_list<double> weights) {
        std::discrete_distribution<int> dist(weights);
        return dist(gen_);
    }

    // Shuffle a container
    template<typename Container>
    void shuffle(Container& c) {
        std::shuffle(c.begin(), c.end(), gen_);
    }

private:
    std::mt19937 gen_; // Mersenne Twister engine – good general-purpose PRNG
};

// Available engines:
// std::mt19937          – Mersenne Twister (32-bit, period 2^19937-1)
// std::mt19937_64       – Mersenne Twister (64-bit)
// std::minstd_rand       – Linear congruential (fast, lower quality)
// std::ranlux24          – Subtract-with-carry (luxury level 3)
// std::ranlux48          – Subtract-with-carry (luxury level 4)
// std::default_random_engine – Implementation-defined

// Available distributions:
// Uniform:    uniform_int_distribution, uniform_real_distribution
// Bernoulli:  bernoulli_distribution, binomial_distribution,
//             geometric_distribution, negative_binomial_distribution
// Poisson:    poisson_distribution, exponential_distribution,
//             gamma_distribution, weibull_distribution,
//             extreme_value_distribution
// Normal:     normal_distribution, lognormal_distribution,
//             chi_squared_distribution, cauchy_distribution,
//             fisher_f_distribution, student_t_distribution
// Sampling:   discrete_distribution, piecewise_constant_distribution,
//             piecewise_linear_distribution

// Usage example: Monte Carlo simulation
double estimatePi(int samples) {
    std::mt19937 gen(42); // Fixed seed for reproducibility
    std::uniform_real_distribution<double> dist(0.0, 1.0);

    int insideCircle = 0;
    for (int i = 0; i < samples; ++i) {

```

```

        double x = dist(gen);
        double y = dist(gen);
        if (x * x + y * y <= 1.0) {
            ++insideCircle;
        }
    }
    return 4.0 * insideCircle / samples;
}

```

9.5.12 String and String View

```

#include <string>
#include <string_view> // C++17
#include <charconv>    // C++17

// std::string_view (C++17) – non-owning view of character data
// Avoids copies when you only need to read a string

// BAD: Unnecessary copies
bool startsWithBad(const std::string& str, const std::string& prefix) {
    return str.substr(0, prefix.size()) == prefix; // substr creates a copy
}

// GOOD: string_view avoids copies
bool startsWith(std::string_view str, std::string_view prefix) {
    return str.substr(0, prefix.size()) == prefix; // substr returns a string_view – no copy
    // Or in C++20: return str.starts_with(prefix);
}

// Works with string literals, std::string, and char* – all zero-copy:
startsWith("hello world", "hello"); // No copies
std::string s = "hello world";
startsWith(s, "hello"); // No copies

// CAUTION: string_view does not own the data – beware of dangling views
// BAD:
std::string_view dangerous() {
    std::string temp = "hello";
    return temp; // DANGLING! temp is destroyed, view points to freed memory
}

// std::charconv (C++17) – fast, locale-independent number conversion
#include <charconv>

// BAD: stoi/stod are locale-dependent and throw exceptions
int valueSlow = std::stoi("42");

// GOOD: from_chars is fast, locale-independent, and non-allocating
int parseIntFast(std::string_view sv) {
    int result = 0;
    auto [ptr, ec] = std::from_chars(sv.data(), sv.data() + sv.size(), result);
    if (ec != std::errc{}) {
        throw std::runtime_error("Parse error");
    }
    return result;
}

```

9.5.13 Chrono (Time Library)

```

#include <chrono>

// C++11 chrono library – type-safe time handling

using namespace std::chrono;

// Clocks
auto now = system_clock::now();           // Wall-clock time
auto steadyNow = steady_clock::now();     // Monotonic (for measuring intervals)
auto highResNow = high_resolution_clock::now(); // Highest available resolution

// Durations – type-safe time intervals
auto tenSeconds = seconds(10);
auto halfSecond = milliseconds(500);
auto tinyInterval = microseconds(100);
auto dur = 2h + 30min + 15s; // C++14 duration literals

// Duration arithmetic
auto total = tenSeconds + halfSecond; // 10500ms

// Duration casting
auto ms = duration_cast<milliseconds>(tenSeconds); // 10000ms
auto secs = duration_cast<seconds>(milliseconds(2500)); // 2s (truncated)

// Benchmarking
auto start = steady_clock::now();
// ... code to benchmark ...
auto end = steady_clock::now();
auto elapsed = duration_cast<microseconds>(end - start);
std::cout << "Elapsed: " << elapsed.count() << " microseconds\n";

// C++20: Calendar and time zones
// Calendar types (C++20)
auto today = year_month_day{floor<days>(system_clock::now())};
auto christmas = 2024y / December / 25;
auto lastDayOfFeb = 2024y / February / last;

bool isLeapYear = year{2024}.is_leap();

// Time zones (C++20)
auto utcNow = system_clock::now();
auto localTime = zoned_time{current_zone(), utcNow};
auto tokyoTime = zoned_time{"Asia/Tokyo", utcNow};

```

9.5.14 Filesystem

```

#include <filesystem> // C++17 (originated from Boost.Filesystem)
namespace fs = std::filesystem;

// Path manipulation
fs::path configPath = fs::current_path() / "config" / "settings.json";
std::cout << "Stem: " << configPath.stem() << "\n"; // settings
std::cout << "Extension: " << configPath.extension() << "\n"; // .json
std::cout << "Parent: " << configPath.parent_path() << "\n"; // ../config

// File operations
if (fs::exists(configPath)) {
    auto size = fs::file_size(configPath);
    auto modified = fs::last_write_time(configPath);
    std::cout << "Size: " << size << " bytes\n";
}

// Directory iteration
for (const auto& entry : fs::directory_iterator("/tmp")) {
    if (entry.is_regular_file()) {
        std::cout << entry.path().filename() << " ("
            << entry.file_size() << " bytes)\n";
    }
}

// Recursive directory iteration
for (const auto& entry : fs::recursive_directory_iterator("src")) {
    if (entry.path().extension() == ".cpp") {
        std::cout << entry.path() << "\n";
    }
}

// Create directories
fs::create_directories("output/logs/2024"); // Creates all intermediate directories

// Copy, rename, delete
fs::copy("src/main.cpp", "backup/main.cpp.bak",
    fs::copy_options::overwrite_existing);
fs::rename("old_name.txt", "new_name.txt");
fs::remove("temp_file.txt");
fs::remove_all("temp_directory"); // Recursive delete

// Permissions
fs::permissions("script.sh",
    fs::perms::owner_exec | fs::perms::group_exec,
    fs::perm_options::add);

// Space information
auto space = fs::space("/");
std::cout << "Capacity: " << space.capacity / (1024*1024*1024) << " GB\n";
std::cout << "Available: " << space.available / (1024*1024*1024) << " GB\n";

```

9.5.15 Optional, Variant, Any (C++17)

```

#include <optional>    // C++17
#include <variant>      // C++17
#include <any>          // C++17

// --- std::optional: a value that may or may not be present ---

// BAD: Using special sentinel values
int findIndexBad(const std::vector<int>& v, int target) {
    for (int i = 0; i < v.size(); ++i) {
        if (v[i] == target) return i;
    }
    return -1; // Magic number meaning "not found"
}

// GOOD: std::optional makes absence explicit
std::optional<int> findIndex(const std::vector<int>& v, int target) {
    for (int i = 0; i < v.size(); ++i) {
        if (v[i] == target) return i;
    }
    return std::nullopt;
}

auto idx = findIndex(data, 42);
if (idx) {
    std::cout << "Found at index " << *idx << "\n";
} else {
    std::cout << "Not found\n";
}

// value_or provides a default
int index = findIndex(data, 42).value_or(-1);

// --- std::variant: type-safe union ---

// BAD: C-style union with type tag
struct ShapeOld {
    enum Type { CircleT, RectangleT, TriangleT };
    Type type;
    union {
        struct { double radius; } circle;
        struct { double width, height; } rect;
        struct { double base, height; } tri;
    };
};

// GOOD: std::variant
using Shape = std::variant<Circle, Rectangle, Triangle>;

// Pattern matching with std::visit
double area(const Shape& shape) {
    return std::visit([](const auto& s) -> double {
        using T = std::decay_t<decltype(s)>;
        if constexpr (std::is_same_v<T, Circle>) {
            return 3.14159 * s.radius * s.radius;
        } else if constexpr (std::is_same_v<T, Rectangle>) {
            return s.width * s.height;
        } else if constexpr (std::is_same_v<T, Triangle>) {
            return 0.5 * s.base * s.height;
        }
    });
}

```

```

    }, shape);
}

// Overloaded visitor pattern (common C++17 idiom)
template<class... Ts> struct overloaded : Ts... { using Ts::operator()...; };
template<class... Ts> overloaded(Ts...) -> overloaded<Ts...>; // C++17 CTAD

double areaVisitor(const Shape& shape) {
    return std::visit(overloaded{
        [](const Circle& c)    { return 3.14159 * c.radius * c.radius; },
        [](const Rectangle& r) { return r.width * r.height; },
        [](const Triangle& t)  { return 0.5 * t.base * t.height; },
    }, shape);
}

// --- std::any: type-safe void* ---
std::any value = 42;
value = std::string("hello");
value = 3.14;

try {
    double d = std::any_cast<double>(value); // OK
    int i = std::any_cast<int>(value);        // Throws std::bad_any_cast
} catch (const std::bad_any_cast& e) {
    std::cerr << "Wrong type: " << e.what() << "\n";
}

// Check type before casting
if (value.type() == typeid(double)) {
    double d = std::any_cast<double>(value);
}

```

9.5.16 Concurrency (C++11 and beyond)

```

#include <thread>
#include <mutex>
#include <future>
#include <atomic>
#include <shared_mutex> // C++17
#include <latch>         // C++20
#include <barrier>       // C++20
#include <semaphore>     // C++20

// --- Threads ---
void worker(int id) {
    std::cout << "Thread " << id << " running\n";
}

std::thread t1(worker, 1);
std::thread t2(worker, 2);
t1.join();
t2.join();

// C++20: jthread – automatically joins on destruction
{
    std::jthread jt(worker, 3);
    // No need to call join() – jt automatically joins when it goes out of scope
}

```

```

// --- Mutexes ---
std::mutex mtx;
int sharedCounter = 0;

void incrementSafely() {
    std::lock_guard<std::mutex> lock(mtx); // RAII lock
    ++sharedCounter;
}

// C++17: std::scoped_lock - locks multiple mutexes without deadlock
std::mutex m1, m2;
void transferFunds() {
    std::scoped_lock lock(m1, m2); // Locks both, avoids deadlock
    // ... transfer ...
}

// C++17: shared_mutex - multiple readers, single writer
std::shared_mutex rwMutex;
std::map<std::string, int> cache;

int readCache(const std::string& key) {
    std::shared_lock lock(rwMutex); // Shared (read) lock - multiple readers OK
    return cache[key];
}

void writeCache(const std::string& key, int value) {
    std::unique_lock lock(rwMutex); // Exclusive (write) lock
    cache[key] = value;
}

// --- Futures and Promises ---
std::future<int> result = std::async(std::launch::async, []() {
    return 42; // expensiveComputation()
});
int value = result.get(); // Blocks until ready

// --- Atomics ---
std::atomic<int> counter{0};

void incrementAtomic() {
    counter.fetch_add(1, std::memory_order_relaxed);
}

// --- C++20 Synchronization Primitives ---

// std::latch - one-shot barrier
std::latch startLatch(1);
std::latch doneLatch(4);

void workerWithLatch(int id) {
    startLatch.wait(); // Wait for the start signal
    // ... do work ...
    doneLatch.count_down(); // Signal completion
}

// std::barrier - reusable synchronization point
std::barrier syncPoint(4, []() noexcept {
    // Completion function runs when all threads arrive
    std::cout << "All threads synchronized\n";
});

```

```

void iterativeWorker() {
    for (int i = 0; i < 10; ++i) {
        // ... do iteration work ...
        syncPoint.arrive_and_wait(); // Sync between iterations
    }
}

// std::counting_semaphore
std::counting_semaphore<4> connectionPool(4); // Max 4 concurrent connections

void useConnection() {
    connectionPool.acquire(); // Wait for a slot
    // ... use connection ...
    connectionPool.release(); // Return the slot
}

```

9.5.17 Concepts (C++20)

```

#include <concepts>

// Concepts replace SFINAE and enable_if with clear, readable constraints

// BAD: SFINAE (pre-C++20)
template<typename T, typename = std::enable_if_t<std::is_arithmetic_v<T>>>
T addSFINAE(T a, T b) { return a + b; }

// GOOD: Concepts (C++20)
template<std::integral T>
T add(T a, T b) { return a + b; }

// Custom concepts
template<typename T>
concept Printable = requires(T t) {
    { std::cout << t } -> std::same_as<std::ostream&>;
};

template<typename T>
concept Hashable = requires(T t) {
    { std::hash<T>{}(t) } -> std::convertible_to<std::size_t>;
};

template<typename T>
concept Container = requires(T c) {
    typename T::value_type;
    typename T::iterator;
    { c.begin() } -> std::same_as<typename T::iterator>;
    { c.end() } -> std::same_as<typename T::iterator>;
    { c.size() } -> std::convertible_to<std::size_t>;
};

// Using concepts
template<Container C>
void printAll(const C& container) {
    for (const auto& item : container) {
        std::cout << item << " ";
    }
    std::cout << "\n";
}

```



```
// Standard library concepts:
// std::same_as, std::derived_from, std::convertible_to
// std::integral, std::floating_point, std::signed_integral, std::unsigned_integral
// std::equality_comparable, std::totally_ordered
// std::movable, std::copyable, std::semiregular, std::regular
// std::invocable, std::predicate
// std::ranges::range, std::ranges::input_range, std::ranges::forward_range, etc.
```

9.5.18 Things to Remember

- The C++ standard library is vast and covers containers, algorithms, iterators, function objects, smart pointers, regular expressions, random numbers, time, filesystem, concurrency, and more. Knowing what's available prevents you from reinventing it.
- Everything from TR1 has been incorporated into the C++ standard (C++11 and later). If you see legacy code using `std::tr1::`, update it to `std::`.
- Modern C++ (C++11 through C++20) has added transformative features: move semantics, lambda expressions, `constexpr`, `auto`, structured bindings, `optional/variant/any`, ranges, concepts, coroutines, and modules. Each release substantially improves what you can express.
- Know the performance characteristics of your containers: `vector` for contiguous storage and cache locality, `unordered_map/unordered_set` for $O(1)$ average lookup, `map/set` for ordered access, `deque` for front/back operations.
- Prefer standard library facilities over hand-written alternatives. They are tested, optimized, portable, and familiar to other C++ programmers.

9.6 Item 55: Familiarize Yourself with Boost

9.7 Visual Summary

ITEM 55: FAMILIARIZE YOURSELF WITH BOOST

1. Need advanced reusable C++ facility → Boost may already provide it.
2. Boost libraries often preview future standard library ideas.
3. Use selectively: quality is high, but dependency and build cost matter.
4. Prefer standard library when equivalent feature exists.
5. Meaning: Boost expands your toolbox, but choose dependencies deliberately.

9.7.1 Core Concept

Boost is a collection of high-quality, peer-reviewed, portable C++ libraries. It serves as a proving ground for standard library additions — many C++ standard library features started as Boost libraries. Understanding Boost helps you leverage battle-tested code and understand the trajectory of the C++ standard.

9.7.2 Boost Libraries That Became Part of the C++ Standard

This is the most important thing to know about Boost: it is the primary feeder for the C++ standard library. Here is a comprehensive list:

Boost Library	Standard Version	Standard Header/Feature
Boost.SmartPtr	C++11	<memory>: shared_ptr, weak_ptr, unique_ptr
Boost.Unordered	C++11	<unordered_map>, <unordered_set>
Boost.Array	C++11	<array>
Boost.Tuple	C++11	<tuple>
Boost.Function	C++11	<functional>: std::function
Boost.Bind	C++11	<functional>: std::bind (superseded by lambdas)
Boost.Ref	C++11	<functional>: std::ref, std::cref
Boost.TypeTraits	C++11	<type_traits>
Boost.Regex	C++11	<regex>
Boost.Random	C++11	<random>
Boost.Thread	C++11	<thread>, <mutex>, <condition_variable>
Boost.Chrono	C++11	<chrono>
Boost.Ratio	C++11	<ratio>
Boost.System	C++11	<system_error>
Boost.Atomic	C++11	<atomic>
Boost.Filesystem	C++17	<filesystem>
Boost.Optional	C++17	<optional>
Boost.Variant	C++17	<variant>
Boost.Any	C++17	<any>
Boost.StringView	C++17	<string_view>
Boost.Math.SpecialFunctions	C++17	<cmath> (special math functions)
Boost.Mp11 (concepts influence)	C++20	<concepts>
Boost.Endian	C++20	<bit>: std::endian
Boost.Span	C++20	
Boost.Outcome	C++23	<expected>
Boost.Stacktrace	C++23	<stacktrace>

9.7.3 Boost Libraries Still Unique (No Standard Equivalent)

Many Boost libraries remain essential because the standard does not provide equivalents:

```
// Boost.Asio is the de facto standard for asynchronous networking in C++
// A Networking TS was proposed but not yet standardized

#include <boost/asio.hpp>

namespace asio = boost::asio;
using tcp = asio::ip::tcp;

// Simple TCP echo server
class EchoServer {
public:
    EchoServer(asio::io_context& ioc, short port)
        : acceptor_(ioc, tcp::endpoint(tcp::v4(), port))
    {
        startAccept();
    }

private:
    void startAccept() {
        auto socket = std::make_shared<tcp::socket>(acceptor_.get_executor());
        acceptor_.async_accept(*socket, [this, socket](boost::system::error_code ec) {
```

```

        if (!ec) {
            handleClient(socket);
        }
        startAccept(); // Accept next connection
    });
}

void handleClient(std::shared_ptr<tcp::socket> socket) {
    auto buf = std::make_shared<std::array<char, 1024>>();
    socket->async_read_some(asio::buffer(*buf),
        [socket, buf](boost::system::error_code ec, std::size_t bytes) {
            if (!ec) {
                asio::async_write(*socket, asio::buffer(*buf, bytes),
                    [](boost::system::error_code, std::size_t) {});
            }
        });
}

tcp::acceptor acceptor_;
};

// Coroutine-based version (Boost.Asio + C++20 coroutines)
asio::awaitable<void> echo(tcp::socket socket) {
    std::array<char, 1024> buf;
    while (true) {
        auto n = co_await socket.async_read_some(
            asio::buffer(buf), asio::use_awaitable);
        co_await asio::async_write(
            socket, asio::buffer(buf, n), asio::use_awaitable);
    }
}

```

9.7.3.1 Boost.Asio (Networking and Asynchronous I/O)

```

// Boost.Spirit lets you write parsers using C++ expression templates
// that look like EBNF grammars

#include <boost/spirit/home/x3.hpp>

namespace x3 = boost::spirit::x3;

// Parse a CSV line
auto parseCSV(const std::string& input) {
    std::vector<std::string> result;
    auto field = x3::lexeme[*(x3::char_ - ',')];
    auto csvRule = field % ',';

    x3::parse(input.begin(), input.end(), csvRule, result);
    return result;
}

// Parse a simple arithmetic expression
// expr = term (('+' | '-') term)*
// term = factor (('*' | '/') factor)*
// factor = number | '(' expr ')'

auto const number = x3::double_;
x3::rule<class factor, double> const factor = "factor";
x3::rule<class term, double> const term = "term";

```

```

x3::rule<class expr, double> const expr = "expr";

auto const factor_def = number | ((' ' >> expr >> ' '));
auto const term_def = factor >> * (('*' >> factor) | ('/' >> factor));
auto const expr_def = term >> * (('+' >> term) | ('-' >> term));

BOOST_SPIRIT_DEFINE(expr, term, factor);

```

9.7.3.2 Boost.Spirit (Parser Framework)

```

// Boost.Graph Library (BGL) – comprehensive graph algorithms

#include <boost/graph/adjacency_list.hpp>
#include <boost/graph/dijkstra_shortest_paths.hpp>
#include <boost/graph/breadth_first_search.hpp>

using Graph = boost::adjacency_list<
    boost::vecS,           // OutEdge list type
    boost::vecS,           // Vertex list type
    boost::directedS,      // Directed graph
    boost::no_property,    // Vertex properties
    boost::property<boost::edge_weight_t, double> // Edge properties
>;

// Build a graph
Graph g(5); // 5 vertices
boost::add_edge(0, 1, 10.0, g);
boost::add_edge(0, 2, 5.0, g);
boost::add_edge(1, 3, 1.0, g);
boost::add_edge(2, 3, 9.0, g);
boost::add_edge(3, 4, 2.0, g);

// Dijkstra's shortest paths
std::vector<double> distances(5);
std::vector<Graph::vertex_descriptor> predecessors(5);
boost::dijkstra_shortest_paths(g, 0,
    boost::distance_map(distances.data())
    .predecessor_map(predecessors.data())
);

for (int i = 0; i < 5; ++i) {
    std::cout << "Distance from 0 to " << i << ": " << distances[i] << "\n";
}

```

9.7.3.3 Boost.Graph

```

// Boost.Serialization – non-intrusive, portable serialization

#include <boost/archive/text_oarchive.hpp>
#include <boost/archive/text_iarchive.hpp>
#include <boost/serialization/vector.hpp>
#include <boost/serialization/string.hpp>
#include <fstream>

class Player {
public:
    Player() = default;
    Player(std::string name, int level, std::vector<std::string> inventory)
        : name_(std::move(name)), level_(level), inventory_(std::move(inventory)) {}

    const std::string& name() const { return name_; }
    int level() const { return level_; }

private:
    friend class boost::serialization::access;

    template<class Archive>
    void serialize(Archive& ar, const unsigned int version) {
        ar & name_;
        ar & level_;
        ar & inventory_;
    }

    std::string name_;
    int level_ = 1;
    std::vector<std::string> inventory_;
};

// Save
void saveGame(const Player& player, const std::string& filename) {
    std::ofstream ofs(filename);
    boost::archive::text_oarchive oa(ofs);
    oa << player;
}

// Load
Player loadGame(const std::string& filename) {
    std::ifstream ifs(filename);
    boost::archive::text_iarchive ia(ifs);
    Player player;
    ia >> player;
    return player;
}

```

9.7.3.4 Boost.Serialization

```

// Arbitrary-precision arithmetic

#include <boost/multiprecision/cpp_int.hpp>
#include <boost/multiprecision/cpp_dec_float.hpp>

using boost::multiprecision::cpp_int;
using boost::multiprecision::cpp_dec_float_50;

// Arbitrary precision integers
cpp_int factorial(int n) {
    cpp_int result = 1;
    for (int i = 2; i <= n; ++i) {
        result *= i;
    }
    return result;
}

auto f100 = factorial(100);
// 9332621544394415268169923885626670049071596826438162146859296389521759999322991560894146397

// Arbitrary precision decimals (50 significant digits)
cpp_dec_float_50 pi = boost::multiprecision::acos(cpp_dec_float_50(-1));
// 3.1415926535897932384626433832795028841971693993751...

```

9.7.3.5 Boost.Multiprecision

```

// Boost.Test – unit testing framework

#define BOOST_TEST_MODULE MyTests
#include <boost/test/unit_test.hpp>

BOOST_AUTO_TEST_SUITE(MathTests)

BOOST_AUTO_TEST_CASE(addition) {
    BOOST_CHECK_EQUAL(2 + 2, 4);
    BOOST_CHECK_CLOSE(3.14, 3.14159, 0.01); // 0.01% tolerance
}

BOOST_AUTO_TEST_CASE(division) {
    BOOST_CHECK_THROW(divide(1, 0), std::domain_error);
    BOOST_CHECK_NO_THROW(divide(10, 2));
}

BOOST_AUTO_TEST_CASE(stringOperations) {
    std::string result = greet("World");
    BOOST_CHECK_EQUAL(result, "Hello, World!");
    BOOST_CHECK(result.find("World") != std::string::npos);
}

BOOST_AUTO_TEST_SUITE_END()

```

9.7.3.6 Boost.Test

```
// Boost.Log – flexible, high-performance logging

#include <boost/log/trivial.hpp>
#include <boost/log/utility/setup/file.hpp>
#include <boost/log/utility/setup/common_attributes.hpp>

void initLogging() {
    boost::log::add_file_log(
        boost::log::keywords::file_name = "app_%N.log",
        boost::log::keywords::rotation_size = 10 * 1024 * 1024, // 10 MB
        boost::log::keywords::format = "[%TimeStamp%] [%Severity%] %Message%"
    );
    boost::log::add_common_attributes();
}

void example() {
    BOOST_LOG_TRIVIAL(trace)    << "Trace message";
    BOOST_LOG_TRIVIAL(debug)    << "Debug message";
    BOOST_LOG_TRIVIAL(info)     << "Info message";
    BOOST_LOG_TRIVIAL(warning)  << "Warning message";
    BOOST_LOG_TRIVIAL(error)    << "Error message";
    BOOST_LOG_TRIVIAL(fatal)    << "Fatal message";
}
```

9.7.3.7 Boost.Log

```
// Boost.PropertyTree – configuration file parsing (INI, JSON, XML)
#include <boost/property_tree/ptree.hpp>
#include <boost/property_tree/json_parser.hpp>

boost::property_tree::ptree config;
boost::property_tree::read_json("config.json", config);
std::string host = config.get<std::string>("server.host");
int port = config.get<int>("server.port", 8080); // Default: 8080

// Boost.Signals2 – thread-safe signals and slots (observer pattern)
#include <boost/signals2.hpp>

boost::signals2::signal<void(const std::string&)> onMessage;

auto conn = onMessage.connect([](const std::string& msg) {
    std::cout << "Received: " << msg << "\n";
});

onMessage("Hello, Boost!"); // Fires all connected slots
conn.disconnect();

// Boost.Interprocess – shared memory and IPC
#include <boost/interprocess/shared_memory_object.hpp>
#include <boost/interprocess/mapped_region.hpp>

namespace bip = boost::interprocess;

bip::shared_memory_object shm(bip::create_only, "SharedMem", bip::read_write);
shm.truncate(1024);
bip::mapped_region region(shm, bip::read_write);
std::memset(region.get_address(), 0, region.get_size());
```

```

// Boost.Coroutine2 / Boost.Context – stackful coroutines
// (C++20 added stackless coroutines; Boost provides stackful alternatives)

// Boost.Geometry – computational geometry
#include <boost/geometry.hpp>
namespace bg = boost::geometry;

using Point = bg::model::point<double, 2, bg::cs::cartesian>;
using Polygon = bg::model::polygon<Point>;

Point p1(0, 0), p2(3, 4);
double dist = bg::distance(p1, p2); // 5.0

// Boost.Hana – metaprogramming library (successor to MPL and Fusion)
#include <boost/hana.hpp>
namespace hana = boost::hana;

auto tuple = hana::make_tuple(1, 'c', 3.14, "hello");
auto filtered = hana::filter(tuple, [](auto x) {
    return hana::trait<std::is_arithmetic>(x);
});
// filtered contains: (1, 3.14)

// Boost.Beast – HTTP and WebSocket (built on Boost.Asio)
// Boost.JSON – JSON parsing and serialization (standardization candidate)
// Boost.Circular_Buffer – fixed-capacity circular buffer
// Boost.Multi_Index – containers with multiple simultaneous orderings
// Boost.Pool – memory pool allocators
// Boost.Intrusive – intrusive containers (no dynamic allocation per element)
// Boost.Lockfree – lock-free data structures

```

9.7.3.8 Other Notable Boost Libraries

9.7.4 How to Use Boost

```

# Installation
# macOS (Homebrew)
brew install boost

# Ubuntu/Debian
sudo apt install libboost-all-dev

# vcpkg (cross-platform, recommended for projects)
vcpkg install boost

# Conan (cross-platform)
conan install boost/1.84.0

# Header-only libraries (many Boost libraries are header-only – no linking required)
# Just add the include path:
g++ -I/usr/include/boost my_program.cpp -o my_program

# Libraries requiring linking:
g++ my_program.cpp -lboost_filesystem -lboost_system -o my_program

# CMake (recommended)
# find_package(Boost REQUIRED COMPONENTS filesystem system regex)
# target_link_libraries(myapp PRIVATE Boost::filesystem Boost::system Boost::regex)

```


9.7.5 Decision Framework: Boost vs. Standard Library vs. Other

Do you need the functionality?

|
v

Is it in the C++ standard library (C++17/20)?

|-- Yes --> Use the standard library version

|-- No --> Is it in Boost?

|-- Yes --> Is there a lighter-weight alternative?

|-- Yes --> Evaluate both (Boost is well-tested but heavy)

|-- No --> Use Boost

|-- No --> Search for a dedicated library or write your own

// Example decisions:

// Need smart pointers? -> std::unique_ptr, std::shared_ptr (standard since C++11)

// Do NOT use boost::shared_ptr in new code

// Need filesystem operations? -> std::filesystem (standard since C++17)

// Do NOT use boost::filesystem in new C++17 code

// Need HTTP client? -> No standard equivalent -> Use Boost.Beast or a dedicated library (libcurl)

// Need JSON? -> No standard equivalent yet (C++26 maybe) -> Use Boost.JSON, nlohmann/json, or...

// Need graph algorithms? -> No standard equivalent -> Use Boost.Graph

// Need async networking? -> No standard equivalent -> Use Boost.Asio

9.7.6 Things to Remember

- Boost is a community of C++ library developers and a collection of peer-reviewed, portable C++ libraries. It serves as a proving ground for additions to the C++ standard.
- Many C++ standard library features originated in Boost, including smart pointers, function, bind, tuple, array, unordered containers, regex, random, filesystem, optional, variant, any, string_view, type traits, and threading primitives. When a Boost library becomes part of the standard, prefer the standard version.
- Boost provides many libraries that have no standard equivalent: Asio (networking), Spirit (parsing), Graph, Serialization, Multiprecision, Test, Log, PropertyTree, Signals2, Interprocess, Geometry, Hana, Beast (HTTP/WebSocket), and many more.
- Boost's peer-review process ensures high quality. Before writing a substantial piece of infrastructure, check if Boost already provides it.
- Use your package manager (vcpkg, Conan, system package manager) to install Boost. Many Boost libraries are header-only and require no linking.

9.8 Summary of Chapter 9

Item	Core Message
53	Compiler warnings catch real bugs. Use maximum warning levels, treat warnings as errors, and leverage modern C++ attributes (<code>override</code> , <code>[[nodiscard]]</code> , <code>[[fallthrough]]</code>) to eliminate entire categories of mistakes.
54	The C++ standard library is vast and powerful. Everything from TR1 is now standard. Know your containers, algorithms, smart pointers, type traits, regex, random numbers, chrono, filesystem, concurrency primitives, and C++20 features like ranges and concepts. Don't reinvent what already exists.
55	Boost is the primary feeder for the C++ standard library. Many Boost libraries are now part of the standard. For functionality not yet standardized (networking, parsing, graphs, serialization), Boost remains the go-to source of high-quality, peer-reviewed code.